Automated Test Data and Test Oracle Generation for REST APIs

Juan Carlos Alonso Valenzuela

PhD dissertation

Supervised by

Dr. Sergio Segura Rueda and

Dr. Antonio Ruiz Cortés



Universidad de Sevilla

May 2025

First published in May 2025 by
Juan Carlos Alonso Valenzuela
Copyright © MMXXV
https://www.javalenzuela.com
javalenzuela@us.es

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

Support: This work has been partially supported by grants PID2021-126227NB-C22 and PID2021-126227NB-C21, funded by MCIN/AEI /10.13039/501100011033/FEDER, UE. Other contributing projects are grant TED2021-131023B-C21, funded by MCIN/AEI/10.13039/501100011033 and by European Union "NextGenerationEU"/PRTR; and "Smart Computer systems Research and Engineering (SCORE)", funded by the Consejería de Transformación Económica, Industria, Conocimiento y Universidades of Junta de Andalucía as a part of the "Programa QUALIFICA relativo al Programa de Ayudas a Acciones Complementarias de I+D+i, en la modalidad de fortalecimiento de Institutos Universitarios de Investigación de las Universidades Andaluzas, Centros e infraestructuras para la adquisición del Sello 'SEVERO OCHOA' o 'MARÍA DE MAEZTU"'.

Dr. Sergio Segura Rueda, Catedrático de Universidad del Área de Lenguajes y Sistemas Informáticos de la Universidad de Sevilla y Dr. Antonio Ruiz Cortés, Catedrático de Universidad del Área de Lenguajes y Sistemas Informáticos de la Universidad de Sevilla,

HACEN CONSTAR

que D. Juan Carlos Alonso Valenzuela, Máster Universitario en Ingeniería del Software: Cloud, Datos y Gestión de las Tecnologías de la Información y Graduado en Ingeniería Informática-Ingeniería del Software, ha realizado bajo nuestra supervisión el trabajo titulado

Automated Test Data and Test Oracle Generation for REST APIs

Una vez revisado, autorizamos el comienzo de los trámites para su presentación como Tesis Doctoral al tribunal que ha de juzgarlo.

Fdo. Dr. Sergio Segura Rueda y Dr. Antonio Ruiz Cortés
en la Universidad de Sevilla
05/05/2025

Yo, D. Juan Carlos Alonso Valenzuela con NIF número 47548996S,

DECLARO

mi autoría del trabajo que se presenta en la memoria de esta tesis doctoral que tiene por título:

Automated Test Data and Test Oracle Generation for REST APIs

Lo cual firmo,

Fdo. D. Juan Carlos Alonso Valenzuela
en la Universidad de Sevilla
05/05/2025

Universidad de Sevilla

The committee in charge of evaluating the dissertation presented by Juan Carlos Alonso Valenzuela in partial fulfillment of the requirements for the degree of Doctor of Philosophy in Software Engineering, hereby recommends _____ of this dissertation and awards the author the grade _____.

Carlos Guillermo Müller Cejás Associate Professor Universidad de Sevilla	Aitor Arrieta Marcos Associate Professor Mondragon Unibertsitatea
Alessandra Gorla Associate Research Professor IMDEA Software Institute	Alberto Martín López Postdoctoral Researcher Università della Svizzera italiana

Ana Belén Sánchez Jerez
Profesora Permanente Laboral
Universidad de Sevilla

To put record where necessary, we sign minutes in

_____, _____

Dedicado a la memoria de mi abuelo José, que a través de su vida y acciones me enseñó lo importante que es estudiar y lo lejos que te puede llevar el esfuerzo.

Acknowledgments

I have been thinking about this moment for a long time, wishing to sit down and write this final section of my doctoral thesis, which brings closure to a research project that began during the final year of my Bachelor's degree in Software Engineering. Although this section did not require an exercise in abstraction or implementation, nor technical knowledge or a detailed analysis of the literature, it may be the one to which I have dedicated the most time—or at least the first one I started working on—because, since the beginning of this project, I have encountered many people who have supported and helped me to achieve my goals in countless ways, always selflessly and often without even realizing it, simply because that's who they are. Since then, I have spent a lot of time thinking about how to show them how grateful I am, but despite all that time, I have the feeling that I won't be able to capture all of it in these pages, but I will do my best.

First, I would like to begin by thanking my supervisors, Sergio Segura and Antonio Ruiz, for everything they have done for me. Sergio, it goes without saying that you are the best thesis advisor one could ask for, for countless reasons that would be difficult to condense in so little space. I don't know how you do it, but you make it look easy from the outside. You have guided me in the best possible way from the beginning, you have a vast number of ideas for research lines, and you always propose research topics that are far ahead of what can be found in the current literature. You focus on contributions with clear practical applications that truly add value to their users, instead of chasing easy publications or following trends. Despite having such great ideas and a clear global vision, you greatly respect the opinions and ideas of your students, giving them full freedom and offering various alternatives if a research line or method doesn't convince them, like when you allowed me to obsess over studying a classic software tool named after a vegetable after discarding trendier technologies like Deep Learning or Reinforcement Learning. But speaking only about research and supervision would fall very short. Above all that, I consider you a friend; you bring out the best in people and have always been an inspiration, not just for the kind of researcher and professor, but for the kind of person I want to become. Thank you for supporting me during the

hardest times of this process, especially when you realized I had doubts about continuing in academia and took the initiative to help me in every possible way. I truly hope to continue collaborating with you in the future. I think almost daily that reaching out to you back then to supervise my Bachelor's Thesis is possibly the best decision I have ever made. I am so glad to have met you. Antonio, thank you very much for giving me the opportunity to be part of the ISA group, where I met the best colleagues I could have ever imagined. Your passion for research and for finding new ways to improve is contagious; you are the eternal optimist and always work selflessly for others. I still remember one of our conversations about the importance of teaching during my first month as a lecturer: you told me that the job of a good professor is to be aware of their students' potential, capabilities, and interests and *to do everything possible to bring out the best in each and every one of them*. It's a piece of advice I have kept in mind in every lecture I have given since that day.

I extend these thanks to all my colleagues in the ISA group, with whom I have shared many moments, not only during working hours. I don't know of another research group that, besides being so multidisciplinary, is also so united. Within this group, I am especially grateful to Alberto Martín, Carlos Müller, Ana Belén Sánchez, and Adela del Río. Alberto, thank you so much for all the help and advice you gave me from my beginnings as a researcher. I have always seen you as the ideal model of a doctoral student (and now postdoctoral researcher) to follow; you do everything well and are someone in a continuous process of improvement. I can't even imagine how far you'll go. Carlos, you are, without a doubt, the best professor I had during my entire degree. You made my project group to enjoy attending every class of the toughest course of the degree, where even the smallest mistake meant failing the whole course, and to this day I can only remember the good things. You were the first person to see my potential as a researcher and gave great references to my supervisors. You are also the first person (after my supervisors) I call whenever I win an award or publish a paper, and I really enjoy our conversations and our book exchanges. Ana, thank you so much for all the advice and help you gave me selflessly during my first steps as a lecturer in Operating Systems. I'm sure it's thanks to you (along with the fantastic course organization and coordination) that it's my favorite course to teach. The same goes for Adela; I greatly enjoyed the AISS practice sessions we taught in parallel, and thanks to you, I have very fond memories of that semester when I taught eight hours on Friday afternoons.

Many thanks to my office mates from E0.41: Antonio Martínez, Miguel Ángel Olivero, Manuel Jesús Jiménez, and Alfonso Bravo, with whom I have shared countless

inside jokes and references. You made a work-from-home advocate like me to look forward to coming to the office every day, making minor details like productivity or getting an extra hour of sleep seem unimportant. The same goes for all the other PhD students from the PSI Miami group: Giuliano Mirabella, Carlos Capitán, Elena Molino, María Salas, and David Romero. David, you have a sense of humor like no one else I've ever met, and somehow you manage to make me laugh no matter how tough things get. You are one of those people who never gives up and exactly the kind of professor the university needs.

I also want to thank Aitor Arrieta and Pablo Valle, with whom I was lucky enough to meet at many conferences and collaborate on several publications, and my internship supervisors, Michael Ernst and Gabriele Bavota, as well as their research groups, who made me feel at home from the very first day. Speaking of my internships, I can't forget Sujit and Sampa, the hosts of my accommodation while I lived in Seattle. It's impossible to find a better place to stay in that city, I truly enjoyed our daily conversations and you made me feel like part of your family.

Thanks to all the students who made my work as a lecturer one of the best experiences of my life. During the final year of my thesis, I had the honor of being voted to appear in the class photo of the second graduating class I taught (despite only teaching a couple of practice groups in their second year). Sadly, I had to turn down countless Bachelor's thesis requests from former students after I "retired" from teaching. In many ways, I consider my time as a professor to be a far more valuable contribution than any of my scientific publications, even if the university doesn't always give it the importance it deserves. Among all these students, I want to especially mention those with whom I continued to have SCRUM meetings at 100 Montaditos after their graduation: Alfonso Alonso, Pablo Caballero, Ricardo Carreño, and Antonio Porcar.

To my group of friends, thank you for always being there (especially on Fridays) and for understanding my lack of availability these past months. Ramón, you are the best friend anyone could have, always there when needed, especially at key moments during my doctoral thesis like the Sujimon League and Dondoko Island. Roberto, I've known you the longest, and maintaining our friendship means a lot to me; you are a lifelong friend, and you've shown me I can count on you for anything. Gabie, you are by far the programmer I admire most, and the only person outside the university with whom I have discussed my research in depth, not to mention our long conversations about how fascinating the academic world is. Pablo Rebollo, thank you, among many other things, for not insulting my limited intelligence during the countless times

I made the most disastrous Mahjong moves thinking I was doing something clever. Alex, thanks for the endless conversations we had at all hours during my stay in the United States. I'm still unclear about your sleep schedule, but it honestly concerns me a little. Guti, thank you for always doing everything possible to stay in touch and for always finding time for us to catch up.

And I leave the best for last. I want to thank my family, especially my parents, my brother, and my grandmother. Grandma, thank you for always taking care of me and encouraging me since I was a child; I will continue doing my best to make you proud. Mom, Dad, without you, literally, this thesis would not exist (nor would I). I will never be able to thank you enough for everything you have done for me, including all the sacrifices you made to make my academic career possible. You have always believed in me—like when I said in primary school that I wanted to study at a high school in Seville and you moved heaven and earth to make it happen, or when you supported me to study in Germany, among many other things. I have tried to make the most of all the countless opportunities you have given me, and although I know I can sometimes be difficult, thank you so much for your understanding and support. I want to keep improving so that you can always be proud of me.

Agradecimientos

Llevaba mucho tiempo pensando en este momento, deseando sentarme y escribir esta última sección de la tesis doctoral, que le da cierre a un proyecto de investigación que comenzó durante mi último año del grado en Ingeniería del Software. Aunque no se trate de una sección que haya requerido de un ejercicio de abstracción o implementación, ni que requiera de conocimientos técnicos o un análisis pormenorizado de la literatura, puede ser a la que más tiempo he dedicado, o como mínimo la primera en la que empecé a trabajar ya que, desde que empezó este proyecto, me he cruzado con muchas personas que me han apoyado y ayudado a conseguir mis objetivos de muchísimas formas, siempre de forma desinteresada y, en muchos casos, quizá sin darse cuenta, porque simplemente es su forma de ser. Desde entonces he dedicado mucho tiempo a pensar en formas de mostrarles lo agradecido que estoy, pero a pesar del tiempo que he empleado, tengo la impresión de que voy a ser incapaz de plasmar todo eso en estas páginas, aunque haré mi mejor esfuerzo.

En primer lugar, me gustaría comenzar agradeciendo a mis supervisores, Sergio Segura y Antonio Ruiz, por todo lo que han hecho por mí. Sergio, sobra decir que eres el mejor director de tesis posible, por una cantidad innumerable de razones que difícilmente podría condensar en tan poco espacio. No sé cómo lo haces, pero consigues que desde fuera parezca fácil, has sabido guiarme de la mejor manera desde el principio, tienes una gran cantidad de ideas para posibles líneas de investigación, y siempre realizas propuestas que se encuentran muy adelantadas con respecto a lo que puede encontrarse en la literatura actual, te centras en realizar contribuciones que tengan claras aplicaciones prácticas y que de verdad puedan aportar valor a sus usuarios, en lugar de buscar realizar publicaciones fáciles o dejarte llevar por modas. A pesar de tener ideas tan buenas y una visión global tan clara de todo, también respetas muchísimo las opiniones e ideas de tus estudiantes, dándoles total libertad y ofreciéndoles varias alternativas si alguna línea de investigación o método no les convence, como cuando me permitiste obsesionarme con estudiar un software clásico con nombre de hortaliza tras haber descartado utilizar tecnologías mucho más de moda y que ya conocía como el Deep learning o el aprendizaje por refuerzo. Pero hablar solo de investigación y su-

pervisión sería quedarse muy corto. Por encima de todo eso, te considero un amigo, sacas lo mejor de las personas y siempre has sido una inspiración no solo de la clase de investigador y profesor, sino de persona que quiero llegar a ser. Muchas gracias por apoyarme en los momentos más duros de todo este proceso, especialmente cuando te diste cuenta de que tenía dudas sobre si continuar en la universidad y tuviste la iniciativa de ayudarme de todas las formas posibles, de verdad que espero seguir colaborando contigo en el futuro. Pienso prácticamente a diario que contactarte en su día para supervisar mi TfG es posiblemente la mejor decisión que he tomado en mi vida, me alegro de haberte conocido. Antonio, muchas gracias por darme la oportunidad de formar parte del grupo ISA, donde he conocido a los mejores compañeros de trabajo que podría imaginar. Tu pasión por la investigación y por buscar nuevas formas de mejorar es contagiosa, eres el eterno optimista y siempre trabajas de forma desinteresada por los demás. Aún recuerdo una de las conversaciones sobre la importancia de la docencia que tuvimos durante mi primer mes como profesor, me dijiste que el trabajo de un buen docente es ser consciente del potencial, capacidades e intereses de sus alumnos, y *hacer todo lo posible para sacar lo mejor de todos y cada uno de ellos*. Es un consejo que he tenido muy presente en todas las clases que he impartido desde ese día.

Extiendo estos agradecimientos a todos mis compañeros del grupo ISA, con los que he compartido muchos momentos, no solo en horario laboral. No conozco a otro grupo de investigación que, además de ser tan multidisciplinar, esté tan unido como este. Dentro de este grupo de personas, estoy especialmente agradecido a Alberto Martín, Carlos Müller, Ana Belén Sánchez y Adela del Río. Alberto, muchas gracias por toda la ayuda y consejos que me has dado desde mis inicios como investigador, siempre te he visto como el mejor modelo posible de estudiante de doctorado (y ahora de investigador posdoctoral) a seguir, lo haces todo bien y eres una persona en un continuo proceso de mejora, no puedo ni imaginar lo lejos que vas a llegar. Carlos, eres, sin duda y por excelencia, el mejor profesor que tuve durante toda la carrera, conseguiste que mi grupo de trabajo disfrutase y fuese con ganas a todas y cada una de las clases de la asignatura más dura de la carrera, donde el más mínimo error significaba un suspenso, y a día de hoy solo puedo recordar las cosas buenas. Fuiste el primero que vio en mí potencial como investigador y diste muy buenas referencias a mis supervisores. Eres el primero (después de mis directores) al que llamo cada vez que consigo un premio o una publicación, y disfruto mucho de nuestras conversaciones e intercambios de material de lectura. Ana, muchas gracias por todos los consejos y ayuda que me aportaste de manera desinteresada durante mis inicios como profesor de Sistemas Operativos, estoy seguro de que es gracias a ti (junto a la fantástica organización y coordinación de

la asignatura) que esta sea mi asignatura favorita para impartir. Algo parecido ocurre con Adela, disfruté mucho de las clases de prácticas de AISS que impartimos en paralelo, y gracias a ti tengo muy buenos recuerdos del cuatrimestre en el que impartía ocho horas de clase los viernes por la tarde.

Muchas gracias a mis compañeros de mi antiguo despacho, el E0.41: Antonio Martínez, Miguel Ángel Olivero, Manuel Jesús Jiménez y Alfonso Bravo, con los que he compartido incontables bromas internas y referencias. Conseguisteis que un defensor del teletrabajo como yo fuese con ganas al despacho todos los días, haciendo que detalles menores como la productividad o dormir una hora más pasasen a un segundo plano. Lo mismo se aplica a todos los demás estudiantes de doctorado del grupo de PSI Miami: Giuliano Mirabella, Carlos Capitán, Elena Molino, María Salas y David Romero. David, tienes un sentido del humor como el de nadie haya conocido, y de alguna manera te las apañas para hacerme reír en cualquier momento. Eres de esas personas que nunca se rinden y la clase de profesor que la universidad necesita.

También quiero agradecer a Aitor Arrieta y Pablo Valle, con quienes he tenido la suerte de coincidir en muchas conferencias y trabajar de forma conjunta en varias publicaciones, a los supervisores de mis estancias, Michael Ernst y Gabriele Bavota, así como a sus grupos de investigación, con los que me sentí como en casa desde el primer día. Hablando de mis estancias, no puedo olvidarme de Sujit y Sampa, los anfitriones de mi alojamiento durante el tiempo que residí en Seattle, es imposible elegir un lugar mejor para alojarse en esa ciudad, disfruté muchísimo de nuestras conversaciones diarias y me hicisteis sentir como parte de vuestra familia.

A todos los alumnos gracias a los cuales recuerdo mi trabajo como profesor como una de las mejores cosas que he hecho en la vida. Durante el último año de tesis, he tenido el honor de ser votado para aparecer en la orla de la segunda promoción de estudiantes a los que enseñé (a pesar de haber impartido solo un par de grupos de prácticas durante segundo curso) y lamentablemente he tenido que rechazar innumerables solicitudes de TfGs por parte de antiguos alumnos por haberme jubilado de la docencia. En muchos aspectos, considero que mi tiempo como profesor es una contribución mucho más valiosa que cualquiera de mis publicaciones científicas, aunque la universidad no le de toda la importancia que se merece. De entre todos estos alumnos, me gustaría destacar a aquellos con los que he realizado (tras su graduación) SCRUM meetings en el 100 Montaditos: Alfonso Alonso, Pablo Caballero, Ricardo Carreño y Antonio Porcar.

A mi grupo de amigos, por estar siempre ahí (especialmente los viernes) y por com-

prender mi falta de disponibilidad durante estos meses. Ramón, eres el mejor amigo que se puede tener y siempre estás ahí cuando se te necesita, especialmente en momentos claves para la consecución de mi tesis doctoral como la liga Sujimon y la isla Dondoko. Roberto, eres al que conozco desde hace más tiempo, y significa mucho para mí mantener tu amistad, eres de esos amigos que son para toda la vida y me has demostrado que puedo contar contigo para cualquier cosa. Gabie, eres sin duda el programador al que más admiro, y la única persona fuera de la universidad con la que he discutido mi investigación de forma detallada, además de mantener largas conversaciones sobre lo fascinante que es el mundo de la academia. Pablo Rebollo, gracias entre otras muchas cosas por no insultar mi limitada inteligencia las innumerables veces que he realizado las jugadas de Mahjong más lamentables posibles creyendo que estaba haciendo algo con sentido. Alex, gracias por las extensas conversaciones que mantuvimos a todas horas durante mi estancia en Estados Unidos, no tengo muy claros cuáles son tus horarios de sueño, pero me preocupan un poco. Guti, muchas gracias por hacer todo lo posible para que sigamos manteniendo siempre el contacto y que siempre podamos encontrar un hueco para ponernos al día.

Dejo lo mejor para el final. Le doy las gracias a mi familia, especialmente a mis padres, mi hermano y mi abuela. Abuela, muchas gracias por cuidarme y animarme siempre desde que era un niño, seguiré haciendo todo lo posible para que estés orgullosa de mí. Mamá, papá, sin vosotros literalmente esta tesis no existiría (ni yo tampoco), nunca podré agradeceros todo lo que habéis hecho por mí, como todos los esfuerzos y sacrificios que habéis hecho para hacer posible mi carrera académica. Siempre habéis creído en mí, como cuando en primaria dije que quería estudiar en un instituto de Sevilla y movisteis cielo y tierra para hacerlo posible, o cuando me apoyasteis para que pudiese realizar mi Erasmus en Alemania, entre otras muchas cosas. He hecho todo lo posible para aprovechar las innumerables oportunidades que me habéis dado y sé que a veces puedo ser una persona difícil, pero muchas gracias por toda vuestra comprensión y apoyo. Quiero seguir mejorando para que siempre podáis estar orgullosos de mí.

Abstract

Web *Application Programming Interfaces (APIs)* enable communication between heterogeneous software systems over the network. Most applications we use daily rely on one or more web APIs to deliver their functionality. These APIs—fundamental components powering services from companies like Google, Meta, and Microsoft—typically adopt the *REpresentational State Transfer (REST)* architectural style and are known as *REST* or *RESTful APIs*. By offering external access to their APIs, companies and independent developers allow third-party developers to integrate their services into their applications. This widespread openness and interoperability have made REST APIs a central pillar in the modern software ecosystem, driving seamless integration across platforms and services.

Given the critical role REST APIs play in software integration, thoroughly testing them is essential: a single fault in an API can compromise the functionality of all systems that depend on it. In recent years, the automated testing of REST APIs has emerged as a vibrant research area, with numerous approaches leveraging API specifications to automatically generate test cases. To support this need, many companies now provide *Testing as a Service (TaaS)* solutions, offering developers a practical way to test APIs without the overhead of managing dedicated testing infrastructure.

The main objective of this dissertation is to address two major limitations commonly found in existing automated testing approaches for REST APIs: the lack of mechanisms for generating realistic input values automatically, and the limited capability of their test oracles, which often focus solely on detecting server errors or validating response syntax. To overcome these challenges, this work introduces three novel contributions: ARTE, an approach that leverages the specification of an API and execution feedback to generate realistic input values; AGORA+, a technique that infers test oracles from previous API executions; and SATORI, a method that analyzes the specification of an API to generate test oracles, without requiring prior API execution. These contributions have been implemented as tools designed to integrate seamlessly with existing automated testing frameworks, thereby enhancing their ability to generate valid API requests and detect a broader range of failures.

We conducted extensive evaluations of our proposed approaches in real-world scenarios, focusing on industrial APIs with millions of active users. Our techniques uncovered over 40 previously undetected failures in widely used commercial platforms such as Amadeus Hotel, YouTube, Vimeo, Foursquare, GitHub, and GitLab. These findings have led to multiple bug fixes and improvements in API documentation, demonstrating the practical impact and effectiveness of our contributions.

RESUMEN

Las *Interfaces de Programación de Aplicaciones (APIs)* web permiten la comunicación entre distintos sistemas software a través de internet. La mayoría de las aplicaciones que usamos a diario dependen de una o más APIs web para ofrecer su funcionalidad. Estas APIs—componentes fundamentales detrás de los servicios ofrecidos por empresas como Google, Meta y Microsoft—adoptan comúnmente el estilo arquitectónico *REpresentational State Transfer (REST)* y son conocidas como APIs *REST* o *RESTful*. Al ofrecer acceso externo a sus APIs, tanto empresas como desarrolladores independientes permiten que terceros integren sus servicios en sus propias aplicaciones. Esta interoperabilidad ha convertido a las APIs REST en un pilar central del ecosistema software moderno, facilitando una integración fluida entre plataformas y servicios.

Dado el papel crítico que desempeñan las APIs REST en la integración software, probarlas de forma exhaustiva es esencial: dado que un solo fallo en una API puede comprometer la funcionalidad de todos los sistemas que dependen de ella. En los últimos años, el desarrollo de técnicas de pruebas automatizadas de APIs REST se ha consolidado como un área de investigación de gran relevancia, con numerosos enfoques que aprovechan las especificaciones de las APIs para generar casos de prueba automáticamente. Para apoyar esta necesidad, muchas empresas ofrecen actualmente soluciones de *Pruebas como Servicio* o *Testing as a Service (TaaS)*, proporcionando a los desarrolladores una forma práctica de probar APIs sin tener que gestionar una infraestructura de pruebas propia.

El principal objetivo de esta tesis es abordar dos limitaciones importantes que suelen encontrarse en las técnicas existentes de pruebas automatizadas de APIs REST: la falta de mecanismos para generar automáticamente valores de entrada realistas, y la capacidad limitada de sus oráculos de prueba, que a menudo se enfocan únicamente en detectar errores del servidor o validar la sintaxis de las respuestas. Para afrontar estos desafíos, este trabajo presenta tres contribuciones: ARTE, un enfoque que analiza la especificación de una API y las respuestas devueltas por esta para generar valores de entrada realistas; AGORA+, una técnica que infiere oráculos de prueba a partir de ejecuciones previas de la API; y SATORI, un método que analiza la especificación de

una API para generar oráculos de prueba sin necesidad de ejecutarla previamente. Estas contribuciones se han implementado como herramientas diseñadas para integrarse de manera fluida con frameworks de pruebas automáticas existentes, mejorando así su capacidad para generar llamadas válidas a las APIs y detectar una gama más amplia de errores.

Como parte de nuestra experimentación, hemos llevamos a cabo evaluaciones exhaustivas de las técnicas propuestas en esta tesis en escenarios realistas, centrándonos en APIs industriales con millones de usuarios activos. Nuestras técnicas revelaron más de 40 fallos previamente no detectados en plataformas comerciales ampliamente utilizadas como Amadeus Hotel, YouTube, Vimeo, Foursquare, GitHub y GitLab. Estos hallazgos han conducido a múltiples correcciones de errores y mejoras en la documentación de las APIs, lo que demuestra el impacto práctico y la efectividad de nuestras contribuciones.

Contents

I Preface **1**

1 Introduction **3**

1.1 Research Context 4

1.1.1 REST APIs 4

1.1.2 Automated Testing of REST APIs 6

1.2 Problem Statement 7

1.3 Dissertation Goal and Research Questions 9

1.4 Research Methodology 10

1.5 Contributions 11

1.5.1 Introduction 11

1.5.2 Publications 14

1.5.3 Developed Tools 19

1.5.4 Datasets 20

1.5.5 Research Stays 21

1.5.6 Awards 22

1.5.7 Other Achievements 23

1.6 Structure of the Dissertation 24

II Background and State of the Art **27**

2 Software Testing **29**

2.1 Introduction . . . 30

2.2 The Software Testing Procedure . . . 30

2.3 Software Testing Taxonomies . . . 31

2.3.1 Software Testing Levels . . . 31

2.3.2 Manual and Automated Testing . . . 32

2.3.3 Black-box and White-box Testing . . . 33

2.3.4 Regression and Progressive Testing . . . 35

2.4 Test Adequacy Criteria . . . 35

2.4.1 Test Coverage . . . 35

2.4.2 Mutation Testing . . . 37

2.5 Test Input Generation Techniques . . . 40

2.6 Test Oracle Generation Techniques . . . 41

2.6.1 Invariant Detection . . . 41

2.7 Summary . . . 44

3 REST APIs **45**

3.1 Introduction . . . 46

3.2 OAS Specification . . . 47

3.3 REST API Testing . . . 49

3.3.1 Manual Testing of REST APIs . . . 52

3.3.2 Automated Testing of REST APIs . . . 52

3.3.3 Testing as a Service of REST APIs . . . 56

3.4 Summary . . . 57

4 Web of Data **59**

4.1 Introduction 60

4.2 Resource Description Framework (RDF) 61

4.3 Knowledge Bases 62

4.4 SPARQL 63

4.5 Summary 64

5 Natural Language Processing **65**

5.1 Introduction 66

5.2 The Transformer Architecture 67

5.3 Large Language Models 69

5.3.1 Prompting Techniques 70

5.3.2 The LLM Training Pipeline 71

5.4 The Hugging Face Platform 73

5.5 Summary 75

III Contributions **77**

6 ARTE: Automated Generation of Realistic Test Inputs for REST APIs **79**

6.1 Introduction 80

6.2 ARTE 82

6.2.1 Processing Test Parameters 84

6.2.2 Search for Predicates 85

6.2.3 Search for Test Inputs 88

6.2.4 Automated Generation of Regular Expressions 89

6.3 Tooling 91

6.4 Evaluation 92
6.4.1 Baselines 93
6.4.2 Experiment 1: Generation of Realistic Test Inputs 95
6.4.3 Experiment 2: Automated Testing 99
6.4.4 Discussion 102
6.5 Threats to Validity 105
6.6 Conclusions 107

7 AGORA+: Dynamic Generation of Test Oracles for REST APIs 109
7.1 Introduction 110
7.2 AGORA+ 113
7.2.1 Beet: A Daikon Front-end for REST APIs 114
7.2.2 Invariant Definition 120
7.2.3 Minimizing False Positives 123
7.2.4 PostmanAssertify: Automated Generation of Test Assertions . . . 124
7.3 Evaluation 128
7.3.1 Experimental Data 128
7.3.2 Experiment 1: Test Oracle Generation 130
7.3.3 Experiment 2: Failure Detection 139
7.4 Detected Failures 143
7.5 Threats to Validity 146
7.6 Conclusions 148

8 SATORI: Static Generation of Test Oracles for REST APIs 149
8.1 Introduction 150
8.2 SATORI 153

8.2.1 Automated Response Field Prompt Generation 153
8.2.2 Process LLM Response 155
8.2.3 Target Oracles 156
8.3 Evaluation 156
8.3.1 Experiment 1: Test Oracle Generation 157
8.3.2 Experiment 2: Artificial Failure Detection 164
8.3.3 Experiment 3: Real Failure Detection 168
8.4 Threats to Validity 170
8.5 Conclusions 171

IV Final Remarks 173

9 Conclusions and Future Work 175
9.1 Conclusions 176
9.2 Discussion 176
9.3 Limitations 178
9.4 Future Work 179

V Appendices 183

A Supplemental Material for: ARTE: Automated Generation of Realistic Test Inputs for Web APIs 185

B Acronyms 191

Bibliography 197

List of Figures

1.1 Summary of publications grouped by year and topic. 19

2.1 Basic testing procedure. 31

3.1 REST API interaction workflow. 46

3.2 Workflow of an automated REST API testing tool. 53

3.3 RESTest workflow. 55

4.1 RDF triple. 61

4.2 Simple RDF graph. 62

5.1 The Transformer model architecture [292]. 68

6.1 Workflow of ARTE. 84

6.2 Graphical representation of the search for test inputs. 88

7.1 Workflow of AGORA+. 113

7.2 Postman collection generated by PostmanAssertify. 125

7.3 Precision and number of invariants reported, depending on the size of the input dataset. 132

7.4 Reported invariants by category in Daikon, AGORA, and AGORA+. . . 135

7.5 How test suite size affects the precision of AGORA+. 138

7.6 How test suite size affects the number of valid invariants reported by AGORA+. 138

7.7 Distribution of the precision of AGORA+ in the 50 test cases subset across 10 executions. 139

7.8 AGORA+ performance detecting each mutation operator. 143

8.1 Workflow of SATORI. 153

8.2 Precision, recall and F1-Score of each model evaluated according to several criteria. Numbers above markers denote average time (in seconds) to generate oracles for a single response field. 160

8.3 Oracle overlapping between SATORI and AGORA+$_U$. 163

8.4 Per API oracle generation overlapping. 164

8.5 FDR overlapping between SATORI and AGORA+. 167

List of Tables

2.1 Test Coverage Model: Levels and Criteria. 37

3.1 CRUD example. 46

4.1 SPARQL query response. 64

6.1 Example of automatically generated regular expression. 90

6.2 ARTE configuration parameters and default values. 91

6.3 Per API breakdown of input values generation for the RapidAPI dataset. O = Operations, P = Parameters. 97

6.4 Per API breakdown of input values generation for the industrial APIs. O = Operations, P = Parameters. 98

6.5 Per API breakdown of valid calls, coverage, and failures detected. P = "Parameters", VC = "Valid calls", C = "Coverage", F = "Failures". 101

7.1 Daikon declaration file format for API Requests and Responses. 114

7.2 Summary of modifications performed on Daikon. The strike-through invariants had to be disabled during our evaluation to avoid a combinatorial explosion of reported false positives. 121

7.3 Test oracle generation. I="Number of likely invariants", P="Precision (% valid test oracles)" 133

7.4 Failure Detection Ratio per API operation. 142

8.1 Prompt structure. 154

8.2 Test oracles supported by SATORI. 157

8.3 Test oracle generation by SATORI and AGORA+$_U$. PI="Positive instances", NI="Negative instances", P="Precision", R="Recall", F1="F1-Score" 162

8.4 Test oracle generation by AGORA+. I="Invariants", P="Precision" . . . 163

8.5 Failure Detection Ratio per API operation achieved by each approach. A="Assertions" 166

8.6 Faults detected by SATORI and AGORA+. 169

List of Listings

1.1 OAS excerpt of the Spotify API. 5

1.2 Example of automatically generated test case. 7

1.3 Spotify API response in JSON format. 8

2.1 Example of program mutation. 38

2.2 Example of JSON mutation. 39

2.3 Java computeSquare function. 42

2.4 Daikon declaration file. 43

2.5 Daikon dtrace file. 43

2.6 Likely invariants of computeSquare. 43

3.1 OAS structure of the DHL API. 48

3.2 OAS excerpt of the find-by-address operation of the DHL API. 50

3.3 DHL OAS response schema with example response. 51

3.4 Example of test case implemented using REST Assured. 52

4.1 SPARQL query to search for books. 63

6.1 OAS excerpt of the DHL API. 83

6.2 SPARQL query used to search for predicates. 86

6.3 SPARQL query for searching test inputs. 89

7.1 Spotify API response in JSON format, for the request GET `https://api.spotify.com/albums/4Em5W5HgYEvhpc/tracks?limit=1&market=ES`. . . . 111

7.2 An excerpt of the getAlbumTracks operation of the Spotify API, expressed in OAS (OpenAPI Specification) format. Listing 7.1 shows a response. . . . 115

7.3 ENTER program point of an API operation. . . . 116

7.4 EXIT program point of an API operation. . . . 116

7.5 Second EXIT nesting level. . . . 117

7.6 ENTER data trace file. . . . 119

7.7 One instance of an EXIT program point in a data trace file. . . . 120

7.8 Invariants detected in the "getAlbumTracks" operation of the Spotify API. 123

7.9 Part of the Postman test script generated by PostmanAssertify for the Spotify "getAlbumTracks" operation. . . . 127

8.1 Yelp API response in JSON format. . . . 150

8.2 OAS excerpt of the Yelp API. . . . 152

8.3 Example of test oracles generated by SATORI. . . . 155

Part I

Preface

1

Introduction

It may seem difficult at first, but all things are difficult at first.

Musashi Miyamoto,
The Book of Five Rings

This dissertation provides a set of techniques and tools aimed at enhancing the effectiveness and failure detection capabilities of existing approaches for the automated testing of REST APIs. This chapter begins by presenting the research context in Section §1.1 and stating the research problem that drives our research in Section §1.2. Section §1.3 enunciates the goal of the dissertation and the research questions that we seek to answer. Section §1.4 provides an overview of the research methodology followed throughout the PhD. In Section §1.5, we summarize our main contributions and publications. Finally, Section §1.6 outlines the structure of the dissertation.

1.1 Research Context

This section provides a brief overview of the key concepts that contextualize this dissertation. Section §1.1.1 introduces REST APIs, while Section §1.1.2 covers automated testing of REST APIs.

1.1.1 REST APIs

Software has become an integral part of modern life, embedded in nearly every device we use daily, including computers, smartphones, vehicles, smart TVs, smartwatches, and home assistants. These devices rarely operate in isolation; instead they often communicate with each other over the internet. This communication is usually performed using web *Application Programming Interfaces (APIs)*, which provide a standardized interface for interacting with services over a network [209]. Web APIs enable both data retrieval and manipulation (*e.g.*, searching for products on AliExpress [8]) as well as the execution of operations (*e.g.*, booking a flight through Amadeus [10]). Notably, API-based services now account for over 80% of all internet traffic [68]. Nearly every application we rely on daily is powered by one or more web APIs. APIs enable a wide range of actions, such as translating text (DeepL API [31]), booking hotel rooms (Amadeus Hotel APIs [11]), tracking package shipments (DHL API [32]), requesting rides (Uber API [105]), managing online payments (PayPal API [76]), and accessing streaming services (Spotify [95] and Twitch [104] APIs), among countless other use cases.

Web APIs have become the cornerstone of software integration, driving new consumption models such as mobile, social, *Internet of Things (IoT)*, or cloud applications. Many companies, including tech giants like Amazon, Google, Meta and Microsoft, have adopted the *API-first model* for software development [77, 98], where APIs are designed as foundational building blocks before any implementation begins. This approach accelerates development by allowing teams to work in parallel, reduces the cost of developing apps, increases the speed to market, and ensures a more consistent and scalable product architecture, among many other advantages [209]. These companies often offer public access to their APIs, enabling developers to integrate their functionality into their own applications.

Modern web APIs typically adhere to the *REpresentational State Transfer (REST)* architectural style, being referred to as REST APIs or RESTful APIs [180, 270]. REST

```yaml
paths:
  "/albums/{id}/tracks":
    get:
      operationId: "getAlbumTracks"
      parameters:
       - name: id
         description: "The Spotify ID for the album"
         in: path
         required: true
         type: string
       - name: market
         description: "An ISO 3166-1 alpha-2 country code"
         in: query
         required: false
         type: string
       - name: limit
         description: "The maximum number of items to return"
         in: query
         required: false
         type: integer
      responses:
        "200":
          description: "OK"
          schema:
            type: object
            properties:
              total:
                description: "The total number of items available to return."
                type: integer
              href:
                description: "A link to the Web API endpoint returning the full
                    result of the request"
                type: string
              items: # Array of objects
                type: array
                items:
                  type: object
                  properties:
                    artists: # Array of objects
                      type: array
                      items:
                        type: object
                        properties:
                          id:
                            type: string
                          name:
                            type: string
                    available_markets: # Array of strings
                      description: "A list of the countries in which the track can
                           be played, identified by their ISO 3166-1 alpha-2 code."
                      type: array
                      items:
                        type: string
                    id:
                      type: string
                    name:
                      type: string
                    explicit:
                      type: boolean
                    duration_ms:
                      description: "The track length in milliseconds."
                      type: integer
                    linked_from: # Nested object
                      type: object
                      properties:
                        id:
                          type: string
                        uri:
                          description: "The Spotify URI for the track."
                          type: string
```

Listing 1.1: OAS excerpt of the Spotify API.

APIs are usually decomposed into multiple REST services, each of which implements one or more create, read, update and delete (CRUD) operations on a resource (*e.g.*, a playlist in the Spotify API). These operations can be invoked by sending specific HTTP requests (generally `POST`, `GET`, `PUT` and `DELETE`) to specific API endpoints. For example, a POST HTTP request to `https://api.spotify.com/v1/users/215/playlists` would create a playlist for the user with ID `215` in the Spotify API. REST APIs are commonly described using the *OpenAPI Specification (OAS)* format [73], arguably the current industry standard. Listing 1.1 illustrates an excerpt of the Spotify OAS that describes the "getAlbumTracks" operation. The specification contains the HTTP method and the path required to call the API operation (lines 1-3), operation ID (line 4), input parameters (lines 5-20), and supported response formats (lines 21-70), each one of them with their corresponding HTTP status code (line 22). This standardized format facilitates various tasks, including the generation of interactive API documentation, automation of workflows, and creation of test cases.

1.1.2 Automated Testing of REST APIs

Software testing is the process of executing a program,usually referred to as *System Under Test (SUT)*, with the intent to find errors [254]. Software testing is carried out through the execution of test cases, which consist of *test inputs*—the data or actions provided to the program necessary to execute it—and *test oracles*, a set of assertions that define the expected outcome, allowing testers to determine whether the program works as expected. Test cases are usually grouped together into collections known as *test suites*. Software testing techniques can be classified by their abstraction level (unit, integration, system and acceptance testing), their purpose (functional and non-functional testing), the automation degree (manual and automated testing), or the knowledge of the source code (black-box and white-box testing), among other criteria [129].

Given the pervasiveness of REST APIs and their central role in software integration, testing them properly is of paramount importance, as a single bug in a REST API may compromise the functionality of all the systems that rely on it. REST APIs are usually tested at the system level, by executing individual operations (*i.e.*, HTTP requests) or sequences of them. For example, to test the functionality of playlists in the Spotify API [95], we would perform the following requests: 1) POST to the `/users/{user_id}/playlists` path to create a playlist; 2) POST to `/playlists/{playlist_id}/tracks` to add songs to the playlist. 3) GET to `/playlists/{playlist_id}` to retrieve playlist

```
1  @Test
2  public void testGetAlbumTracks() {
3
4      // Send a GET request to retrieve album tracks and check status code
5      given()
6          .baseUri("https://api.spotify.com/v1") // Set the base API URL
7              .header("Authorization", "Bearer " + token) // Authorization header
8              .param("market", "ES") // market query parameter
9              .param("limit", 1) // limit query parameter
10         .when()
11             .get("/albums/" + "4Em5W5HgYEvhpc2elrpKES" + "/tracks") // API endpoint
12         .then()
13             .statusCode(200); // Verify HTTP 200 OK response
14 }
```

Listing 1.2: Example of automatically generated test case.

details. 4) DELETE to `/playlists/{playlist_id}/tracks` to remove songs from the created playlist. Each request requires providing the API with valid input values (for parameters such as `user_id` or `playlist_id`), and verifying different aspects of the API responses, such as checking that they are successful (2XX HTTP status codes) and syntactically valid according to the response structure defined in the OpenAPI Specification.

Test cases for REST APIs can be implemented manually, using libraries specific to programming languages or through graphical interfaces like the one offered by Postman [78]. However, because REST APIs adhere to common standards and can be described using OAS specifications, their testing is highly automatable, allowing the same testing techniques to be applied across different APIs. Automated approaches for testing REST APIs have gained significant traction in recent years [122, 130, 131, 132, 164, 190, 191, 200, 212, 215, 216, 217, 218, 219, 231, 234, 243, 261, 277, 284, 294, 301, 303, 308]. These approaches usually operate on black-box mode, leveraging the OAS specification of an API to automatically generate up to millions of test cases. For example, Listing 1.2 presents an automatically generated test case for the "getAlbumTracks" operation of the Spotify API (shown in Listing 1.1). Test cases like this can be created by automated tools like RESTest [240, 243] (described in Section §3.3.2), by assigning values to the `id`, `market` and `limit` input parameters defined in the OAS (lines 11, 8 and 9, respectively) and verifying that the response was successful (line 13).

1.2 Problem Statement

REST APIs have become the backbone of modern software ecosystems, powering critical functionalities in industries such as finance, healthcare, and e-commerce, facilitating everything from payment processing to data synchronization and cloud inte-

gration. Their widespread adoption underscores the need for robust testing, essential to prevent failures that could disrupt interconnected systems. Despite the substantial advances in automating different aspects of the testing workflow, existing tools are still considerably limited in two vital aspects: their limited abilities for generating *realistic input values* and the simplicity of their *test oracles*.


```
{
    "total": 14,
    "href": "https://api.spotify.com/albums/4Em5W5HgYEvhpc/tracks?limit=1&market=ES",
    "items": [
        {
            "artists": [
                {
                    "id": "2CvCyf1gEVhI0mX6aFXmVI",
                    "name": "Paul Simon"
                },
                {
                    "id": "70cRZdQywnSFp9pnc2WTCE",
                    "name": "Arthur Garfunkel"
                }
            ],
            "available_markets": [ "ES", "US", "JP" ],
            "id": "0gFvkiT2afIcJwNxXQ7W51",
            "name": "Mrs. Robinson",
            "explicit": false,
            "duration_ms": 234346,
            "linked_from": {
                "id": "98cZPdKywnMGp8fnw2XTYU",
                "uri": "https://spotify.com/artist/98cZPdKywnMGp8fnw2XTYU"
            }
        }
    ]
}
```


Listing 1.3: Spotify API response in JSON format.

Test input values generation. Effectively testing REST APIs requires *realistic input values* for parameters such as country names, URLs, coordinates, and addresses. For instance, successfully executing the operation in Listing 1.1 requires providing real country codes for the `market` parameter, which could be inferred by analyzing its name and description in the OAS. Similarly, APIs for travel booking often require valid airport codes (*e.g.*, "JFK" or "LHR"), while map-based services may expect geographic coordinates in latitude and longitude format (*e.g.*, `47.6206006,-122.3492248`). E-commerce APIs might require currency codes (*e.g.*, "USD" or "EUR"), and weather services often expect city names or postal codes as input. Supplying realistic values is essential not only for achieving successful execution but also for uncovering issues related to validation, data type handling, and response content. Generating meaningful values through random methods alone is rarely effective. Even if a test data generator bypasses syntactic validation constraints, the likelihood of producing API requests that return results and accurately assess API operations remains low. Some approaches rely on random (fuzzing) or default values, but these are often insufficient for thor-

ough testing. Instead, most methods rely on data dictionaries—curated sets of input values maintained by testers—a manual effort that hinders full automation.

Test oracle generation. Existing tools for automated API testing are limited by the simplicity of their *test oracles*, with the types of failures they can detect being mainly crashes (5XX HTTP status code), inconsistencies with the OAS (*e.g.*, missing output JSON properties), regressions, and violations of API best practices (*e.g.*, ensuring idempotent operations return identical responses upon repeated calls). For example, Listing 1.3 shows a response of the "getAlbumTracks" operation of the Spotify API (described in Listing 1.1). While it conforms to the API specification and would be considered correct by existing tools, it may still contain errors that would go unnoticed by existing tools, such as incorrect field length (*e.g.*, the country codes of the `available_markets` response field, line 16, should have length 2), invalid formats (*e.g.*, `href` and `uri`, lines 2 and 23, should be valid URLs), numerical constraints infringement (*e.g.*, `total` and `duration_ms`, lines 1 and 20, cannot be negative), or inconsistencies in array properties (*e.g.*, `total`, line 1, should be greater than or equal to the size of the `items` array, line 4). However, since these errors do not violate syntax or the formal API specification, existing approaches are unable to detect them.

1.3 Dissertation Goal and Research Questions

Given the research context and the identified problems, the main goal of this dissertation is defined as follows:

Goal of this dissertation

Development of techniques and tools for the automated generation of realistic test inputs and test oracles for REST APIs.

To achieve this goal, this dissertation seeks to answer the following Research Questions (RQs). The contributions outlined in Section §1.5 aim at partially or fully answering them.

RQ_1 **How effectively can realistic test input values be automatically generated for REST APIs?** Existing input generation techniques (usually tailored for desktop and mobile apps) rely on GUI labels, making them unsuitable for REST APIs, where parameters may follow different naming conventions, and key information is often found in their natural language descriptions or in machine-readable

information such as expected patterns and sample values. The proposed technique should generate test input values by leveraging various components of the OAS specification while also refining and expanding these values through execution feedback obtained from API response analysis.

RQ_2 To what extent is it possible to automatically generate test oracles for REST APIs? Our goal is to develop techniques capable of detecting failures in syntactically valid responses (*i.e.*, OAS-conformant responses with 2XX HTTP status codes). These techniques should generate test oracles both by analyzing request-response pairs and by leveraging OAS information alone, without requiring prior API execution.

1.4 Research Methodology

Throughout this PhD, we have followed the Design Science Research Methodology (DSRM) [265], which focuses on developing and refining artifacts such as systems, methods, procedures, and tools. These artifacts are created to address specific problems and are continuously evaluated and improved through iterative refinement. The DSRM outlines six key steps that must be followed to apply the methodology effectively. Below, we define these steps in the context of this dissertation:

1. *Problem identification and motivation:* This phase involves defining the specific research problem and justifying the value of its solution. This dissertation addresses the lack of approaches and tools for the automated generation of realistic input values and test oracles in REST API testing. These limitations have been identified by previous surveys and tool comparisons [193, 214, 244], and a literature review confirmed the lack of solutions in the research community. Automating these aspects would enhance the level of test automation and significantly improve the fault detection capabilities of existing API testing tools.

2. *Definition of the objectives for a solution:* This step involves outlining the goals aimed at addressing, and ideally solving, the identified problem. The primary objective of this dissertation is to develop techniques and tools for the automated generation of realistic input values and test oracles for REST APIs. From this main objective, several sub-objectives are derived, each designed to answer the research questions posed in Section §1.3.

3. *Design and development:* This phase focuses on creating the artifact. In this dissertation, we propose three main artifacts: ARTE (an approach for the automated generation of realistic test inputs), AGORA+ (an approach that derives test oracles from previous API executions), and SATORI (an approach that generates test oracles through documentation analysis without requiring prior API execution). These artifacts have been iteratively refined and improved to better address the identified challenges over time.

4. *Demonstration:* This stage aims to showcase the effectiveness of the developed artifacts in addressing the identified problem. In this dissertation, it has often served as a preliminary assessment to determine whether an artifact has the potential to solve a specific problem before conducting a more formal evaluation.

5. *Evaluation:* This phase aims to assess how effectively the artifact addresses the identified problem. Given that this dissertation focuses on automated testing techniques for REST APIs, the evaluation has been conducted using real-world case studies, particularly on commercial APIs such as DHL, GitHub, and YouTube.

6. *Communication:* The final phase of the DSRM process involves disseminating the work and its findings to the research community and other relevant audiences. Our research has been published in high-impact journals and presented at relevant international conferences.

1.5 CONTRIBUTIONS

This section summarizes the main contributions stemming from our research. These contributions have been published in relevant journals and conferences, and led to the development of several tools and datasets that pave the way for future research.

1.5.1 Introduction

Our contributions fall into two distinct research topics, each aimed at answering one of the Research Questions stated in Section §1.3. In what follows, we outline these topics by providing relevant research context, enunciating the problems tackled, and the proposed solutions to address them.

Generation of realistic test input values

Context: Current API testing tools lack mechanisms for automatically generating realistic test data, often relying on fuzzing, default values or manually curated data dictionaries. These limitations restrict their ability to produce valid requests (*i.e.*, 2XX status codes) that effectively exercise core API functionality, preventing full automation. Existing test data generation techniques, designed for GUI systems, primarily use label names, whereas generating realistic input values for REST APIs demands a more comprehensive analysis of the various elements in the OAS specification.

Problem statement: Thorough REST API testing requires realistic inputs, such as country codes, coordinates, hotel names and addresses, which are often manually specified.

Contribution: We present ARTE (Automated generation of Realistic TEst inputs), an approach for automatically generating realistic test input values for REST APIs. ARTE combines Natural Language Processing (NLP), search-based and knowledge extraction techniques to infer meaningful input values based on the specification of the API parameters. Additionally, it incorporates execution feedback to refine and expand these values.

A preliminary version of this work was presented at the ACM Student Research Competition (undergraduate category) of the *29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (ESEC/FSE 2021) [119]. This contribution has been published in the Journal *IEEE Transactions on Software Engineering* (TSE 2022) [122]—awarded as the Best scientific publication of January 2023 at the School of Informatics of the University of Seville. It was also presented at the Journal First track of the *30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (ESEC/FSE 2022) [14], and as a relevant paper at the *XXVII Jornadas de Ingeniería del Software y Bases de Datos* (JISBD 2023) [124].

Generation of domain-specific test oracles

Context: Research on REST API testing has largely focused on automating test case generation, yet the sophistication of these techniques contrasts with the simplicity of their test oracles, which are typically limited to detecting crashes, regressions, and violations of API specifications or design guidelines. Recent studies [193, 214, 244] identify test oracle generation as a major limitation to achieving full automation in REST API test case generation. Existing test oracle generation techniques proposed in the literature are designed for white-box testing at the method level, focusing on specific program-

ming languages, making them unsuitable for black-box REST API testing, which operates at the system level. These approaches rely on method internals, structured documentation, or dataflow analysis—none of which are applicable in a black-box context. Additionally, techniques that infer test oracles from previous program executions depend on code mutation, prior static analysis, or human intervention, further limiting their applicability to REST API testing.

Problem statement: Existing API testing tools lack mechanisms for the automated generation of domain specific test oracles, relying solely on syntax validation.

Contribution 1: We propose AGORA+ (Automated Generation of test Oracles for REST APIs), the first approach for automatically generating test oracles for REST APIs, which derives test oracles by detecting invariants—output properties that should always hold—in API responses. Operating in a black-box setting, it learns the expected behavior of an API by analyzing request-response pairs. To achieve this, we enhanced the Daikon tool for dynamic detection of likely invariants [179], adding new invariant types and developing Beet [16], a tool that translates any OAS specification and set of API requests and responses into Daikon-compatible inputs. AGORA+ supports 106 different invariant types for REST APIs and it cam be seamlessly integrated into existing API testing tools supporting OAS specifications. Overall, this approach uncovered 31 bugs across 11 commercial APIs with millions of users, that would go undetected by any other technique proposed in the literature.

AGORA+ was presented in a Journal article published in *ACM Transactions on Software Engineering and Methodology* (TOSEM 2025) [128] as an extension of AGORA, published at the *32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (ISSTA 2023) [125], where it received the Distinguished Artifact Award. A preliminary version of AGORA obtained the first prize (graduate category) in the *ACM Student Research Competition* of the *30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (ESEC/FSE 2022) [120] and the second prize in the *ACM SRC Grand Finals* [96]. It was also presented as a relevant paper at the *XXVIII Jornadas de Ingeniería del Software y Bases de Datos* (JISBD 2024) [126]. The Journal extension is the result of an international collaboration with Professor Michael D. Ernst, creator of Daikon, after a two-month research stay at the University of Washington.

Contribution 2: We introduce SATORI (Static API Test ORacle Inference), a black-box approach that derives test oracles directly from the API specification, eliminating the need for prior API execution. SATORI leverages Large Language Models (LLMs) to

generate test oracles by analyzing the properties of the response fields of API operations, such as their names and descriptions. SATORI addresses the primary limitation of AGORA+, its dependence on a diverse test suite that thoroughly exercises the API functionality to report accurate invariants. If the test suite lacks diversity or contains faulty responses, AGORA+ may report invalid or incomplete invariants. By eliminating the need for previous API execution, SATORI offers a more cost-effective solution. However, performance comparisons reveal that both approaches are complementary. SATORI can generate a substantial portion of the test oracles without executing the API and even surpasses AGORA+ in detecting certain oracle types. Nonetheless, in many cases, API execution remains necessary to infer specific test oracles. SATORI identified 18 bugs across 7 commercial APIs, leading to bug fixes and documentation updates by the API developers.

This work is the result of an international collaboration with Professor Gabriele Bavota, after a one-month stay at Università della Svizzera Italiana, and it is going to be submitted to the *40th IEEE/ACM International Conference on Automated Software Engineering* (ASE 2025).

1.5.2 Publications

Below is the complete list of publications resulting from our research, presented in chronological order. When available, we include bibliometric indicators for each publication. Specifically, for conference papers, we provide the class and rating according to the *ICORE* conference rating [53, 54]. For journal articles, we report the journal *Impact Factor (IF)* from the *Journal Citation Reports (JCR)*, and the rank of the journal within the *Computer Science, Software Engineering* category. In both cases, we report the number of citations according to Google Scholar as of April 2025.

2021

We began tackling the two Research Questions (Section §1.3) of this dissertation during the final year of my bachelor's degree. Our initial contribution to $\mathbf{RQ_1}$ (*How effectively can realistic test input values be automatically generated for REST APIs?*) was a preliminary version of ARTE, our approach for generating realistic input values, which we presented at the ACM Student Research Competition of ESEC/FSE 2021 (undergraduate category). Regarding $\mathbf{RQ_2}$ (*To what extent is it possible to automatically generate test oracles for REST APIs?*), we published a short paper at a national conference introducing an approach for the automated inference of metamorphic relations for testing

query-based systems.

- **ESEC/FSE 2021-SRC** [119]. J. C. Alonso. Automated generation of realistic test inputs for web APIs. *29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering - Student Research Competition*, pages 1666–1668. Athens, Greece. 2021. https://doi.org/10.1145/3468264.3473491

  Citation count: 10.

- **JISBD 2021** [121]. J. C. Alonso, A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés. Identificación Automática de Relaciones Metamórficas en Sistemas de Búsqueda. *XXV Jornadas de Ingeniería del Software y Bases de Datos*. Málaga, Spain. 2021.

2022

During my master's degree, we expanded on the two works presented in 2021. ARTE was accepted for publication in the journal *IEEE Transactions on Software Engineering* (presented at the Journal First track of ESEC/FSE 2022), while our approach for inferring metamorphic relations in query-based systems was published in an international workshop (MET 2022). In the first semester of the PhD (second half of 2022), we introduced the initial version of AGORA, our approach for generating domain-specific test oracles through invariant detection, at the ACM Student Research Competition of ESEC/FSE 2022 (graduate category), where it won the first prize. These preliminary results were also published as a short paper at a national conference (JISBD 2022). Additionally, we delivered a tutorial on various techniques for testing REST APIs at ICSOC 2022.

- **TSE 2022** [122]. J. C. Alonso, A. Martin-Lopez, S. Segura, J. M. García, and A. Ruiz-Cortés. ARTE: Automated Generation of Realistic Test Inputs for Web APIs. *IEEE Transactions on Software Engineering*. 2022. https://doi.org/10.1109/TSE.2022.3150618 **[Presented at the Journal First track of ESEC/FSE 2022] [Best scientific publication of January 2023 at the School of Informatics of the University of Seville]**

  JCR IF: 9.322.
  Rank: 3/110 (Top 2.27%, **Q1**).
  Citation count: 43.

- **MET 2022** [278]. S. Segura, J. C. Alonso, A. Martin-Lopez, A. Durán, J. Troya, and A. Ruiz-Cortés. Automated Generation of Metamorphic Relations for Query-Based Systems. *7th International Workshop on Metamorphic Testing*. Pittsburgh, USA. 2022. https://doi.org/10.1145/3524846.3527338

  Citation count: 12.

- **ESEC/FSE 2022-SRC** [120]. J. C. Alonso. Automated Generation of Test Oracles for RESTful APIs. *30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering - Student Research Competition*, pages 1808–1810, Singapore, Singapore. 2022. https://doi.org/10.1145/3540250.3559080 [**First prize in the graduate category**]

  Citation count: 6.

- **JISBD 2022** [123]. J. C. Alonso, S. Segura, and A. Ruiz-Cortés. Generación Automática de Oráculos de Prueba para APIs RESTful. *XXVI Jornadas de Ingeniería del Software y Bases de Datos*. Santiago de Compostela, Spain. 2022.

- **ICSOC 2022-Tutorial** [237]. A. Martin-Lopez and J. C. Alonso. Testing of RESTful Web APIs. *20th International Conference on Service-Oriented Computing - Tutorials*, pages 411–413. Sevilla, Spain. 2022. https://doi.org/10.1007/978-3-031-26507-5_43

2023

This year, we continued working on AGORA, which was accepted at an international conference (ISSTA 2023) and honored with the Distinguished Artifact Award. Additionally, we received the second prize in the ACM SRC Grand Finals for our contributions on automated generation of test oracles. We also completed a two-month research stay at the University of Washington under the supervision of Professor Michael D. Ernst. During this time, we worked on enhancing the Daikon tool for dynamic invariant detection and refining AGORA by incorporating heuristics to improve its precision, along with an additional step to transform the reported invariants into executable assertions.

- **ISSTA 2023** [125]. J. C. Alonso, S. Segura, and A. Ruiz-Cortés. AGORA: Automated Generation of Test Oracles for REST APIs. *32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 1018–1030. Seattle, WA,

USA, 2023. https://doi.org/10.1145/3597926.3598114 **[Distinguished Artifact Award]**

ICORE Rank: A.
Citation count: 16.

- **ACM SRC Grand Finals 2023** [96]. J. C. Alonso. Automated Generation of Test Oracles for REST APIs. *ACM Student Research Competition Grand Finals*, 2023. **[Second prize in the graduate category]**

2024

During this year, we contributed to research areas beyond the core topics of this dissertation. Our work on SaaS pricing analysis was accepted at an international workshop (SOC4AI), and we began exploring AI-driven testing of Large Language Models as part of the TRUST4AI European project [103]. We also submitted AGORA+, the result of our collaboration with Professor Michael D. Ernst, to the *ACM Transactions on Software Engineering and Methodology* (TOSEM) journal. Finally, we completed a one-month research stay at Università della Svizzera Italiana under the supervision of Professor Gabriele Bavota, where we worked on SATORI—our approach that leverages Large Language Models to automatically infer test oracles for REST APIs through specification analysis.

- **SOC4AI 2024** [154]. F. J. Cavero, J. C. Alonso, and A. Ruiz-Cortés. From Static to Intelligent: Evolving SaaS Pricing with LLMs. *1st International Workshop on Service-Oriented Computing for AI Applications*. Tunis, Tunisia. 2024.

2025

In the final months of the PhD, AGORA+ was accepted for publication at TOSEM 2025 and we intend to submit the SATORI paper to the international conference ASE 2025. Our contributions on AI-driven testing of Large Language Models were published at the Fairness 2025 international workshop, and we are currently extending this work for submission to a Journal publication.

- **TOSEM 2025** [128]. J. C. Alonso, M. D. Ernst, S. Segura, and A. Ruiz-Cortés. Test Oracle Generation for REST APIs. *ACM Transactions on Software Engineering and Methodology*. 2025. https://doi.org/10.1145/3726524

JCR IF: 6.6.
Rank: 8/132 (Top 5.7%, **Q1**).

- **ASE 2025** [127]. J. C. Alonso, A. Martin-Lopez, S. Segura, G. Bavota, and A. Ruiz-Cortés. SATORI: Static Test Oracle Generation for REST APIs. *40th IEEE/ACM International Conference on Automated Software Engineering*. Seoul, South Korea. 2025. [**To be submitted**]

  ICORE Rank: A*.

- **Fairness 2025** [271]. M. Romero-Arjona, J. A. Parejo, J. C. Alonso, A. B. Sánchez, A. Arrieta, and S. Segura. AI-Driven Fairness Testing of Large Language Models: A Preliminary Study. *1st International Workshop on Fairness in Software Systems*. Montréal, Québec, Canada. 2025.

Summary of Publications

Figure §1.1 provides an overview of all publications produced during the PhD. Contributions are organized by publication year (columns), and topic (rows). For each publication, we also show the author position (black = first, gray = second, white = third), as well as its relevance, with large circles denoting the most relevant publications, namely full papers in rank A or A* conferences, and JCR journals. A summary of the publications presented each year is shown at the bottom of the figure.

As illustrated in the graph, our research journey has followed a clear path. During the final year of the bachelor's degree, we initiated work on the two main topics of this dissertation, proposing a test input generation technique that leverages natural language processing and knowledge-extraction techniques (ESEC/FSE 2021-SRC), and a test oracle generation technique based on metamorphic testing (JISBD 2021). In early 2022, these contributions were extended into a JCR journal article (TSE 2022) and an international workshop paper (MET 2022), respectively. Subsequently, we shifted focus to the automated generation of test oracles through invariant detection, leading to preliminary publications in the graduate category of an ACM Student Research Competition (ESEC/FSE 2022-SRC, awarded with the first prize winner of the graduate category) and a national conference (JISBD 2022). This work was later expanded into a full paper in the main track of an international conference (ISSTA 2023) and was further recognized with the second prize at the ACM SRC Grand Finals 2023 (graduate category). In collaboration with Professor Michael D. Ernst from the University of Washington, we extended this research into a journal article, accepted for publication in TOSEM 2025. In parallel, we explored an alternative approach for automated test

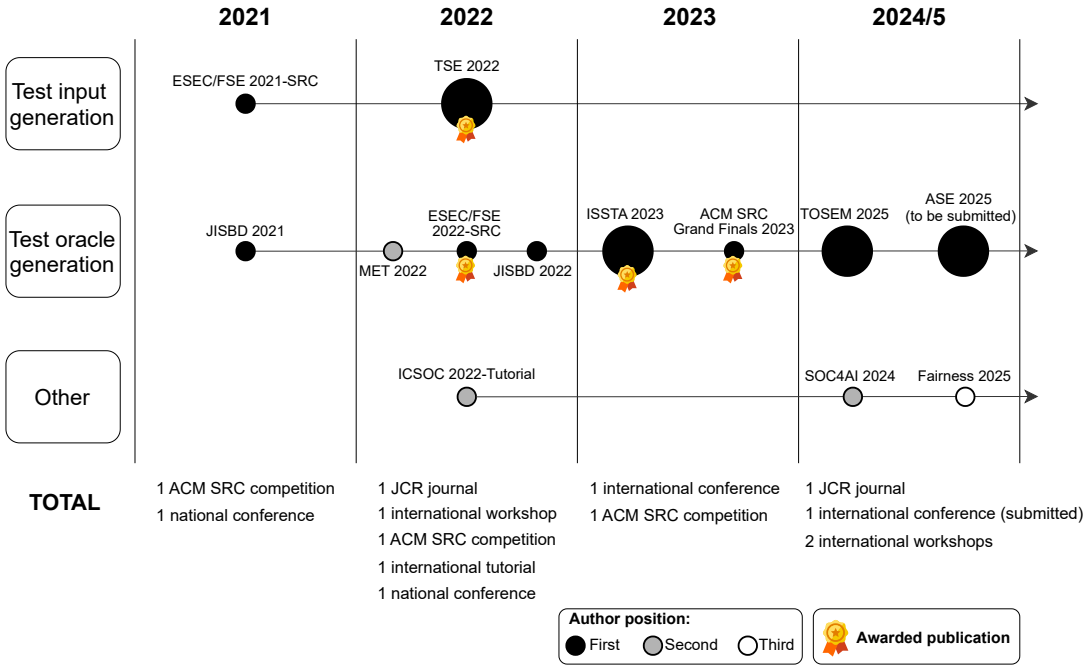


Figure 1.1: Summary of publications grouped by year and topic.

oracle generation that employs Large Language Models to generate test oracles statically, without requiring prior API execution. This work, conducted in collaboration with Professor Gabriele Bavota from Università della Svizzera Italiana, is going to be submitted to ASE 2025. Additionally, we presented a tutorial on Testing of REST APIs at ICSOC 2022 and contributed to research beyond our core topics, collaborating on SaaS pricing analysis (SOC4AI 2024), and AI-driven testing of Large Language Models (Fairness 2025).

1.5.3 Developed Tools

This thesis is complemented by several open-source tools that implement the research approaches developed during the PhD. These tools are publicly available on GitHub.

- **Beet** [16]. Beet is a Daikon *front-end* (also known as *instrumenter*) that transforms any OpenAPI specification and a collection of API requests and responses into a format compatible with the Daikon invariant detector [179]. Beet is one of the core components of AGORA+ (**A**utomated **G**eneration of test **O**racles for **R**EST **A**PIs). AGORA+ infers the expected behavior of an API by analyzing previous

API requests and their corresponding responses, making it applicable to any REST API and easy to integrate into existing API testing tools. This repository also includes a reference to a Docker image containing a modified and extended version of Daikon specifically adapted for REST APIs [23, 25]. Beet has been implemented using Java and, as of April 2025, Beet has received over 45 stars on GitHub.

- **SATORI** [89]. SATORI (**S**tatic **A**PI **T**est **OR**acle **I**nference) is a black-box approach that generates test oracles for REST APIs by leveraging LLMs to analyze API specifications–without requiring API execution. It infers the expected behavior of an API by examining the properties of response fields, such as their names and descriptions. SATORI is built with Python.

- **PostmanAssertify** [79]. PostmanAssertify automates the generation of test assertions for REST APIs by transforming the invariants reported by AGORA+ and the test oracles inferred by SATORI into executable assertions. It creates a Postman collection containing sample API requests for each status code of the operations under test, embedding the corresponding invariants and test oracles within each request as assertions. PostmanAssertify is programmed in Java.

1.5.4 Datasets

The contributions of this PhD have also led to the creation of several datasets designed to facilitate the replication and further extension of our research.

- **ARTE dataset** [29]. This dataset was created as part of our research on automated generation of realistic input values. It includes the source code of all the developed projects, experimental reports, documentation, video tutorials, and a collection of 40 REST APIs from diverse application domains, systematically gathered from the RapidAPI platform. For each API, we generated its OAS specification through web scraping.

- **AGORA+ dataset** [28]. This dataset was created as part of our work on test oracle generation for REST APIs using invariant detection techniques and was honored with the Distinguished Artifact Award at ISSTA 2023. It has been expanded as part of the journal extension. The dataset includes the source code of the developed projects, an Ubuntu Virtual Machine preconfigured with all the projects,

video tutorials, a Docker image, scripts to replicate our experiments, experimental reports, datasets categorizing all reported invariants, API requests (and their corresponding responses) used as input for AGORA+, screenshots of bug reports along with responses from the API providers, and comprehensive documentation.

- **OKAMI dataset** [74]. OKAMI (Oracle Knowledge of API Methods for Innovation) is one of the core contributions of the SATORI paper. It consists of a dataset containing the ground truth of all the test oracles from 17 operations across 12 industrial APIs. The dataset includes over 10.5k annotated oracle instances covering more than 1.8k unique response fields. OKAMI is publicly available on Hugging Face and is intended to serve as a benchmark for future research.

1.5.5 Research Stays

During the PhD, we conducted two research stays at international institutions. The first stay took place at the University of Washington (Seattle, United States) under the supervision of Professor Michael D. Ernst. The second stay was at Università della Svizzera Italiana (Lugano, Switzerland), supervised by Professor Gabriele Bavota.

University of Washington

From July 1 to September 10, 2023 (two months and ten days), we collaborated closely with Professor Michael D. Ernst at the University of Washington. Professor Ernst leads the Programming Languages and Software Engineering (PLSE) research group and is the creator of the Daikon tool for dynamic invariant detection used in our work. His research blends strong theoretical foundations with practical experimentation, aiming to transform the way software developers work. Professor Ernst was previously a tenured Professor at MIT and, before that, a researcher at Microsoft Research. During our stay, we worked on extending and modifying Daikon, as well as on enhancing AGORA for its journal extension.

Università della Svizzera Italiana

From April 19 to May 22, 2024 (one month and three days), we conducted a research stay at Università della Svizzera Italiana, collaborating with Professor Gabriele Bavota. Professor Bavota leads the Software Analytics Research Team (SEART). His research interests encompass AI for software engineering, software maintenance and evolution, mining software repositories, and empirical software engineering. Currently, Professor

Bavota is the Principal Investigator of the DEVINTA ERC Starting Grant, focusing on developing an artificial assistant for software developers. During our stay, we worked on designing an approach that leverages specification analysis to automatically generate test oracles for REST APIs.

1.5.6 Awards

Throughout this dissertation, our contributions have been recognized with several awards, highlighting their quality and impact.

- **First prize of the ACM SRC at ESEC/FSE 2022.** I won the Student Research Competition (graduate category) organized by the *Association for Computing Machinery (ACM)* and sponsored by KAIST (Korea Advanced Institute of Science and Technology) at the *30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. During this competition, I presented our key findings on automated generation of test oracles for REST APIs. My research paper [120], poster, and technical presentation were evaluated by a panel of over a dozen experts in the field, who selected my work as the best among all the participants. This achievement was featured in the news portal of the University of Seville [37].

- **Second prize of the ACM SRC Grand Finals 2023.** Obtaining the first prize in the ACM Student Research Competition (SRC) at ESEC/FSE 2022 qualified me to compete in the ACM SRC Grand Finals 2023. This prestigious competition brings together first-place winners from all SRCs held throughout the year, where they submit their research to a separate panel of judges. The judges select three undergraduate and three graduate students as the Grand Finals winners. As the culmination of a year-long competition, the Grand Finals featured over 249 computer science students presenting their research at 19 major ACM conferences. I was awarded second prize in the graduate category [96] for my work on automated generation of test oracles. The University of Seville published an article on its news portal acknowledging this accomplishment [97].

- **Distinguished Artifact Award at ISSTA 2023.** The supplemental material provided with our ISSTA 2023 paper [125] received this award for demonstrating outstanding quality in terms of usability, reproducibility, and potential for advancing future research or practice. This recognition highlights that our con-

tribution goes beyond theoretical research, offering tangible, practical resources that benefit the software engineering community.

- **Best scientific publication of January 2023 at the School of Informatics of the University of Seville.** In 2023, we received this local recognition for our *IEEE Transactions on Software Engineering* [122] article.

1.5.7 Other Achievements

Teaching Experience

I worked as a full-time teaching assistant (24 ECTS credits per year) for a total of three academic years—while pursuing my master's degree and during two of the three years of my PhD—acounting for a total of 600 teaching hours. During this period, I delivered lectures and supervised projects on various technologies and topics, including Docker, Linux commands, web-scraping, web development with Django and Spring Boot, software testing, software architecture and integration, among others.

Many of the courses and technologies I taught directly relate to the topics of my dissertation, particularly software testing, web development and software integration. Additionally, I contributed to developing lecture materials for laboratory sessions and designed the practical software integration project for the *Architecture and Integration of Software Systems* course of the Software Engineering degree program.

Throughout this period, I served as a lecturer for the following courses:

- **Architecture and Integration of Software Systems.** Software Engineering degree, 2nd year. University of Seville.
  - 2021-2023: Laboratory sessions (English and Spanish groups).
- **Data and Algorithm Analysis and Design.** Information Technologies degree, 2nd year. University of Seville.
  - 2021-2022: Laboratory sessions.
- **Intelligent Access to Information.** Software Engineering degree, 4th year. University of Seville.
  - 2021-2022: Laboratory sessions.
- **Operating Systems.** Software Engineering degree, 2nd year. University of Seville.

  - 2021-2024: Laboratory sessions.

- **Requirements Engineering.** Software Engineering degree, 3rd year. University of Seville.
  - 2022-2024: Laboratory sessions.

Mentoring Programs

I participated as a mentee in the mentoring program of the International Mentoring Foundation for the Advancement of Higher Education (IMFAHE) [55], a non-profit organization dedicated to fostering innovation and global education by connecting talent worldwide. The International Mentor Program (IMP) provides personalized mentorship by pairing students and professionals with experts from prestigious institutions such as Harvard University and Google.

My mentor was Dr. Yannet Interian [109], an Associate Professor at the University of San Francisco, whose research focuses on applying machine learning and deep learning techniques to medical data. She earned her PhD from Cornell University in 2006 and, before joining academia, she worked as a data scientist at Google for five years, where she tackled problems related to advertising, viral media, and online social networks. Additionally, she co-founded Akualab, a startup that helped organizations develop data-driven products using machine intelligence.

Throughout our mentoring sessions, we explored the potential applications of my research in an industrial setting and discussed possible future research directions.

1.6 Structure of the Dissertation

This dissertation is structured into five parts, each consisting of one or more chapters.

- **Part I: Preface**. This initial part comprises this introduction chapter, which establishes the foundation of the dissertation. It presents the research context, defines the problem statement, outlines the primary goals of the dissertation, the research methodology and the key contributions.

- **Part II: Background and State of the Art**. In this part, we provide the reader with the theoretical and contextual background, along with an overview

of the current state of the art, necessary to contextualize our contributions. Chapter §2 introduces background information on software testing; Chapter §3 covers REST APIs; Chapter §4 explores the Web of Data; and Chapter §5 presents background on Natural Language Processing.

- **PART III: CONTRIBUTIONS**. This part is organized into three chapters, each one of them presenting one of the main contributions that stem from this dissertation. Chapter §6 introduces ARTE, our technique for automatically generating realistic input values for REST APIs. Chapter §7 details AGORA+, a dynamic approach for deriving test oracles by identifying invariants in previous API requests. Chapter §8 presents SATORI, a static technique that infers test oracles directly from the API specification.

- **PART IV: FINAL REMARKS**. It consists of a single chapter (Chapter §9), which concludes the dissertation and explores potential future research directions.

- **PART V: APPENDICES**. The appendices include supplementary information to support our research. Appendix §A, originally published as supplementary material for the ARTE Journal article, contains detailed algorithmic descriptions of the functionality of ARTE. Appendix §B clarifies the acronyms used throughout the dissertation.

Part II
Background and State of the Art

2

Software Testing

We have as many testers as we have developers. And testers spend all their time testing, and developers spend half their time testing. We're more of a testing, a quality software organization than we're a software organization.

Bill Gates,
Co-founder of Microsoft

This chapter provides background on software testing and presents the state of the art knowledge necessary to contextualize the contributions of this dissertation. Section §2.1 introduces the fundamentals of software testing, followed by Section §2.2, which outlines the key steps involved in the testing process. Section §2.3 reviews various taxonomies used to classify software testing based on different criteria, while Section §2.4 focuses on different techniques for assessing the quality of test suites. Sections §2.5 and §2.6 discuss recent advances in automated test input generation and test oracle generation, respectively. Lastly, Section §2.7 concludes the chapter with a summary.

2.1 Introduction

According to the IEEE Standard for Software and System Test Documentation (IEEE 829-2008) [1], software testing is defined as *"The process of analyzing a software item to detect the differences between existing and required conditions (that is, bugs) and to evaluate the features of the software item"*. Software testing is essential in software development, ensuring that a program meets its functional and non-functional requirements while increasing confidence in its reliability. Notably, a significant portion of the resources of a software project (often exceeding 50%) is dedicated to fault detection and correction [141, 254]. However, despite this substantial investment, the inherent complexity of software makes it impractical, and often impossible, to identify all faults within a program [141, 163, 254].

This chapter provides background on software testing, with a focus on the techniques relevant in the context of this dissertation. Additionally, it explores the state of the art in automated test input and test oracle generation techniques across various software testing contexts.

2.2 The Software Testing Procedure

Software testing involves executing test cases, which may be grouped into test suites, on a *System Under Test* (SUT). A test case consists of predefined inputs and expected outputs intended to verify compliance with a specific requirement [163, 254]. Figure §2.1 illustrates the two fundamental steps of the testing procedure. First, the program is executed with a set of inputs, returning an output. Then, this output is analyzed to determine whether the program is behaving as expected. The correctness of the output is assessed through a mechanism known as a *test oracle*. Test oracles can take various forms, including specifications and documentation, statistical properties of the program, or expert knowledge.

In some cases, the oracle may be unavailable or too complex to apply. This limitation is known in the testing literature as the *oracle problem*—the challenge of determining output correctness during software testing. While test inputs can often be generated systematically, defining a test oracle (*i.e.*, verifying output correctness) is not always straightforward. The oracle problem is recognized as a significant challenge in automated testing [137], as the absence of well-defined oracles significantly

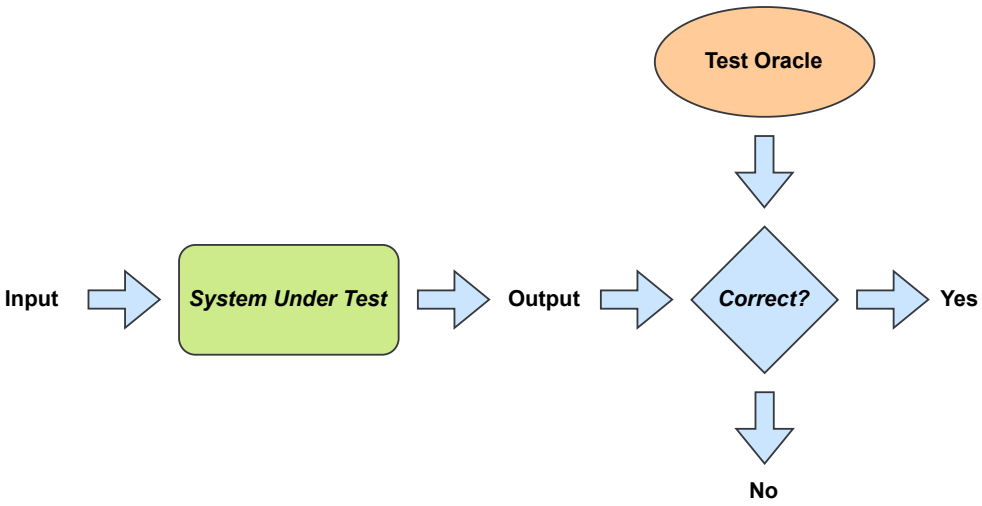


Figure 2.1: Basic testing procedure.

reduces the fault-detection capabilities of testing tools, thereby hindering full automation. To address this challenge, researchers have explored techniques such as metamorphic testing [276], oracles derived from formal specifications [186, 208], and machine learning-based predictions [169, 204, 300, 304], among others.

2.3 Software Testing Taxonomies

Software testing encompasses various approaches and methodologies, each serving distinct purposes in ensuring software quality. This section provides an overview of key software testing taxonomies, including the different levels of testing, testing based on code visibility (black-box and white-box), testing with different automation levels (manual and automated), and strategies for validating new or existing features (regression and progressive testing). Each of these categories plays a critical role in the testing process, helping to identify defects and ensure the software meets its intended requirements.

2.3.1 Software Testing Levels

Software testing levels define the different stages at which testing is conducted throughout the software development process [129]. These levels ensure that defects are identified early and that the system functions as expected. Generally, software testing is structured into four progressive levels, each focusing on a different aspect of the

system:

- **Unit Testing.** This is the most granular level of testing, where individual components of the software—such as methods, functions, or classes—are tested in isolation. The goal is to verify that each unit performs correctly before integration. Developers typically write and execute unit tests using frameworks like JUnit (Java) [60], pytest (Python) [80], or GoogleTest (C++) [43]. For example, in an e-commerce application, unit tests might check if a function correctly calculates the total price of items in a shopping cart.
- **Integration Testing.** This level focuses on verifying interactions between integrated components. It ensures that different parts of the software, such as modules, databases, or external APIs, communicate correctly. For instance, an integration test might validate whether a payment gateway properly processes transactions in an online store.
- **System Testing.** At this stage, the entire software application is tested as a whole to ensure it meets functional and non-functional requirements. *Functional testing* ensures that the system performs its intended tasks correctly (*e.g.*, verifying that users can create an account, reset their password, or search for products in an online store). *Non-functional testing* evaluates aspects such as performance, security, and usability (*e.g.*, measuring response times under high traffic, ensuring encryption of sensitive user data, or checking if the interface is accessible to users with disabilities). For example, in a ride-sharing app, system testing might check whether users can successfully book a ride, receive driver details, and process payments while also ensuring the app remains stable under heavy user load.
- **Acceptance Testing.** This is the final stage before deployment, where the system is validated against end-user requirements. It ensures that the software performs correctly in real-world conditions and is ready for release. Acceptance testing can take different forms, such as *User Acceptance Testing (UAT)* [224], where actual users test the system, or *Beta Testing* [181], where a limited group of external users evaluates the software before a full launch. For instance, a new social media app might undergo beta testing to gather user feedback before its public release.

2.3.2 Manual and Automated Testing

Manual testing requires testers to execute test cases manually, interact with the application, compare actual and expected results, and document findings. It is commonly

used for exploratory, usability, and ad-hoc testing, where human intuition and observation are essential. However, it is time-consuming, prone to human error, and inefficient for large-scale, repetitive tests.

In contrast, *automated testing* relies on scripts and tools to execute test cases automatically, significantly improving speed, accuracy, and efficiency. It is particularly well-suited for regression, performance, and large-scale testing, where thousands of test cases can be executed in a short time. While it requires an initial investment in tools and script development, it becomes cost-effective over time.

Automated testing can be categorized into two levels. The first level involves tools that require manually written test scripts, using libraries and tools such as JUnit [60], pytest [80], Selenium [91, 92], or Cypress [22]. Once implemented, these scripts can be executed repeatedly. The second level includes tools that *generate* and *execute* test cases automatically. For example, fuzz testing tools like AFL (American Fuzzy Lop) [6, 7] and libFuzzer [61] generate random inputs to uncover security vulnerabilities. Property-based testing tools such as QuickCheck (Haskell) [159] and Hypothesis (Python) [51, 52] automatically create test cases based on specified properties. Model-based testing tools like GraphWalker [46] derive test cases from system models, while AI-powered tools such as Testim [99] and Mabl [64] dynamically analyze application behavior to generate tests.

2.3.3 Black-box and White-box Testing

This section explains how software testing can be classified based on whether or not it utilizes information from the source code and implementation details of the system under test [254].

Black-box testing focuses entirely on the external behavior of a system without considering its internal structure or implementation details. The tester does not need access to the underlying code, logic, or data flow. Instead, they evaluate the software by providing inputs and verifying whether the output matches the expected results. Black-box testing is widely applied in system, integration, and acceptance testing, where the primary concern is whether the software functions correctly rather than how it achieves those results. Common techniques include *equivalence partitioning* (grouping test cases into valid and invalid input classes), *boundary value analysis* (testing the edges of input ranges), and *fuzz testing* (inputting large amounts of random or unexpected data to uncover vulnerabilities). Many automated tools apply black-box testing

techniques. For example, fuzzers like AFL [6, 7] and libFuzzer [61] generate randomized inputs to test software robustness by identifying crashes or unexpected behavior. Similarly, model-based testing tools, such as Spec Explorer [94], generate test cases automatically based on system specifications, ensuring that the software under test behaves correctly under different conditions.

White-box testing examines the internal workings of the software, including its code structure, logic, and data flow. It requires knowledge of the implementation details and is typically performed by developers or testers with programming expertise. This approach ensures that every code path, branch, and function behave as expected under different conditions, helping to identify logic errors and security flaws. White-box testing is commonly used for unit testing, code coverage analysis, and security testing. Techniques include *statement coverage* (ensuring every line of code is executed at least once), *branch coverage* (testing all possible decision points in the code), and *path testing* (executing every possible route through the program). Automated white-box testing tools leverage these techniques to improve efficiency. Search-based testing approaches, for instance, generate test cases automatically by analyzing the source code to maximize coverage. Tools like EvoSuite [183] use genetic algorithms to create test cases for Java programs, while KLEE [152] applies symbolic execution to explore execution paths in C programs.

In addition to these two approaches, *gray-box testing* [213] offers a middle ground that combines elements of both black-box and white-box testing. Testers have partial knowledge of the internal workings of the system under test, such as access to algorithms, database structures, or API documentation, but do not examine the full source code. This method allows for more targeted testing than black-box testing while avoiding the complexity of full white-box analysis. This approach is particularly useful in *integration testing* and *penetration testing* (a security assessment where testers simulate attacks to identify vulnerabilities), where understanding system architecture helps uncover security vulnerabilities, data flow issues, and integration errors. By blending internal insights with functional testing, gray-box testing improves efficiency and helps identify defects that may not be easily discovered using black-box or white-box methods alone. Security tools like Burp Suite [17] assist in penetration testing by identifying weaknesses in web applications.

2.3.4 Regression and Progressive Testing

As software evolves, it is crucial to validate both existing functionality and newly introduced features to maintain quality and prevent defects. Regression testing and progressive testing address these needs by focusing on different aspects of the software testing process [129].

Regression testing is performed to ensure that recent code changes (such as bug fixes, enhancements, or new features) do not negatively affect the existing functionality of the software. The goal is to detect unintended side effects by re-running previously executed test cases. Regression testing is essential in agile and continuous integration (CI/CD) environments, where frequent code updates occur. For example, in an e-commerce website, regression testing would ensure that adding a new payment method does not break the checkout process.

Progressive testing focuses on testing new or modified functionalities in a software system. Instead of verifying previously existing features, this approach is designed to ensure that new changes are working as intended and integrate well with the existing system. For instance, when introducing a recommendation engine in a video streaming platform, progressive testing would validate whether the new algorithm correctly suggests content based on user preferences.

2.4 Test Adequacy Criteria

This section discusses two of the main criteria used for measuring the quality of a test suite, namely test coverage and mutation testing.

2.4.1 Test Coverage

Test coverage [129] is one of the key metrics used to evaluate the thoroughness of a test suite. It measures the extent to which the source code of the system under test is exercised by the test cases and is typically expressed as a percentage. The higher the test coverage, the greater the confidence that the code has been tested for potential defects. By highlighting which parts of the code have been executed and which have not, test coverage helps ensure that critical paths, conditions, and edge cases are explored.

There are several types of test coverage, each offering a different perspective on

how thoroughly the code has been tested. *Statement coverage* checks whether each line or statement in the code has been executed at least once. *Branch coverage* ensures that every possible branch of conditional statements, such as if and else, has been taken. *Function coverage* verifies that all functions or methods have been invoked during testing. *Path coverage* measures whether all possible execution paths through a given part of the code have been followed. *Loop coverage* focuses on whether loops in the code are executed zero times, once, or multiple times. *Condition coverage* checks that each boolean sub-expression in a compound condition evaluates to both true and false at least once.

High test coverage indicates that more of the code has been exercised by tests, which reduces the risk of undetected bugs and improves software reliability. It also helps identify parts of the code that may remain untested. However, high coverage alone does not guarantee the absence of defects, since coverage metrics do not account for the quality or effectiveness of the tests themselves. For example, a test suite that achieves 100% coverage but contains no assertions would not be capable of detecting incorrect behavior—only crashes or runtime errors would surface.

Test Coverage Criteria for REST APIs

REST APIs can be tested using either black-box or white-box approaches. White-box techniques can be easily compared using source code coverage metrics. However, this is often not feasible for black-box testing, especially when the system under test is a commercial product with unavailable source code. To overcome this limitation and enable meaningful comparisons between testing techniques, Martin-Lopez et al. [239] proposed a catalogue of ten test coverage criteria designed to measure how thoroughly a test suite exercises the inputs (*i.e.*, requests) and outputs (*i.e.*, responses) of an API. These criteria are grouped into eight Test Coverage Levels (TCLs), ranging from TCL0 (the weakest) to TCL7 (the strongest). Together, these levels form a standardized framework—known as the Test Coverage Model (TCM)—for evaluating and comparing REST API testing techniques. An evaluation using open-source APIs with real bugs demonstrated a strong correlation between these coverage levels and both code coverage and fault detection. The results indicate that higher TCLs are associated with increased fault detection capability and broader code coverage.

Table §2.1 presents the different Test Coverage Levels (TCLs) along with their associated test coverage criteria, distinguishing between input and output aspects. To achieve a specific TCL, all criteria from the preceding levels must be satisfied.

Table 2.1: Test Coverage Model: Levels and Criteria.

TCL	Input criteria	Output criteria
0		
1	Paths	
2	Operations	
3	Content-type	Content-type
4	Parameters	Status code classes
5		Status codes
6		Response body properties
7	Operation flows	

- **TCL0.** Represents a test suite that does not fully satisfy any coverage criterion.
- **TCL1.** Requires that all paths of the API are exercised—that is, at least one request must target each distinct path.
- **TCL2.** Requires coverage of all operations, meaning that each API operation must be exercised by at least one request.
- **TCL3.** Requires testing all supported content types for every operation, both in requests (inputs) and responses (outputs).
- **TCL4.** Requires testing of all parameters values (only for those with a finite number of possible values, such as Booleans and enums) and status code classes (*e.g.*, 2XX, 4XX).
- **TCL5.** Extends TCL4 by requiring coverage of parameter values and all specific status codes supported by the operations.
- **TCL6.** Requires coverage of all distinct output properties in the response fields of each operation.
- **TCL7.** Represents the highest and most challenging level to achieve, requiring test coverage of operation flows—that is, sequences of dependent API calls.

2.4.2 Mutation Testing

One way to assess the quality of a test suite is to measure how effectively it detects artificially introduced faults in the system under test. This technique, known as mu-

tation testing, is a fault-based testing approach that measures the effectiveness of test cases by analyzing their capacity to identify faults [263].

Mutation testing operates by introducing small syntactic modifications, called mutants, into the source code of a program. Each mutant represents a faulty version of the original program, generated by applying predefined mutation operators. These operators are tailored to specific programming paradigms and languages. There are two main types [210, 263]: traditional operators, which apply to procedural languages like C (*e.g.*, replacing `i++` with `i--`), and class-level operators, designed for object-oriented features such as inheritance or polymorphism (*e.g.*, removing the super keyword). Listing 2.1b depicts an example of mutant created by replacing the operator $>$ of the original program in Listing 2.1a with the operator $<$.

```
def is_greater(a, b):
    return a > b
```

(a) Original code

```
def is_greater(a, b):
    return a < b
```

(b) Mutant

```
def is_greater(a, b):
    return a > (b + 0)
```

(c) Equivalent mutant

Listing 2.1: Example of program mutation.

Once mutants are created, test cases are executed against them. If a test case differentiates a mutant from the original program, the mutant is considered killed, indicating the effectiveness of the test. If no test case detects the mutant, it remains alive. Some mutants are equivalent, meaning they do not alter program behavior and cannot be detected by any test case. Listing 2.1c shows an example of an equivalent mutant generated by adding 0 to one of the variables. The mutation score, calculated as the percentage of killed mutants (excluding equivalent ones), serves as an adequacy metric for the test suite.

A key challenge in mutation testing is the detection of equivalent mutants, as determining program equivalence is undecidable. Although various techniques aim to mitigate this issue [233, 262, 274], they remain imperfect. Despite this limitation, mutation testing is grounded in two key hypotheses: the Competent Programmer Hypothesis [166], which assumes that programmers make small, simple mistakes similar to mutants, and the Coupling Effect [258], which suggests that test cases effective at detecting simple faults, such as mutants, will also catch more complex defects.

Beyond traditional programming languages, mutation testing has been applied to various domains, including SQL, aspect-oriented programming, network protocols, and web services. Several automated tools for mutation testing are available, facilitat-

ing its adoption in software testing research and practice. For a comprehensive survey on mutation testing, we refer the reader to [263].

JSONMutator

Traditional mutation testing tools are primarily designed for white-box testing, as they require access to the source code of the system under test to introduce mutations. However, in many scenarios, such as automated testing of REST APIs (where researchers often test industrial APIs deployed online), testers do not have access to the source code. In these cases, traditional mutation testing is not feasible, and real faults are typically too rare to enable a systematic evaluation of the fault detection techniques of a proposed technique.

```
{
   "id": "1234",
   "track_number": 1,
   "type": "track"
}
```

(a) Original JSON

```
{
   "id": "1234",
   "track_number": -1,
   "type": "track"
}
```

(b) Mutated JSON

Listing 2.2: Example of JSON mutation.

To address this limitation, JSONMutator [59] was introduced as a black-box mutation testing tool specifically tailored for evaluating the fault detection capabilities of test suites or approaches designed for REST APIs. Unlike traditional mutation testing, which modifies the source code, JSONMutator applies mutations directly to the *responses* returned by the API, which are typically in JSON format.

JSONMutator integrates a total of 23 different mutation operators, such as altering string values (*e.g.*, replacing with boundary values or adding special characters), modifying numerical fields (by adding deltas or replacing with random values), reordering or removing array elements, and flipping Boolean values. For instance, Listing 2.2b shows an example of a mutated JSON generated by applying the "replace" mutation operator over the `track_number` response field of the original JSON (Listing 2.2a). These mutation operators are inspired by real-world faults discovered in the Spotify and YouTube APIs.

2.5 Test Input Generation Techniques

Numerous approaches have been proposed in the literature for the automated generation of test input values. These techniques vary in complexity, ranging from simple methods that use fixed or random strings (*e.g.*, "sampleString") [196, 227, 305], to more sophisticated strategies based on heuristic rules [201, 285], semantic information retrieval [236, 299], concolic execution [221], search engines [246], web service APIs [149], or data retrieved from the database of the system under test [160, 161]. More recent approaches leverage machine learning models, such as Recurrent Neural Networks (RNNs) [229] and, more recently, Large Language Models (LLMs) [167, 232]. These approaches have been applied across a wide range of contexts, such as GUI applications—including desktop [236], mobile [158, 170, 196, 201, 227, 229, 232, 299, 305], and web applications [118, 285]—autonomous driving systems [205, 288, 296], and machine learning programs [167, 225, 310], among others.

Semantic information retrieval techniques have been applied in the context of GUI testing. Mariani et al. [236] presented Link, an approach to retrieve realistic test inputs for web, desktop, and mobile applications from DBPedia. Wanwarang et al. [299] introduced SAIGEN, which follows the same principles of Link, but is specifically tailored for mobile apps. Besides semantic-enabled approaches, other authors advocate for extracting realistic test inputs from other sources. Shahbaz et al. [279, 280] proposed generating valid and invalid string test data based on Web searches and predefined regular expressions (*e.g.*, following the format of an email address). Bozkurt and Harman [149] relied on web service composition [174] to generate realistic test data, *i.e.*, by finding a web service that returns as output the data required by a different service. Clerissi et al. [160, 161] presented DBInputs, an approach to testing web applications by reusing data stored in the system database (*e.g.*, a resource identifier). Recent approaches leverage LLMs for test input generation. Liu et al. [232] introduced `QTypist`, an LLM-based method that generates meaningful text inputs for mobile GUI testing by understanding the contextual information of the application. Similarly, Alian et al. [118] proposed FormNexus, a technique that employs LLMs to automate test input generation for web forms. FormNexus focuses on extracting semantic insights from individual form elements and their interrelations by analyzing textual content, DOM tree structures, and visual proximity.

2.6 Test Oracle Generation Techniques

The automated generation of test oracles is a widely studied research area. Existing approaches can be categorized based on their input sources and application domains. Test oracles have been derived from various inputs, including source code [169, 204, 245, 253, 300, 304], formal specifications [186, 208], semi-structured documentation [145, 146, 147, 192, 253, 306], previous program executions [148, 156, 172, 207, 211, 249, 250, 251, 287], as well as combinations of these sources. Their application spans multiple contexts, such as Java projects [145, 147, 153, 169, 251], machine learning programs [150], databases [172], and cyber-physical systems [133].

Automated test oracle generation techniques primarily operate at the method level and are designed for specific programming languages, predominantly Java [145, 146, 147, 148, 153, 156, 169, 192, 204, 207, 211, 249, 250, 251, 253, 287, 300, 304, 306]. These techniques generate unit test assertions by leveraging information from the method under test, semi-structured procedure specifications (*e.g.*, Javadoc), assertion-less unit test methods (test prefixes), or a combination of these sources. Documentation-based approaches [145, 146, 147, 192, 306] infer test oracles from variable names and dataflow analysis. Approaches that derive test oracles from previous program executions require generating valid and invalid instances of program states [249, 250, 251], usually generated through code mutation, or rely on prior static analysis [156], a human in the loop [148, 211], or previously annotated false positives and false negatives [287] to refine automatically generated assertions. Recent LLM-based techniques [169, 204, 252, 253, 300, 304] also focus on specific programming languages and operate at the method level.

Other related techniques include metamorphic testing and regression testing. Metamorphic testing [261, 276, 277, 278] identifies relationships between inputs and outputs across multiple executions of a program to detect inconsistencies, requiring manual specification of these metamorphic relations.

2.6.1 Invariant Detection

A common approach for the generation of test oracles is through the detection of invariants. An invariant is a property that is always satisfied at one or more points of the execution of a program [178]. For example, in a procedure that takes an array and returns the same array with an additional element, an invariant could state

that the output array is always larger than the input, *i.e.*, `size(return.array[]) > size(input.array[])`. These invariants can serve as test oracles to verify program correctness.

Invariants can be detected statically—by analyzing the source code, without executing it [165, 188]—or dynamically—by observing program behavior across multiple executions [178, 179, 199, 222]. Statically detected invariants are usually less numerous and less specific than those detected by dynamic techniques [177, 256, 257]. However, dynamic invariant detection techniques may result in overfitting, especially if the analyzed executions lack variety. Since it is infeasible to run the program with all possible inputs, dynamically detected invariants are referred to as *likely invariants*, until they are confirmed by a domain expert. Likely invariants can be categorized based on where in the program execution they are observed. For instance, *likely preconditions* apply to the state before a method or function is executed, while *likely postconditions* describe the state after its completion. Both are specialized forms of likely invariants, scoped to specific program components such as functions or methods.

The automated detection of likely invariants has shown promising results in various domains, including Java programs [251], relational databases [162], automated program repair [309], WS-BPEL composition testing [260], cyber-physical systems [117], and cloud-based [264] and distributed [194] systems.

Daikon

Daikon [179] is an open-source tool that detects likely invariants in programs by monitoring test executions. This monitoring process involves observing the program state at designated *program points*, initially considering all possible invariants as valid. Those invariants that are not violated by any execution are reported as likely invariants. Daikon operates by analyzing an instrumented version of a software execution, generated by a *front-end* or *instrumenter*. This instrumentation produces a declaration file and a data trace file. The *declaration* file describes the variables that exist at each program point. The *data trace* file contains the values in each execution. Front-ends are available for various programming languages and data formats, including Java, Perl, C++, and CSV [24].

```
1 public Result computeSquare(int inputValue) {
2         return new Result(inputValue * inputValue);
3 }
```

Listing 2.3: Java computeSquare function.

```
ppt main.computeSquare(int):::EXIT1
ppt-type subexit
variable inputValue
  var-kind variable
  dec-type int
  rep-type int
variable return
  var-kind return
  dec-type Result
  rep-type hashcode
variable return.square
  var-kind field square
  enclosing-var return
  dec-type int
  rep-type int
```

Listing 2.4: Daikon declaration file.

Listing 2.3 shows a Java method that squares an input integer. For this method, a Daikon front-end generates an ENTER program point for method entry and an EXIT program point for method exit. Listing 2.4 depicts the content of a Daikon declaration file for the EXIT program point. As illustrated, the definition of the program point includes the variables representing the input and output parameters to be observed. Each variable definition includes information about its name, datatype in the original program (`dec-type`), datatype in the data trace file (`rep-type`), and whether the variable is a property of another variable (`enclosing-var`), among others. In the example, Listing 2.4 contains the int-type variable `inputValue` (input parameter), and the object-type variable `return`, containing the integer field `return.square` (method output).

```
main.computeSquare(int):::EXIT1
inputValue
10
1
return
1458849419
1
return.square
100
1
```

Listing 2.5: Daikon dtrace file.

```
=== main.computeSquare(int):::EXIT
return.square >= 0
inputValue <= return.square
```

Listing 2.6: Likely invariants of computeSquare.

Listing 2.5 shows the data trace file of one execution of the EXIT program point. For each variable, the data trace contains its name, the value observed in the program execution, and the modified bit. This bit specifies whether a variable value has been assigned or not since the last time it was observed. After processing these files, assuming

a larger data trace file, Daikon would return a set of invariants like the one presented in Listing 2.6.

2.7 SUMMARY

This chapter provided the necessary background and state of the art insights to contextualize our contributions. We began by explaining the core components of software testing, explored various criteria for categorizing testing approaches, and introduced test coverage and mutation testing as techniques for assessing test suite quality. We then examined the latest advancements in automated test input and test oracle generation.

3

REST APIs

The medium is the message.

Marshall McLuhan,
Canadian philosopher

This chapter provides background on REST APIs and the state of the art in REST API testing to contextualize our contributions. Section §3.1 introduces REST APIs, followed by Section §3.2, which discusses the OpenAPI Specification. Section §3.3 examines both manual and automated approaches to REST API testing, including Testing as a Service (TaaS) tools. Section §3.4 summarizes the chapter.

3.1 INTRODUCTION

REST APIs, also referred to as RESTful APIs, are a specific type of Web APIs that adhere to the Representational State Transfer (REST) [180] architectural style, a set of constraints that promote scalability, statelessness, and uniform interfaces in distributed systems. Clients interact with REST APIs by sending HTTP requests to a web service and processing the returned responses, typically in JSON format. Figure §3.1 provides an overview of this communication process.



Figure 3.1: REST API interaction workflow.

By sending requests, the client interacts with different resources managed by the API provider. A resource represents any type of data that the client can access and manipulate using CRUD (Create, Read, Update and Delete) operations, which typically correspond to the HTTP methods POST, GET, PUT and DELETE, respectively. Requests are directed to API endpoints, uniquely identified by a combination of an HTTP method and a URL path. Table §3.1 illustrates examples of different operations performed on the *Playlist* resource of the Spotify API [95].

Table 3.1: CRUD example.

HTTP method	Path	Action
POST	`/users/{user_id}/playlists`	Create a new playlist
GET	`/me/playlists`	Return playlists of the logged in user
PUT	`/playlists/{playlist_id}`	Edit playlist
DELETE	`/playlists/{playlist_id}/tracks`	Remove items from a playlist

When making HTTP requests, the client specifies the HTTP method and path while also providing parameters in various parts of the request, such as the body, headers, query string, or path. Once the web service processes the request, it returns a response, typically in JSON format, accompanied by a three-digit HTTP status code that indicates the outcome of the request. HTTP status codes fall into five groups based on their first digit [48]:

- **1XX codes - Informational.** These codes indicate that the server has received the request and is processing it. They serve as provisional responses, indicating the client to wait for the final response.
- **2XX codes - Success.** This class of status code indicates that the request has been successfully received and processed by the server.
- **3XX codes - Redirection.** This class of status code indicates that the client must take additional action to complete the request. For example, a 302 status code signals that the requested resource has been temporarily moved to a different URL, and the client should redirect to that location.
- **4XX codes - Client Error.** These codes indicate errors caused by the client when attempting to process the request. They are returned in situations such as when the user provides invalid parameter values (400 code), is not authenticated to perform an action (401 code), or tries to access a resource that does not exist (404 code).
- **5XX codes - Server Error.** This class of status code indicates cases in which the server has encountered an error or is unable to fulfill the request.

3.2 OAS SPECIFICATION

The OpenAPI Specification (OAS) [73], originally introduced as Swagger by SmartBear Software in 2011 [93] and managed by the OpenAPI Initiative since 2015 [71], is a widely adopted standard for describing REST APIs in a machine-readable format. It enables both humans and machines to discover, understand and interact with an API without requiring prior knowledge of its implementation details or underlying technology. By providing a structured and language-agnostic definition, OAS reduces the efforts required for API consumers for interacting with the service.

An OAS specification is structured as a JSON object and can be represented in either JSON or YAML format. This standardized structure enables a wide range of applications, including generation of interactive API documentation for developers, automated code generation (*e.g.*, API gateways, client SDKs, server stubs, and API mocks in multiple programming languages), security analysis, API governance, CI/CD integration, database modelling, and automated test case generation.

OAS follows a `major.minor.patch` versioning scheme. At the time of writing this

dissertation, the latest release is version 3.1.1 [72], published on October 24, 2024. Listing 3.1 presents the root of the OAS specification of the DHL API [33]. The root element of an OAS specification is the OpenAPI object, which defines key characteristics of the API through several fields. The main fields are:

```yaml
openapi: 3.0.1
info:
    title: DHL Location Finder API
    version: 1.10.1
    description: "The DHL Location Finder API provides Service Point locations of Deutsche
        Post DHL Group. This service is also known as pick up and drop off (PUDO)."
tags:
- name: locations
  description: "Retrieve information about DHL Service Point locations"
servers:
- url: https://api.dhl.com
  description: Production Server
security:
- ApiKeyAuth: [ ]
paths:
    /location-finder/v1/find-by-address:
        get:
            ...
    /location-finder/v1/find-by-geo:
        get:
            ...
components:
    securitySchemes:
        ApiKeyAuth:
            type: apiKey
            in: header
            name: DHL-API-Key
    schemas:
        BadRequest:
            ...
        InternalError:
            ...
        locations:
            ...
```

Listing 3.1: OAS structure of the DHL API.

- **`openapi` (line 1).** Specifies the version of the OpenAPI Specification being used.
- **`info` (lines 2-6).** Contains metadata about the API, such as its name, description, terms of service, license, or version.
- **`tags` (lines 7-9).** Provides a list of tags with additional metadata for organizing API operations.
- **`servers` (lines 10-12).** Lists server objects that provide connectivity information for different target servers.
- **`security` (lines 13-14).** Declares the security mechanisms available for the API, such as API keys or OAuth 2.0 authentication.

- **`paths` (lines 15-21).** Defines the available API endpoints and their corresponding operations.

- **`components` (lines 22-34).** Contains reusable elements, such as shared response formats, request schemas, and security schemes.

In the OAS specification, the `paths` array contains relative URLs. Each URL may list one or more HTTP methods (verbs) that define how to interact with that endpoint. Every combination of URL and HTTP method represents a single API operation. Listing 3.2 presents one of the operations from the DHL API. This operation, identified by the operation ID "getLocation-finderV1Find-by-address" (line 5), is accessed by performing an HTTP request to the relative path `/location-finder/v1/find-by-address` (line 2) using the GET HTTP method (line 3).

OAS defines the input parameters (lines 6-53) and responses (lines 54-72) of the operation. Each input parameter includes its name (line 7), location within the API request (line 8), description (line 9), datatype (lines 11-12), and whether it is mandatory (line 10). Additionally, OAS may provide optional details such as example values (line 13), default values (line 46), and constraints like minimum (line 48) and maximum values (line 49), among others.

Supported responses are categorized by their HTTP response codes (lines 55, 61, and 67). Each response includes a natural language description (lines 56, 62, and 68) and a specified format for the expected output (lines 57-60, 63-66, and 69-72). In this case, the response schemas (lines 60, 66, and 72) reference predefined components within the `components` section of the API specification, enabling reuse across multiple API operations that share the same response structure. Listing 3.3 presents a simplified version of the expected response schema for the operation when the API returns a 200 status code (left-hand side), along with a corresponding JSON-formatted response (right-hand side) that adheres to this schema.

3.3 REST API Testing

This section presents background information on the state of the art in both manual and automated REST API testing techniques.

```yaml
paths:
  /location-finder/v1/find-by-address:
    get:
      description: "This method will find DHL locations based on an address."
      operationId: "getLocation-finderV1Find-by-address"
      parameters:
      - name: countryCode
        in: query
        description: "A two-letter ISO 3166-1 alpha-2 code specifying the country."
        required: true
        schema:
          type: string
        example: DE
      - name: addressLocality
        in: query
        description: "Text specifying the name of the locality, for example a city."
        required: false
        schema:
          type: string
        example: Mannheim
      - name: postalCode
        in: query
        description: "Text specifying the postal code for an address."
        required: false
        schema:
          type: string
      - name: streetAddress
        in: query
        description: "The street address is expressed as free form text."
        required: false
        schema:
          type: string
      - name: providerType
        ...
      - name: locationType
        ...
      - name: serviceType
        ...
      - name: radius
        ...
      - name: limit
        in: query
        description: "Maximum amount of locations included in the results."
        required: false
        schema:
          default: 15
          type: number
          minimum: 0
          maximum: 50
      - name: hideClosedLocations
        ...
      - name: currentDate
        ...
      responses:
        "200":
          description: "DHL Service Point location collection"
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/locations"
        "400":
          description: "Bad request, for example invalid country"
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/BadRequest"
        "500":
          description: "Internal Server Error."
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/InternalError"
```

Listing 3.2: OAS excerpt of the find-by-address operation of the DHL API.

```yaml
components:
  schemas:
    locations:
      type: object
      properties:
        locations:
          type: array
          items:
            type: object
            properties:
              url:
                type: string
              location:
                type: object
                properties:
                  ids:
                    type: array
                    items:
                      type: object
                      properties:
                        locationId:
                          type: string
                        provider:
                          type: string
                  type:
                    type: string
              name:
                type: string
              place:
                type: object
                properties:
                  countryCode:
                    type: string
                  addressLocality:
                    type: string
                  latitude:
                    type: number
                  longitude:
                    type: number
              serviceTypes:
                type: array
                items:
                  type: string
```

(a) OAS response schema

```json
{
   "locations": [
      {
         "url": "/locations/8003-4321406",
         "location": {
            "ids": [
               {
                  "locationId": "8003-4321406",
                  "provider": "parcel"
               }
            ],
            "type": "servicepoint"
         },
         "name": "Q-Mobile",
         "place": {
            "countryCode": "DE",
            "addressLocality": "Mannheim",
            "latitude": 49.488414,
            "longitude": 8.468105
         },
         "serviceTypes": [
            "parcel:pick-up",
            "parcel:drop-off-unlabeled",
            "parcel:drop-off-firstmile-unlabeled",
            "parcel:drop-off-firstmile",
            "parcel:drop-off-return-unlabeled",
            "parcel:drop-off-return",
            "parcel:drop-off"
         ]
      }
   ]
}
```

(b) Response conforming to schema

Listing 3.3: DHL OAS response schema with example response.

3.3.1 Manual Testing of REST APIs

Test cases for REST APIs can be implemented manually, using libraries such as REST Assured [83] for Java, pytest [80] for Python, and Jest [58] for JavaScript, or by utilizing tools like Postman [78], which provides a graphical interface for building, testing, and interacting with APIs.

Listing 3.4 shows an example of a REST Assured test case for the "getLocation-finderV1Find-by-address" operation of the DHL API (Listing 3.2), which contains a single API request. Typically, test cases for REST APIs are implemented at the system level. Each test case involves at least one request to specific API operations (line 17), where input values are assigned to their parameters (lines 13-15). Then, various verifications are performed on the returned responses, such as checking the HTTP status code (line 19) and validating that the response JSON meets specified criteria (line 20).

```
1  @Test
2  public void testFindLocationByAddress() {
3
4      // Input parameters
5      String countryCode = "DE"; // Germany
6      String addressLocality = "Mannheim"; // City in Germany
7      int limit = 10; // Limit the number of results to 10
8
9      // Send a GET request to find DHL locations
10     Response response = given()
11             .baseUri("https://api.dhl.com/location-finder/v1") // Set the base API URL
12                 .header("DHL-API-Key", apiKey) // Set API Key in the header
13                 .param("countryCode", countryCode) // Add country code parameter
14                 .param("addressLocality", addressLocality) // Add address locality parameter
15                 .param("limit", limit) // Limit results to 10
16             .when()
17                 .get("/find-by-address") // API endpoint
18             .then()
19                 .statusCode(200) // Verify that the response status code is 200 (OK)
20                 .body("locations.size()", greaterThan(0)) // Ensure at least one location is found
21             .extract()
22             .response(); // Store the response
23
24     // Print the response for debugging
25     System.out.println("Response Body: " + response.getBody().asString());
26 }
```

Listing 3.4: Example of test case implemented using REST Assured.

3.3.2 Automated Testing of REST APIs

REST API testing is highly automatable. Given that all REST APIs follow the same standards and design principles—functioning as a set of defined operations over resources—the same testing techniques can be applied to all APIs. Moreover, the functionality of any REST API can be formally described using an OAS specification, providing a structured interface that is both human and machine-readable.

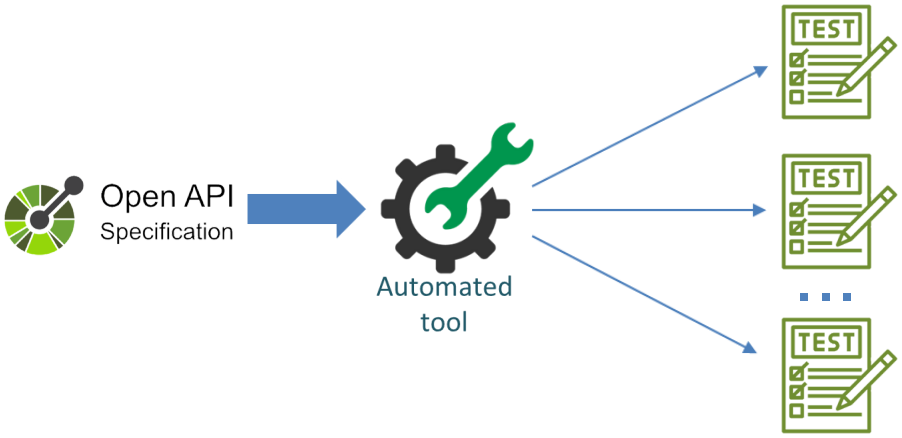


Figure 3.2: Workflow of an automated REST API testing tool.

Recent years have seen a surge in automated REST API testing techniques. Most of these approaches follow a workflow similar to the one presented in Figure §3.2, where an automated tool utilizes the OAS specification of the API under test to generate hundreds, thousands, or even millions of test cases as the one depicted in Listing 3.4. These test cases are created by assigning values to input parameters across different API operations and verifying responses (*i.e.*, applying test oracles) by ensuring, for instance, that the returned responses are not server errors (5XX codes) and are conformant to the OAS-defined syntax.

Most of these approaches operate in black-box mode, requiring only access to the OAS specification without relying on the source code of the API. This makes them applicable to any REST API, regardless of its implementation details. The approaches differ in the way they generate API requests, employing techniques such as model-based testing [231, 243, 298], property-based testing [200, 212, 277, 278], and constraint-based testing [240, 242]. Some methods focus on testing individual API operations by generating single API requests, while others construct sequences of API requests to enable stateful testing [131, 164, 200, 216, 218, 294]. More recent approaches leverage Large Language Models (LLMs) [217, 218, 219, 223] to identify dependencies between parameters and operations. In contrast, white-box approaches require access to the API source code and depend on the underlying technology stack, making them less widely applicable than black-box techniques. Most white-box approaches incorporate search algorithms to optimize failure detection and maximize code coverage [130, 284, 308].

Input values for test case generation can be obtained using various strategies. Some authors rely on random (fuzzing) [131] or default [175] values, but these are rarely sufficient for thorough API testing. Generating meaningful values for parameters such as

hotel names, airport codes, or ISO currency codes randomly is often impractical. Even if a test data generator produces syntactically valid inputs, the likelihood of constructing API requests that return some results—and thus exercise the core functionality of the API—would be remote. A similar limitation applies to default or sample values, which are primarily intended to illustrate API behavior rather than support exhaustive testing. Hence, most test case generation approaches resort to data dictionaries—predefined sets of input values manually collected and curated by testers [240]. For instance, to test the `postalCode` parameter of the DHL API (Listing 3.2), a tester might compile a list of valid postal codes to use as inputs. However, creating and maintaining data dictionaries for every non-trivial parameter is a labor-intensive and costly manual process.

When it comes to their supported test oracles, existing tools are all limited by the types of failures they can detect, namely crashes (responses labelled with a 5XX HTTP status code) [130, 131, 200, 212, 218, 219, 240, 294, 301], discrepancies with the API specification (*e.g.*, missing properties in the output JSON) [130, 200, 212, 240, 294], regressions [187, 191], and violations of API best practices (*e.g.*, ensuring that repeated calls to idempotent operations return identical results) [132, 136, 291, 307]. However, many errors can still go undetected by current tool support, such as incorrect field lengths or formats, violations of numerical constraints, or issues with array properties. Recent surveys [193] and tool comparisons [214, 244] have identified the generation of test oracles as one of the major challenges in automated REST API testing.

RESTest

RESTest [240, 243] is an open-source black-box testing framework for REST APIs that employs a model-based approach, automatically deriving test cases from the OAS specification of the API under test. It integrates advanced testing techniques such as fuzzing, adaptive random testing, and constraint-based testing to generate diverse test cases, ensuring comprehensive API validation. RESTest can be easily extended with new test case generation strategies, test data generators, and test writers, enabling developers to customize the tool to their specific requirements. The framework is publicly available on GitHub [88].

Figure §3.3 illustrates the workflow of RESTest, which consists of the following steps:

1. **Test model generation.** RESTest takes the OAS specification of the API under test as input, treating it as the *system model*. From this, it automatically generates a *test*

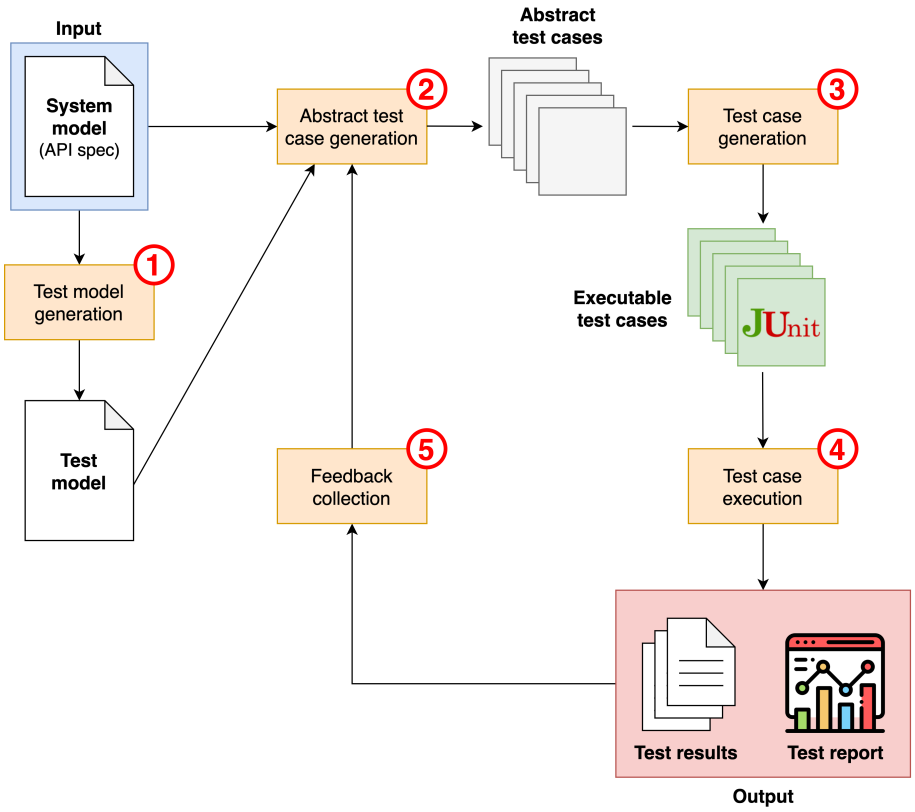


Figure 3.3: RESTest workflow.

model that incorporates test-specific configuration data. This test model includes all relevant testing configurations and can be manually refined with additional details, such as authentication credentials (*e.g.*, API keys) or test data generation settings (*e.g.*, random number or date generation, or random English words).

2. **Abstract test case generation.** The system and test models guide the creation of *abstract test cases* based on user-defined test generation strategies, such as random or constraint-based testing. These test cases remain abstract and platform-independent, allowing for later transformation into executable test cases for various testing frameworks and programming languages.

3. **Test case generation.** Abstract test cases are converted into *executable test cases* using a test writer, which translates them into a specific programming language or testing framework. RESTest currently supports the generation of test cases for REST Assured [83] and Postman [78].

4. **Test case execution.** The generated test cases are executed, producing reports and machine-readable statistics. Test results can be visually analyzed using a user-friendly GUI built with Allure [9], providing clear insights into testing outcomes.

5. **Feedback collection.** Test case generators and other components leverage test outputs (*i.e.*, the statistics generated in the previous step) to refine and enhance test case generation, enabling more sophisticated testing strategies.

3.3.3 Testing as a Service of REST APIs

Driven by significant advancements and the widespread adoption of cloud computing in recent years, a new set of business model—generally based on paid subscriptions—has emerged and gained popularity under the umbrella term *Anything as a Service* (*XaaS*). These models provide services to customers without requiring them to purchase, install, or maintain physical infrastructure or proprietary software. Originating from the concept of *Software as a Service* (*SaaS*)—which delivers applications and software through the cloud without the need for local installation (*e.g.*, Google Workspace [44], or Microsoft 365 [65])—this business model has expanded into other domains. One such example is *Infrastructure as a Service* (*IaaS*), which offers on-demand computing resources such as virtual servers, storage, and networking (*e.g.*, Amazon Web Services EC2 [12], Google Compute Engine [42], and Microsoft Azure Virtual Machines [67]). Similarly, *Platform as a Service* (*PaaS*) provides cloud-based development and execution environments to streamline application creation and deployment (*e.g.*, Microsoft Azure App Service [66]). This approach has further diversified into specialized models, including *Backend as a Service* (*BaaS*), *DataBase as a Service* (*DBaaS*), and *Games as a Service* (*GaaS*), among many others.

Another emerging model within the XaaS ecosystem is *Testing as a Service* (*TaaS*), which leverages cloud computing to provide on-demand software testing solutions. TaaS enables organizations to perform automated and manual testing without investing in dedicated infrastructure, reducing costs and improving scalability. Popular TaaS offerings integrate seamlessly with continuous integration and deployment (CI/CD) pipelines (such as TravisCI [102], Jenkins [57], CircleCI [19], or GitHub Actions [41]), ensuring efficient quality assurance throughout the software development lifecycle.

Several TaaS providers specialize in REST API testing, including Rapid [82] (formerly known as RapidAPI), Datadog [27], and Sauce Labs [90]. These platforms offer a range of subscription-based services, including tools for manually creating test cases—often through graphical interfaces—automated test execution with scheduled or condition-based triggers, and features that enhance collaboration among developers. Additionally, they integrate seamlessly with CI/CD platforms, provide notification systems via platforms such as email, Slack, or Microsoft Teams, and offer interac-

tive dashboards with statistical insights into test results.

Martin-Lopez et al. [244] conducted an empirical study involving 13 industrial APIs to evaluate the use of automated test case generation methods for online testing (*i.e.*, testing performed directly on production software) of REST APIs, resembling the model of TaaS offered by popular industrial platforms. The goal was to assess the practical viability of techniques proposed in academic research within an industrial setting. The study, which involved the generation of over one million test cases in a period of 15 days, uncovered approximately 390K failures. These were triaged into 254 distinct bugs, 65 of which were acknowledged and subsequently fixed by the API developers.

3.4 SUMMARY

This chapter provided background on REST APIs and the state of the art in REST API testing. We introduced the key characteristics of REST APIs, presented the OAS specification, and examined recent advancements in both manual and automated testing, including Testing as a Service (TaaS) platforms.

4

Web of Data

Data is a precious thing and will last longer than the systems themselves.

Tim Berners-Lee,
Inventor of the World Wide Web, 2016 Turing Award

This chapter presents background information on the Web of Data. Section §4.1 introduces the concept of the Web of Data, followed by Section §4.2, which describes the RDF framework for representing relationships between resources. Section §4.3 explores knowledge bases, while Section §4.4 explains the SPARQL query language used to retrieve information from RDF-based knowledge bases. Finally, Section §4.5 summarizes the chapter.

4.1 INTRODUCTION

The Web of Data [144, 203] is a continuously expanding global data space that contains billions of interlinked, queryable data. Its primary objective is to facilitate the publication of structured data on the Web in formats that machines can efficiently process, enabling a higher level of automation and integration than human-readable content alone. By making the Web more machine-readable, the Web of Data allows for the automation of complex tasks, such as discovering, cross-referencing, filtering, and organizing vast amounts of information across different websites in real time. This not only enhances data interoperability but also enables the development of more powerful and intelligent Web applications. These applications can seamlessly integrate and analyze data from diverse sources, providing users with richer insights and more efficient decision-making processes.

This data is structured and published following the Linked Data principles [144], proposed by Tim Berners-Lee, which establish guidelines for making data more accessible, interoperable, and machine-readable on the Web:

1. **Use URIs as names for things.** Each entity (or resource) should have a unique Uniform Resource Identifier (URI) to ensure unambiguous referencing across different datasets. This enables consistent identification and retrieval of information.

2. **Use HTTP URIs so that people can look up those names.** By employing HTTP URIs, resources become directly accessible on the Web. When a URI is accessed, it should return meaningful data about the resource, facilitating both human and machine interpretation.

3. **When someone looks up a URI, provide useful information, using the standards (RDF, SPARQL).** When a URI is queried, the server should return structured and meaningful information, typically formatted using standards such as RDF (Resource Description Framework) and queried via SPARQL. These technologies ensure data can be effectively understood and processed by automated systems.

4. **Include links to other URIs, so that they can discover more things.** Connecting datasets by linking to other relevant URIs enhances discoverability and interoperability. This interconnection forms a Web of Data, allowing both humans and machines to seamlessly navigate between related information sources.

As the Web of Data continues to grow, it plays an increasingly crucial role in fields such as artificial intelligence, semantic search, data analytics, and knowledge graphs, paving the way for a more intelligent and context-aware digital ecosystem.

4.2 Resource Description Framework (RDF)

In the Web of Data, relationships between resources are expressed using the *Resource Description Framework* (*RDF*) [197], a W3C standard that defines how to represent and structure data in the form of triples. Each RDF triple consists of three components: a subject, a predicate, and an object, formally denoted as $\langle subject, predicate, object \rangle$. The predicate specifies the relationship (or link) between the subject and object entities.

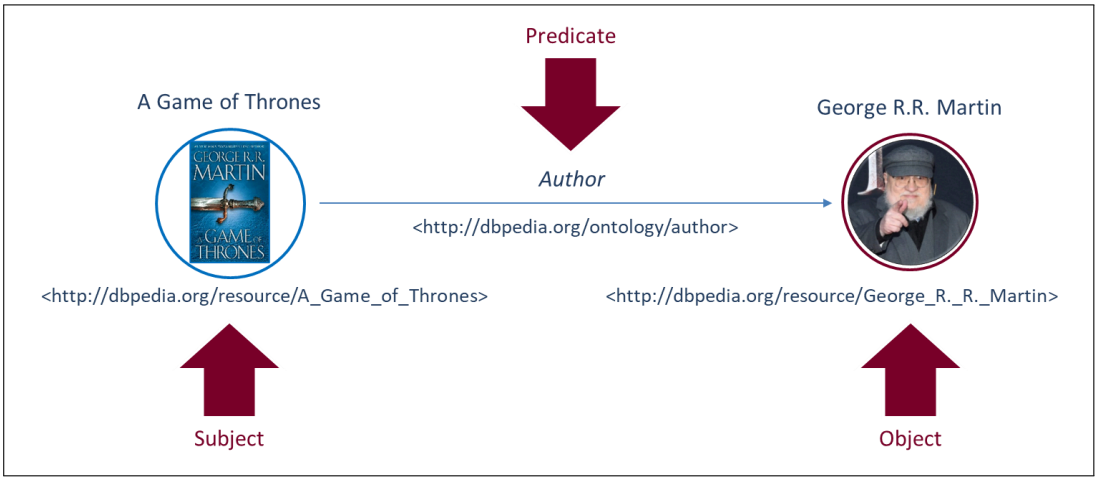


Figure 4.1: RDF triple.

For example, the triple depicted in Figure §4.1 indicates that "A Game of Thrones" (subject) is authored by (predicate) George R.R. Martin (object). In RDF, subjects and predicates are typically represented by URIs to ensure unambiguous identification of entities and relationships. The object can either be another URI (representing another resource) or a literal value (such as a string, number, or date).

Figure §4.2 illustrates a simple RDF graph representing this structure. In the diagram, a subject (identified by a URI) is associated with multiple predicates, each linking it to different objects. Some objects are URIs (`http://dbpedia.org/resource/George_R._R._Martin` and `http://dbpedia.org/resource/A_Song_of_Ice_and_Fire`), signifying links to other resources, while others ("978-0553573404" and "694") are literals, representing data values such as names, dates, or textual descriptions. This graph-based structure enables seamless data integration and linkage across distributed

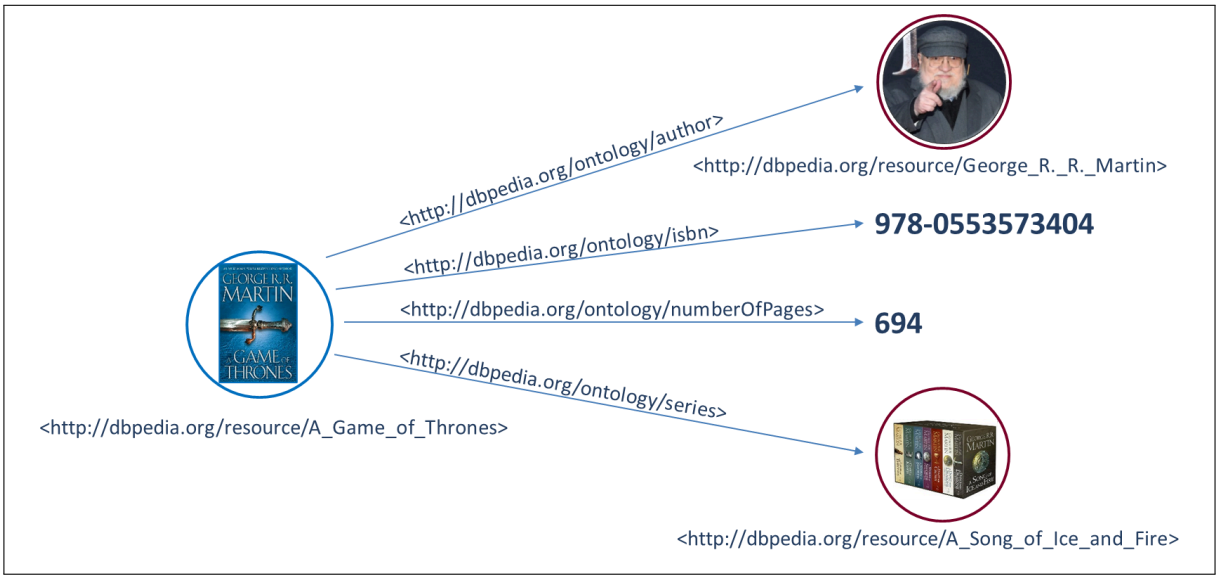


Figure 4.2: Simple RDF graph.

datasets, forming the foundation of the Semantic Web. RDF graphs can be queried using SPARQL (Section §4.4).

4.3 Knowledge Bases

Knowledge bases [220] like DBpedia [143] and Wikidata [297] are structured repositories of information that store, organize, and manage data in a way that supports automated reasoning, decision-making, and efficient information retrieval. Knowledge bases often consist of RDF graphs where triples generated from various sources are interlinked and can be queried using SPARQL queries (Section §4.4). They play a critical role in a variety of applications, ranging from artificial intelligence and expert systems to semantic search and data analytics. By organizing data in a structured and interconnected manner, they enable systems to infer new relationships and insights that go beyond the explicit information stored.

DBpedia [143] is one of the largest and most widely used knowledge bases available on the Web. It was created by extracting structured information from Wikipedia and transforming it into a machine-readable format. This structured data includes entities identified by unique URIs, rich RDF descriptions, hierarchical classifications, and interlinked relationships between entities. By leveraging these features, DBpedia enables users to perform semantic queries on Wikipedia content, allowing them to explore relationships, properties, and connections to external datasets. The DBpe-

dia project was initially launched by researchers at the Free University of Berlin and Leipzig University, in collaboration with OpenLink Software. Today, it is maintained by the University of Mannheim and Leipzig University, ensuring its continued development and integration within the Linked Data ecosystem.

4.4 SPARQL

SPARQL (*SPARQL Protocol and RDF Query Language*) [2] is a query language designed for retrieving and manipulating data from knowledge bases and datasets represented as RDF triples. It enables users to perform structured queries on Semantic Web data, facilitating access to interlinked information across multiple sources.

```
SELECT DISTINCT ?title ?author ?pages ?isbn WHERE {

 ?subject <http://dbpedia.org/property/title> ?title ;
       <http://dbpedia.org/property/author> ?author ;
       <http://dbpedia.org/ontology/numberOfPages> ?pages ;
       <http://dbpedia.org/ontology/isbn> ?isbn .

 FILTER (?pages >= 100)
 FILTER regex(str(?isbn), "^([0-9]*[-| ]){4}[0-9]*$")

} LIMIT 5
```

Listing 4.1: SPARQL query to search for books.

SPARQL supports various query forms, with `SELECT` being one of the most commonly used. This query type returns results in different formats, such as structured tables, RDF triples, or RDF graphs. Listing 4.1 presents a sample SPARQL query that retrieves book titles along with their corresponding authors, ISBN numbers and number of pages. The query includes two essential SPARQL clauses:

- **`FILTER(condition).`** Restricts results based on a Boolean condition. This can be used to match regular expressions or impose arithmetic constraints (*e.g.*, filtering values above or below a threshold).
- **`LIMIT.`** Defines the maximum number of results to return, ensuring that only a specified subset of matching data is retrieved.

In this example, the query selects books that contain the predicates “title”, “author”, “numberOfPages”, and “ISBN”, while applying `FILTER` and `LIMIT` conditions. The filter ensures that the ISBN follows a five-group numeric pattern separated by

hyphens and that the book contains at least 100 pages. The limit ensures that the maximum number of results to return is 5. Table §4.1 presents the query results from Listing 4.1, showcasing the retrieved book information.

Table 4.1: SPARQL query response.

Title	Author	Number of pages	ISBN
The Last Wish	http://dbpedia.org/resource/Andrzej_Sapkowski	352	978-0-575-08244-1
The Martian	http://dbpedia.org/resource/Andy_Weir	416	978-0-8041-3902-1
The Name of the Wind	http://dbpedia.org/resource/Patrick_Rothfuss	672	978-0-7564-0407-9
An Oracle	http://dbpedia.org/resource/Sakyo_Komatsu	224	978-4-7700-2039-0
The Black Tulip	http://dbpedia.org/resource/Alexandre_Dumas	288	978-0-14-044892-4

4.5 Summary

In this chapter, we provided background information on the Web of Data to contextualize our contributions. We introduced the Web of Data and the Linked Data principles that define its structure, the Resource Description Framework (RDF) for representing relationships between resources, the concept of knowledge bases, and SPARQL, a query language for querying RDF-based knowledge bases.

5

Natural Language Processing

Words are pale shadows of forgotten names. As names have power, words have power. Words can light fires in the minds of men. Words can wring tears from the hardest hearts.

Patrick Rothfuss,
The Name of the Wind

This chapter presents the key concepts of Natural Language Processing (NLP) needed to contextualize our contributions. Section §5.1 introduces NLP, covering both traditional and modern techniques. Section §5.2 discusses the Transformer architecture, a fundamental framework behind recent generative machine learning models, followed by Section §5.3, which covers Large Language Models. Section §5.4 presents the Hugging Face platform. Section §5.5 summarizes the chapter.

5.1 INTRODUCTION

Natural Language Processing (NLP) is a field of artificial intelligence that focuses on enabling computers to understand, interpret, generate, and manipulate human language, thus narrowing the gap between how humans communicate and how machines process data.

Before the advent of deep learning and Large Language Models (LLMs), NLP primarily relied on traditional, rule-based and statistical techniques. These approaches typically involved a pipeline composed of handcrafted rules or shallow machine learning components for processing and analyzing text. Some of these techniques were *tokenization* [195], where text is split into individual words or phrases, and *lemmatization* [134], which reduces words to their base or dictionary form to normalize inputs (*e.g.*, converting "learning" into "learn"). *Part-of-speech (POS) tagging* [289] was used to assign grammatical categories (*e.g.*, noun, verb, adjective) to each token of the text based on its context, while *Named Entity Recognition (NER)* [282] aimed to identify specific entities such as people, locations, or organizations. To represent textual data numerically, techniques like *Bag of Words (BoW)* [273] or *TF-IDF (Term Frequency-Inverse Document Frequency)* [272] were commonly used. These representations, however, ignored word order and contextual information, treating text as unordered collections of tokens. Despite their simplicity, such pipelines enabled early successes in applications like information retrieval, document classification, and rule-based chatbots. Nevertheless, they often struggled with ambiguity, lacked domain generalization, and failed to capture deeper semantic relationships in language.

The limitations of traditional NLP techniques—particularly their inability to model word meaning and context—spurred the development of more advanced methods rooted in *representation learning*. A pivotal milestone came with the introduction of *Word2Vec* [247, 248], which represented words as dense, continuous vectors learned from their surrounding context in large corpora. This marked a shift from symbolic to *distributional semantics*, capturing nuanced semantic relationships such as analogies. Building on this, *Recurrent Neural Networks (RNNs)* [176] and their variants like *Long Short-Term Memory (LSTM)* [202] and *Gated Recurrent Units (GRU)* [157] networks, were proposed to model sequential data. These architectures maintained memory over time, enabling more context-aware processing of text and advancing tasks like machine translation and text generation.

The next major breakthrough came with the introduction of the *Transformer architec-*

ture [292], which replaced recurrence with self-attention mechanisms, enabling models to capture global dependencies in parallel and at scale. This architecture laid the foundation for pretrained LLMs, trained on massive corpora with billions of parameters, capable of performing a wide range of NLP tasks with little to no task-specific supervision, thanks to techniques like transfer learning, fine-tuning, and more recently, prompting and in-context learning. The rise of LLMs has fundamentally transformed NLP, enabling systems to generate coherent text, answer questions, translate languages, and reason about language with unprecedented fluency and accuracy.

5.2 The Transformer Architecture

The Transformer architecture, introduced by Vaswani et al. in 2017 [292], was originally proposed for machine translation but has since revolutionized the field of deep learning. Designed to convert an input sequence into an output sequence, the Transformer is well-suited for a wide range of tasks such as machine translation, text summarization, and question answering. Its flexibility, scalability, and performance have made it applicable far beyond its original purpose, becoming the foundational architecture behind most cutting-edge generative models, including ChatGPT [45, 70, 116], DeepSeek [198, 228], Gemini [38, 39], and Llama [63, 173].

At the core of the Transformer is the *self-attention mechanism*, which enables the model to consider the relevance of every token in the sequence relative to every other token, regardless of their position. This contrasts with earlier approaches like Recurrent Neural Networks (RNNs) [157, 176, 202], which process tokens sequentially and struggle with long-range dependencies. By processing input tokens in parallel, Transformers achieve both speed and the ability to model complex relationships within the data.

Figure §5.1 shows the architecture of a standard Transformer model. The architecture consists of two main components: an *encoder*, which processes the input sequence, and a *decoder*, which generates the output sequence. Each component is composed of stacked layers that include multi-head self-attention mechanisms, feed-forward networks, residual connections, and layer normalization. To compensate for the lack of inherent sequence order, Transformers use *positional encodings* to inject information about the position of tokens in the sequence.

There are three main architectural variants of the Transformer [290]:

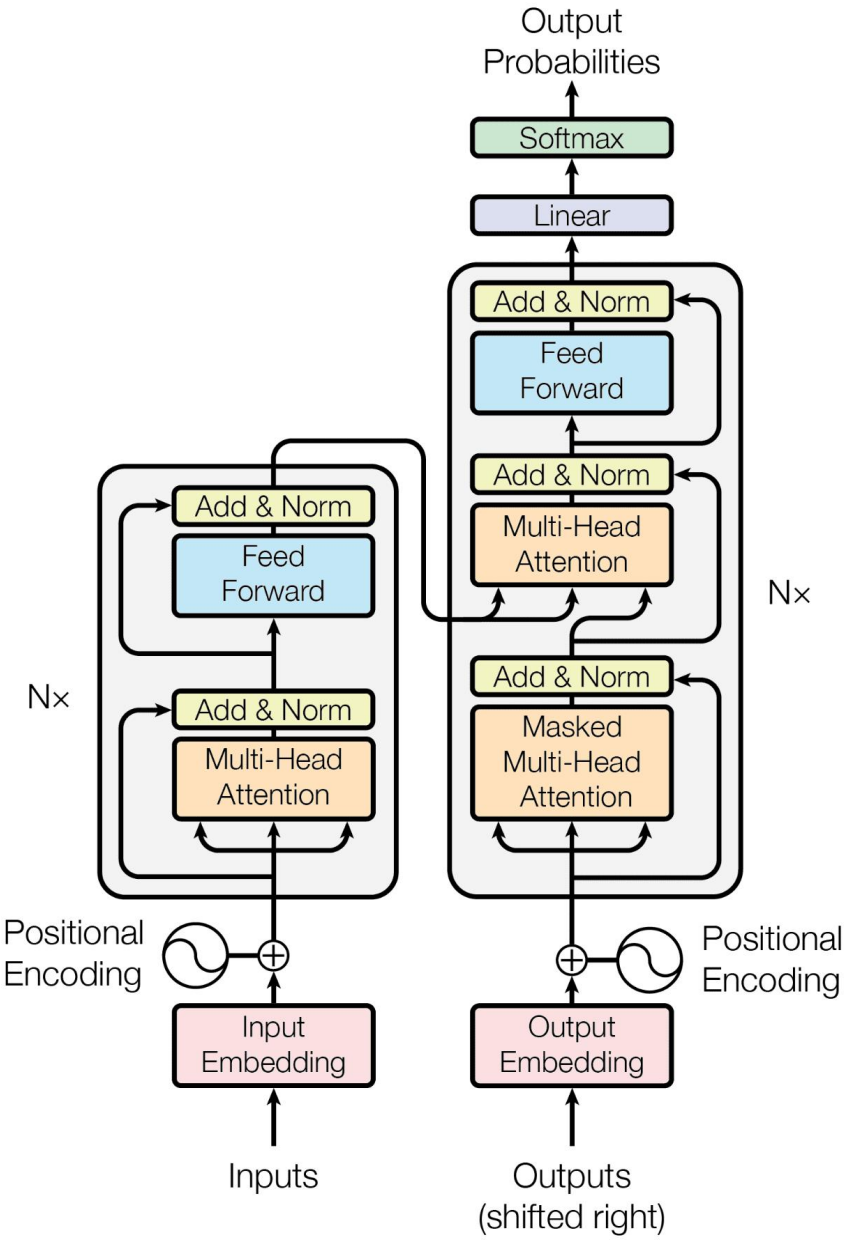


Figure 5.1: The Transformer model architecture [292].

- **Encoder-decoder models,** like the original Transformer and T5 [269], are used for sequence-to-sequence tasks where both an input and an output sequence are involved, such as translation or summarization.
- **Encoder-only models,** such as BERT [168] and RoBERTa [230], are optimized for understanding and classification tasks (*e.g.*, sentiment analysis, named entity recognition), where the goal is to generate contextual embeddings for the input.
- **Decoder-only models,** such as the GPT [45, 116, 151, 267, 268] and Llama [63, 173] families, are designed for generative tasks, where the model predicts the next token in a sequence. These models have become the dominant architecture for LLMs and underpin most state-of-the-art systems today.

Although originally developed for NLP, Transformer-based models have also been applied successfully in other domains. In computer vision, models like Vision Transformer (ViT) [171] have shown competitive performance on image classification tasks. In bioinformatics, they are used for protein structure prediction [155] and genomic sequence analysis [255].

5.3 Large Language Models

Large Language Models are built upon the Transformer architecture. Early breakthroughs in this field, such as the GPT series from OpenAI [45, 116, 151, 267, 268] and BERT from Google [168], laid the foundation for modern LLMs. The release of ChatGPT in November 2022 marked a significant turning point; its advanced conversational capabilities captured public attention and demonstrated how interactive dialogue could transform access to AI. This breakthrough spurred numerous companies to release a variety of LLMs—both proprietary and open-source. For example, Meta introduced the Llama series [63, 173], DeepSeek launched its R1 model [198], Alibaba presented Qwen [266], Anthropic developed Claude [20], and Microsoft unveiled Phi [115]. Google further diversified its offerings with the proprietary Gemini family [38, 39] alongside the open-source Gemma series [286]. Notably, these providers continuously release new and improved versions of their models that significantly outperform the capabilities of earlier iterations, surpassing models released only months—or even weeks—prior. This rapid pace of innovation underscores the dynamic and competitive nature of the field, driving ongoing advancements in AI performance.

LLMs vary widely in size—from models that require sophisticated datacenters to smaller variants that can even operate on smartphones—enabling diverse applications across different environments. Proprietary models are typically accessed through graphical interfaces or REST APIs and often come with usage limits based on the number of prompts or tokens, requiring payment for extended or higher-tier access. In contrast, open-source LLMs are freely available on platforms like Hugging Face [49] (Section §5.4), where users can download, fine-tune, and deploy them without significant restrictions beyond the required computational resources. Open-source LLMs are usually noted for their permissive licenses, allowing broad usage in commercial settings. For instance, according to the license terms of the Llama 3 models [62], organizations with fewer than 700 million active users can use the models under the Meta Llama 3 Community License Agreement (*i.e.*, at no cost), and are only required to request a commercial license if their user base exceeds this threshold.

Recent trends in LLM development include retrieval-augmented generation (RAG) [185], which integrates external data to improve the accuracy of responses; multi-agent systems [226] that allow models to collaborate and solve complex problems; multimodal approaches [302] that combine text, image, audio, and video inputs; and reasoning-focused models [70, 198] designed to perform more sophisticated logical inferences. These innovations have led to the integration of LLMs into many everyday applications, from chatbots and virtual assistants to translation services, programming aids, and advanced search engines.

The upcoming subsections cover different techniques for prompting LLMs, as well as the different stages of their training process.

5.3.1 Prompting Techniques

Prompting techniques define how tasks are presented to LLMs and can significantly impact the quality of their outputs [293]. Among the most widely used techniques is zero-shot prompting, where the model receives only a task description and is expected to produce a relevant response, without any examples. Few-shot prompting includes a small number of input-output examples in the prompt to illustrate the desired behavior. Another influential method is chain-of-thought prompting, which encourages the model to articulate intermediate reasoning steps before arriving at a final answer—an approach that usually improves performance on complex reasoning tasks.

5.3.2 The LLM Training Pipeline

This subsection discusses the various techniques employed in training LLMs.

Pretraining

The pretraining step [290] is the foundational stage in the training of an LLM, in which the model learns to understand and generate human language by predicting tokens in massive, unlabeled text corpora. During this process, the model is exposed to billions of words from diverse sources—such as books, articles, code, and web pages—which it processes using a Transformer-based architecture. Depending on the model type, the objective is typically to predict the next token in a sequence (as in autoregressive models like GPT [45, 116, 151, 267, 268]) or to recover randomly masked tokens (as in masked language models like BERT [168]). Through this self-supervised learning task, the model develops a deep statistical understanding of grammar, syntax, semantics, and even world knowledge. Pretraining does not require annotated data, it relies on the inherent structure of language to generate training signals.

Due to the massive scale of the data and the size of the models—often involving billions of parameters—pretraining is by far the most expensive phase in terms of both time and computational resources, typically requiring large clusters of GPUs or TPUs and weeks or even months of continuous training. For example, the pre-training of DeepSeek-V3 [228]—a model with 685 billion parameters—required approximately 2.66 million GPU hours (equivalent to over 110k days), while all subsequent training phases combined consumed only 124k GPU hours (just over 5k days). The result of this phase is a general-purpose language model capable of capturing rich linguistic patterns, but not yet tailored to perform specific tasks or follow human instructions.

Supervised Fine-Tuning

Supervised fine-tuning adapts a pretrained language model to perform specific tasks by training it on labeled datasets [290]. Unlike pretraining, which is general and task-agnostic, fine-tuning introduces supervision in the form of input-output pairs, allowing the model to specialize in tasks such as sentiment analysis, question answering, text summarization, or named entity recognition. During this phase, the weights of the model are updated using a smaller, task-specific dataset, often with a lower learning rate to preserve the general language understanding acquired during pretraining. Supervised fine-tuning can significantly improve performance on a target task, especially when sufficient labeled data is available. This phase requires careful design of training

objectives, datasets, and evaluation metrics to ensure that the model learns the desired behaviors without overfitting or forgetting its broader language capabilities. In terms of execution time and computational requirements, this training step is much less demanding than pretraining.

Instruction Tuning

This training step is designed to make language models more responsive to human instructions expressed in natural language. Unlike supervised fine-tuning, which focuses on optimizing performance for specific tasks, instruction tuning exposes the model to a broad set of tasks framed as input-output pairs accompanied by natural language instructions. These instructions describe what the model is expected to do (*e.g.*, "Translate this sentence into Japanese" or "Improve the writing of this paragraph"), helping the model generalize across diverse tasks and better follow user prompts. The data used for instruction tuning is typically curated from existing benchmarks, synthetic generation, or human-authored examples, covering a wide variety of domains and tasks. As a result of this process, the model becomes instruction-tuned—capable of not just completing a given text sequence, but of interpreting and responding to a wide range of user commands in a way that resembles human-like conversation. This enables the model to better handle open-ended queries, follow multi-step instructions, and engage more naturally and helpfully with users in interactive settings. For this reason, providers of open-source or open-weight LLMs often release two versions of each model: a base model, trained only through pretraining, which excels at text completion; and an instruction-tuned variant, specifically adapted for conversational or task-oriented use through additional fine-tuning on instruction-following datasets.

Reinforcement Learning from Human Feedback (RLHF)

This training technique is used to further align language models with human preferences and values, particularly in open-ended or subjective tasks [259]. While supervised fine-tuning teaches a model to imitate labeled examples by minimizing a loss function over fixed input-output pairs, RLHF goes a step further by incorporating human judgment into the training process. Instead of relying on static labels, RLHF uses human feedback to rank multiple model outputs for the same prompt based on qualities like helpfulness, safety, or clarity. These rankings are used to train a reward model, which estimates how desirable a given output is. Then, reinforcement learning—typically via the Proximal Policy Optimization (PPO) algorithm [275]—is used to fine-tune the model to maximize the predicted reward. This iterative process encourages the model to generate responses that better reflect human preferences,

making it especially valuable for conversational agents, where there is no single correct answer and alignment with user expectations is critical.

Reasoning Models

Recent advances have shown that simply increasing the size of LLMs (in terms of billions of parameters) or expanding the training data is no longer sufficient to yield substantial performance improvements, suggesting that current models may have reached a plateau. In response, researchers have explored *test-time compute* techniques [283], which allow models to perform additional computation during inference. This process emulates human reasoning by giving the model time to think step by step and consider multiple possible solutions before generating a final response. By generating intermediate reasoning steps—often referred to as chains-of-thought—models can significantly improve their performance on complex tasks such as mathematical problem solving and code generation, at the cost of longer inference times.

Various training techniques have been developed to equip LLMs with reasoning capabilities. For example, DeepSeek R1 [198] uses reinforcement learning to refine its chain-of-thought outputs. During training, the model generates step-by-step reasoning for a given problem, and a reward function evaluates each reasoning path based on correctness and clarity. Through a policy gradient method—specifically, Group Relative Policy Optimization (GRPO) [281]—the model receives feedback and is updated to prefer reasoning strategies that produce accurate and well-structured answers.

Another core contribution of the DeepSeek R1 paper is a series of distilled models derived by fine-tuning open-weight architectures—such as Qwen [266] and Llama [63, 173]—using an extensive synthetic dataset generated from chain-of-thought outputs generated by R1. This distillation process transfers the sophisticated reasoning skills learned through reinforcement learning to smaller models. Benchmark evaluations reported in the paper indicate that these distilled models achieve a significant performance improvement, sometimes even outperforming larger counterparts.

5.4 The Hugging Face Platform

Hugging Face [49], founded in 2016 by French entrepreneurs Clément Delangue, Julien Chaumond, and Thomas Wolf, is a leading machine learning platform that provides an open-source ecosystem for building, sharing, and deploying machine learning models. Hugging Face democratizes access to artificial intelligence by providing tools

and resources that enable developers and researchers worldwide to collaborate and innovate. This commitment to openness fosters transparency and inclusivity in AI development. In August 2023, Hugging Face raised $235 million in Series D funding, elevating its valuation to $4.5 billion [50].

The platform offers a suite of services designed to facilitate various aspects of machine learning:

- **Model Sharing and Hosting.** Users can upload, share, and access a vast repository of pre-trained models across multiple modalities, such as text, image, and audio processing. Many companies release their models on Hugging Face, leveraging its extensive community and infrastructure to maximize impact and accessibility.
- **Dataset Creation and Sharing.** Users can create, share, and access datasets, facilitating the development and benchmarking of machine learning models.
- **Model Fine-Tuning.** Developers can fine-tune pre-trained models on specific datasets to tailor them for particular applications, enhancing performance for specialized tasks.
- **Model Evaluation.** Tools are available to assess model performance, aiding in the selection of the most suitable models for given tasks.
- **Code Sharing.** Hugging Face supports the sharing of codebases, promoting collaboration and the dissemination of best practices within the AI community.
- **Learning Resources.** The platform serves as an educational hub, offering tutorials, documentation, and community support to help users at all levels advance their machine learning skills.

One of the central offerings of Hugging Face is the `Transformers` library [100, 101], a Python package that provides open-source implementations of transformer models for text, image, and audio tasks, simplifying the process of utilizing state-of-the-art models. It offers an abstraction layer that streamlines interactions with models, allowing developers to load pre-trained models and fine-tune them with minimal code.

5.5 SUMMARY

In this chapter, we provided the foundational knowledge of Natural Language Processing necessary to contextualize the contributions of this dissertation. We covered both traditional and modern NLP techniques, the Transformer architecture, and Large Language Models, detailing their key characteristics, prompting strategies, and training techniques. Additionally, we introduced the Hugging Face platform.

Part III

Contributions

ARTE: Automated Generation of Realistic Test Inputs for REST APIs

When asked, 'How do you write?' I invariably answer, 'one word at a time'.

Stephen King,
American writer

In this chapter, we present ARTE, an approach for the automated extraction of realistic test data for REST APIs from knowledge bases like DBpedia. Specifically, ARTE leverages the specification of the API parameters to automatically search for realistic test inputs using natural language processing, search-based, and knowledge extraction techniques. ARTE has been integrated into RESTest, fully automating the test case generation process. Evaluation results on 140 operations from 48 real-world REST APIs show that ARTE can efficiently generate realistic test inputs for 64.9% of the target parameters, outperforming the state-of-the-art approach SAIGEN (31.8%). More importantly, ARTE supported the generation of over twice as many valid API calls (57.3%) as random generation (20%) and SAIGEN (26%), leading to a higher failure detection capability and uncovering several real-world bugs. These results show the potential of ARTE for enhancing existing REST API testing tools, achieving an unprecedented level of automation.

The work discussed in this chapter was presented in a Journal article published in IEEE Transactions on Software Engineering [122].

6.1 INTRODUCTION

Testing REST APIs adequately requires using realistic test inputs such as country names, codes, coordinates, or addresses. As an example, the "Find by address" operation of the DHL API [33] (Listing 6.1) requires users to provide valid country codes (*e.g.*, "`DE`" for Germany), postal codes (*e.g.*, "`68161`"), street addresses (*e.g.*, "`Planken Straße`"), and locality names (*e.g.*, "`Mannheim`"), among others. Generating meaningful values for these types of parameters randomly is rarely feasible. Even if a test data generator could bypass the syntactic validation generating values with the right format, the chances of constructing API requests that return some results—and therefore exercise the core functionality of the API—would be remote. To address this issue, most test case generation approaches resort to *data dictionaries*: sets of input values collected by the testers, either manually [240] or, when possible, automatically [294]. This means a major obstacle for automation since data dictionaries must be created and maintained for each non-trivial input parameter, on each API under test. Other authors propose using the default or sample values included in the API specification as test inputs, if any, but those are solely intended to explain the behavior of the API, and they are insufficient to test it thoroughly [175].

Several authors have addressed the problem of generating realistic test inputs for desktop, web, and mobile apps using semantic knowledge discovery techniques [236, 299]. Specifically, they propose to query knowledge bases like DBpedia [143] to discover realistic test input values for the graphical user interface (GUI) elements of the application under test. For example, if the GUI includes a text field with the label "`DOI`", their approaches would search for "DOI" identifiers in DBpedia. To the best of our knowledge, this strategy—based on querying knowledge bases for realistic test inputs—has not been applied to the context of REST APIs yet, despite its potential.

Automatically generating realistic test data for REST APIs requires facing some unique challenges since, unlike GUIs, APIs are intended for developers, rather than for users. To start with, unlike GUI labels, API parameters may follow many different naming conventions. Hence, for example, different APIs could refer to the concept *country code* using very different parameter names such as `country` (Asos API [113]), `countryCode` (DHL API [33]), `country_code` (Numverify API [69]), `cc` (Foursquare API [36]), `c`, or even a less explanatory term, such as `location` (Domainr API [34]) or `excludedCountryIds` (GeoDBCities API [40]). This may make it necessary to resort to the description of each parameter in the search for further information that helps to

identify the key concept. However, descriptions are given in natural language and, as names, they can be very heterogeneous both in style and length. Finally, API specifications occasionally include further information such as expected patterns (*i.e.*, regular expressions) and sample values. Exploiting all these aspects for the generation of realistic test data is the key challenge addressed in our work.

In this chapter, we present an approach for the Automated generation of Realistic TEst inputs (ARTE) for REST APIs. Specifically, our approach focuses on RESTful APIs as the de facto standard, but it could be applied to any REST API provided that there is an API specification. ARTE leverages semantic knowledge discovery for the generation of realistic test inputs. In particular, ARTE exploits the specification of the various elements of the API under test, such as the name and description of its parameters, by querying knowledge bases to automatically generate realistic test inputs. In contrast to related approaches, ARTE includes a novel step for the automated inference of regular expressions from previously generated inputs, increasing the accuracy of the semantic queries and the overall performance of the approach. Furthermore, ARTE has been integrated into RESTest [243], making our approach fully automated and publicly available.

We evaluated the effectiveness of ARTE by comparing its performance with existing random techniques and the related tool SAIGEN—proposed in the context of automated testing of mobile apps [299]—in two scenarios: (1) on the generation of realistic test inputs for 48 REST APIs; and (2) on the generation of valid API calls, API coverage, and the detection of failures in 6 industrial REST APIs. Experimental results show that ARTE can generate meaningful valid inputs for 64.9% of the target API parameters (137 out of 211), outperforming SAIGEN (31.8%). More importantly, ARTE supported the generation of over twice as many valid API calls (57.3%) as random generation (20%) and SAIGEN (26%). As a result, ARTE achieved higher coverage and revealed more failures in more APIs, detecting confirmed bugs in the APIs of Amadeus [11] and DHL [33] not detected by related techniques. These results show the potential of ARTE to enhance current specification-driven REST API testing tools.

This chapter provides the following original research and engineering contributions:

- ARTE, a novel approach for the automated extraction of realistic test inputs for REST APIs from knowledge bases like DBPedia (Section §6.2).
- Integration of ARTE into the open-source testing framework RESTest, making

our approach readily applicable in practice (Section §6.3).

- An empirical comparison of ARTE with random data generation techniques and the state-of-the-art approach SAIGEN [299] on the generation of realistic test inputs and its impact on testing real-world APIs (Section §6.4).
- A publicly available replication package [29] containing the source code and datasets discussed in the chapter. We trust that this will also serve as a benchmark for future contributions in the topic.

We discuss the threats to validity in Section §6.5, and conclude the chapter in Section §6.6.

6.2 ARTE

In this section, we present ARTE, our approach for the Automated generation of Realistic TEst inputs for REST APIs. Specifically, ARTE leverages the information in the API specification to search for syntactically and semantically valid test data in knowledge bases like DBpedia. We define syntactically and semantically valid inputs as follows:

Definition 6.1 - Syntactically valid input.
An input value is *syntactically valid* if it satisfies the syntactic constraints defined in the API specification and it is accepted by the API under test without returning an error. For example, "`Germany`" is not a syntactically valid value for the parameter `countryCode` in the API of DHL (Listing 6.1) because, although it conforms to the specification (string value), the API only accepts two-letter ISO 3166-1 alpha-2 codes, as explained in the description of the parameter. Conversely, the value "`DE`" (ISO 3166-1 alpha-2 code for Germany) is syntactically valid.

Definition 6.2 - Semantically valid input.
An input value is *semantically valid* if it is coherent with the API domain. For example, "`Berlin`" is a semantically valid test input value for the parameter `addressLocality` in the API of DHL (Listing 6.1), whereas "`dog`" is not.

```yaml
paths:
  "/find-by-address":
    get:
      operationId: findByAddress
      description: Find DHL locations based on an address.
      produces:
        - application/json
      parameters:
        - name: countryCode
          in: query
          description: "A two-letter ISO 3166-1 alpha-2 code."
          required: true
          type: string
          x-example: "DE"
        - name: postalCode
          in: query
          description: "Postal code for an address."
          required: false
          type: string
          x-example: "53113"
        - name: limit
          in: query
          description: "Maximum of results to return."
          required: false
          type: number
          default: 15
          maximum: 50
          x-example: 20
        - name: locationType
          in: query
          description: "Type of the DHL Service Point location."
          required: false
          type: string
          enum:
            - servicepoint
            - locker
            - postoffice
            - postbank
        - name: streetAddress ...
        - name: serviceType ...
        - name: radius ...
        - name: addressLocality ...
        - name: providerType ...
      responses:
        "200":
          description: "List of DHL Service Point locations."
          schema:
            $ref: "#/definitions/supermodelIoLogisticsPUDOLocations"
```

Listing 6.1: OAS excerpt of the DHL API.

A value can be syntactically valid, but semantically invalid, and vice versa. For example, "dog" is a syntactically valid value for the parameter `addressLocality` in the API of DHL—it conforms to the specification (string value) and is processed by the API without errors—but it is not coherent with the semantics of the parameter. Conversely, "DEU" (ISO 3166-1 alpha-3 code for Germany) is a semantically valid value for the parameter `countryCode`, but it is not syntactically valid, since the API expects two-letter codes, *i.e.*, "DE" for Germany.

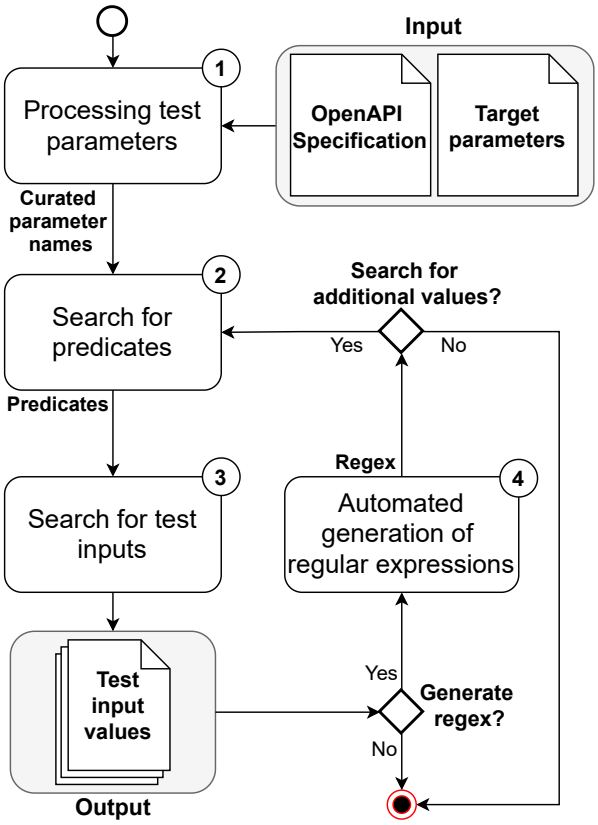


Figure 6.1: Workflow of ARTE.

In what follows, we describe the main steps of ARTE, highlighted in Figure §6.1. A more formal description of our approach using pseudocode is provided in Appendix §A.

6.2.1 Processing Test Parameters

In this first step, the information of the test parameters is collected from the input API specification including their name, description and, if available, extra syntactic constraints like regular expressions and min/max values. As an additional input, the

user must specify the list of parameters for which ARTE should generate meaningful data. This is helpful to exclude trivial parameters like dates, or domain-specific identifiers (*e.g.*, a YouTube video ID), unlikely to be found in general-purpose knowledge bases like DBpedia. For such parameters, a different test data generation strategy may be used (*e.g.*, data dictionaries).

Some popular APIs include parameters with a single character in their names. This is the case, for example, of the Open Movie Database (OMDb) API [4] and the dblp computer science bibliography API [30]. When this happens, it is very common that this single character is the first letter of the full name of the parameter, *e.g.*, 't' for the movie *title* in the OMDb API. Based on this idea, ARTE tries to infer the full parameter name from its description using natural language processing (NLP) techniques [142, 235]. After using part-of-speech tagging [289], stop words removal [295], and lemmatization [134], we are left with only the nouns included in the description. We observed that the full parameter name often matches the shortest noun from the list that begins with the same letter as the one-character original name of the parameter *e.g.*, 'q' for search *query* in the DBLP search API [30] given the description "The query string to search for". Thus, we used this as the default heuristic for selecting the full parameter name from the list of nouns. When multiple candidate words have the same length, ARTE selects the first one in alphabetical order, and if no candidate word is found, the letter is used as keyword. Implementing other heuristics would be straightforward.

6.2.2 Search for Predicates

In this step, a user-defined knowledge database (*e.g.*, DBpedia) is queried to obtain predicates that are representative of the target input parameters. For each target parameter, the search for predicates is performed in two iterative steps. First, a representative keyword of the target parameter is generated from the name and the description of the parameter, by applying the matching rules presented later on in this section. Second, for each candidate keyword, a SPARQL query is constructed as shown in Listing 6.2, where the string `keyword` is replaced by the selected keyword. This query conducts a search for predicates (line 3) that contain the provided keyword (filter by regular expression, line 5, where the flag `'i'` means "case-insensitive") and ordered by length in ascending order (line 7).

After executing the query, if successful, a list of predicates containing the keyword is returned. As an example, the following are a subset of the predicates found in DB-

```
SELECT DISTINCT ?predicate WHERE {

 ?predicate a rdf:Property

 FILTER regex(str(?predicate), 'keyword', 'i')

} ORDER BY strlen(str(?predicate))
```

Listing 6.2: SPARQL query used to search for predicates.

pedia when using the keyword "currency":

- `http://dbpedia.org/property/`**`currency`**
- `http://dbpedia.org/ontology/`**`currency`**
- `http://dbpedia.org/property/`**`currency`**`Iso`
- `http://dbpedia.org/ontology/`**`currency`**`Code`

Predicates are ordered by length in ascending order, with all of them containing the exact keyword. ARTE selects the first n predicates returned by the query (5 in our experiments), computing the support of each of them. Similarly to Mariani et al. [236], we define the *support* of a predicate as the number of unique RDF triples that contain it. The support of a predicate is obtained by running a SPARQL query as the one shown in Listing 6.3 including the `COUNT` set function (see Appendix §A for an example). This query will include filters with the syntactic constraints of the API specification (regex and min/max values), if any, to exclude invalid values. A predicate is accepted only if its support is greater than a user-defined threshold (20 in our evaluation).

To identify relevant keywords, several *matching rules* are applied to the name and description of the parameters. These rules are ordered by priority, meaning that, as soon as a keyword is found with a rule, it is used to search for predicates. If a predicate is accepted (*i.e.*, its support is greater than the configured threshold), the search for predicates for that parameter terminates, and the process starts over again for the next parameter, otherwise ARTE tries to identify new keywords with the remaining rules. In our preliminary experiments, we found that the description of the parameter, and not only its name, often includes key information for the generation of meaningful test inputs. Therefore, the matching rules implemented in ARTE exploit the description of the parameter first (rules 1-3 below) and then its name (rules 4-6).

In what follows, we describe the matching rules currently used in ARTE, ordered from highest to lowest priority:

1. If the name of the parameter appears in its description followed by the words "`code`" or "`id`", both words are used concatenated. For example, if the parameter name is `country` and its description is "A valid country code", this rule will be matched and the keyword `countryCode` will be used for the search for predicates.

2. If a word *K* with the same initial characters as the name of the parameter appears in the description followed by the words "`code`" or "`id`", both the name of the parameter and *K* are concatenated with "`code`" or "`id`" and used for the search of predicates. For example, if the parameter name is `lang` and its description is "A language code", the keywords `langCode` and `languageCode` will be used in the search for predicates.

3. If a noun or an unknown word (*e.g.*, an acronym or a non-English word) is found in the description of the parameter, followed by the words "`code`" or "`id`", both words are concatenated and used for the search of predicates. If multiple matches are found, the keywords are considered in order of appearance. For example, if the parameter name is `origin` and its description is "A valid country code or airport code", the keywords `countryCode` and `airportCode` will be used for the search for predicates.

4. This rule has no condition. Whenever it is reached, the unmodified parameter name is used for the search for predicates, *e.g.*, `streetAddress` (Listing 6.1).

5. If the name of the parameter is in snake case format (*i.e.*, word1_word2) or in kebab case format (*i.e.*, word1-word2), the parameter name is converted to camel case format (*i.e.*, word1Word2) and used in the search for predicates. For example, the operation for matching businesses in the Yelp API [110] contains a parameter called `zip_code`; using this as a keyword (rule 4) returns 0 candidate predicates in DBpedia, whereas using `zipCode` returns a list of 5 predicates.

6. If the parameter name is in snake case, kebab case or camel case format, it is split into multiple words, and each one is used as a keyword to search for predicates. For example, several operations of the Great Circle Mapper API [47] contain a parameter called `icao_iata`, which accepts both IATA and ICAO codes. Using the unmodified parameter name (rule 4) produces 0 results, and so does converting it to camel case format (rule 5). However, when searching for predicates with the keywords `icao` and `iata`, multiple predicates are obtained.

Matching rules 4 to 6 are based on the common naming convention for REST API

parameters, whereas rules 1 to 3 are based on collocations we found during our preliminary work with 10 randomly selected APIs from different domains [29] (not included in our evaluation dataset to avoid overfitting). We remark that the list of matching rules is not exhaustive and new patterns could be readily targeted in the future by adding new rules. If no predicates are found for any of the matching rules, the parameter is ignored and no input values will be generated for it.

6.2.3 Search for Test Inputs

At this stage, the predicates obtained during the previous phase are used to drive the search for meaningful test inputs. When searching test inputs for a specific parameter, it is important to consider it in the context of all the parameters supported by the API operation. For example, a search for `title` may return hundreds or even thousands of title-type inputs belonging to very diverse entities (*e.g.*, movies, people, videogames, books, etc.). However, if we accompany such predicate with another semantically related one such as `director`, the search is significantly narrowed, increasing the chances of generating semantically valid test inputs, *e.g.*, movie titles. For this reason, instead of evaluating the predicates in isolation, our approach starts by looking for entities that contain all the predicates simultaneously and, if not enough inputs are obtained, then the predicates are progressively discarded.

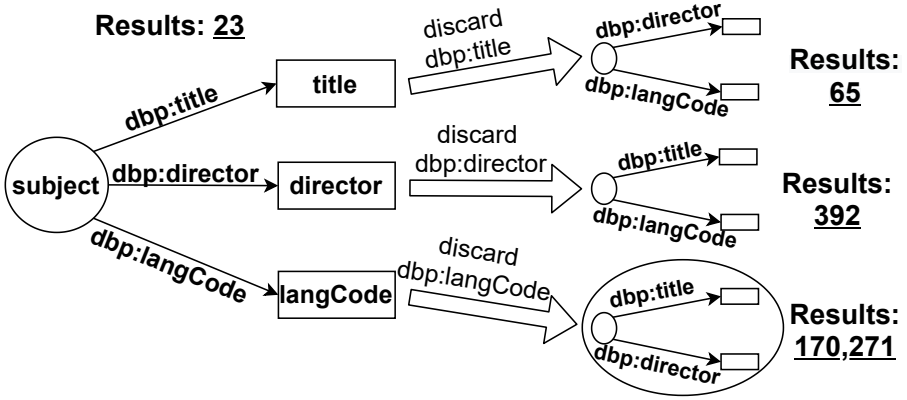


Figure 6.2: Graphical representation of the search for test inputs.

As an example, Listing 6.3 shows the SPARQL query constructed for the search of test inputs (*i.e.*, objects in RDF triples) for the parameters `title`, `director` and `langCode`. This query returned 23 matches in DBpedia. Each match contains three values, one for each parameter (*i.e.*, `title`, `director` and `langCode`). However, some values may be repeated, therefore less than 23 *unique* values could be obtained for each

parameter. A threshold is established to define the minimum number of unique values that should be obtained (100 in our experiments). Suppose that, for a subset of parameters, the threshold is not reached (*e.g.*, there are enough values for `langCode`, but not for `director` and `title`). Then, ARTE would repeat the query depicted in Listing 6.3 considering only the predicates related to the parameters for which the threshold has not been achieved (*i.e.*, `director` and `title`), thus widening the search.

```
SELECT DISTINCT ?title, ?director, ?langCode WHERE {

 ?subject <http://dbpedia.org/property/title> ?title ;
    <http://dbpedia.org/property/director> ?director ;
    <http://dbpedia.org/property/langcode> ?langCode .

}
```

Listing 6.3: SPARQL query for searching test inputs.

Figure §6.2 (left-hand side) depicts an example for the query in Listing 6.3. A search is performed for entities including the predicates `director`, `title`, and `langCode`, obtaining 23 results. Suppose that such search does not return the minimum threshold for any of the three parameters. This being the case, ARTE would execute a number of queries equal to the number of parameters, each containing all predicates but one (right-hand side of Figure §6.2). As illustrated, if `title` or `director` are discarded, 65 and 392 results are returned, respectively, but if `langCode` is discarded, the resulting query would return 170,271 results. Values for the `title` and `director` parameters would be extracted from this last query (since it is the one returning the highest number of results), while values for the `langCode` parameter would be obtained as a result of using its predicate in isolation.

This query-decomposition process continues until all parameters acquire the established threshold of input values, or until all predicates are executed in isolation.

6.2.4 Automated Generation of Regular Expressions

Finally, ARTE can optionally generate regular expressions for each target parameter based on the input values obtained in the previous phases, by integrating the tool RegexGenerator++ [138, 139, 140] (details in Section §6.3). This allows to refine the search for syntactically valid inputs, improving the effectiveness of the approach. Table §6.1 depicts an example of this process for the parameter `market` in the Spotify API [95]. Specifically, the regular expression in Table §6.1 matches strings that are two-characters long, where the second character is not a digit; this pattern conforms to the format of ISO alpha-2 country codes, and allows to filter out values of different length

Table 6.1: Example of automatically generated regular expression.

Valid values	Invalid values	Regex
CN	ITA	
ES	DOM	
IT	BGR	^\w[^\d]$
JP	92	
AD	HUN	

(*e.g.*, "ITA") and numerical values (*e.g.*, "92"). The basic steps of the process are as follows:

1. A set of input values is generated as described in the previous sections, and classified as syntactically valid or invalid (first and second columns in Table §6.1). Inputs can be classified as valid or invalid either manually or, as in our work, automatically based on the API responses (see Section §6.3 for details). The number of valid and invalid values must reach a minimum threshold defined by the user (5 in our experiments).

2. Based on the generated valid and invalid inputs, a regular expression is generated (column "Regex" in Table §6.1). This regular expression should match and not match as many valid and invalid inputs as possible, respectively.

3. If the percentage of valid values matching the regular expression (*i.e.*, recall) exceeds a minimum threshold (90% in our evaluation), the generation of the expression is considered successful. In that case, the list of input values generated until that moment is filtered such that only those values matching the regular expression are kept. Invalid values matching the regular expression (*i.e.*, false positives) may still be helpful to identify potential bugs. For instance, the regular expression generated for the parameter `countryCode` in the DHL API matched both "`UK`" and "`FR`", however, the former was rejected by the API (*i.e.*, classified as invalid). This may reveal unintended behavior, *e.g.*, codes that should be supported by the API but are not.

If a regular expression is successfully generated, ARTE extracts more input values by using the predicates that have not been leveraged yet (obtained as described in Section §6.2.2) adding the generated regular expression as a filter clause in the SPARQL query. These inputs will match the regular expression and therefore they should be more likely to be syntactically valid. On the contrary, if a regular expression cannot be

inferred, the process restarts in step 1, *i.e.*, ARTE will try to generate a regular expression with more input values classified as valid or invalid.

The process ends once the maximum number of consecutive attempts to generate regular expressions is reached (2 in our evaluation), or after ARTE has extracted input values from a maximum number of extra predicates leveraging the generated regular expressions (set to 3 in our experiments).

6.3 TOOLING

We implemented ARTE in Java, leveraging existing libraries for specific tasks, namely: (1) Jena [56], for the creation of SPARQL queries; (2) Stanford CoreNLP [21], for NLP related tasks; and (3) RegexGenerator++ [138, 139, 140], for the generation of regular expressions. RegexGenerator++ uses search-based techniques for automatically generating context-aware regular expressions based on strings tagged as *valid* or *invalid* (*i.e.*, *matching* or *not matching* the regular expression, respectively) within a corpus, namely, a text that provides some context. We slightly modified RegexGenerator++ to make it not context-aware, by adding the anchors '^' and '$' at the start and end of the generated regular expressions, respectively. This modification significantly improves the performance of ARTE for the generation of regular expressions.

Table 6.2: ARTE configuration parameters and default values.

#	Parameter	Value
1	Number of predicates selected when searching for predicates	5
2	Min predicate support	20
3	Min number of unique parameter values	100
4	Min recall for accepting a regex	90%
5	Min number of valid and invalid values for generating a regex	5
6	Max number of extra predicates to leverage using regex	3
7	Max number of consecutive attempts to generate a regex	2

Table §6.2 summarizes the configuration parameters of ARTE, described in the previous section, and the default values used in our evaluation. These values were selected based on our preliminary work with the tool. They provide a balance between performance and effectiveness. Hence, for example, setting low values for parameters 1-3 would result in faster execution, but lower probabilities of finding good values. Analogously, setting low values for parameters 4 and 5 would make it easier for ARTE

to generate a regular expression, with the risk of it not being sufficiently accurate. On the contrary, setting a high value for parameters 6 and 7 would result in a slower execution with the risk of obtaining overfitted regular expressions.

We integrated ARTE into RESTest [243], a state-of-the-art black-box test case generation framework for RESTful APIs. More precisely, ARTE automatically generates data dictionaries (*i.e.*, sets of valid and invalid values) for the selected API parameters, releasing testers from that burden. This makes ARTE easily applicable to any of the test case generation algorithms implemented in RESTest.

For the generation of regular expressions, input values must be labeled as valid or invalid. By integrating ARTE into RESTest, we automate this step as follows:

- An input parameter value is classified as *valid* if it was present in an API call for which a successful response (2XX status code) was obtained.
- An input parameter value is classified as *invalid* if it was present in an API call for which a "client error" response (4XX status code) was obtained and either: (1) it was the only parameter used in the API call; or (2) it was accompanied by other parameters whose values were already classified as valid.
- Input values not meeting any of the previous constraints are labeled as *unclassified* and ignored when inferring regular expressions.

RESTest generates and executes a fixed number of test cases by iterations. After each iteration, ARTE automatically classifies the generated values as described above and tries to infer a regular expression (Section §6.2.4).

6.4 EVALUATION

We aim to answer the following research questions:

- **RQ_1:** *How effective is ARTE in generating realistic test inputs for REST APIs?* We aim to measure the effectiveness of ARTE in generating syntactically and semantically valid test inputs for real-world REST APIs.
- **RQ_2:** *What is the impact of ARTE on the automated generation of test cases for REST APIs?* The final goal of our approach is to improve current state-of-the-art methods for test case generation of REST APIs. Therefore, we wish to compare the

effectiveness of existing methods and ARTE in terms of valid API requests generated and test coverage achieved.

- **RQ$_3$:** *Does ARTE improve the failure-detection capability of existing test case generation techniques?* We aim to investigate whether input values generated by ARTE reveal more failures than those generated by related techniques.

In the next sections, we explain the two experiments performed for answering the research questions. In both cases, we used the default configuration of ARTE, and we relied on DBpedia as the selected knowledge base, more specifically, the 2016-10 core dataset.[1]

The experiments were performed in a desktop machine equipped with Intel i7-6700 CPU@3.40GHz, 16GB RAM, and 125GB SSD running Windows 10 Pro 64 bit and Java 8.

6.4.1 Baselines

We compared ARTE against four baselines, three (pseudo)-random test data generation strategies (experiment 2) and SAIGEN [299] (experiments 1 and 2), described below.

Random test data generation techniques

We implemented three related, but different random test data generation approaches recently used in the context of automated test case generation for REST APIs. All the generators were implemented using RESTest. Next, we present them, from the most naive to the most sophisticated one.

Fuzzing This technique aims at finding implementation bugs (especially security-related) by using random, malformed or unexpected input data [189]. In our experiments, we use the *fuzzing* test case generator implemented in RESTest [243], which generates random values including out-of-range numerical values, long strings with unusual characters, empty strings, and null values.

Data dictionaries This approach proposes to use a small set of predefined values for each parameter type. Specifically, we used the small data dictionaries proposed by Atl-

[1] http://downloads.dbpedia.org/2016-10/

idakis et al. [131], namely: “sampleString” and “” (empty string) for string parameters, “0” and “1” for integers, and “1.23” for doubles.

Data generators This approach uses specific test data generators for each parameter type [212, 243]. We used the default test data generators integrated into RESTest, which also leverage the regular expressions and min/max constraints included in the API specification, if any. Basically, we generated random English words for string parameters, numbers between 1 and 100 for integers, and floating numbers between 1 and 100 for doubles.

SAIGEN

To the best of our knowledge, our work is the first to leverage the Web of Data for driving test data generation for REST APIs. However, semantic information retrieval techniques have already been successfully applied in the context of GUI testing of desktop, web, and mobile applications [236, 299]. In this chapter, we compare ARTE against the most recent of these contributions, SAIGEN (Semantic Aware Input GENerator) [299], a related tool for the automated generation of realistic test inputs for mobile apps. Evaluation results on 12 mobile applications showed that SAIGEN was able to find inputs for 50% of the GUI labels on average. Out of these, 94% of inputs were semantically valid.

Both approaches, ARTE and SAIGEN, exploit knowledge bases for the generation of realistic test inputs, but with significant differences. First, SAIGEN searches for test inputs based on the GUI labels and potential synonyms, whereas ARTE exploits the API specification, applying NLP techniques to the names and descriptions of the parameters. This means that ARTE can leverage further information than SAIGEN, but in practice this also imposes new challenges since, as explained in Section §6.2, parameters can use different naming conventions (*e.g.*, parameters with a single letter) and very heterogeneous descriptions in natural language. Second, SAIGEN searches for predicates by adding the selected keyword to the namespace prefix of the knowledge base, whereas ARTE constructs specific SPARQL queries. In practice, this means that searches in SAIGEN are restricted to predicates containing exactly the specified keyword (*e.g.*, `currency`), whereas ARTE widens the search space by looking for predicates containing the keyword (*e.g.*, `currencyIso`). Lastly, ARTE integrates a novel step for the refinement of the test inputs extracted with automatically generated regular expressions (Section §6.2.4).

Since SAIGEN targets GUI elements only, in our experiments we ran the tool using the name of the API parameters as if they were GUI labels. We used the default settings of SAIGEN, configuring it to work with the selected version of the knowledge base.

6.4.2 Experiment 1: Generation of Realistic Test Inputs

In this experiment, we aim to answer $\mathbf{RQ_1}$ by evaluating the effectiveness of ARTE in generating realistic test inputs for real-world REST APIs. In what follows, we describe the setup and the results of the experiment.

Experimental setup

For this experiment, we resorted to RapidAPI [81], a popular online repository containing more than 40k REST APIs, classified in 46 different categories. Specifically, we created a dataset of 40 APIs as follows. We selected the first 10 APIs of each category, *i.e.*, $46 \times 10 = 460$ APIs. APIs and categories were sorted in the same order as they were displayed on the platform at the time of performing the search on May, 2021. Then, we selected the first 40 APIs of the list, filtering out those meeting any of the following exclusion criteria: (1) APIs containing exclusively domain-specific parameters (*e.g.*, database IDs) or trivial parameters (such as `limit`, `offset` or enumerated values), for which random generation would suffice, (2) APIs for which there was no validation for any of the parameters (*i.e.*, any value would be considered valid), (3) APIs containing only confidential parameters (*e.g.*, username, password, email, or credit card number), (4) APIs not returning any response, (5) APIs without parameters, (6) paid APIs (so-called *premium* in RapidAPI), and (7) APIs whose documentation was not written in English. The resulting dataset contains 40 APIs from 11 different categories. Table §6.3 shows, for each API, its name, category, number of operations, and number of parameters used in our evaluation. Parameters used in more than one operation within the same API are considered only once, *e.g.*, `iso_a2` in the Referential API [114]. In total, the dataset includes 173 different parameters from 122 API operations. These parameters span multiple concepts such as website URLs, city names, ingredient names, and currency codes, among many others.

RapidAPI does not provide a publicly available OAS specification for the APIs in the repository. To address this issue, we generated the OAS specification of each API using the web scraping libraries Selenium [91] and BeautifulSoup [15] on the web user interface of RapidAPI, which displays, among others, the name and type of each input parameter for every API operation.

To further evaluate our approach, we used a second dataset of 8 industrial popular RESTful APIs from different domains, depicted in Table §6.4. The dataset includes 38 parameters from 18 API operations. The OAS specification of Spotify [95] was downloaded from the APIs.guru repository [13]. The specifications of Yelp Fusion [110] and RESTCountries [85] were manually created from the documentation available on the official website. The OAS specification of the remaining APIs were downloaded from their official websites. The specification of the Amadeus Hotel API [11] was the only one including regular expressions for some of its parameters.

We compared ARTE against SAIGEN, since it is the only baseline specifically tailored for the automated generation of realistic test inputs. Additionally, we measured the performance of ARTE before applying the refinement with automatically generated regular expressions, denoted as ARTE NR (ARTE No Regex).

ARTE and SAIGEN may generate hundreds or even thousands of values for each parameter, making it infeasible to manually test all of them. To evaluate whether syntactically and semantically valid values were generated, we tested the APIs with 10 randomly selected values per parameter, manually checking whether at least one of them was valid. A similar approach was followed by the authors of SAIGEN [299]. We refer the reader to Section §6.2 for the definition of syntactically and semantically valid test inputs.

Experimental results

The results of ARTE and SAIGEN in the RapidAPI dataset are shown in the last nine columns of Table §6.3. On average, ARTE found syntactically valid inputs for 78% of the parameters (135 out of 173), whereas SAIGEN generated syntactically valid inputs for 42.8% of them (74 out of 173). Analogously, ARTE generated semantically valid inputs for 64.2% of the parameters (111 out of 173), while SAIGEN generated semantically valid inputs for 31.8% (55 out of 173). Regarding realistic values (both syntactically and semantically valid), ARTE generated realistic values for 63% of the parameters (109 out of 173), whereas SAIGEN only generated realistic values for 30.6% of them (53 out of 173). ARTE generated realistic inputs for 100% of the parameters in 9 out of 40 APIs (4 with SAIGEN), and 50% or more in 23 of them (13 with SAIGEN). It is noteworthy that the generation of regular expressions allowed to increase the percentage of realistic test inputs in 11 out of the 25 APIs for which ARTE did not initially obtain 100% of syntactically valid parameters. Among others, ARTE automatically generated regular expressions for parameters such as `season` (API-FOOTBALL and API-BASKETBALL APIs), `currency` (Skyscanner and Alpha Vantage APIs) and `state`

(RedLine and Realty Mole APIs).

Table 6.3: Per API breakdown of input values generation for the RapidAPI dataset. O = Operations, P = Parameters.

API	Category	O	P	Syntactically valid (%)			Semantically valid (%)			Syntactically and semantically valid (%)		
				SAIGEN	ARTE NR	ARTE	SAIGEN	ARTE NR	ARTE	SAIGEN	ARTE NR	ARTE
AeroDataBox	Transportation	6	8	37.5	62.5	75	37.5	62.5	75	37.5	62.5	75
Airport info	Transportation	1	2	50	100	100	50	100	100	50	100	100
AirportIX	Transportation	6	6	33.3	16.7	50	16.7	0	33.3	16.7	0	33.3
Alpha Vantage	Finance	3	5	0	0	40	0	0	40	0	0	40
API-BASKETBALL	Sports	5	6	83.3	83.3	100	33.3	16.7	33.3	33.3	16.7	33.3
API-FOOTBALL	Sports	5	4	50	75	100	50	50	75	50	50	75
Astronomy	Science	1	2	100	100	100	100	100	100	100	100	100
Aviation Reference Data	Transportation	5	6	66.7	66.7	83.3	66.7	66.7	83.3	66.7	66.7	83.3
CarbonFootprint	Science	1	3	33.3	33.3	33.3	33.3	33.3	33.3	33.3	33.3	33.3
Countries Cities	Location	4	4	75	100	100	75	100	100	75	100	100
Currency Converter	Finance	2	4	50	75	75	0	50	50	0	50	50
Domainr	Business	2	3	66.7	66.7	66.7	0	33.3	33.3	0	33.3	33.3
Face Detection	V. Recognition	1	1	0	0	0	0	0	0	0	0	0
Fixer Currency	Finance	3	4	0	100	100	0	100	100	0	100	100
Football Prediction	Sports	1	1	0	0	0	0	0	0	0	0	0
GeoDB Cities	Data	4	8	0	62.5	75	0	62.5	75	0	62.5	75
Google Maps Geocoding	Location	2	3	66.7	66.7	66.7	33.3	33.3	33.3	33.3	33.3	33.3
Great Circle Mapper	Travel	3	3	0	33.3	33.3	0	33.3	33.3	0	33.3	33.3
Hotels	Travel	3	3	66.7	100	100	0	33.3	33.3	0	33.3	33.3
Hotels Com Provider	Travel	1	2	100	100	100	100	100	100	100	100	100
Movie Database (Imdb alt.)	Entertainment	2	2	50	100	100	0	50	50	0	50	50
NAVITIME Route (car)	Transportation	2	3	0	0	0	0	0	0	0	0	0
NAVITIME Route (totalnavi)	Transportation	1	2	0	0	0	0	0	0	0	0	0
Periodic Table of Elements	Science	1	1	0	0	0	0	0	0	0	0	0
Priceline com Provider	Travel	2	4	50	100	100	50	100	100	50	100	100
Realty in US	Business	4	8	62.5	87.5	100	50	75	87.5	50	75	87.5
Realty Mole Property	Business	3	7	57.1	71.4	85.7	85.7	85.7	100	57.1	71.4	85.7
Recipe - Food - Nutrition	Food	8	7	14.3	71.4	71.4	14.3	85.7	85.7	14.3	71.4	71.4
RedLine Zipcode	Location	6	8	25	75	87.5	25	75	87.5	25	75	87.5
Referential	Data	8	18	27.8	72.2	72.2	27.8	44.4	44.4	27.8	44.4	44.4
Rent Estimate	Data	1	3	100	100	100	100	100	100	100	100	100
Restb.ai Watermark Detection	V. Recognition	1	1	0	100	100	0	100	100	0	100	100
Skyscanner Flight Search	Transportation	8	5	40	40	80	0	0	40	0	0	40
Spott	Location	1	4	100	100	100	50	50	50	50	50	50
Subtitles for YouTube	Data	1	1	100	100	100	100	100	100	100	100	100
TrailAPI	Travel	2	5	40	100	100	40	80	80	40	80	80
Travel Advisor	Transportation	3	8	50	100	100	25	87.5	87.5	25	87.5	87.5
UPHERE.SPACE	Science	2	2	50	50	50	0	0	0	0	0	0
US Restaurant Menus	Food	6	5	80	100	100	60	80	80	60	80	80
Yahoo Finance	Finance	1	1	0	0	0	0	0	0	0	0	0
Total		**122**	**173**	**42.8**	**69.9**	**78**	**31.8**	**56.1**	**64.2**	**30.6**	**54.9**	**63**

Table §6.4 shows the results for the set of industrial APIs, where the results of ARTE are even better than those obtained with the RapidAPI dataset. As illustrated, ARTE generated syntactically and semantically valid inputs for 73.7% of the parameters (28 out of 38), in contrast with SAIGEN, which generated realistic inputs for only 36.8% of the parameters (14 out of 38). Furthermore, ARTE generated realistic inputs for 100% of the target parameters in 3 out of 8 APIs (DHL, OMDb and RESTcountries), and over 50% in 6 of them, whereas SAIGEN did not manage to generate realistic inputs for all the parameters in any of the APIs, generating 50% or more in 3 of them (DHL,

RESTCountries and Yelp). The automated generation of regular expressions yielded an improvement for the parameter `currency` of the RESTCountries API. Additionally, it improved the automated generation of valid test cases in the APIs of RESTCountries, Spotify and DHL, as explained in Section §6.4.3.

Table 6.4: Per API breakdown of input values generation for the industrial APIs. O = Operations, P = Parameters.

API	O	P	Syntactically valid (%)			Semantically valid (%)			Syntactically and semantically valid (%)		
			SAIGEN	ARTE NR	ARTE	SAIGEN	ARTE NR	ARTE	SAIGEN	ARTE NR	ARTE
Amadeus Hotel	2	7	42.9	85.7	85.7	42.9	85.7	85.7	42.9	85.7	85.7
Deutschebahn StaDa	1	4	0	25	25	0	25	25	0	25	25
DHL Location Finder	2	6	50	100	100	50	100	100	50	100	100
Marvel	1	5	60	100	100	40	40	40	40	40	40
OMDb	1	3	33.3	100	100	0	100	100	0	100	100
RESTCountries	4	4	75	75	100	100	100	100	75	75	100
Spotify	5	4	25	75	75	25	75	75	0	50	50
Yelp Fusion	2	5	60	80	80	80	80	80	60	80	80
Total	**18**	**38**	**44.7**	**81.6**	**84.2**	**44.7**	**76.3**	**76.3**	**36.8**	**71.1**	**73.7**

In total, considering both the RapidAPI dataset and the industrial APIs, ARTE and SAIGEN generated realistic inputs for 64.9% (137 out of 211) and 31.8% (67 out of 211) of the parameters, respectively. It is worth recalling that SAIGEN was specifically designed for mobile apps, and therefore its poor performance on REST APIs was expected. This supports the ability of ARTE to address the specific characteristics of REST APIs.

Regarding performance, ARTE took on average 21.3 seconds (standard deviation σ = 30.6 seconds) to generate the set of input values for all the parameters of each API, whereas SAIGEN required 11.8 seconds on average per API (σ = 11.2 seconds).

In view of these results, we can answer $\mathbf{RQ_1}$ as follows:

Answer to $\mathbf{RQ_1}$: How effective is ARTE in generating realistic test inputs for REST APIs?

ARTE is effective in generating realistic test inputs for real-world REST APIs. With a sample of 10 values per parameter, ARTE generated syntactically and semantically valid inputs for 64.9% of the parameters (137 out of 211), approximately twice as many as the baseline, SAIGEN, 31.8% (67 out of 211).

6.4.3 Experiment 2: Automated Testing

In this experiment, we aim to answer **RQ$_2$** and **RQ$_3$** by evaluating how ARTE can contribute to the automated generation of valid API calls, API coverage, and detection of failures. Next, we describe the setup and the results of the experiment.

Experimental setup

For this experiment, we used 14 operations from 6 industrial APIs, depicted in the first column of Table §6.5.[2] For each operation, we generated and executed 1k API calls (20 iterations of 50 test cases) with each data generation strategy, leading to 14 (operations) $\times$ 6 (strategies) $\times$ 1k = 84k calls in total. Then, we computed the percentage of valid API calls generated, the API coverage achieved and the number of failures detected by each approach. For the computation of valid API calls generated, we resorted to the REST best practices [270], which dictate that valid API calls should obtain 2XX HTTP status codes, whereas invalid calls should obtain 4XX status codes. For computing the API coverage, we considered the test coverage criteria for REST APIs defined in [239]. These criteria are classified into input criteria—elements covered by the API requests (*e.g.*, operations and parameter values)—and output criteria—elements covered by the API responses (*e.g.*, status codes and response body properties). Input elements (*e.g.*, a parameter) are considered covered if they are included in at least one API request obtaining a successful response (*i.e.*, 2XX status codes).

We used RESTest [243] for the generation and execution of test cases. In this experiment, each test case comprises a single API call. REST APIs often impose inter-parameter dependencies that restrict the way in which parameters can be combined to form valid API requests. For instance, the use of a parameter may require the inclusion of some other parameter. To handle these dependencies, we used the constraint-based test case generator integrated into RESTest [240], which supports the generation of API calls satisfying all the inter-parameter dependencies of the API operation under test [238, 242]. Therefore, when obtaining an error response, we are confident that it is due to individual input values, and not due to violation of the dependencies. The only exception is fuzzing, where dependencies are intentionally ignored to test the API with any potential input combination. Failures were automatically detected using the built-in test oracles in RESTest, mostly based on the detection of server errors (5XX status codes) and the identification of inconsistencies between the API specification and

[2] We excluded the Marvel API and the OMDb API from this experiment because the Marvel API always returns successful responses and the OMDb API does not use 4XX status codes for error responses.

the API responses.

Test data was generated in six fashions: using the four baselines (Section §6.4.1), ARTE without automated generation of regular expressions (again denoted as ARTE NR), and ARTE. Exceptionally, we used small data dictionaries (15-20 values) for domain-dependent parameters (*e.g.*, album identifier in Spotify) in all the test data generation strategies excluding fuzzing.

Whenever ARTE or SAIGEN could not generate values for a parameter, it was assigned a random value using the data generators integrated into RESTest for a fair comparison among all the techniques under study. In practice, however, it may be sensible omitting the parameter from the API call, assuming it is optional, to increase the chances of generating a valid API request. Overall, ARTE could not generate values for 3 out of 37 parameters, whereas SAIGEN could not generate values for 9 of them.

Experimental results

We next describe the experimental results related to the automated test case generation ($\mathbf{RQ_2}$) and the detection of failures ($\mathbf{RQ_3}$).

Automated test case generation Table §6.5 shows the percentage of valid calls generated and the coverage achieved by each test data generation strategy (columns "VC (%)" and "C (%)", respectively). The highest values of each row are highlighted in boldface. On average, fuzzing achieved 1.2% of valid API calls, data dictionaries achieved 12%, data generators achieved 20%, SAIGEN achieved 26%, ARTE NR achieved 39.9%, and ARTE achieved 57.3%. ARTE obtained better results in 13 out of 14 API operations. The results obtained for the DHL API are especially significant: ARTE obtained between 59.8% and 98.9% of valid API calls, whereas the random approaches got 0%. This is because its `latitude` and `longitude` parameters are defined as `string` instead of `float`, making random generation useless. SAIGEN was able to generate valid values for `latitude` and `longitude`, but it did not generate valid values for country codes, which explains the 0% of valid calls generated for the operation "Find by address". The automated generation of regular expressions played a key role in the generation of valid API calls in 6 out of 14 API operations under test, with a difference of up to 64.3% in the RESTCountries API. Regular expressions showed to be effective in generating test inputs for parameters such as country codes, markets, currency codes and language names.

In terms of API coverage, ARTE achieved an average coverage of 86.5% and 87.9%

Table 6.5: Per API breakdown of valid calls, coverage, and failures detected. P = "Parameters", VC = "Valid calls", C = "Coverage", F = "Failures".

		Fuzzing			Data dictionaries			Data generators			SAIGEN			ARTE NR			ARTE		
API - Operation	**P**	**VC (%)**	**C (%)**	**F**	**VC (%)**	**C (%)**	**F**	**VC (%)**	**C (%)**	**F**	**VC (%)**	**C (%)**	**F**	**VC (%)**	**C (%)**	**F**	**VC (%)**	**C (%)**	**F**
Amadeus Hotel - Find hotels	7	0	3.3	0	1.6	68.4	0	8.1	72.2	0	3.7	73.6	0	8.6	75	7	**9.6**	**75.5**	9
Amadeus Hotel - View hotel rooms	2	0	3.6	0	17.5	65.1	0	43.4	64.1	0	30.8	**66.1**	0	51.5	65.6	0	**62.2**	65.1	0
Deutschebahn StaDa - Get stations	4	0.3	96.3	3	9.4	**97**	0	15.5	**97**	0	10.9	**97**	0	17.2	**97**	0	**19.2**	**97**	0
DHL - Find by address	4	0	3.8	0	0	3.8	0	0.1	24.1	0	0	3.8	0	13.7	**81**	138	**59.8**	**81**	140
DHL - Find by geo	2	0	3.9	0	0	3.9	0	0	3.9	0	94.1	**80.5**	0	**98.9**	**80.5**	0	97.7	**80.5**	0
RESTCountries - Capital	1	0	25	0	0	25	0	0.5	**100**	0	**19.9**	**100**	0	4.5	**100**	0	4.6	**100**	0
RESTCountries - Code	1	11.4	**100**	0	0	22.2	0	1.8	88.9	1	41.4	**100**	0	91.1	**100**	0	**99.2**	**100**	0
RESTCountries - Currency	1	0	22.2	0	0	22.2	0	0.7	88.9	0	0	22.2	0	3	**100**	1	**67.3**	88.9	0
RESTCountries - Language	1	0	22.2	0	0	22.2	0	0.2	88.9	0	16.5	88.9	0	18.3	**100**	2	**48.1**	88.9	0
Spotify - Get album	1	0	3.7	0	49.4	76.8	0	50.4	76.8	0	50.6	81.7	0	56.5	**89**	0	**92**	**89**	0
Spotify - Get category	2	0	20	0	47.5	80	0	49.3	85	0	53.4	80	0	53.6	85	0	**86.9**	**90**	0
Spotify - Get featured playlists	3	5.4	86.3	0	24.4	86.3	0	26.6	86.3	0	26.3	86.3	0	28	**88.2**	0	**42.6**	**88.2**	0
Yelp Fusion - Search business	5	0	8.8	0	18.3	94.1	0	37.9	94.1	0	16.9	94.1	0	**48.9**	**97.1**	0	44.8	94.1	0
Yelp Fusion - Search transactions	3	0	16.7	0	0	16.7	0	45.8	**72.2**	0	0	16.7	0	65.1	**72.2**	0	**67.7**	**72.2**	0
TOTAL	**37**	**1.2**	**29.7**	**3**	**12**	**48.8**	**0**	**20**	**74.5**	**1**	**26**	**70.8**	**0**	**39.9**	**87.9**	**148**	**57.3**	**86.5**	**149**

with and without the use of regular expressions, respectively, outperforming all other approaches by a margin of up to 58.2% (compared to fuzzing), 32% on average. This difference is mainly due to two reasons: (1) the poor performance of the related techniques in generating valid requests (*e.g.*, fuzzing), a prerequisite to cover most API elements; and (2) the less varied data used in API requests, which results insufficient to cover some output elements. This was the case, for example, of the Spotify API, where ARTE covered between 8% and 12% more response body properties than data dictionaries, data generators, and SAIGEN.

Test case generation and execution using data dictionaries and fuzzing took, on average, about 11 minutes per API operation, data generators 19 minutes, SAIGEN 14 minutes, ARTE NR 18 minutes, and ARTE 17 minutes. These times are mainly influenced by the response time of the APIs under test, usually longer in valid calls. Thus, approaches generating more valid calls typically took longer to execute.

In view of these results, $\mathbf{RQ_2}$ can be answered as follows:

Answer to $\mathbf{RQ_2}$: What is the impact of ARTE on the automated generation of test cases for REST APIs?

ARTE improved the automated generation of test cases for 13 out of the 14 REST API operations under test. Specifically, ARTE generated over twice as many valid requests (57.3%) as SAIGEN (26%) and about three times as many as the best of the random approaches (20%). The superiority of ARTE was also reflected in the API coverage (86.5%).

Failure detection Table §6.5 shows the number of failures detected by each approach. Data dictionaries and SAIGEN uncovered no failures, data generators uncovered 1 failure, fuzzing uncovered 3 failures, and ARTE uncovered a total of 149 failures (one less without using regular expressions). Failures can occur due to multiple reasons, for example, 5XX status codes (server errors), inconsistencies between the API responses and the API specification, or client error responses (*i.e.*, 4XX status codes) obtained after valid API calls, or vice versa, when an invalid input value results in a 2XX response code. Data generators and fuzzing only uncovered failures in the form of 5XX status codes, whereas ARTE also uncovered failures in the form of inconsistencies in the API and unexpected API responses.

ARTE uncovered two issues not detected by any of the other approaches. In the "Find hotel rooms" operation of the Amadeus API, the documentation states that the `hotelName` parameter should contain 4 keywords maximum. However, there were 16 API calls violating this condition which obtained successful responses, revealing a fault. This bug was reported and confirmed by the API providers. Also, in the "Find by address" operation of the DHL API, we found that the documentation was not exhaustive, since the API accepted 27 country codes not listed in the API documentation. Additionally, the API returns a 400 code when using the country code UK, despite this being a valid ISO 3166-1 alpha-2 code, as indicated in the API documentation. After reporting these issues to DHL, they confirmed that it was the intended behavior and updated the documentation to reflect it. This shows the potential of ARTE to reveal disconformities between the API specification (or documentation) and its behavior.

In view of these results, we can answer $\mathbf{RQ_3}$ as follows:

Answer to $\mathbf{RQ_3}$: Does ARTE improve the failure-detection capability of existing test case generation techniques?

ARTE revealed more failures in more APIs than related approaches. In particular, it uncovered 149 failures in 2 API operations, unveiling issues not detected by SAIGEN and random approaches.

6.4.4 Discussion

In what follows, we further explore the results and what they tell us about the research questions.

RQ_1: Generation of realistic test inputs

The results of the first experiment show that, with just a sample of 10 values per parameter, ARTE managed to generate syntactically and semantically valid values for 64.9% of them, outperforming SAIGEN (33.1%). We may remark that the results obtained in this experiment are a pessimistic approximation, since we are only considering 10 input values per parameter, out of the hundreds or thousands of values generated by ARTE. Had we considered a larger sample (*e.g.*, 100 values), we would have probably obtained better results. It is noteworthy that in all APIs, except for those of Recipe - Food - Nutrition and Realty Mole Property, the number of syntactically valid values is greater than or equal to the number of semantically valid values. This confirms that the main difficulty in test data generation lies in the generation of semantically valid inputs.

One of the distinctive features of ARTE compared to previous approaches is the automated generation of regular expressions (Section §6.2.4). Regular expressions did have a positive impact in the generation of test inputs for 11 out of 25 APIs from the RapidAPI dataset in which ARTE did not achieve 100% of syntactically valid values. In fact, automatically generated regular expressions were key to equal or outperform the results of SAIGEN in the APIs of AirportIX and API-BASKETBALL. Regular expressions worked well for parameters with unambiguous names, whose values follow a specific format (*e.g.*, `countryCode`). However, they were not so effective for parameters with a more generic name (*e.g.*, `code`) or not following any particular format (*e.g.*, `hotelName` in the Amadeus API), therefore more research will be required for improving the generation of realistic inputs in those cases.

There are two main reasons why ARTE failed to generate valid inputs for some parameters: (1) the name and the description of the parameter are not descriptive enough; and (2) the parameter is too specific. The first reason mostly applied to the APIs from the RapidAPI dataset. We noticed that, in many cases, the parameters expecting a code do not explicitly state it (*e.g.*, textually in the description or by using a name such as `countryCode`). In this scenario, ARTE may generate both names and codes, but these may not follow the correct format or simply be invalid. Industrial APIs, on the other hand, generally provide a more exhaustive documentation, but they tend to have very specific parameters such as `Ril-100` (identifier of German train stations in the Deutschebahn StaDa API) or `ean` (European Article Number in the Marvel API), hard to find in general-purpose knowledge bases like DBpedia. Despite this limitation, it is worth highlighting that ARTE successfully generated realistic inputs for German

federal state names, ingredients names, website URLs, addresses, postal codes, country codes, currency codes, and language codes, among others.

Domain-specific parameters (*e.g.*, database identifiers) are unlikely to be found in general-purpose knowledge bases. For such parameters, we resorted to manually-created data dictionaries, and therefore we could not achieve full automation. This limitation is shared by related techniques [135, 212, 240]. We see potential in combining ARTE with learning input values from previous responses [190, 294].

Regarding performance, ARTE took about 10 seconds more than SAIGEN (21.3 vs. 11.9) to generate test inputs for all parameters of each API, on average. This is because ARTE applies NLP techniques not only to the name of the parameter but also to its description. In addition, ARTE searches for predicates in DBpedia once for each matching rule (Section §6.2.2). However, we deem this as a negligible toll considering the gain in effectiveness, *i.e.*, ARTE generated input values for 205 out of 211 parameters, as opposed to SAIGEN, which generated values for 144 of them.

RQ_2: Automated generation of test cases

ARTE generated between 2 and 48 times more valid calls than the baselines. However, the improvement in the Amadeus Hotel API and the Deutschebahn StaDa API was not as significant. There were several reasons behind these results worth mentioning, since they could be extrapolated to other APIs, namely:

1. *Regular expressions in the specification*. The OAS specification of the Amadeus API contains regular expressions describing the format of some parameters (*e.g.*, an ISO code for parameter `lang`). Therefore, the approach using data generators can also leverage these regular expressions, thus obtaining similar results.

2. *REST bad practices*. When an API call uses invalid input data, it should obtain a "client error" response. Conversely, when it uses valid data, it should obtain a successful response. This is not the case in the Amadeus API. Some parameters require inputs such as language codes, but the API returns successful responses as long as the value used conforms to the expected format (*e.g.*, two capital letters).

3. *Semantic inter-parameter dependencies*. These are dependencies that constrain the values that different parameters can take based on their meaning. For instance, even if ARTE generates valid values for the parameters `cityCode` and `hotelName`, the API will return an error if there is no hotel with the specified name in the

provided city and vice versa. These errors may also arise for trivial parameters, not targeted by ARTE. For example, if the number of rooms provided as input (an integer parameter) is greater than those available at the specified hotel.

ARTE outperformed all the baseline approaches in 13 out of 14 API operations under test. SAIGEN obtained better results in the operation "Capital" of the REST-Countries API because it leveraged a predicate that yielded mainly country capitals (those accepted by the API), whereas ARTE leveraged one that yielded capitals of both countries and regions. ARTE also outperformed the related approaches in terms of coverage, by a margin that ranged between 13.4% and 58.2%.

RQ_3: Failure detection

The increase in the number of valid API calls generated by ARTE translates into more diverse tests that exercise different parts of the API under test and, consequently, uncover more failures. However, different test data generation strategies may uncover different types of failures, and therefore they are complementary, rather than exclusive. As an example, fuzzing uncovered a 500 status code in the Deutschebahn StaDa API, caused when using parameter values containing unexpected characters such as '%'. This bug could not have been uncovered by techniques leveraging realistic input data exclusively.

The integration of ARTE into RESTest enables leveraging not only valid test inputs but also invalid ones, since RESTest can automatically classify the values generated by ARTE into valid or invalid, according to the API responses. For instance, in the Spotify API, ARTE detected that the API returned a successful response for 6 markets that were not present in the documentation. Conversely, the value "`LY`" (Libya) for parameter `country` was classified as invalid, since it was rejected by the API. The same happened with 24 language names in the RESTCountries API. Are these values really not supported by the API, or may these responses be caused by a bug in their implementation? ARTE can reveal unexpected behaviors like these.

6.5 Threats to Validity

In this section, we discuss the possible internal and external validity threats that may have influenced our work, and how these were mitigated.

Internal validity. *Are there factors that might affect the results of our evaluation?* For the

experiments, we used the OAS specifications of the APIs under test. When possible, we resorted to the API specifications publicly available. However, for the RapidAPI dataset and the Yelp and RESTCountries APIs such specifications were missing. We generated them manually (Yelp and RESTCountries) and using web scraping (RapidAPI), based on the information provided in the online documentation of each API. It is therefore possible that some of the OAS specifications have errors and deviate from the API documentation. To mitigate this threat, each of the OAS specification files was carefully reviewed by at least two authors.

The results obtained for the first experiment—ability of ARTE to generate realistic inputs—are based on a sample of 10 values per parameter. Given the random nature of this experiment, it should have been repeated several times (*e.g.*, 10-30) and analyze the results with statistical tests. However, this was a manual process involving 211 (parameters) $\times$ 10 (values) $\times$ 2 (approaches) = 4,220 API calls, and so repeating it would be an extremely costly endeavor. Furthermore, we emphasize that the results obtained are simply a pessimistic approximation of what could be achieved with ARTE when considering all the inputs generated per parameter (instead of simply 10 values). The same threat applies to the second experiment. Industrial APIs impose restrictive quota limitations [184, 240], thus making it infeasible to execute thousands of requests without exceeding the quota and rate limits of the services under test. In spite of this, the high number of test cases generated and executed (84k) make us confident about the validity of the results.

Faults in the implementation of the tools used used—RESTest and SAIGEN—could compromise the validity of the results. To mitigate this threat, we carefully checked and tested the implementation of each test data generator and their results leveraging the existing regression test suites of RESTest and SAIGEN.

External validity. *To what extent can we generalize the findings of our investigation?* We evaluated our approach on a subset of APIs and therefore our conclusions could not generalize beyond that. To mitigate this threat, we evaluated ARTE on a set of 140 operations from 48 real-world RESTful APIs, including popular industrial APIs with millions of users worldwide. Additionally, we selected APIs belonging to different application domains and various sizes in terms of number of operations and parameters.

ARTE applies several heuristics to infer realistic values based on the name and description of API parameters. In particular, we proposed six matching rules to generate predicates that are likely to return semantically valid test inputs (Section §6.2.2). These

rules are based on common naming conventions for API parameters and our work with a subset of APIs, and therefore could not generalize further. However, we may remark that this does not invalidate our results and that new matching rules could be readily added in the future.

6.6 Conclusions

This chapter presented ARTE, an approach for the automated generation of realistic test inputs for REST APIs. ARTE analyzes the specification of the API under test to extract semantically related concepts for every API parameter. Then, those concepts are used to query a knowledge base from which to extract test inputs. As a distinctive feature, ARTE implements an iterative process for the refinement of test inputs through the automatic generation of regular expressions. Valid and invalid parameter values—those accepted and rejected by the API, respectively—are used to create regular expressions according to them. This allows ARTE to filter undesired values in subsequent queries to the knowledge base. ARTE has been integrated into RESTest, a black-box testing framework for RESTful APIs. In practice, this allows to automate the whole testing process: test data generation using ARTE, test case generation, test case execution, and assertion of test outputs. Evaluation results on 140 operations from 48 REST APIs show the effectiveness of ARTE to generate realistic test inputs and its potential to boost the fault detection capability of test case generation tools for REST APIs.

AGORA+: Dynamic Generation of Test Oracles for REST APIs

Simple can be harder than complex: You have to work hard to get your thinking clean to make it simple. But it's worth it in the end because once you get there, you can move mountains.

Steve Jobs,
Co-founder of Apple

This chapter introduces AGORA+, an approach for generating test oracles for REST APIs through the detection of invariants—output properties that should always hold. AGORA+ learns the expected behavior of an API by analyzing API requests and their corresponding responses. We enhanced the Daikon tool for dynamic detection of likely invariants, adding new invariant types and creating a front-end called Beet. Beet translates any OpenAPI specification and a set of API requests and responses into Daikon inputs. AGORA+ can detect 106 different types of invariants in REST APIs. We also developed PostmanAssertify, which converts the invariants identified by AGORA+ into executable JavaScript assertions. AGORA+ achieved a precision of 80% on 25 operations from 20 industrial APIs. It also identified 48% of errors systematically seeded in the outputs of the APIs under test. AGORA+ uncovered 31 bugs in popular APIs, including Amadeus, Deutschebahn, GitHub, Marvel, NYTimesBooks, and YouTube, leading to fixes and documentation updates.

The work discussed in this chapter was presented in a Journal article published in ACM Transactions on Software Engineering and Methodology [128]. A preliminary version of this work appeared in the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023 [125] (Distinguished Artifact Award) and in the ACM Student Research Competition [96, 120] (first prize winner in ESEC/FSE 2022, second prize winner in SRC Grand Finals 2023).

7.1 INTRODUCTION

The importance of REST APIs has driven the creation of numerous techniques and tools for the automated detection of failures within these APIs [193, 214]. These tools are all limited in the types of failures that they can detect, namely crashes (responses with a 5XX HTTP status code) [130, 131, 200, 212, 218, 219, 240, 294, 301], disconformities with the API specification (*e.g.*, a missing output JSON property) [130, 200, 212, 240, 294], regressions [187, 191], and violations of API best practices (*e.g.*, checking that the results of multiple calls to idempotent operations are identical) [132, 136, 291, 307]. As an example, Listing 7.1 shows the response of the "getAlbumTracks" operation of the Spotify API for the request "GET `https://api.spotify.com/albums/4Em5W5HgYEvhpc/tracks?limit=1&market=ES`". The response conforms to the API specification and therefore would be considered as a correct output by existing tools. However, the response could still contain errors that would go unnoticed by current tool support, including incorrect field length (*e.g.*, the country codes of the `available_markets` response field should have length 2) or format (*e.g.*, the values of the `href` and `uri` response fields should be URLs) and violations of numerical constraints (*e.g.*, the `total` and `duration_ms` response fields cannot be negative) or array properties (*e.g.*, the `total` response field should be greater than or equal to the size of the `items` array), among others. None of these errors would be detected by any existing approach proposed in the literature, since all the responses are syntactically valid and conform to the corresponding API specifications. Recent surveys [193] and tool comparisons [214, 244] have identified the generation of test oracles as one of the major challenges in the generation of test cases for REST APIs. This is the problem that motivates our work.

This chapter presents AGORA+, a black-box approach for the Automated Generation of Oracles for REST APIs through the detection of likely invariants. AGORA+ was created by extending and modifying the Daikon [179] system for dynamic invariant detection in two directions. Firstly, we present a novel software tool—a Daikon *front-end* called Beet—that converts an OAS specification, plus a set of API requests and responses, into a format processable by Daikon. This makes our approach easy to integrate into existing API testing tools supporting OAS specifications. Secondly, we further enhanced the capabilities of Daikon by customizing and expanding its set of invariant templates. Currently, AGORA+ supports the detection of 106 distinct types of invariants in REST APIs. Finally, to foster its use in practice, we introduce PostmanAssertify. This is a tool that transforms the invariants detected by AGORA+ into executable JavaScript scripts with assertions, written using the Chai library [18], that

```
{
    "total": 14,
    "href": "https://api.spotify.com/albums/4Em5W5HgYEvhpc/tracks?limit=1&market=ES",
    "items": [
        {
            "artists": [
                {
                    "id": "2CvCyf1gEVhI0mX6aFXmVI",
                    "name": "Paul Simon"
                },
                {
                    "id": "70cRZdQywnSFp9pnc2WTCE",
                    "name": "Arthur Garfunkel"
                }
            ],
            "available_markets": [ "ES", "US", "JP" ],
            "id": "0gFvkiT2afIcJwNxXQ7W51",
            "name": "Mrs. Robinson",
            "explicit": false,
            "duration_ms": 234346,
            "linked_from": {
                "id": "98cZPdKywnMGp8fnw2XTYU",
                "uri": "https://spotify.com/artist/98cZPdKywnMGp8fnw2XTYU"
            }
        }
    ]
}
```


Listing 7.1: Spotify API response in JSON format, for the request GET `https://api.spotify.com/albums/4Em5W5HgYEvhpc/tracks?limit=1&market=ES`.

are compatible with Postman [78], a popular API platform in industry.

Evaluation results using 25 operations from 20 industrial APIs showed that just 50 API requests and responses are sufficient for AGORA+ to learn hundreds of accurate invariants (test oracles), achieving a precision of 68%. This precision improves to 80% when learning from 10k API requests. These results surpass those obtained using the default set of invariants in Daikon, whose precision is less than 40%. We also evaluated the effectiveness of the generated test oracles in detecting failures by automatically seeding 2.5M faults in the outputs of the API operations under test. The test oracles generated by AGORA+, learned from only 50 API requests, were able to detect 48% of the incorrect outputs, supporting the cost-effectiveness of our approach.

During our evaluation, AGORA+ generated several invariants that, when manually analyzed, indicated issues within the target APIs. One example was the invariant `return.room.typeEstimated.beds >= 0`, which revealed a confirmed bug in the Amadeus API where certain hotel offers included rooms with *zero* beds. Overall, AGORA+ resulted in the detection of 31 faults (17 confirmed, 12 fixed) in 13 operations of 11 industrial APIs, that led to bug fixes in many of them, including updates in the documentation of GitHub. All these bugs would have passed unnoticed by current test case generators. This highlights the value of AGORA+ not only as a test oracle

generation approach, but also as a testing technique on its own.

This chapter presents the following original research and engineering contributions:

- AGORA+, a black-box approach for the automated generation of test oracles for REST APIs based on the analysis of the API specification and previous requests and responses (Section §7.2).

- Beet (Section §7.2.1), a Daikon front-end for REST APIs seamless to integrate into existing test case generation tools for REST APIs supporting OAS specifications. Beet is open-source and available on GitHub [16].

- Several Daikon extensions [23, 25], including reporting invariants in CSV format, support for the detection of 22 new types of invariants (Section §7.2.2), new heuristics for automatically suppressing common false positive patterns (Section §7.2.3), as well as a new output format that returns executable assertions compatible with the JavaScript Chai library (Section §7.2.4).

- PostmanAssertify, a tool that converts the invariants reported by AGORA+ into executable JavaScript test assertions compatible with the popular API platform Postman [78]. PostmanAssertify is open-source and publicly available on GitHub [79].

- An empirical evaluation of AGORA+ in terms of precision and fault detection in 25 operations from 20 industrial APIs (Section §7.3), including discovery of 31 real-world bugs (Section §7.4).

- A publicly available replication package including the source code and the data used in our work, as well as a pre-configured virtual machine to ease reproducibility and replicability [28].

Section §7.5 addresses threats to validity, and Section §7.6 concludes.

7.2 AGORA+

Figure §7.1 overviews AGORA+, our approach for the automated generation of test oracles for REST APIs. At the core of the approach is Beet,[1] a novel Daikon front-end. Beet receives three inputs: 1) the OAS specification of the API under test, 2) a set of API requests, and 3) the corresponding API responses. Beet returns a declaration file (describing the format of the API operation inputs and outputs) and a data trace file (specifying the values assigned to each input parameter and response field in each API call). This instrumentation is then processed by our customized version of Daikon, resulting in a set of likely invariants that, once confirmed by developers, can be used as test oracles. Optionally, these confirmed invariants can be used as input to PostmanAssertify, which generates a Postman collection with the invariants implemented as executable assertions in JavaScript.

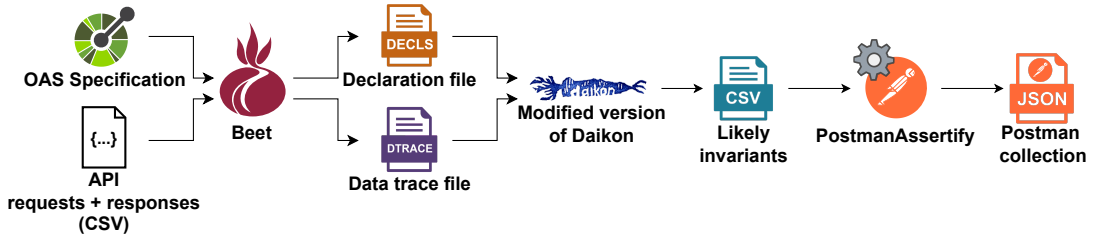


Figure 7.1: Workflow of AGORA+.

AGORA+ works at the operation level, that is, it learns invariants from API requests testing individual API operations, as this is the most basic and common testing practice [122, 212, 214, 240, 242, 243, 244, 277]. In AGORA+, the detected likely invariants fall into two categories: likely preconditions, which are properties satisfied by the input parameters of an operation, and likely postconditions, which are properties consistently satisfied by the API responses. Learning invariants for sequences of API calls (*e.g.*, creating a resource, then updating it, then deleting it) could be implemented in a similar way and remains for future work. AGORA+ currently supports JSON as the de facto standard data format. Supporting other languages should be straightforward using existing converters, *e.g.*, XML to JSON.

We now describe the Beet front-end, the types of invariants currently supported, the heuristics applied by AGORA+ to minimize the number of false positives reported, and the tool PostmanAssertify.

[1] Daikon is an Asian radish. Previous Daikon front-ends have adopted the name of root vegetables, such as Celeriac, Chicory, and Takuan [24]. We decided to follow this convention.

7.2.1 Beet: A Daikon Front-end for REST APIs

This section explains how Beet generates a declaration and a data trace file from an OAS specification and a set of API requests and responses.

Table 7.1: Daikon declaration file format for API Requests and Responses.

API Request	API operation	`ppt <operationName>&<statusCode>():::ENTER`
		`ppt-type enter`
		`variable input`
		`  var-kind variable`
		`  dec-type <operationName>&Input`
		`  rep-type hashcode`

	Input param	`variable input.<parentVariable>.<paramName>`
		`  var-kind field <paramName>`
		`  enclosing-var input.<parentVariable>`
		`  dec-type <primitiveType> | <ppt-name>&Input&<paramName>`
		`  rep-type <primitiveType> | hashcode`

	Input array	`variable input.<parentVariable>.<paramName>`
		`  var-kind field <paramName>`
		`  enclosing-var input.<parentVariable>`
		`  dec-type <primitiveType>[] | <paramName>[]`
		`  rep-type hashcode`
		`variable input.<parentVariable>.<paramName>[..]`
		`  var-kind array`
		`  enclosing-var input.<parentVariable>.<paramName>`
		`  array 1`
		`  dec-type <primitiveType>[] | <paramName>[]`
		`  rep-type <primitiveType>[] | hashcode[]`

API Response	API operation	`ppt <operationName>&<statusCode>():::EXIT<exitNumber>`
		`ppt-type subexit`
		`variable input`
		`-- Input variables --`
		`variable return`
		`  var-kind return`
		`  dec-type <ppt-name>&Output&<statusCode>`
		`  rep-type hashcode`

	Response field	`variable return.<parentVariable>.<fieldName>`
		`  var-kind field <fieldName>`
		`  enclosing-var return.<parentVariable>`
		`  dec-type <primitiveType> | <ppt-name>&Output&<fieldName>`
		`  rep-type <primitiveType> | hashcode`

	Response array	`variable return.<parentVariable>.<fieldName>`
		`  var-kind field <fieldName>`
		`  enclosing-var return.<parentVariable>`
		`  dec-type <primitiveType>[] | <fieldName>[]`
		`  rep-type hashcode`
		`variable return.<parentVariable>.<fieldName>[..]`
		`  var-kind array`
		`  enclosing-var return.<parentVariable>.<fieldName>`
		`  array 1`
		`  dec-type <primitiveType>[] | <fieldName>[]`
		`  rep-type <primitiveType>[] | hashcode[]`

Declarations

The declaration files provide a description of the inputs and outputs for each API operation. Table §7.1 summarizes how these files are generated from the information in the API specification. For each operation, an ENTER program point is created, followed by the definition of input parameters, if any. These input parameters are defined as a single `input` variable representing the whole input with as many properties/fields as input parameters (`input.<paramName>`). Similarly, an EXIT program point is created for each operation, including an identical definition of the input parameters, a `return` variable representing the whole output, and as many properties/fields of the return variable as response fields (`return.<paramName>`). The value of `<primitiveType>` in Table §7.1 can be java.lang.String, int, double, or boolean. Variables of type object are represented using hash codes.

Two cases require special consideration: JSON objects and arrays of objects. JSON

```yaml
paths:
  "/albums/{id}/tracks":
    get:
      operationId: "getAlbumTracks"
      parameters:
       - name: id
         description: "The Spotify ID for the album"
         in: path
         required: true
         type: string
       - name: market
         description: "An ISO 3166-1 alpha-2 country code"
         in: query
         required: false
         type: string
       - name: limit
         description: "The maximum number of items to return"
         in: query
         required: false
         type: integer
      responses:
        "200":
         description: "OK"
         schema:
           type: object
           properties:
             total:
               type: integer
             href:
               type: string
             items: # Array of objects
               type: array
               items:
                 type: object
                 properties:
                   artists: # Array of objects
                     type: array
                     items:
                       type: object
                       properties:
                         id:
                           type: string
                         name:
                           type: string
                   available_markets: # Array of strings
                     type: array
                     items:
                       type: string
                   id:
                     type: string
                   name:
                     type: string
                   explicit:
                     type: boolean
                   duration_ms:
                     type: integer
                   linked_from: # Nested object
                     type: object
                     properties:
                       id:
                         type: string
                       uri:
                         type: string
```

Listing 7.2: An excerpt of the getAlbumTracks operation of the Spotify API, expressed in OAS (OpenAPI Specification) format. Listing 7.1 shows a response.

```
GET https://api.spotify.com/albums/{id}/tracks?limit={limit}&market={market}
```

(a) API request

```
ppt getAlbumTracks&200():::ENTER
ppt-type enter
variable input
  var-kind variable
  dec-type getAlbumTracks&Input
  rep-type hashcode
variable input.id
  var-kind field id
  enclosing-var input
  dec-type java.lang.String
  rep-type java.lang.String
variable input.market
  var-kind field market
  enclosing-var input
  dec-type java.lang.String
  rep-type java.lang.String
variable input.limit
  var-kind field limit
  enclosing-var input
  dec-type int
  rep-type int
```

(b) Declaration file

Listing 7.3: ENTER program point of an API operation.

```
responses:
'200':
  description: 'OK'
  schema:
    type: object
    properties:
      total:
        type: integer
      href:
        type: string
      items:
        type: array
        items:
          type: object
          properties:
            artists:
              type: array
              items:
                type: object
                properties:
                  id:
                    type: string
                  name:
                    type: string
            available_markets:
              type: array
              items:
                type: string
...
```

(a) YAML response schema

```
ppt getAlbumTracks&200():::EXIT1
ppt-type subexit
variable input
... // variables from the entry point are repeated here
variable return
  var-kind return
  dec-type getAlbumTracks&Output&200
  rep-type hashcode
variable return.total
  var-kind field total
  enclosing-var return
  dec-type int
  rep-type int
variable return.href
  var-kind field href
  enclosing-var return
  dec-type java.lang.String
  rep-type java.lang.String
variable return.items
  var-kind field items
  enclosing-var return
  dec-type items[]
  rep-type hashcode
variable return.items[..]
  var-kind array
  enclosing-var return.items
  array 1
  dec-type items[]
  rep-type hashcode[]
```

(b) Declaration file

Listing 7.4: EXIT program point of an API operation.

```yaml
responses:
'200':
  description: 'OK'
  schema:
    type: object
    properties:
      total:
        type: integer
      href:
        type: string
      items:
        type: array
        items:
          type: object
          properties:
            artists:
              type: array
              items:
                type: object
                properties:
                  id:
                    type: string
                  name:
                    type: string
            available_markets:
              type: array
              items:
                type: string
            id:
              type: string
            name:
              type: string
            explicit:
              type: boolean
            duration_ms:
              type: integer
            linked_from:
              type: object
              properties:
                id:
                  type: string
                uri:
                  type: string
```

(a) YAML response schema

```
ppt getAlbumTracks&200&items():::EXIT2
ppt-type subexit
variable input
... // variables from the entry point are repeated here
variable return
  var-kind return
  dec-type getAlbumTracks&Output&200&items
  rep-type hashcode
variable return.artists
...
variable return.artists[..]
...
variable return.available_markets
  var-kind field available_markets
  enclosing-var return
  dec-type java.lang.String[]
  rep-type hashcode
variable return.available_markets[..]
  var-kind array
  enclosing-var return.available_markets
  array 1
  dec-type java.lang.String[]
  rep-type java.lang.String[]
variable return.id
...
variable return.name
...
variable return.explicit
...
variable return.duration_ms
...
variable return.linked_from
  var-kind field linked_from
  enclosing-var return
  dec-type getAlbumTracks&Output&200&items&linked_from
  rep-type hashcode
variable return.linked_from.id
  var-kind field id
  enclosing-var return.linked_from
  dec-type java.lang.String
  rep-type java.lang.String
variable return.linked_from.uri
...
```

(b) Declaration file

Listing 7.5: Second EXIT nesting level.

objects are flattened and each property is treated as a separate parameter. On the other hand, in Daikon, the elements of an array of objects can only be specified using their hashcode, limiting the types of invariants that can be identified to changes in the array. Unlike JSON objects, arrays of objects cannot be flattened by creating additional entries in the EXIT with indexed positions (*e.g.*, `return.items[1].name`) because the number of array elements varies across responses, and the declaration file cannot dynamically accommodate a changing number of variable entries. Even if such flattening were feasible, comparing all properties across all elements in nested arrays would lead to a combinatorial explosion of reported invariants, many of which would be false positives or irrelevant, while also incurring a substantial performance overhead. To support more informative array-related output invariants, Beet implements a recursive strategy by creating a new EXIT[2] program point (that we define as a new nesting level) for each distinct array element, describing its properties as independent response fields.

As an example, Listings 7.3 and 7.4 present the declarations of the ENTER (*i.e.*, API request) and EXIT (*i.e.*, API response) program points for the "getAlbumTracks" operation of the Spotify API (Listing 7.2). The "`&`" in the program point names has no semantic significance to Daikon. In Listing 7.3, the definition of the ENTER program point is followed by the definition of the input parameters. Specifically, an input variable representing the entire input, which has three properties, each representing a distinct input parameter (`input.id`, `input.market` and `input.limit`). Similarly, in Listing 7.4, the definition of the EXIT program point is followed by the definition of the input parameters (omitted for brevity), a return variable representing the entire output, and as many properties of the return variable as response fields (*e.g.*, `return.total` and `return.href`). The response includes an array of objects, `items`, including the set of music albums matching the search criteria. This is transformed into two distinct variables, one of type object (hashcode) that represent the whole array (lines 19–23), and another of type array containing the hashcodes of the array elements (lines 24–29). Besides this, an additional EXIT program point is created—a new nesting level—defining the properties of each array item (*i.e.*, Spotify album), as shown in Listing 7.5. The left part of the Listing also includes the corresponding fragment of the response format being represented. The details of the variables of type string (`return.id`, `return.name` and `return.linked_from.uri`), integer (`return.duration_ms`), boolean (`return.explicit`) and array of objects (`return.artists`) have been omitted for brevity.

[2]ENTER and EXIT program points must be defined in pairs in Daikon. Each EXIT program point is paired with a renamed copy of the ENTER program point.

The use of nested structures improves the relevance of reported invariants and avoids performance overheads. Placing all response fields as variables in a single program point would likely cause a combinatorial explosion of reported invariants by comparing variables that are potentially unrelated, leading to false positives and irrelevant results. By using nesting, variables are assigned scopes based on their nesting level, preventing unnecessary comparisons and improving efficiency.

Data Traces

Data trace files contain the actual input and output values observed during the execution of the API. Each data trace record must have the same variables as the corresponding declaration. Listing 7.6 shows the data trace file corresponding to a request to the "getAlbumTracks" operation of the Spotify API with input parameters `id="4Em5W5HgYEvhpc"`, `market="ES"` and `limit=1`, along with the API request being represented. Analogously, Listing 7.7 depicts, along with the fragment of the response in JSON format being represented, the main data trace file of the corresponding response, containing, among other properties, an array of objects. For each array item, Beet generates a new pair of trace files (*i.e.*, an ENTER and an EXIT) with the values of each object (*i.e.*, Spotify Album), omitted for brevity.

Beet is implemented in Java and is open-source. We refer the reader to GitHub for a more exhaustive description of the instrumentation process and additional examples [16].

```
GET https://api.spotify.com/albums/4Em5W5HgYEvhpc/tracks?limit=1&market=ES
```

(a) API request

```
1  getAlbumTracks&200():::ENTER
2  input
3  1242334637
4  1
5  input.id
6  "4Em5W5HgYEvhpc"
7  1
8  input.market
9  "ES"
10 1
11 input.limit
12 1
13 1
```

(b) Data trace file

Listing 7.6: ENTER data trace file.

```
{
    "total": 14,
    "href": "https://api.spotify.com/albums/4Em...",
    "items": [
        {
            "artists": [
                {
                    "id": "2CvCyf1gEVhI0mX6aFXmVI",
                    "name": "Paul Simon"
                },
                {
                    "id": "70cRZdQywnSFp9pnc2WTCE",
                    "name": "Arthur Garfunkel"
                }
            ],
            "available_markets": [ "ES", "US", "JP" ],
            "id": "0gFvkiT2afIcJwNxXQ7W51",
...
```

(a) API response in JSON format

```
getAlbumTracks&200():::EXIT1
input
...
return
2043815652
1
return.total
14
1
return.href
"https://api.spotify.com/albums/4Em..."
1
return.items
1534143414
1
return.items[..]
[313805079]
1
```

(b) Data trace file

Listing 7.7: One instance of an EXIT program point in a data trace file.

7.2.2 Invariant Definition

This section describes the changes made to Daikon for enabling the detection of likely invariants in REST APIs. In order to identify classes of invariants that could be used as effective test oracles, we used a benchmark of 40 APIs (702 operations) from the RapidAPI repository [81], that were systematically collected for the evaluation of ARTE [122]. None of the APIs of this benchmark were used for the evaluation of the approach, in order to avoid bias. This dataset contains real-world APIs which spam multiple application domains, such as finance, sports, travel, and visual recognition, among many others.

We followed a systematic procedure to identify the types of invariants present in REST APIs. For each API operation, we began by reviewing its name and description, as well as analyzing its input parameters, to understand its intended behavior and usage. Next, we examined each response field to identify invariant templates to add (*i.e.*, define a new template), enable or disable, as follows:

1. **Unary invariants.** These involve a single variable, such as the "scalar.LowerBound" invariant, which specifies the minimum allowable value for a numerical value. For each response field, we assessed whether existing Daikon invariants adequately described its characteristics and were sufficiently generic for application across different APIs. When gaps were identified, new invariants were defined. This resulted in the addition of 22 new invariants and disabling 6 invariants.

Table 7.2: Summary of modifications performed on Daikon. The strike-through invariants had to be disabled during our evaluation to avoid a combinatorial explosion of reported false positives.

Added invariants (22)	**Unary (22)**	string.IsUrl	stringsequence.ElementsAreUrl
		string.FixedLengthString	stringsequence.FixedLengthString
		string.IsNumeric	stringsequence.ElementsAreNumeric
		string.IsEmail	stringsequence.ElementsAreEmail
		string.IsDateYYYYMMDD	stringsequence.ElementsAreDateYYYYMMDD
		string.IsDateDDMMYYYY	stringsequence.ElementsAreDateDDMMYYYY
		string.IsDateMMDDYYYY	stringsequence.ElementsAreDateMMDDYYYY
		string.IsTime	stringsequence.ElementsAreTime
		string.IsTimeWithSeconds	stringsequence.ElementsAreTimeWithSeconds
		string.IsTimeAMPM	stringsequence.ElementsAreTimeAMPM
		string.IsTimestamp	stringsequence.ElementsAreTimestamp
Disabled invariants (35)	**Unary (6)**	scalar.NonZero	sequence.EltNonZero
		scalar.NonZeroFloat	sequence.EltNonZeroFloat
		scalar.RangeInt.PowerOfTwo	sequence.EltRangeInt.PowerOfTwo
	Binary (27)	twoScalar.IntNonEqual	~~twoSequence.PairwiseStringGreaterThan~~
		twoScalar.FloatNonEqual	~~twoSequence.PairwiseStringLessEqual~~
		twoScalar.LinearBinary	~~twoSequence.PairwiseStringGreaterEqual~~
		twoScalar.LinearBinaryFloat	twoScalar.NumericInt.Divides
		~~twoString.StringNonEqual~~	twoScalar.NumericInt.Square
		~~twoString.StringLessThan~~	twoScalar.NumericFloat.Divides
		~~twoString.StringGreaterThan~~	twoScalar.NumericFloat.Square
		~~twoString.StringLessEqual~~	twoSequence.PairwiseNumericInt.Divides
		~~twoString.StringGreaterEqual~~	twoSequence.PairwiseNumericInt.Square
		~~twoSequence.SeqSeqStringLessThan~~	twoSequence.PairwiseNumericFloat.Divides
		~~twoSequence.SeqSeqStringGreaterThan~~	twoSequence.PairwiseNumericFloat.Square
		~~twoSequence.SeqSeqStringLessEqual~~	twoSequence.PairwiseLinearBinary
		~~twoSequence.SeqSeqStringGreaterEqual~~	twoSequence.PairwiseLinearBinaryFloat
		~~twoSequence.PairwiseStringLessThan~~	
	Ternary (2)		
		threeScalar.LinearTernary	LinearTernaryFloat
Enabled invariants (9)	**Binary (9)**	twoString.StdString.SubString	twoSequence.SuperSequence
		twoSequence.SubSequence	twoSequence.SuperSequenceFloat
		twoSequence.SubSequenceFloat	twoSequence.SuperSet
		twoSequence.SubSet	twoSequence.SuperSetFloat
		twoSequence.SubSetFloat	

2. **Binary invariants.** These involve two variables, such as "twoString.StringEqual", which enforces that two string variables always have the same value. We analyzed potential relationships between the response field under analysis, input parameters and the remaining response fields. This process led to enabling 9 previously disabled invariants and disabling 27 invariants that were prone to false positives or combinatorial explosions of reported invariants.

3. **Ternary invariants.** These involve three variables, such as identifying linear relationships among numerical variables. A similar analysis as the one conducted for binary invariants was performed for ternary relationships, but no meaningful logical relationships involving three variables were observed in the APIs. Consequently, existing Daikon ternary invariants were disabled as they were not observed in any of the APIs.

Overall, we implemented 22 new types of invariants, disabled 35 default Daikon invariants, and activated 9 invariants disabled by default in Daikon. Table §7.2 lists all the added, disabled, and enabled invariants in our modified version of Daikon. In the context of naming invariants, the term "sequence" refers to array variables, while "scalar" refers to numerical variables, booleans (0 or 1) and objects (represented as hashcodes). The new invariants detect string patterns such as URLs, dates, and length constraints. The disabled invariants would most likely (according to our analysis of the input and output format of the benchmark operations) provide irrelevant or misleading information in our context, such as comparing the scalar value of strings or linear relations between numerical variables. Finally, we activated 9 invariants related to detecting subsets and supersets when comparing array variables (*e.g.*, `x[] is a subsequence of y[]`) and detecting substring relations between string variables (*e.g.*, `input.id is a substring of return.href`). Overall, our customized version of Daikon supports 106 types of invariants for REST APIs, classified into five categories:

- *Arithmetic comparisons (48 invariants).* Specify numerical bounds (*e.g.*, `size(return.artists[]) >= 1`) and relations between numerical fields (*e.g.*, `input.limit >= size(return.items[])`).

- *Array properties (23 invariants).* Represent comparisons between arrays, such as subsets, supersets, or fields that are always member of an array (*e.g.*, `return.hotel.hotelId in input.hotelIds[]`).

- *Specific formats (22 invariants).* Specify restrictions regarding the expected format (*e.g.*, `return.href is Url`) or length (*e.g.*, `LENGTH(return.id)==22`) of string fields.
- *Specific values (9 invariants).* Restrict the possible values of fields (*e.g.*, `return.visibility one of {"public", "private"}`).
- *String comparisons (4 invariants).* Specify relations between string fields, such as equality (*e.g.*, `input.name == return.name`) or substrings (*e.g.*, `input.id is a substring of return.href`).

We refer the reader to the AGORA+ GitHub repository [16] for a more detailed description of each type of invariant, including examples. More types of invariants could be considered in the future. Listing 7.8 shows some of the likely invariants inferred by Daikon for the "getAlbumTracks" operation of the Spotify API used as running example.

```
1  === getAlbumTracks&200():::ENTER
2  LENGTH(input.id)==14
3  input.limit >= 1
4  LENGTH(input.market)==2
5  === getAlbumTracks&200():::EXIT
6  return.href is Url
7  input.limit >= size(return.items[])
8  return.total >= size(return.items[])
9  return.total >= 1
10 input.market is a substring of return.href
11 input.id is a substring of return.href
12 === getAlbumTracks&200&items():::ENTER
13 ...
14 === getAlbumTracks&200&items():::EXIT
15 size(return.artists[]) >= 1
16 All the elements of return.available_markets[] have LENGTH=2
17 input.market in return.available_markets[]
18 LENGTH(return.id)==22
19 LENGTH(return.linked_from.id)==22
20 return.linked_from.uri is Url
21 LENGTH(return.linked_from.uri)==54
22 return.linked_from.id is a substring of return.linked_from.uri
23 return.duration_ms > size(return.artists[])
24 return.duration_ms > size(return.available_markets[])
25 === getAlbumTracks&200&items&artists():::ENTER
26 ...
27 === getAlbumTracks&200&items&artists():::EXIT
28 LENGTH(return.id)==22
```

Listing 7.8: Invariants detected in the "getAlbumTracks" operation of the Spotify API.

7.2.3 Minimizing False Positives

In the previous version of our approach [125], invariants of the arithmetic comparison category caused 3 out of every 4 false positives. Most false positives were

invariants comparing two unrelated output properties. For example, the invariant `return.duration_ms > size(return.artists[])` states that the duration of a song in milliseconds should always be greater than the number of artists in the song. While this may be true in most cases, it does not characterize the API specification and we consider it a false positive. Finding counterexamples to automatically rule out these false positives is extremely unlikely, and the number of reported false positives can become unbearable if the API has multiple numerical response fields.

To address this, AGORA+ incorporates a novel heuristic to reduce false positives caused by arithmetic comparisons, leveraging the following observation: parameters with different value ranges are unlikely to be related. For example, the duration of a song typically spans thousands of milliseconds (*e.g.*, more than 30k ms), whereas the number of artists on a song may vary from 1 to around 10. Our heuristic evaluates the observed value ranges for variables involved in arithmetic inequalities (*e.g.*, `x<y`, `x<=y`, `x>y`, `x>=y`). Specifically, it discards invariants if the maximum observed value of one variable (*e.g.*, 10 artists) is less than the minimum observed value of the other variable (*e.g.*, 30k ms). For example, the invariant `return.duration_ms > size(return.artists[])` would be discarded if the respective value ranges do not overlap, such as duration of 30k-300k ms versus 1-10 artists. When value ranges overlap, the heuristic allows the invariant to be retained, as the relationship might be meaningful. To implement this heuristic, we modified 32 arithmetic comparison invariants in Daikon, ensuring they are not reported if there is no overlap between the value ranges of the two variables being compared. This heuristic enhances the relevance of the invariants generated by Daikon and has potential for application in the detection of invariants in other domains.

7.2.4 PostmanAssertify: Automated Generation of Test Assertions

This section presents PostmanAssertify, a tool that converts the invariants reported by AGORA+ into executable assertions in JavaScript that are compatible with Postman. Postman [78] is a widely used API development and testing platform that offers a graphical interface to create, send, and analyze HTTP requests to REST APIs. One of its standout features is the use of collections, which allow developers to group and organize API requests. This enables developers to run entire suites of tests with a single click.

PostmanAssertify receives the API specification and the CSV file containing the set of confirmed invariants generated by AGORA+ as inputs. PostmanAssertify returns

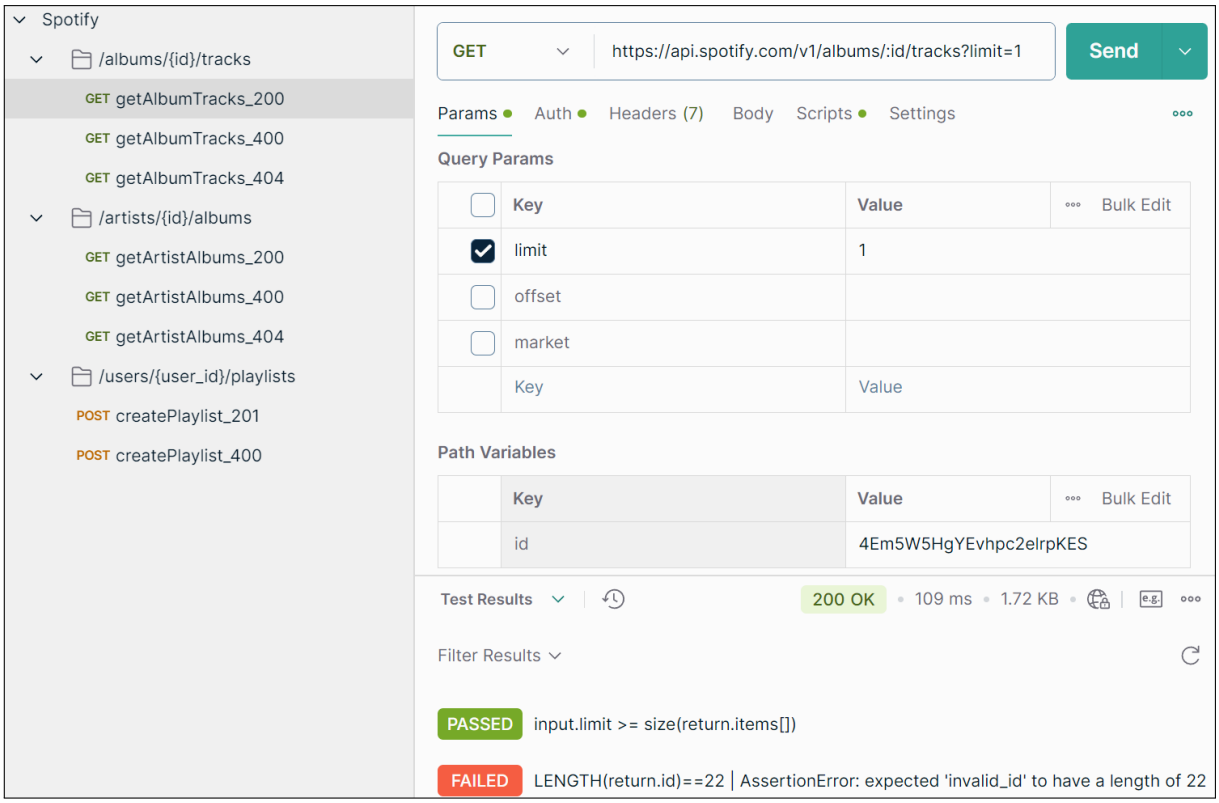


Figure 7.2: Postman collection generated by PostmanAssertify.

a Postman collection, including a sample API request for each status code of the API operations under test and—attached to each request—all the invariants as executable test assertions.

Figure §7.2 shows an example of a Postman collection generated by PostmanAssertify running on the Postman desktop application. The collection contains a request for each response code of every API operation where AGORA+ has detected invariants. These operations are listed in the directories on the left sidebar of Figure §7.2. Each request, as shown in the right pane of Figure §7.2, contains the following:

- The URI of the API operation with the input parameters configured.
- A test script written in JavaScript using the Chai library [18, 108].
- Assertions for all the confirmed invariants implemented in the test script.

Every time this script is executed, Postman generates a report such as the one shown in the bottom pane of Figure §7.2, in which it indicates whether each assertion passed or failed, with an explanatory message in case of failure. The collection reports can be exported in JSON format for further analysis.

Listing 7.9 contains a snippet of the test script generated by PostmanAssertify for the Spotify "getAlbumTracks" operation used as running example. For the sake of brevity, the test script only contains the assertions generated for two of the invariants: `input.limit >= size(return.items[])`, reported at the base nesting level; and `LENGTH(return.id)==22`, reported at the `items` nesting level.

PostmanAssertify first generates the code that initializes the variables storing the values of the API request input parameters (lines 1–18), the special values to consider as null (line 20), and the returned API response (line 23). Then, it generates the code containing all the test cases (one per confirmed invariant) of the base nesting level (lines 25–36). Each test case starts by obtaining the values of the variables involved in the assertions (lines 26–30), which can be either input parameters of properties of the response. Then, if none of the variables is null (lines 32–33), the assertion is executed (line 34).

After creating the tests of the base nesting level, PostmanAssertify generates the code for accessing the base variable of the subsequent nesting level (lines 38–42), and all the test cases of this new nesting level, if any (lines 44–51). This process is iteratively repeated for all the subsequent nesting levels.

```
// Getting value of the id path parameter
input_id = pm.request.url.path[2];
if (input_id != null) {
    input_id = decodeURIComponent(input_id);
}

// Getting value of the market query parameter
input_market = pm.request.url.query.get("market");
if (input_market != null) {
    input_market = decodeURIComponent(input_market);
}

// Getting value of the limit query parameter
input_limit = pm.request.url.query.get("limit");
if (input_limit != null) {
    input_limit = decodeURIComponent(input_limit);
    input_limit = parseInt(input_limit);
}

valuesToConsiderAsNull = [];

// Base nesting level (getAlbumTracks&200 program point)
response = pm.response.json();

pm.test("input.limit >= size(return.items[])", () => {
    // Getting value of variable: return_items_size_array
    return_items_size_array = response["items"];
    if (return_items_size_array != null) {
        return_items_size_array = return_items_size_array.length;
    }

    if ((input_limit != null) && (!valuesToConsiderAsNull.includes(input_limit)) &&
    (return_items_size_array != null) && (!valuesToConsiderAsNull.includes(return_items_size_array))) {
        pm.expect(input_limit).to.be.at.least(return_items_size_array);
    }
})

// Second nesting level (getAlbumTracks&200&items program point)
response_items = response["items"]
if (response_items != null) {
    for (response_items_index in response_items) {
        response_items_element = response_items[response_items_index];

        pm.test("LENGTH(return.id)==22", () => {
            // Getting value of variable: return_id
            return_id = response_items_element["id"];

            if ((return_id != null) && (!valuesToConsiderAsNull.includes(return_id))) {
                pm.expect(return_id).to.have.length(22);
            }
        })

    } // Closing for response_items
} // Closing if response_items
```

Listing 7.9: Part of the Postman test script generated by PostmanAssertify for the Spotify "getAlbumTracks" operation.

To implement the Postman assertions (lines 34 and 49), we extended Daikon with a new output format [26] so it reports the invariants supported by AGORA+ as assertions of the Chai JS library. PostmanAssertify has been implemented in Java and is publicly available on GitHub [79].

7.3 EVALUATION

We aim to answer the following research questions:

- **RQ$_1$:** *How effective is AGORA+ in generating test oracles?* We measure the precision of AGORA+ in generating invariants that properly model the expected API behavior.
- **RQ$_2$:** *How does the size of the input dataset affect the performance of AGORA+?* The precision and the number of detected invariants usually depends on the quality and diversity of the input datasets (*i.e.*, API requests and responses).
- **RQ$_3$:** *How effective are the generated test oracles in revealing failures?* The final goal is generating test oracles that can be used during testing to identify erroneous responses caused by defects. Thus, we investigate the effectiveness of the generated oracles for detecting failures in REST APIs.

7.3.1 Experimental Data

For our experiments, we used a set of 25 operations from 20 industrial APIs (Table §7.3) tested by previous authors [122, 243, 244, 277]. All these APIs are freely accessible. Our replication package [28] provides detailed information on the specific authentication requirements for each API. The OAS specification of the APIs were obtained from their official websites. For those APIs that do not provide access to the official specification, we either generated them manually (Foursquare, iTunes, Ohsome, OMDb, RESTCountries, and Yelp) or used the version available in APIs.guru [13] (Spotify, Stripe, Vimeo, and YouTube), modifying them according to the latest version of the web docs. Some of the existing specifications were either incorrect (parameters of type array defined as strings) or incomplete (missing response fields). We manually fixed those specifications to ensure that Beet could process them. This process was carried out using a semi-automated process with RESTest, which leverages the Atlassian OAS

syntactic validator [112]. During test case generation, RESTest identified missing response fields, which were then manually added to the OAS in an iterative process that took only a few minutes per API operation.

The Vimeo API offers various authentication scopes [107], including public (exclusively returns public video data), and private (grants access to private video data, provided the user is the video owner). We opted for the public scope due to our limited access to private video metadata. When using the public scope, the API does not return certain response fields that are exclusive to the private scope, and therefore AGORA+ reports that these fields are always null. We removed these fields from the OAS specification to avoid bloating the results.

For each operation, the RESTest [240] framework generated and executed API calls until obtaining 10k valid API calls per operation, at least 9k of which must be non-empty results (*i.e.*, the result is not a zero-length list), generating 250k calls in total. According to REST best practices [270], we consider an API response as valid if it is labeled with a 2XX HTTP status code. The OMDb API does not adhere to REST best practices, returning a 200 response containing the `error` response field when the user provides invalid data. For this API, we consider as valid those API responses that do not contain this field.

Input data generation

The invariants detected by AGORA+ largely depend on the diversity of the API requests provided as input. To foster such diversity, we configured RESTest to use specific data generation strategies for each of the input parameters of the API operations of our study. For each one of the 224 different input parameters of the operations of our analysis, we used one of the following strategies:

- *Manually created data dictionaries (31.3% of the parameters).* Collection of input values created manually. For example, a list of database IDs (such as Spotify Album IDs), country codes or city names, among others.
- *Enum values (19.2% of the parameters).* List of all the possible values accepted for a specific input parameter (usually specified in the documentation). For example, when creating a GitHub repository, the value of the `visibility` parameter can only be either "public" or "private".
- *Numbers (19.2% of the parameters).* Randomly generates a number within minimum and maximum bounds. This strategy is used, for instance, in pagination

parameters.

- *English words (17% of the parameters).* Randomly generates a word or sequence of words using the extJWNL (Extended Java WordNet Library) Java library [35]. Used for free text fields, such as when setting the name of a Spotify playlist.
- *Booleans (10.3% of the parameters).* Randomly generates either "true" or "false".
- *Dates (3% of the parameters).* Randomly generates a string following a specific date format. The generated date is within a start and end date.

When manually creating the data dictionaries, we followed current practices and selected values based on an analysis of the API specification and documentation. For instance, for the `location` parameter of the Yelp API, we created a list of cities located in different continents. Our replication package [28] contains the RESTest configuration files and the data dictionaries used for generating the test cases.

7.3.2 Experiment 1: Test Oracle Generation

This experiment answers $\mathbf{RQ_1}$ (precision of the approach) and $\mathbf{RQ_2}$ (impact of the dataset size) by evaluating the effectiveness of AGORA+ in generating test oracles.

Experimental setup

For each API operation, we randomly divided the 10k automatically-generated requests into 10 random subsets of each of the sizes 50, 100, 500, and 1k requests. We report the performance in terms of average precision (*i.e.*, $precision = TP/(TP + FP)$) and average number of reported invariants across the 10 runs. For each run, the inferred invariants were classified as true positives or false positives based on manual analysis. The cost of manually verifying invariants was minimal for most cases, as they could be confirmed within seconds from the name and the description included in the API specification (*e.g.*, a field named `image_url` is clearly expected to be a URL) or by analyzing the API documentation, which typically specifies the expected format or constraints of response fields in natural language. If the documentation was not clear enough, we consulted the API providers about the expected API behavior. For instance, in the RESTCountries API, we noticed that one of the *idds* (international dialing code) of Canada was an empty string (""), instead of a numerical code. Upon seeking clarification, the API providers confirmed that this behavior was intentional [84]. A true positive invariant describes a property of the output that should always hold;

they are valid test oracles. A false positive reflects a pattern that has been observed in all the API requests and responses provided as input, but does not represent the expected behavior of the API. For example, AGORA+ may report that the visibility of a GitHub repository is always "public" (`return.visibility == "public"`), because it never observes the "private" value, or that the number of stars in a repository is always greater than its number of forks (`return.stargazers_count > return.forks_counts`). Although these were true for all the test cases used to infer the invariants, they do not represent the intended behaviour of the API, and therefore they are false positives. As part of our analysis, we measured the proportion of reported true positives and false positives of each one of the invariant categories presented in Section §7.2.2.

Daikon also detects invariants among input parameters (*i.e.*, likely preconditions in ENTER program points). These invariants can offer insights about the API behavior regarding input parameter values. For example, they may report that all the supported values of an input parameter have been used in at least one request that returned a valid response (*e.g.*, `input.direction one of { "asc", "desc" }`) or that the numerical values are within a valid range (*e.g.*, `input.limit >= 1`). A violation of these invariants or an incomplete version of them could reveal either a limitation in the test suite used as input for AGORA+ or a bug in the API implementation. For example, the invariant `input.direction == "asc"` (an incomplete version of `input.direction one of { "asc", "desc" }`) reveals that there are no API responses for requests using the "desc" value for the `direction` parameter. This could mean either that the test suite is not diverse enough or that there are no valid responses when this input parameter value is used due to a bug in the API implementation. However, invariants about inputs do not provide information about the output and hence we did not use them to calculate precision, focusing only on likely postconditions.

We compared the effectiveness of AGORA+ against the default version of Daikon and the previous version of our approach, AGORA [125]. AGORA supports the same invariants as AGORA+, but it does not apply the heuristic presented in Section §7.2.3 for reducing false positives. In all cases, our novel front-end, Beet, transformed API specifications, requests, and responses into inputs for Daikon. Our preliminary experiments revealed that 13 of Daikon default invariants resulted in a combinatorial explosion of string comparisons and a high number of false positives. For a fair comparison, we disabled these problematic invariants in the default version of Daikon used as baseline, as detailed in our supplementary material [28]. These disabled invariants are strike-through in Table §7.2.

We compared the performance of these three approaches in terms of precision. We do not compare them in terms of recall due to the large number of response fields returned by many of the API operations (up to several hundreds). Generating a ground truth for all the supported invariants would require manually annotating a dataset of up to tens of thousands of invariants per API operation.

The experiment was performed on a laptop equipped with Intel i7-11800H @2.30GHz, 32GB RAM, and 1TB SSD running Windows 11 and Java 17.

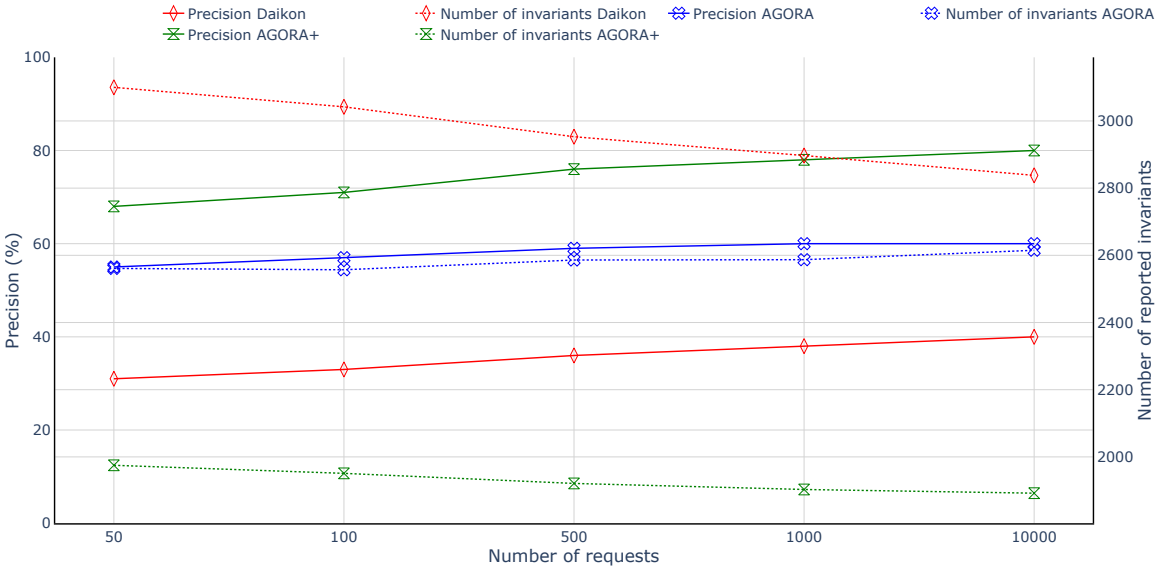


Figure 7.3: Precision and number of invariants reported, depending on the size of the input dataset.

Experimental results

RQ_1: Effectiveness of the approach Table §7.3 shows the results for each API operation, set of API requests size (50, 100, 500, 1k, 10k), and approach (AGORA+ vs. AGORA vs. default Daikon). The reported values for the random subsets of 50, 100, 500, and 1k requests represent the average precision and the average number of reported invariants across 10 runs. Figure §7.3 shows the evolution of the precision and the number of invariants reported by each approach when the input test suite size increases.

Next, we analyze the results obtained from the entire dataset of 10k requests ($\mathbf{RQ_1}$), shown in the last two columns of the table. The following section compares different sizes of the input dataset ($\mathbf{RQ_2}$).

When learning from the whole dataset (10k API requests), AGORA+ obtained a

Table 7.3: Test oracle generation. I="Number of likely invariants", P="Precision (% valid test oracles)"

API - Operation	50 API calls						100 API calls						500 API calls						1K API calls						10K API calls					
	Daikon		AGORA		AGORA+		Daikon		AGORA		AGORA+		Daikon		AGORA		AGORA+		Daikon		AGORA		AGORA+		Daikon		AGORA		AGORA+	
	I	P (%)	I	P (%)	I	P (%)	I	P (%)	I	P (%)	I	P (%)	I	P (%)	I	P (%)	I	P (%)	I	P (%)	I	P (%)	I	P (%)	I	P (%)	I	P (%)	I	P (%)
AmadeusCitySearch-GETCities	34	84	34	97	32	100	38	84	37	94	33	100	41	80	38	93	33	99	41	80	38	93	34	98	39	85	37	95	33	100
AmadeusHotel-getMultiHotelOffers	132	17	121	54	120	55	131	18	121	56	120	57	118	21	115	62	114	62	113	23	110	66	110	66	99	26	107	68	107	68
Deutschebahn-getStations	202	22	127	30	91	41	164	28	114	36	86	48	110	45	103	45	82	57	102	49	102	46	81	58	94	53	97	48	76	62
DHL-getLocation-finderV1Find-by-address	54	40	26	50	19	64	52	41	26	52	18	67	49	44	25	55	18	69	48	46	24	58	18	71	45	49	22	64	17	76
FDIC-searchInstitutions	450	14	407	32	235	48	430	16	408	33	226	52	487	17	466	32	231	57	503	17	486	32	229	60	589	16	601	29	249	62
Foursquare-placeSearch	267	13	145	32	85	43	308	13	164	32	95	43	339	13	188	30	94	50	332	14	190	31	94	50	270	17	154	35	90	48
GitHub-createOrganizationRepository	83	94	200	97	194	99	81	96	199	98	193	100	81	96	199	98	193	100	80	96	198	98	192	100	80	96	198	98	192	100
GitHub-getOrganizationRepositories	44	40	149	86	139	92	42	43	146	88	138	93	39	46	147	89	140	94	39	46	149	89	142	92	38	47	148	89	142	93
GitLab-getApiV4ProjectsIdBadges	5	23	6	57	4	92	5	28	6	58	4	85	5	43	6	67	4	100	4	47	6	67	4	100	4	50	6	67	4	100
GitLab-listProjectJobs	142	12	152	32	91	54	136	13	142	36	80	63	119	15	135	38	71	72	116	16	134	39	70	74	113	17	130	41	67	79
iTunes-search	120	12	98	30	54	54	127	12	101	32	57	56	121	14	93	36	54	62	118	14	88	38	50	66	111	17	78	44	44	77
LanguageTool-checkText	123	54	65	67	57	74	116	61	66	71	59	78	120	68	69	75	63	80	120	72	71	76	65	82	120	74	73	78	67	84
Marvel-getComicIndividual	223	24	121	46	103	52	199	28	114	51	96	58	174	34	108	57	92	65	160	38	104	60	89	68	140	46	96	66	88	69
NYTimesBooks-GET_lists-format	44	62	44	68	43	70	42	70	40	77	40	78	35	86	34	89	34	89	34	87	34	90	34	90	34	88	32	91	32	91
Ohsome-elementsAggregation	11	81	18	88	15	99	11	82	18	89	15	100	11	82	17	88	15	100	11	82	17	88	15	100	11	82	17	88	15	100
OMDB-byIdOrTitle	7	55	17	87	16	92	7	56	16	91	16	97	7	54	16	92	15	98	7	54	16	92	15	98	7	57	16	94	15	100
OMDB-bySearch	5	90	5	95	5	97	6	65	6	78	6	92	5	75	6	82	5	98	5	79	6	83	5	100	6	83	7	86	6	100
RESTCountries-v31ListOfCodes	139	27	68	40	60	45	139	31	56	50	52	53	126	39	50	61	47	65	125	39	49	63	46	67	124	40	49	63	46	67
Spotify-createPlaylist	22	100	41	100	41	100	22	100	41	100	41	100	22	100	41	100	41	100	22	100	41	100	41	100	22	100	41	100	41	100
Spotify-getAlbumTracks	48	43	65	83	59	92	47	45	67	86	61	94	43	49	66	88	60	97	42	50	66	88	60	98	41	54	66	89	60	98
Spotify-getArtistAlbums	55	40	53	82	50	89	50	47	53	86	49	92	38	64	51	90	48	96	32	75	50	92	47	98	31	84	52	92	49	98
Stripe-PostProducts	58	67	41	83	31	99	58	67	42	83	31	100	58	67	42	83	32	98	58	67	42	83	31	99	64	61	40	88	31	100
Vimeo-search_videos	548	39	344	55	287	66	546	41	349	56	295	66	524	46	349	59	299	69	508	50	343	61	295	71	514	53	330	66	286	76
Yelp-getBusinesses	68	25	33	36	23	52	59	29	32	37	21	56	49	35	25	48	14	85	45	37	24	51	13	91	41	39	22	50	12	92
YouTube-listVideos	218	33	181	62	122	73	224	34	192	61	118	78	230	36	198	64	121	85	230	36	200	64	121	86	201	42	196	66	123	86
TOTAL	**3100**	**31**	**2561**	**55**	**1975**	**68**	**3042**	**33**	**2557**	**57**	**1951**	**71**	**2953**	**36**	**2586**	**59**	**1921**	**76**	**2897**	**38**	**2587**	**60**	**1903**	**78**	**2838**	**40**	**2615**	**60**	**1892**	**80**

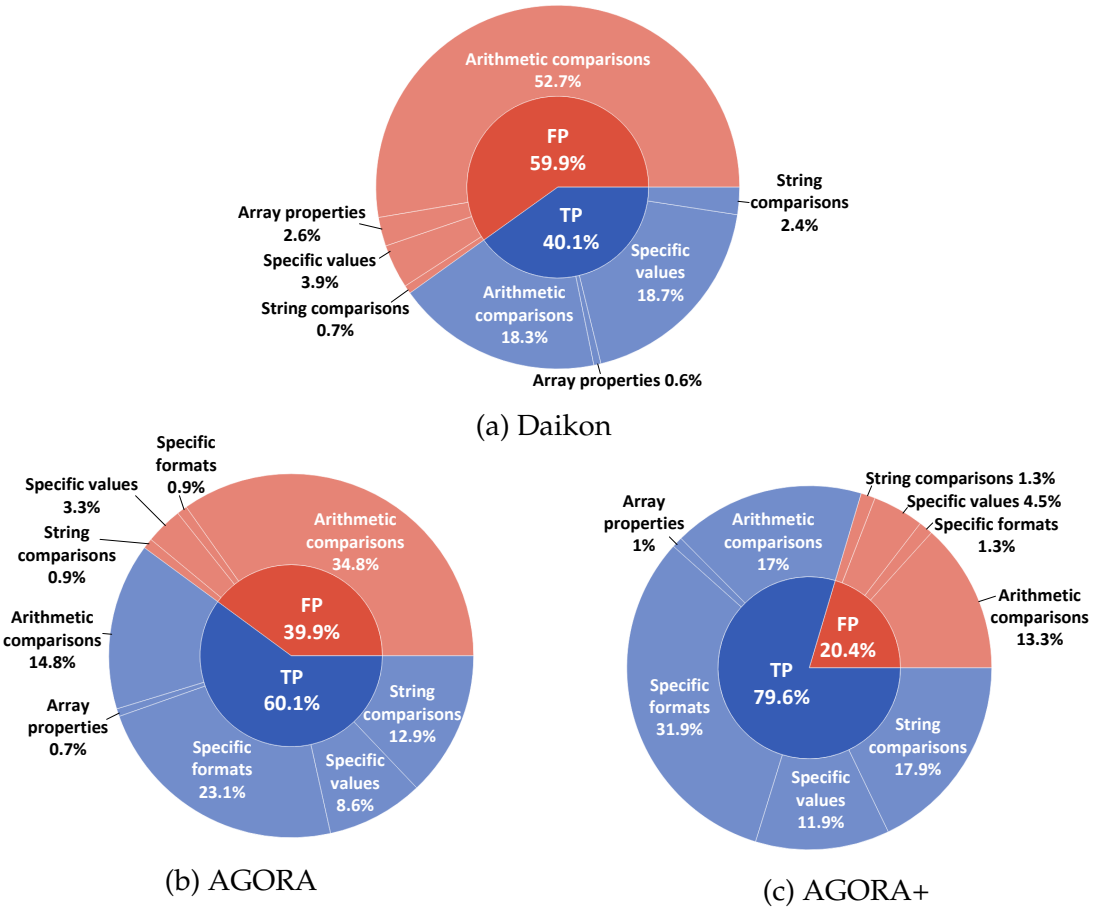
total precision of 80% (1506 out of 1892 invariants are valid test oracles), outperforming the precision of both AGORA (60%) and the default configuration of Daikon (40%) by 33% and 100%, respectively. AGORA+ equaled or outperformed all the baselines in all the API operations.

The precision achieved by AGORA+ ranged between 48% in the Foursquare API and 100% for 8 API operations (AmadeusCitySearch-GETCities, GitHub-createOrganizationRepository, GitLab-getApiV4ProjectsIdBadges, OMDb-byIdOrTitle, OMDb-bySearch, Ohsome, Spotify-createPlaylist, and Stripe). 24 of the 29 false positives in Deutschebahn are related to station opening and closing times; 12 of these FPs state that the opening (or closing) times of all the workdays of the week are the same (*e.g.*, `monday.from_time == tuesday.from_time`). This fact is true for all the stations returned by the API, but we consider it possible that it would not be true for every station, so we conservatively marked it as a false positive. Twelve more FPs are about the length of the station closing time: it is always 5 characters long, as in "18:00". Because it is conceivable that a station closes before 10:00am, we again marked these as false positives.

The heuristic applied by AGORA+ to reduce the number of false positives (detailed in Section §7.2.3) led to a precision increase over AGORA of up to 114% (FDIC API), yielding a precision increase in 22 out of 25 operations. Most false positives in the FDIC API arise from arithmetic comparisons involving response fields that serve as numerical flags (values of 0 or 1) with other numerical variables. The heuristics of AGORA+ significantly reduced the number of false positives for this operation, lowering them from 415 to 81. In the Yelp API, 10 out of the 11 false positives reported by AGORA are arithmetic comparisons stating that the value of the numeric parameters `open_at` (an integer representing an Unix time) and `radius` (search radius in meters) is always greater than all the numeric response fields, including the size of all the arrays in the response. The heuristic applied by AGORA+ automatically suppressed all these false positives.

Statistical analysis confirms significant performance differences among the approaches. The Friedman test ($p\text{-}value < 0.05$) rejected the null hypothesis of equivalent precision among AGORA+, AGORA, and Daikon. Post-hoc analysis with Holm correction further validated these findings.

The number of invariants reported per operation varied between 4 in the GitLab-getApiV4ProjectsIdBadges operation and 286 in the Vimeo API. We observed a strong positive correlation between the number of response fields and the number of reported



(a) Daikon

(b) AGORA

(c) AGORA+

Figure 7.4: Reported invariants by category in Daikon, AGORA, and AGORA+.

invariants, with a Spearman correlation coefficient of 0.92.

Figure §7.4 breaks down the true positives and false positives of each approach (Daikon, AGORA and AGORA+). In the default version of Daikon, the largest portion of false positives (88%) were found in the arithmetic comparisons category, followed by specific values (7%), array properties (4%), and string comparisons (1%). The invariants reported by AGORA and AGORA+ are the same, except those belonging to the arithmetic comparisons category. Thanks to the heuristics applied in AGORA+ (Section §7.2.3), the percentage of false positives in this category decreased from 87% (909 invariants) to 65% (252 invariants), suppressing a total of 657 false positives.

False positives in the arithmetic comparison category typically occur when comparing numerical fields with values in different magnitude orders, such as the duration of a Spotify song in milliseconds and its number of artists. The remaining false positives are either invariants that report an object as always null or that limit a response field to only a specific value or set of values, but the API supports more (*e.g.*, reporting that the

visibility of a GitHub repository is always "public"). These invariants indicate either a lack of diversity in the test suite or bugs in the API (c.f. Section §7.4).

The heuristic applied by AGORA+ to reduce false positives also suppressed 66 true positives. This happened in cases in which the variables involved in the invariant had clearly defined minimum and maximum values that did not overlap (*i.e.*, they guarantee that the invariant always holds and thus it is not a false positive). For example, the automatically suppressed invariant `input.maxWidth > size(return.items[])`, reported in the YouTube API is true because, according to the documentation [111], the minimum value of `input.maxWidth` is 72, whereas the maximum value of `size(return.items[])` is 50.

Overall, arithmetic comparisons remain the most common type of false positives reported by AGORA+ (65% of false positives), followed by specific values (22%), string comparisons (6%) and specific formats (6%), with no false positives of the array properties category.

We now present an analysis of the performance of the sets of added, enabled, and disabled invariants, specified in Table §7.2. AGORA+ reported 627 invariants belonging to the set of added invariants, spanning 13 invariant types and achieving precision ranges between 91%, and 100% for 11 invariant types. Among enabled invariants, it detected 272 instances of a single type ("twoString.StdString.SubString") with 99% precision. For disabled invariants, the original configuration of Daikon reported 789 instances across 11 invariant types, with precision ranging from 0% (for 7 types) to 3%, except for two high-performing types: "scalar.NonZero" (240 instances, 98% precision) and "sequence.EltNonZero" (74 instances, 99% precision). These invariants specify whether numerical variables or array elements are nonzero but are also used to indicate non-null objects. These invariants were intentionally disabled in AGORA+ to prevent redundancy or bloating the results, as nullability can be effectively specified using the OpenAPI Specification (OAS) nullable property.

Beet took between 1.4 seconds (50 requests) and 67 seconds (10k requests) to generate the instrumentation of the target API operations. Daikon (customized and default version) took between 1.8 seconds (50 requests) and 30 seconds (10k requests) to detect the reported invariants. Overall, AGORA+ took less than 2 minutes to generate the invariants using the complete dataset of 10k requests.

Answer to **RQ$_1$**: How effective is AGORA+ in generating test oracles?

AGORA+ is effective in generating test oracles, achieving a precision of 80% when learning from 10k API requests. This is a precision improvement of 100% over the default configuration of Daikon.

RQ$_2$: Impact of the size of the input dataset. Figure §7.5 shows the evolution of the precision of AGORA+ with respect to the number of API requests, as detailed in Table §7.3. The total precision improved from 68% with 50 API requests (an average of 1345 valid invariants, *i.e.*, test oracles), to 80% with the complete dataset (1506 valid invariants). This means a drop in precision of just 12 percentage points when using the smallest dataset (50 API requests). The greatest precision increase takes place between 100 and 500 requests (5 percentage points). Conversely, the number of reported invariants decreases, mostly due to the suppression of false positives. The suppression of invariants classified as true positives when increasing the size of the input test suite reveals that a counterexample (*i.e.*, failure) has been found (c.f. Section §7.4). For instance, in the NYTimesBooks API, the suppression of the confirmed invariant `LENGTH(return.isbn10)==10` revealed the presence of ISBNs with invalid values in the test suite. Figure §7.6 shows the evolution of the number of reported valid invariants (*i.e.*, true positives) with respect to the number of API requests. When using the 50 requests datasets, AGORA+ detected, on average, 91% of all the true positives reported by the 10k dataset, with a standard deviation of 8 percentage points. The increase in the number of reported true positives ranged between 0 percentage points in 8 API operations (GitHub-createOrganizationRepository, GitLab-getApiV4ProjectsIdBadges, NYTimesBooks-GET_lists-format, Ohsome-elementsAggregation, OMDb-byIdOrTitle, Spotify-createPlaylist, Stripe-PostProducts and Yelp-getBusinesses), and 38 percentage points in the "searchInstitutions" operation of the FDIC API. This shows the applicability of AGORA+ even when provided with a small set of test cases as input.

Figure §7.7 shows a boxplot with the distribution of the precision achieved by AGORA+ on all the operations across multiple divisions on random subgroups of 50 test cases. For all the API operations, the difference between the median and the first and third quartiles was less than 5 percentage points, with the exceptions of the NYTimesBook API, with a difference of 6 percentage points between the median and the third quartile, and the "getApiV4ProjectsIdBadges" operation of the GitLab API, with a difference of 17 percentage points between the median the first quartile. This significant difference in the GitLab API was due to the suppression, in 8 out of 10 seeds,

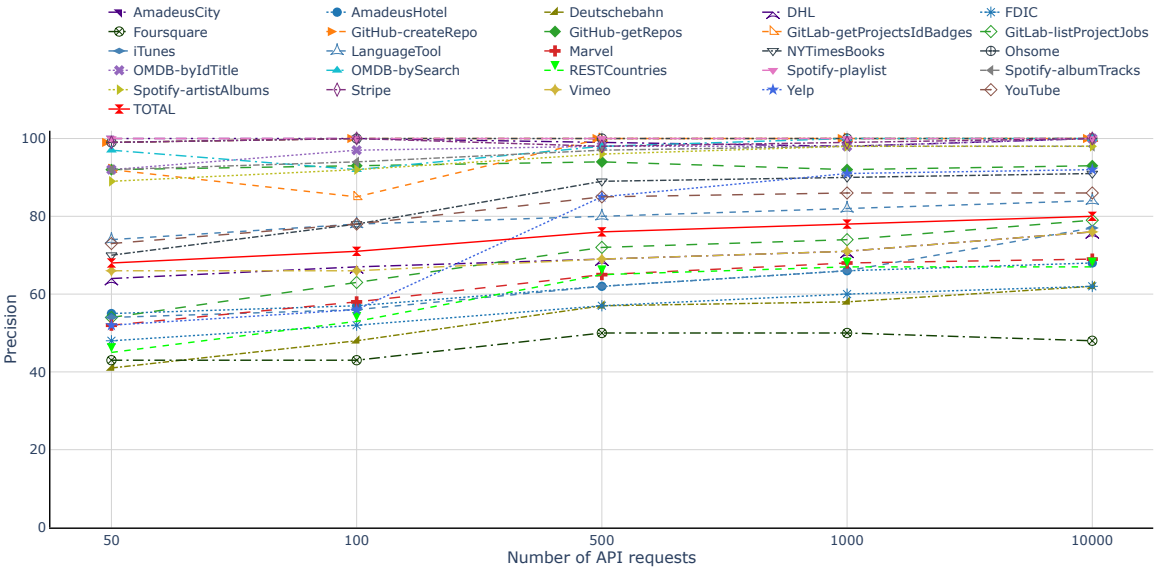


Figure 7.5: How test suite size affects the precision of AGORA+.

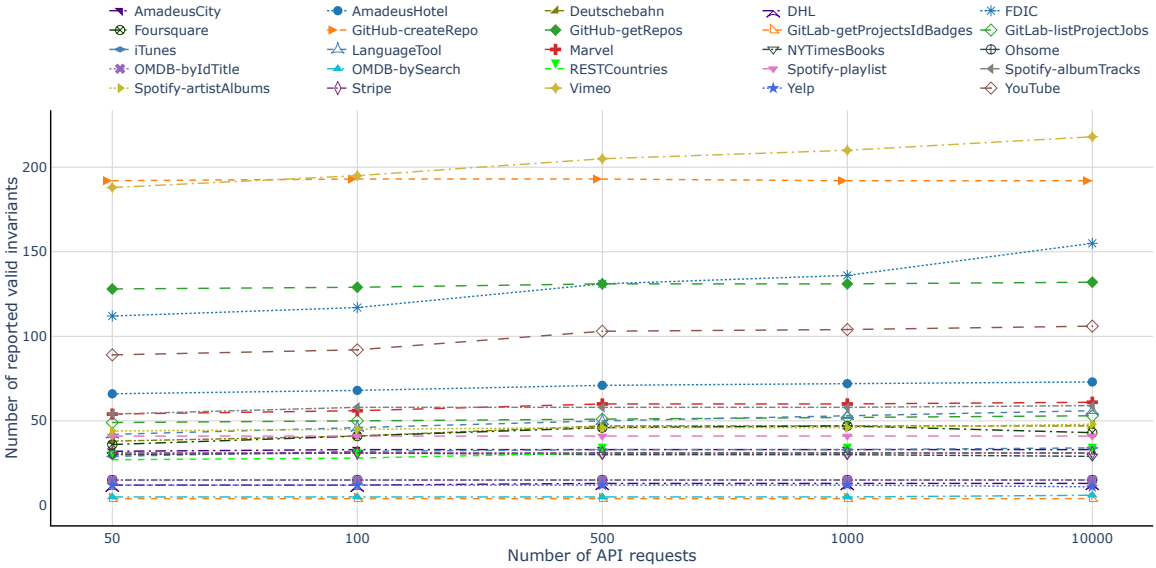


Figure 7.6: How test suite size affects the number of valid invariants reported by AGORA+.

of 2 valid invariants (out of an average of 4 reported invariants) which revealed the presence of real bugs (Section §7.4). Overall, these results show that the effectiveness of AGORA+ is stable across different datasets.

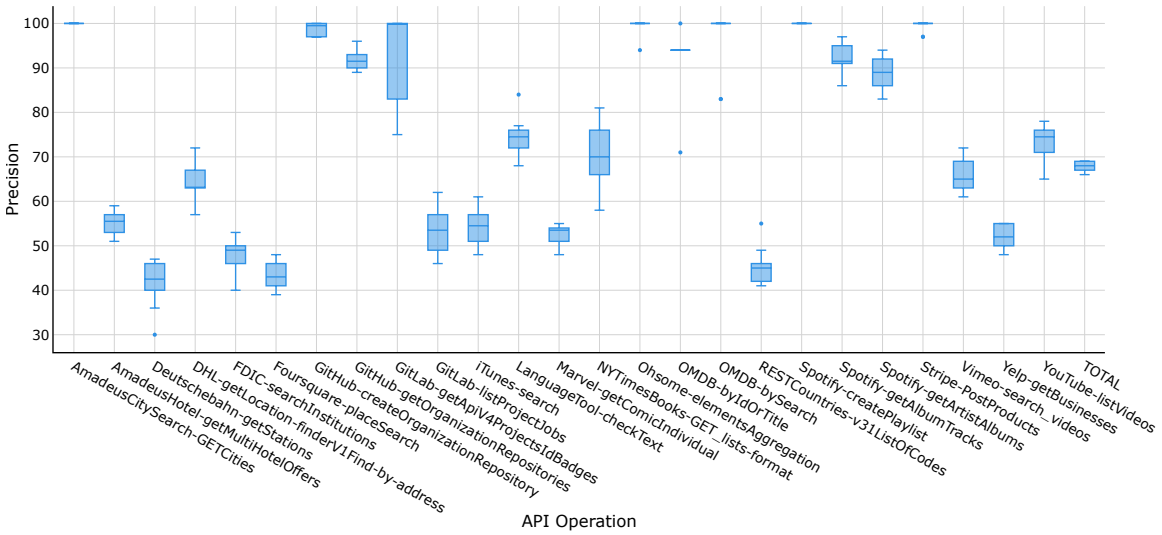


Figure 7.7: Distribution of the precision of AGORA+ in the 50 test cases subset across 10 executions.

Answer to **RQ$_2$**: How does the size of the input dataset affect the performance of AGORA+?

Increasing the size of the test suite causes an increase in the precision achieved and a decrease in the number of invariants reported. The suppressed invariants are mostly false positives, with only a small increase in the number of valid invariants (*i.e.*, true positives). With a small but diverse set of 50 API requests, AGORA+ achieves a precision of 68% (12 percentage points less than when using a dataset 200 times bigger), and detects 91% of the valid invariants detected when using the 10k dataset.

7.3.3 Experiment 2: Failure Detection

This experiment aims to answer **RQ$_3$** by analyzing the effectiveness of the test oracles generated by AGORA+ in detecting failures.

Experimental setup

To address **RQ$_3$**, we evaluated the effectiveness of the generated test oracles in detecting failures (*i.e.*, erroneous outputs) in the APIs under test. To this end, we sys-

tematically seeded *errors* in API responses using JSONMutator [59], an open-source mutation tool that applies different mutation operators on JSON data, *e.g.*, removing an array item. This approach differs from traditional mutation testing, where *defects* are seeded in the source code of the program under test. The motivation behind our strategy is to assess the failure detection capabilities of the generated test oracles on large-scale industrial APIs, for which source code is not available. Although open-source APIs exist, they are generally less complex compared to the APIs used in our study [214, 241]. Also, we argue that this strategy—introducing errors in API responses—is appropriate since our goal is assessing the effectiveness of the test oracles, not the test inputs, which has already been thoroughly investigated in previous studies. This simulates a regression testing scenario where previously inferred test oracles are reused to ensure that code changes do not unintentionally impact existing functionalities. In case of changes in the API implementation or functional behavior, it would be necessary to run AGORA+ again to generate updated test oracles that align with the revised API behavior.

For each API operation, we selected the test oracles derived from the set of 50 test cases since, as revealed in our previous experiment, this was the most conservative input dataset. Test oracles were transformed into executable assertions (1369 in total) using PostmanAssertify (Table §7.4). Then, for each API operation, we randomly selected 1k API responses from the set of 10k test cases generated by RESTest meeting the following constraints: (1) they were not part of the 50-requests set used for detecting the invariants, (2) they contained at least one result item (we cannot apply mutation operators on empty arrays), and (3) they revealed no failures (Section §7.4).

We used JSONMutator to introduce a single error on each API response, simulating a failure. Then, we ran the assertions and marked the failure as detected if at least one of the test assertions (*i.e.*, test oracles) was violated. We repeated this process 100 times per operation to minimize the effect of randomness computing the average percentage of failures detected. In total, the results are based on 2.5M seeded errors: 25 operations x 1,000 API responses x 100 repetitions. For each mutated test suite, we used PostmanAssertify to generate a Postman collection in which the mutated outputs were used as response mocks.

For our experiments, we configured JSONMutator to apply mutation operators that resulted in syntactically valid mutants, *i.e.*, conform to the API specification. Syntactically invalid mutants that would result in violations of the API specification (*e.g.*, adding a new property to a JSON object) can be detected by existing approaches and

therefore are out of the scope of AGORA+. Specifically, we enabled the mutation operators that consist of changing boolean, double, long and string values (*e.g.*, adding or removing characters) and altering array values (*e.g.*, removing and disordering elements), using a total of 12 mutation operators. All the mutations resulted in a distinguishable change in the API response and therefore there were no equivalent mutants [233, 263].

For string response fields, the "boundary" operator replaces the original string with boundary values, such as an empty string, an all-uppercase or all-lowercase string, or strings of minimum and maximum length (*e.g.*, 1 or 10 characters). The "mutate" operator modifies strings by adding, removing, or replacing a single character, while the "addSpecialCharacters" operator introduces special characters like "/" or "*" into the string. Lastly, the "replace" operator completely substitutes the string with a random value.

For numerical response fields (doubles, longs, and integers), the "mutate" operator alters values by adding or subtracting a small delta, which is 0.5 for decimal numbers and 1 for integers. The "replace" operator, on the other hand, substitutes the number with a completely random value.

For arrays, the "disorderElements" operator changes the order of the elements within the array, while the "removeElement" operator randomly removes one or more elements. The "empty" operator clears the entire array, leaving it with no elements.

For Boolean fields, the "mutate" operator simply flips the Boolean value, changing true to false or vice versa.

We disabled the mutation operators that produced mutants non-conformant with the OAS specification. Also, we disabled operators that converted response fields into null values, since null values are easily detected as a violations of the `nullable` property of OAS.

Experimental results

Table §7.4 shows the number of test assertions (*i.e.*, test oracles) and the percentage of detected failures for each API operation. Overall, test oracles generated by AGORA+ identified 48% of the failures. This percentage ranged between 15% in the RESTCountries API and 93% in the operations "createOrganizationRepository" of the GitHub API and "createPlaylist" of the Spotify API.

Figure §7.8 illustrates the percentage of mutants generated by each mutation op-

Table 7.4: Failure Detection Ratio per API operation.

API - Operation	Assertions (test oracles)	FDR (%)
AmadeusCitySearch-GETCities	33	51
AmadeusHotel-getMultiHotelOffers	69	60
Deutschebahn-getStations	38	21
DHL-getLocation-finderV1Find-by-address	12	48
FDIC-searchInstitutions	119	47
Foursquare-placeSearch	40	23
GitHub-createOrganizationRepository	193	93
GitHub-getOrganizationRepositories	130	65
GitLab-getApiV4ProjectsIdBadges	3	30
GitLab-listProjectJobs	53	37
iTunes-search	32	38
LanguageTool-checkText	42	29
Marvel-getComicIndividual	54	37
NYTimesBooks-GET_lists-format	31	50
Ohsome-elementsAggregation	15	74
OMDB-byIdOrTitle	15	36
OMDB-bySearch	5	21
RESTCountries-v31ListOfCodes	26	15
Spotify-createPlaylist	41	93
Spotify-getAlbumTracks	43	70
Spotify-getArtistAlbums	43	75
Stripe-PostProducts	30	58
Vimeo-search_videos	199	47
Yelp-getBusinesses	12	24
YouTube-listVideos	91	67
TOTAL	**1369**	**48**

erator killed by AGORA+. AGORA+ showed its lowest performance with the "disorderElements" mutation operator (6% detection rate), as only 3 out of 25 API operations had test oracles related to array ordering. Conversely, string response fields are the most common in the API responses, accounting for 72% of mutations (1.8M out of 2.5M). AGORA+ performed best on string-type response fields, detecting 54% of mutations. Overall, AGORA+ proves effective in detecting failures related to incorrect formats (*e.g.*, URLs or dates), predefined values, or relational patterns (*e.g.*, equality or substring). However, it struggles to identify incorrect values that depend solely on the semantics of the application or underlying data. For instance, the primary source of undetected errors was modifying unique string values (*e.g.*, for movie titles, "title=Alien" $\rightarrow$ "title=Aliens") introduced by the string-replace mutation operator. Detecting such semantic bugs is extremely challenging and falls outside the scope of this work.

In view of these results, we can answer $\mathbf{RQ_3}$ as follows:

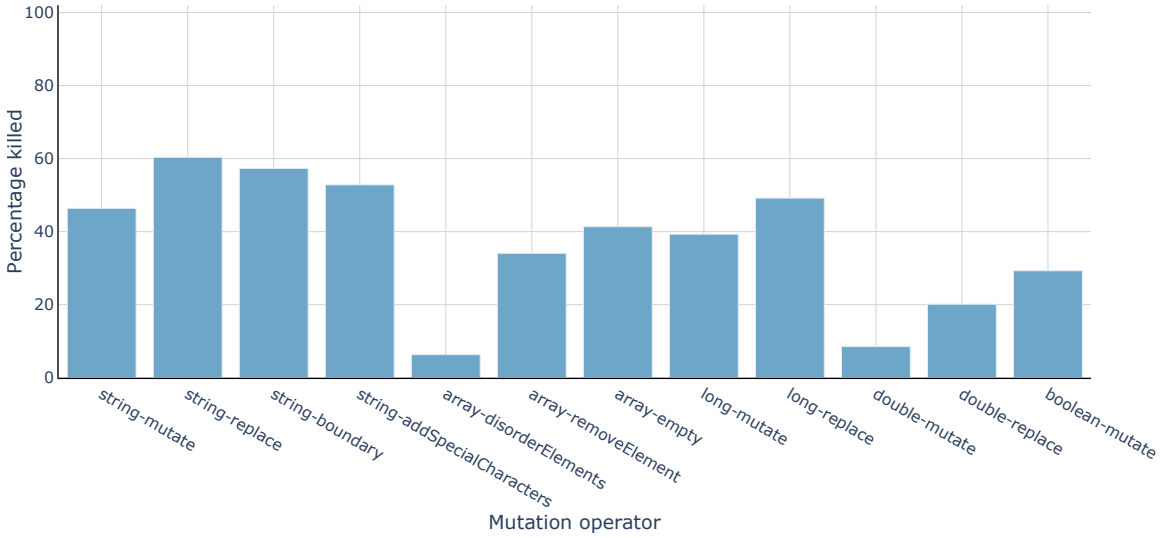


Figure 7.8: AGORA+ performance detecting each mutation operator.

Answer to **RQ$_3$**: How effective are the generated test oracles in revealing failures?

The test oracles generated by AGORA+ are effective in detecting failures, catching 1 out of every 2 errors systematically seeded in API responses.

7.4 Detected Failures

The invariants detected by AGORA+ revealed real-world bugs in some of the APIs, showing the potential of the approach as a testing technique on its own. Some of the invariants revealed inconsistent behavior, *e.g.*, hotel rooms with *zero* beds in the Amadeus Hotel API. We also found cases where a confirmed invariant (*i.e.*, test oracle) was discarded when increasing the size of the input dataset, meaning that a counterexample (*i.e.*, failure) had been detected. Therefore, the invariants reported by AGORA+ play a dual role in failure detection: invalid invariants reveal failures observed during the invariant detection process, whereas violated valid invariants indicate failures observed in production. Invalid variants require manual inspection, whereas the violation of confirmed invariants can be automatically detected during testing. Overall, AGORA+ detected 31 domain-specific bugs (17 confirmed) in 13 operations from 11 APIs with millions of users worldwide, namely Amadeus Hotel, Deutschebahn, FDIC, Foursquare, GitHub, GitLab, Marvel, NYTimesBooks, OMDb, RESTCountries, and YouTube. Our supplementary material contains videos showing the replication

process of these bugs, as well as anonymized screenshots of our reports and the received responses [28]. Next, we detail the detected bugs.

Amadeus Hotel. During our initial experiments, one of the detected invariants in the Amadeus Hotel API led to the identification of 55 hotel offers in which the offered room had zero beds (`return.room.typeEstimated.beds>=0`). This bug has been confirmed and fixed by Amadeus developers. Also, according to the description provided in the OAS specification, the property `code` of the `taxes` response field of this operation should be a two-letter country code, but the API always returns values such as "TOTAL_TAX" or "SERVICE_CHANGE". This bug has not been confirmed yet.

GitHub. In the "createOrganizationRepository" operation, the violation of the confirmed invariant `input.license_template==return.license.key` revealed 15 test cases in which the repository was created with an incorrect license. More specifically, in some cases, when creating a repository with the "ncsa" or "postgresql" license, GitHub created a repository with the "other" license. This bug has been confirmed by the API providers. Also, contrary to what is stated in the API specification and the documentation, AGORA+ detected that the field `template_directory` was never included in the responses of the "getOrganizationRepositories" operation (`return.template_repository==null`). Developers confirmed the issue and updated the documentation of GitHub accordingly.

GitLab. The suppression of the confirmed invariant `return.rendered_image_url is Url` of the "getApiV4ProjectsIdBadges" operation uncovered badges with URLs containing whitespaces, rendering them invalid. This bug was confirmed and fixed by the API providers [3].

Foursquare. AGORA+ identified 9 bugs in this API, all of which have been confirmed by the API providers. The suppression of the confirmed invariants `return.menu is URL` and `return.website is URL` exposed locations with invalid URLs for menus and websites, such as "http://gh". These bugs have already been fixed. The suppression of the valid invariant `input.radius > return.distance` revealed cases where the API returned places outside the user-defined search radius. This was traced back to incorrect user-entered coordinates and has been corrected. Similarly, the suppression of `input.radius >= return.context.geo_bounds.circle.radius` exposed a similar issue, this bug has been fixed too. The suppression of `input.min_price <= return.price` and `input.max_price >= return.price` uncovered test cases where the API returned places outside the specified price range, these bugs have not been fixed yet. Additionally, the suppression of `return.stats.total_photos >= 0` revealed a flaw where neg-

ative values were assigned to the `total_photos` field when photos were demoted by the system or the Placemarker community. The API providers confirmed this as a bug and are working on a fix. Similarly, the suppression of `return.stats.total_photos >= size(return.photos[])` and `return.stats.total_tips >= size(return.tips[])` exposed test cases where the reported totals for photos and tips were less than the actual number of items in the corresponding arrays. These two bugs have been resolved.

FDIC. The API documentation specifies that the values of the `CONSERVE` and `LAW_SASSER_FLG` response fields should be a numerical flag (0 or 1), but the invariants `return.data.CONSERVE one of {"N","Y"}` and `return.data.LAW_SASSER_FLG one of {"N","Y"}` revealed that the documentation was inconsistent. The same happens with the TRUST response field, AGORA+ reported the invariant `return.data.TRUST one of {"0", "1"}`, whereas the documentation describes "00", "10" and "11" as the expected values. None of these reports have been confirmed yet.

RESTCountries. AGORA+ detected that the area in square kilometers of the Norwegian territories Svalbard and Jan Mayen was indicated as -1.0 (invariant `return.area>=-1.0`). This bug has been confirmed and fixed by the API providers [86, 87]. Also, the response fields documentation [5], indicates that the value of the `startOfWeek` response field can be either Sunday or Monday. However, the invariant `return.startOfWeek one of {"monday","saturday","sunday"}` identified "Saturday" as the value for Iran. This report has been confirmed and the developers updated the documentation accordingly.

YouTube. When performing a search using the `regionCode` input parameter, the returned videos must be available in the provided region. However, a violation of the confirmed invariant `input.regionCode in return.contentDetails.regionRestriction.allowed[]`, led us to detect 81 cases in which the API returned videos that were not available in the provided region. This error has been confirmed by YouTube developers.

Deutschebahn. The violation of the confirmed invariant `return.localServiceStaff.availability.sunday.toTime is Time` led to the detection of a train station (Coburg) with a value following an incorrect format for the end time of the availability of the service staff on Sundays (recorded as 18.30, instead of 18:30). This bug has been confirmed and fixed by the API providers.

Marvel. In the "getComicById" operation, AGORA+ detected +3.1k comics with 0 pages (`return.pageCount>=0`), invalid date formats, comics with an invalid Diamond

code (violations of the invariant `LENGTH(return.diamondCode)==9`), and invalid values for the EAN code (violations of the invariant `LENGTH(return.ean)==20`). For example, we found a case where the EAN code had the value of the Diamond code. The OAS specification states that the value of the `data.results.stories.items.type` response field should be either "interior" or "cover", but the invariant `return.type one of {"cover","interiorStory"}` revealed that this description is imprecise. Further analysis of the remaining API requests uncovered 32 different values for this response field. These reports have not been confirmed yet.

NYTimesBooks. The `isbn10` response field of this API is expected to store a string of 10 characters. However, the violation of the `LENGTH(return.isbn10)==10` invariant revealed two different failures: a book with an isbn10 of 9 characters, and a book with the string "isbn10" as value for the `isbn10` response field. These reports have not been confirmed yet.

OMDb. The `type` parameter of the OMDb API operations allows users to filter results by media type—namely "movie", "series" or "episode"—as specified in the documentation. However, one of the invariants (`return.Type one of {"game","movie","series"}`) revealed a new value for this parameter that was not specified in the documentation: "game". Moreover, we detected that the operations "byIdOrTitle" and "bySearch" do not support filtering by "episode", despite being specified as a valid filtering value in the documentation. API developers have not confirmed these issues yet.

7.5 Threats to Validity

In this section, we discuss the potential validity threats that may have influenced our work, and how these were mitigated.

Internal validity. *Are there factors that might affect the results of our evaluation?* For our experiments, we used the OAS specification of the APIs under test. When possible, we used the publicly available API specifications. However, the specifications of the Foursquare, iTunes, Ohsome, OMDb, RESTCountries, and Yelp APIs were unavailable, so we generated them manually based on an analysis of the web documentation. Therefore, it is possible that these specifications have errors and deviate from the API documentation. To mitigate this, we used the state-of-the-art Atlassian OAS syntactic validator [112] to ensure the OAS was consistent with the returned API responses. The

updated OAS specifications were thoroughly reviewed by at least two authors.

The effectiveness of our approach largely depends on the diversity of the input API requests and responses. To maximize input diversity, we manually selected a set of varied test inputs for each parameter based on an analysis of the documentation. This may be considered a naive and conservative approach. Using more systematic or automated means (*e.g.*, adaptive random testing [206]) could probably yield even better results.

The classification of the reported invariants as true positives or false positives may be affected by human biases or errors. To mitigate this threat, each invariant was checked by at least two authors, analyzing the API documentation. If the documentation was not clear enough, we consulted the API providers about the expected API behavior.

Finally, the division of the dataset into random subgroups may have also affected the results. To mitigate this threat, we performed this division into subsets 10 times, computing the average performance between runs.

External validity. *To what extent can we generalize the findings of our investigation?* We evaluated AGORA+ on a set of 25 operations from 20 different APIs, and therefore our conclusions might not generalize beyond that. To mitigate this threat, we evaluated the approach with a set of popular industrial APIs of different domains and various sizes used in related papers.

The completeness of the OAS specification has a direct impact in the quality of the reported invariants. An incomplete OAS (*e.g.*, with missing response fields) leads to an incomplete set of invariants. However, most existing tools for automated API testing provide features that ensure the syntactic validity of the OAS with respect to the returned API responses.

AGORA+ relies on test suite diversity to ensure accurate invariant detection. A lack of diversity in the test suite can lead to invalid or incomplete invariants that reflect only the API behavior within the test suite, without generalizing to the expected functionality. However, test suite diversity is a fundamental requirement in testing and can typically be achieved by applying standard test case design techniques.

The novel types of invariant proposed might not generalize beyond the selected APIs. To mitigate this threat, these invariants were created based on an analysis of a systematically collected dataset of 40 realistic APIs (702 operations) belonging to dif-

ferent domains [122]. We remark, however, that this set of invariants is not intended to be complete and new invariant types could be proposed in the future.

7.6 Conclusions

This chapter introduces AGORA+, an approach for generating test oracles for REST APIs through the detection of likely invariants. Invariants are detected by analyzing the API specification and a set of API requests with their corresponding responses. The approach is implemented using Daikon, an open-source tool for dynamic invariant detection. In particular, we created Beet, a novel Daikon front-end for REST APIs described using OAS, and a customized version of Daikon supporting the detection of 106 distinct types of invariants in REST APIs. AGORA+ also integrates PostmanAssertify, a tool that converts the reported invariants into executable Postman assertions, making our approach readily applicable. Evaluation results on a set of 25 operations from 20 industrial APIs show that AGORA+ can generate hundreds of effective test oracles with just 50 requests in seconds, outperforming the default version of Daikon and AGORA—a previous version of our approach—by a margin of 100% and 33%, respectively. In addition, AGORA+ helped identify 31 issues in industrial APIs with millions of users, contributing to fixes and documentation updates, demonstrating its potential as a standalone testing technique. Since it operates in a black-box mode, AGORA+ can be easily integrated into existing API testing tools supporting the OAS specification format.

SATORI: Static Generation of Test Oracles for REST APIs

Difficulties mastered are opportunities won.

Winston Churchill,
Statesman and former Prime Minister of the United Kingdom

This chapter introduces SATORI (Static API Test ORacle Inference), a black-box approach for generating test oracles for REST APIs by analyzing their specification. SATORI uses LLMs to infer the expected behavior of an API by analyzing the properties of the response fields of its operations, such as their name and descriptions. We also extended the PostmanAssertify tool to automatically convert the test oracles reported by SATORI into executable assertions. SATORI achieved an F1-Score of 74.5% on 17 operations from 12 industrial APIs, outperforming the related approach, AGORA+ (69.2%), which requires executing the API to generate test oracles. SATORI uncovered 18 bugs in popular APIs—Amadeus Hotel, Deutschebahn, FDIC, GitLab, Marvel, OMDb and Vimeo—leading to bug fixes and documentation updates.

We intend to submit an extended version of this chapter to the 40th IEEE/ACM International Conference on Automated Software Engineering, ASE 2025.

8.1 INTRODUCTION

To the best of our knowledge, AGORA+ is the only existing approach in the literature that addresses the automated generation of test oracles for REST APIs, which generates test oracles through the detection of likely invariants. Although effective, the main limitation of AGORA+ is its reliance on a sufficiently diverse test suite that thoroughly exercises the API functionality to report accurate invariants. If the test suite lacks diversity or contains faulty responses, the reported invariants may be invalid or incomplete.

Listing 8.1 shows a response for the "getBusinesses" operation of the Yelp API which conforms to the API specification (Listing 8.2) and would be considered correct by existing tools. However, this response may still contain errors that would go undetected by test case generators, such as incorrect field length (*e.g.*, `country` should have 2 characters), format (*e.g.*, `image_url` should be a valid URL), specific values (*e.g.*, `price` adheres to the following allowed values: "$", "$$", "$$$", "$$$$"), or violations of numerical constraints (*e.g.*, `latitude` should range from -90 to 90), among others. Most of these constraints, which can be implemented as test oracles, can be inferred by analyzing certain characteristics of the OAS specification of the API under test (Listing 8.2), such as the name and description of the different response fields.

```
{
    "total": 1,
    "businesses": [
        {
            "id": "7dzGDH1BtzEjhZh1FeeaqA",
            "name": "Caipirinha Corner",
            "image_url": "https://s3-media1.fl.yelpcdn.com/bphoto/zrG.jpg",
            "rating": 4.0,
            "coordinates": {
                "latitude": 37.3968404980258,
                "longitude": -5.97877264022827
            },
            "price": "$",
            "location": {
                "city": "Seville",
                "country": "ES"
            }
        }
    ]
}
```

Listing 8.1: Yelp API response in JSON format.

This chapter presents SATORI (Static API Test ORacle Inference), a black-box static approach for automatically generating test oracles for REST APIs by analyzing their OAS specification, without requiring prior API execution. SATORI leverages LLMs to infer test oracles from the unstructured components of the OAS specification, such as

response field names and descriptions, making it compatible with existing API testing tools that support OAS specifications. Currently, SATORI supports a catalog of 29 test oracles, which can be easily extended by users. To ease its use in practice, we extended the PostmanAssertify tool to transform the test oracles reported by SATORI into executable JavaScript assertions, written using the Chai library [18], that are compatible with Postman [78], a widely used API platform in industry with over 35 million users.

The results of an evaluation conducted on 17 operations from 12 industrial APIs show the capabilities of SATORI for automatically generating up to hundreds of valid test oracles per API operation, achieving an F1-Score of 74.5%. SATORI identified 18 real bugs across 7 widely used industrial APIs, which would have passed unnoticed by existing test cases generators. Our findings lead to bug fixes and documentation updates in GitLab and Vimeo APIs, respectively. Since it does not require prior API execution, SATORI offers a more cost-effective solution than AGORA+$_{\mathrm{U}}$ (configuration of AGORA+ that only reports invariants involving one variable), which obtained an F1-Score of 69.2%. However, their performance differences show that each approach excels in identifying distinct oracle types, making them complementary and resulting in a more effective combined solution.

This chapter presents the following original research and engineering contributions:

- SATORI (Static API Test ORacle Inference): A black-box static approach for automatically generating test oracles for REST APIs through specification analysis (Section §8.2). SATORI is open-source and publicly available on GitHub [89].
- An extended version of the PostmanAssertify tool [79], which transforms the test oracles generated by SATORI into executable JavaScript assertions compatible with the widely used Postman API platform [78]. This tool is open-source and publicly available on GitHub.
- An empirical evaluation of SATORI, using AGORA+ as a baseline, in terms of precision, recall, F1-Score and failure detection across 17 operations from 12 industrial APIs (Section §8.3), including discovery of 22 real-world bugs.
- OKAMI (Oracle Knowledge of API Methods for Innovation) [74], a dataset containing the annotated ground truth of all the test oracles of the API operations used in our evaluation (over 10.5K test oracles from more than 1.8K response fields), enabling reliable benchmarking and future comparisons.

```yaml
paths:
  "/businesses/search":
    get:
      operationId: getBusinesses
      parameters:
       - name: term
         description: "Search term, e.g. food or restaurants."
         in: query
         schema:
           type: string
       - name: location
         description: "Geographic area for business search."
         in: query
         schema:
           type: string
      responses:
        "200":
          description: "Returns all businesses"
          content:
            application/json:
              schema:
                type: object
                properties:
                  total:
                    type: integer
                    description: "Total number of businesses found."
                  businesses:
                    type: array
                    items:
                      type: object
                      properties:
                        id:
                          type: string
                        name:
                          type: string
                        image_url:
                          type: string
                        rating:
                          type: number
                          description: "Business rating (ranges from 1... 5)."
                        coordinates:
                          type: object
                          properties:
                            latitude:
                              type: number
                            longitude:
                              type: number
                        price:
                          type: string
                          description: "Price level. Value is one of $, $$, $$$ and $$$$."
                          example: "$$"
                        location:
                          type: object
                          properties:
                            city:
                              type: string
                            country:
                              type: string
                              description: "ISO 3166-1 alpha-2 country code."
```

Listing 8.2: OAS excerpt of the Yelp API.

Section §8.4 discusses threats to validity, and Section §8.5 concludes the chapter.

8.2 SATORI

Figure §8.1 outlines SATORI, our approach for automatically generating test oracles for REST APIs through specification analysis. Starting from an OAS specification, SATORI extracts the schema of each response field of all the target operations and generates prompts for a (configurable) LLM. The outputs of the LLM (*i.e.*, test oracles) are then processed into a machine-readable format and, following an optional human verification step, are provided to an extended version of PostmanAssertify [79] to produce a Postman collection with executable test assertions.

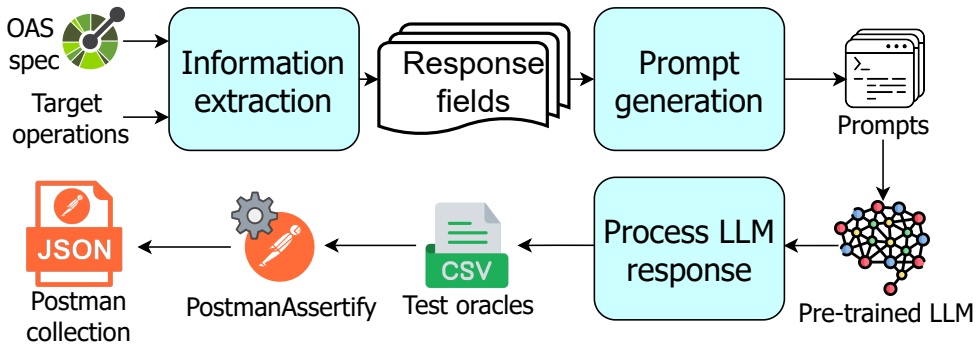


Figure 8.1: Workflow of SATORI.

We now describe the complete SATORI workflow in detail, as well as the supported set of test oracles.

8.2.1 Automated Response Field Prompt Generation

This subsection explains how SATORI generates prompts to infer test oracles for each response field of an API operation.

Information extraction

First, we extract contextual information to generate input prompts. Specifically, we gather details for each response field—such as name, description, and examples—along with global context like the API name and operation ID, providing the LLM with richer context to infer accurate test oracles.

Table 8.1: Prompt structure.

Complete prompt	
⟨*ContextPrompt*⟩ + ⟨*PropertiesPrompt*⟩ + ⟨*OraclesPrompt*⟩	
Context prompt: ⟨*ContextPrompt*⟩	
I am going to give you a response field of the ⟨*operation_id*⟩ operation of the ⟨*api_name*⟩ API. The name of this response field is "⟨*response_field_name*⟩" and it is of type ⟨*response_field_type*⟩.	I am going to give you a response field of the getBusinesses operation of the Yelp API. The name of this response field is "price" and it is of type string.
Properties prompt: ⟨*PropertiesPrompt*⟩	
- - - This response field has the following properties: "name": "⟨*response_field_name*⟩" "⟨*property_key*⟩": "⟨*property_value*⟩" - - -	- - - This response field has the following properties: "name": "price" "type": "string" "description": "Price level. Value is one of $, $$, $$$, $$$$." "example": "$$" - - -
Oracles prompt: ⟨*OraclesPrompt*⟩	
⟨*TaskIntroductionPrompt*⟩ +[⟨*SingleOraclePrompt*⟩]{⟨*response_field_datatype_test_oracles*⟩}+⟨*ResponseFormatPrompt*⟩	
Task introduction prompt: ⟨*TaskIntroductionPrompt*⟩	
Given this information, I want you to answer the following questions about some properties of this response field:	
Single oracle prompt: ⟨*SingleOraclePrompt*⟩	
⟨*oracle_number*⟩ - ⟨*oracle_prompt*⟩ JSON property: "⟨*oracle_name*⟩", of type ⟨*oracle_datatype*⟩⟨*no_value_instruction*⟩	3 - Should this response field have a set of specific values? JSON property: "string_specific_values", of type array of string, if there are no specific values, the array is empty
Response format prompt: ⟨*ResponseFormatPrompt*⟩	
I want the response to be a single JSON object with the properties indicated in each question (⟨*response_field_datatype_test_oracles*⟩). I don't want any kind of additional natural language explanation, only the JSON object.	I want the response to be a single JSON object with the properties indicated in each question (string_is_url, string_is_numeric, string_specific_values, string_is_email, string_is_date, string_fixed_length, string_is_time). I don't want any kind of additional natural language explanation, only the JSON object.

Prompt generation

Using information from the previous step, SATORI generates one prompt per response field, following the structure shown in Table §8.1. The left-hand side of the table displays the prompt template, while the right-hand side provides an example for the `price` response field from Listing 8.2. To set the overall behavior of the model and ensure consistent outputs, a *System prompt* is used to guide the approach of the model to inferring test oracles. Each prompt begins by establishing context for the task, followed by the necessary information, and then a detailed task description. Consequently, each prompt is structured into three sections: *Context prompt*, *Properties prompt*, and *Oracles prompt*.

System prompt We use a role-playing approach, instructing the model to act as an expert software engineer. The system prompt is: "You are a highly skilled software engineer with extensive experience in designing and testing REST APIs. Answer to your questions simply by generating a JSON object, without providing any additional information or explanation."

Context Prompt This section provides the model with essential context, including the name of the target response field, the API, and the specific operation under test.

Properties Prompt This section includes the name of the response field along with additional properties, such as descriptions or examples, which the LLM can analyze to identify potential test oracles.

Oracles Prompt This section directs the model to generate test oracles for the response field and is structured in three parts. First, the *Task introduction prompt* outlines the task. Next, individual *Single oracle prompts* are presented as questions, guiding the model to infer specific test oracles based on the datatype of the response field (Section §8.2.3). Finally, the *Response format prompt* instructs the model to return the test oracle values in a structured JSON format. Each single oracle prompt consists of a question (denoted as ⟨*oracle_prompt*⟩) and specifies the expected response in terms of JSON property name (⟨*oracle_name*⟩) and datatype (⟨*oracle_datatype*⟩). Additionally, instructions define a default JSON property value if no oracle is identified (⟨*no_value_instruction*⟩).

8.2.2 Process LLM Response

SATORI processes the responses of the LLM with a post-processing step to ensure syntactic accuracy, handling issues like transforming responses into valid JSON, standardizing formats, merging multiple JSON files, and removing extraneous text. Listing 8.3 shows the response returned by SATORI for the response field in Table §8.1.

```json
{
   "string_is_url": false,
   "string_is_numeric": false,
   "string_specific_values": [ "$", "$$", "$$$", "$$$$" ],
   "string_is_email": false,
   "string_is_date": false,
   "string_fixed_length": null,
   "string_is_time": false
}
```

Listing 8.3: Example of test oracles generated by SATORI.

Processed responses are then consolidated into a single CSV file, where each line represents a response field and its corresponding test oracles. This CSV file, along with the OAS specification, serves as input to an enhanced version of PostmanAsser-

tify [79]. PostmanAssertify converts the test oracles into executable JavaScript assertions compatible with Postman [78], a popular platform for API development and testing. PostmanAssertify produces a Postman collection that includes sample API requests for each one of the tested operations, attaching the test oracles to each request as executable assertions. For instance, PostmanAssertify would generate the assertion: `pm.expect(["$", "$$", "$$$", "$$$$"].includes(price)).to.be.true` for the response field in Table §8.1.

8.2.3 Target Oracles

SATORI supports all 49 unary invariants (*i.e.*, test oracles evaluating a single variable) supported by the dynamic approach AGORA+ [125]. We simplified these 49 AGORA+ invariants into 29 SATORI test oracles (showcased in Table §8.2), by merging similar numeric and format-specific checks. For instance, float and integer invariants of AGORA+ (like `OneOfFloat` and `OneOfScalar`) are combined into a single SATORI test oracle (*e.g.*, `number_specific_values`). Likewise, date and time format checks are unified, simplifying multiple formats into `string_is_date` and `string_is_time` test oracles. The resulting oracles assess properties such as string formats (*e.g.*, `string_is_url`), numerical boundaries (*e.g.*, `number_max_value`), and array sizes (`array_min_size`). SATORI allows user customization of the test oracles, adapting to specific requirements.

8.3 Evaluation

We aim to answer the following research questions:

- **RQ_1:** *How do different LLMs perform in generating test oracles with SATORI?* We analyze the performance and cost of different LLMs as the backbone of SATORI, considering model size, code specialization and reasoning capabilities.
- **RQ_2:** *What is the effectiveness of SATORI in generating test oracles and how does it compare against dynamic oracle generation approaches?* We compare the oracle generation capabilities of SATORI equipped with the LLM chosen in the previous RQ with respect to AGORA+ as a representative dynamic approach.
- **RQ_3:** *How effective is SATORI in detecting artificially seeded faults and how does it compare against dynamic approaches?* We evaluate the effectiveness of the oracles

Table 8.2: Test oracles supported by SATORI.

Datatype	**Test oracles**	
String	`is_url`	`is_numeric`
	`specific_values`	`is_email`
	`is_date`	`fixed_length`
	`is_time`	
Boolean	`always_true`	`always_false`
Number	`min_value`	`max_value`
	`specific_values`	
Array[*datatype*]	**Oracles of *datatype***	
	`min_size`	`max_size`
	`specific_sizes`	
Array[number]	**Oracles of Array[number]**	
	`ascending_order`	`descending_order`

generated by SATORI in detecting faults (mutations) in API responses, comparing it against AGORA+.

- **RQ$_4$:** *Can SATORI uncover real faults not discoverable by dynamic approaches?* We evaluate the ability of SATORI to detect real faults in API responses that are not identified by AGORA+.

8.3.1 Experiment 1: Test Oracle Generation

With this experiment, we aim to answer **RQ$_1$** and **RQ$_2$** by measuring the performance achieved by SATORI with different LLMs, and comparing it against dynamic oracle generation approaches.

Experimental setup

Next we describe the dataset used as a benchmark, the LLMs and baselines experimented with, and the metrics considered for evaluation.

Dataset As a contribution of this work, we present the OKAMI (Oracle Knowledge of API Methods for Innovation) dataset, a reliable benchmark for evaluating test oracle generation techniques for REST APIs. OKAMI is composed of 17 operations from 12

industrial APIs, 11 of which were used for the evaluation of the AGORA conference paper [28, 125]. We added five more operations from industrial APIs used in previous papers [122, 131, 244] and one from the Vimeo API [106], a popular video platform. When necessary, we updated the OAS specifications of these APIs according to the latest version of the web docs. We manually created the ground truth of all the oracles supported by SATORI for all the response fields of these API operations (*e.g.*, labeling `href` response fields as URLs), resulting in a dataset of 1,816 response fields and 10,645 test oracles. To avoid human bias or errors during the labeling process, each response field was checked by at least two authors, analyzing the API specification (OAS) or consulting the API providers in case of discrepancies. OKAMI is publicly available on Hugging Face [74] to serve as a benchmark for future studies.

LLMs and Baselines To answer $\mathbf{RQ_1}$, we selected a set of 21 LLMs according to different criteria, namely: (i) model size (from 1B to 33B parameters), (ii) code specialization (explicitly trained on code or not), and (iii) reasoning capabilities (explicitly trained to reason about their answers or not). We selected models from six different vendors, *i.e.*, Google, Meta, Microsoft, Alibaba, DeepSeek and OpenAI. We aim to analyze the impact of the aforementioned criteria on the performance for automated test oracle generation, and to select the best model for the subsequent experiments.

Regarding configuration parameters, we use the default settings for all models and a temperature of 0 (greedy decoding), thus making their outputs mostly deterministic and avoiding the need for multiple executions.

To answer $\mathbf{RQ_2}$, we compare the performance of SATORI against AGORA+, a dynamic approach that requires prior API execution to infer test oracles. In particular, we consider two versions of AGORA+: unary and binary. The unary version, denoted as AGORA+$_{\mathrm{U}}$, reports the same test oracles as SATORI (*i.e.*, those involving a single variable), while the binary version, denoted as AGORA+, generates also test oracles involving two variables (*e.g.*, `input.limit >= size(return.items[])`).

AGORA+ requires a sufficiently diverse set of API requests and responses to detect invariants effectively, with 50 being enough. For a fair evaluation, we used the same sets of 10K requests used in the AGORA+ paper. Since the performance of AGORA+ depends on these (randomly generated) sets of request-response pairs, we used 10 sets of 50 pairs for each operation and computed averages.

Metrics For both **RQ$_1$** and **RQ$_2$**, we report the overall precision, recall and F1-Score. For **RQ$_2$**, we report also the same metrics per oracle type for further analyses. For AGORA+ (binary), we report only the overall precision, since the OKAMI dataset contains only unary oracles. This is because creating the ground truth for all the possible binary oracles would lead to a combinatorial explosion, requiring the manual annotation of a dataset of up to tens of thousands of instances per API operation.

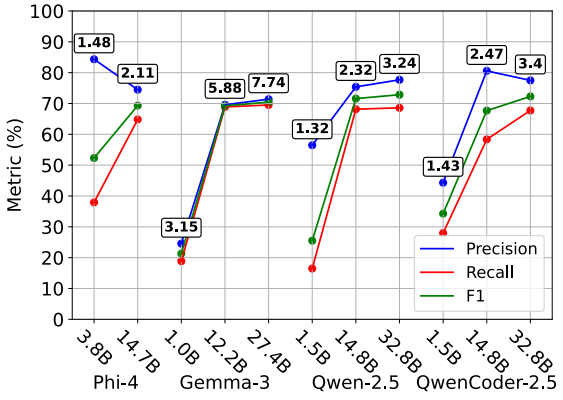
We also report the average time (in seconds) required to generate the test oracles for each response field of the API operations, and the cost (in dollars) for OpenAI models. The open source LLMs were executed on a single NVIDIA A100 GPU with 80GB of VRAM, while OpenAI models were invoked via their web API [75]. AGORA+ does not require GPU resources, therefore it was executed on a desktop computer equipped with an Intel i9-12900K @3.20GHz, 64GB RAM, and 2TB SSD running Windows 11.

RQ$_1$: Experimental results

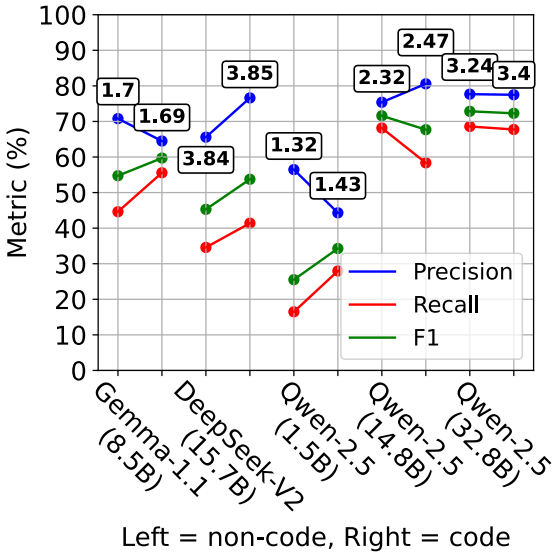
Figure §8.2 shows the precision, recall and F1-Score achieved by each model. The figure is split in four subfigures according to the criteria previously mentioned, *i.e.*, size (8.2a), code specialization (8.2b), reasoning capabilities (8.2c) and best model of each vendor considered (8.2d). The numbers shown on top of markers denote the average time (in seconds) to generate all possible oracles of a single response field in the OAS.

As expected, model size plays a role, as confirmed in Figure 8.2a. Here we evaluated four families featuring the same model in different sizes, namely, Phi-4 (3.8B, 14.7B), Gemma 3 (1B, 12.2B, 27.4B), Qwen2.5 and Qwen2.5-Coder (1.5B, 14.8B, 32.8B). As observed, models under 5B parameters exhibit significantly lower performance compared to the rest. However, it is worth noting that models between 12-15B parameters achieve performance (69.3-71.6%) mostly on par with that achieved by models with $\sim$30B parameters (70.4-72.2%). This may be relevant if cutting costs is desirable (*e.g.*, cheaper GPUs and faster inference times).

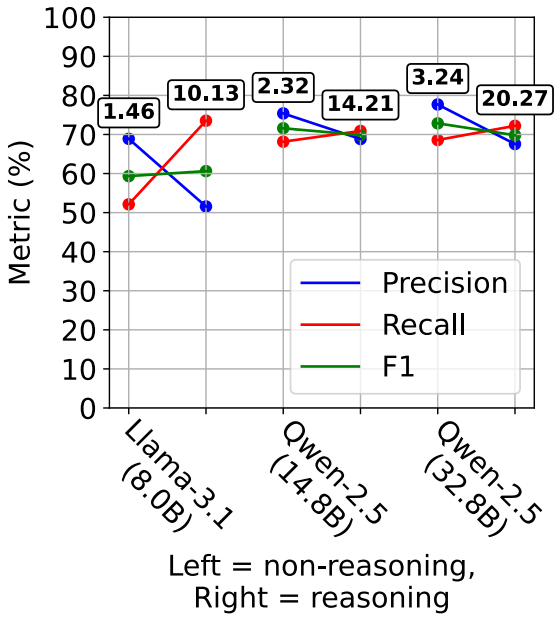
To evaluate the impact of code specialization on the task of oracle generation for REST APIs, we evaluated five models which offer a base version and a *code-specialized* version (*i.e.*, the same model further trained on code), namely, Gemma 1.1 (8.5B), DeepSeek-V2-Lite (15.7B) and Qwen2.5 (1.5B, 14.8B, 32.8B). As shown in Figure 8.2b, results are mixed. While code specialization seems to help for Gemma, DeepSeek and Qwen 1.5B models (7.6% higher F1 on average), it has a slightly negative impact on Qwen 14.8B (-3.9% F1) and no significant impact on Qwen 32.8B. Execution times are not significantly affected by code specialization.



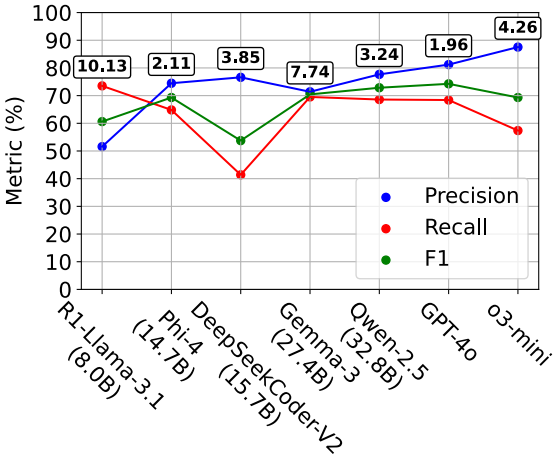
(a) Size.



(b) Code specialization.



(c) Reasoning capabilities.



(d) Best model of each vendor.

Figure 8.2: Precision, recall and F1-Score of each model evaluated according to several criteria. Numbers above markers denote average time (in seconds) to generate oracles for a single response field.

Regarding reasoning capabilities, we evaluated three models which offer versions distilled from (*i.e.*, fine-tuned with the answers generated by) the reasoning model DeepSeek R1 [198], namely, Llama 3.1 (8B) and Qwen2.5 (14.8B, 32.8B). Figure 8.2c highlights two interesting aspects. First, reasoning distillation does not significantly affect the overall F1-Score of models, although it worsens precision and improves recall, meaning that distilled models tend to generate more oracles, resulting in more false positives (wrong oracles) but also less false negatives (less correct oracles missed). On the other hand, reasoning models are significantly slower than their non-distilled counterparts, taking 6-7$\times$ longer to generate oracles.

Figure 8.2d shows the comparison between the best model of each vendor and the closed source models GPT-4o and o3-mini. The average F1-Score ranges from 53.7% (DeepSeekCoder-V2-Lite) up to 74.5% (GPT-4o). The low performance of DeepSeekCoder-V2-Lite for this task may be attributed to the fact that is a *Mixture of Experts* (MOE) model originally designed with over 200B parameters, thus its lite version may not be able to unleash the full potential of the MOE architecture.

Answer to **RQ$_1$**: How do different LLMs perform in generating test oracles with SATORI?

The best LLM for SATORI is GPT-4o, with an average F1-Score of 74.5% and the fastest execution time (1.96s per response field). Other cost-effective alternatives are Phi-4 (14.7B parameters, 69.3% F1-Score, 2.11s) and Qwen2.5 (32.8B parameters, 72.8% F1-Score, 3.24s).

RQ$_2$: Experimental results

Table §8.3 shows the performance, in terms of precision (P), recall (R) and F1-Score (F1), for each type of test oracle achieved by SATORI and by AGORA+$_U$. For each test oracle, we report the count of response fields of its datatype that are positive instances (the oracle is present) and negative instances (the oracle is absent). Table §8.3 combines primitive and array test oracles. For instance, `string_is_url` and `array_string_is_url` are reported as `string_is_url`. Table §8.4 presents the precision achieved by AGORA+ on each operation, with an overall precision of 68.8%, ranging from 40.7% to 100%.

Figures §8.3 and §8.4 show the overlapping between SATORI and AGORA+$_U$ in terms of positive instances (PI) of test oracles generated, grouped by test oracle (Figure §8.3) and API operation (Figure §8.4). Out of the 1167 positive instances of oracles generated, 374 (32%) were identified exclusively by SATORI, 329 (28.2%) only by AGORA+$_U$, and 464 (39.8%) by both approaches. Both approaches achieve similar

Table 8.3: Test oracle generation by SATORI and AGORA+$_U$. PI="Positive instances", NI="Negative instances", P="Precision", R="Recall", F1="F1-Score"

Oracle name	# PI	# NI	Total	AGORA+$_U$			SATORI (GPT4o)		
				P (%)	R (%)	F1 (%)	P (%)	R (%)	F1 (%)
string_is_url	220	804	1024	99.9	80	88.9	94.3	98.6	**96.4**
string_fixed_length	208	816	1024	81.9	83.6	**82.7**	59.6	28.9	38.9
number_min_value	206	26	232	81.4	43.6	56.7	82	91.9	**86.6**
string_specific_values	201	823	1024	47.6	42.5	44.9	65.1	85	**73.8**
number_max_value	82	150	232	66.2	44.2	52.9	90	44.4	**59.5**
string_is_numeric	82	942	1024	99.1	77.8	**87.2**	84.8	81.7	83.2
string_is_time	80	944	1024	100	55	70.9	97.4	93.8	**95.5**
string_is_date	62	962	1024	100	72.9	84.3	95.4	100	**97.6**
number_specific_values	54	178	232	65.6	66.3	**65.9**	78.6	20.4	32.4
array_min_size	34	82	116	55.7	62.5	**58.9**	100	8.8	16.2
array_max_size	24	92	116	43	57.4	**49.1**	0	0	-
array_specific_sizes	21	95	116	47.8	77.1	**59**	0	0	-
boolean_always_false	7	126	133	10.5	14.3	**12**	-	0	-
boolean_always_true	7	126	133	35.2	27.1	**30.3**	-	0	-
string_is_email	7	1017	1024	100	42.9	59	100	100	**100**
array_number_ascending_order	2	1	3	100	100	**100**	-	0	-
TOTAL	**1297**	**7184**	**8481**	**76.6**	**63.1**	**69.2**	**81.2**	**68.7**	**74.5**

performance in certain oracles (*e.g.*, `string_is_url` or `string_is_numeric`). In these cases, SATORI is more cost-effective, as it only requires access to API specification, unlike AGORA+$_U$, which needs a diverse test suite to verify API behavior. However, their performance differences show that they are complementary. SATORI outperforms AGORA+$_U$ in oracles where AGORA+$_U$ is limited by its heuristics. For example, SATORI can precisely infer enum values (`string_specific_values`) from descriptions (*e.g.*, lines 50-51 of Listing 8.2), whereas AGORA+$_U$ considers as free-text fields those response fields showing more than three unique values. Similarly, SATORI detects domain-specific minimum values (*e.g.*,-90 for `latitude`), whereas AGORA+$_U$ is limited to the (-1, 2) range. As shown in Figure §8.3, SATORI performed best in 7 test oracles, generating up to 100% of positive instances of a test oracle, while AGORA+$_U$ excelled in 8 oracles. Both were complementary in generating `string_is_numeric`.

On the other hand, in many cases, is necessary to execute the API to infer certain test oracles. For instance, regarding the `string_fixed_length` oracle, SATORI can infer the length of certain well known formats such as ISO country codes, but for more specific response fields, such as database IDs, it is necessary to execute the API and analyze the response fields values, which causes AGORA+$_U$ to obtain a higher performance. As shown in Figure §8.3, AGORA+$_U$ was the only approach able to generate positive instances of certain test oracles (`array_max_size`, `array_specific_sizes`,

Table 8.4: Test oracle generation by AGORA+. I="Invariants", P="Precision"

API - Operation	# I	P (%)
AmadeusHotel-getMultiHotelOffers	119.6	55
Deutschebahn-getStations	90.2	40.7
DHL-getLocation-finderV1Find-by-address	19	63.9
FDIC-searchInstitutions	235	48
GitHub-createOrganizationRepository	193.9	99.1
GitHub-getOrganizationRepositories	139	91.8
GitLab-getApiV4ProjectsIdBadges	4	91.7
GitLab-listProjectJobs	91	53.6
Marvel-getComicIndividual	102.9	52.6
OMDB-byIdOrTitle	16.4	92.2
OMDB-bySearch	5.2	96.7
Spotify-createPlaylist	41	100
Spotify-getAlbumTracks	58.9	90
Spotify-getArtistAlbums	49.7	88.6
Vimeo-search_videos	286.7	65.8
Yelp-getBusinesses	22.7	52.1
YouTube-listVideos	122.2	72.9
TOTAL	**1597.4**	**68.8**

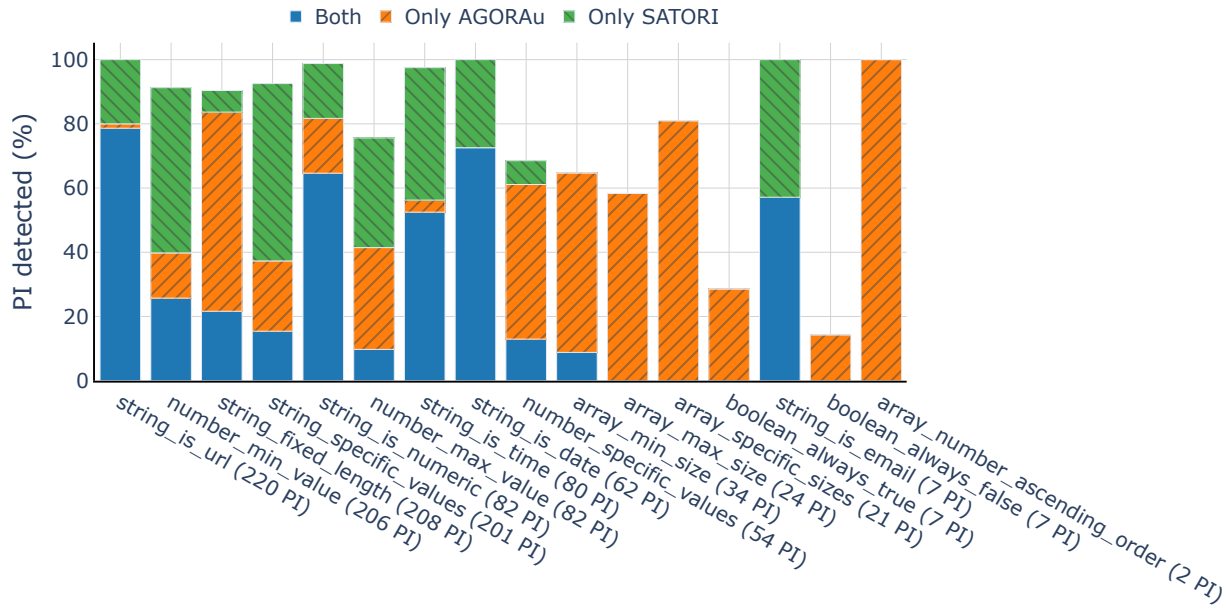


Figure 8.3: Oracle overlapping between SATORI and AGORA+$_U$.

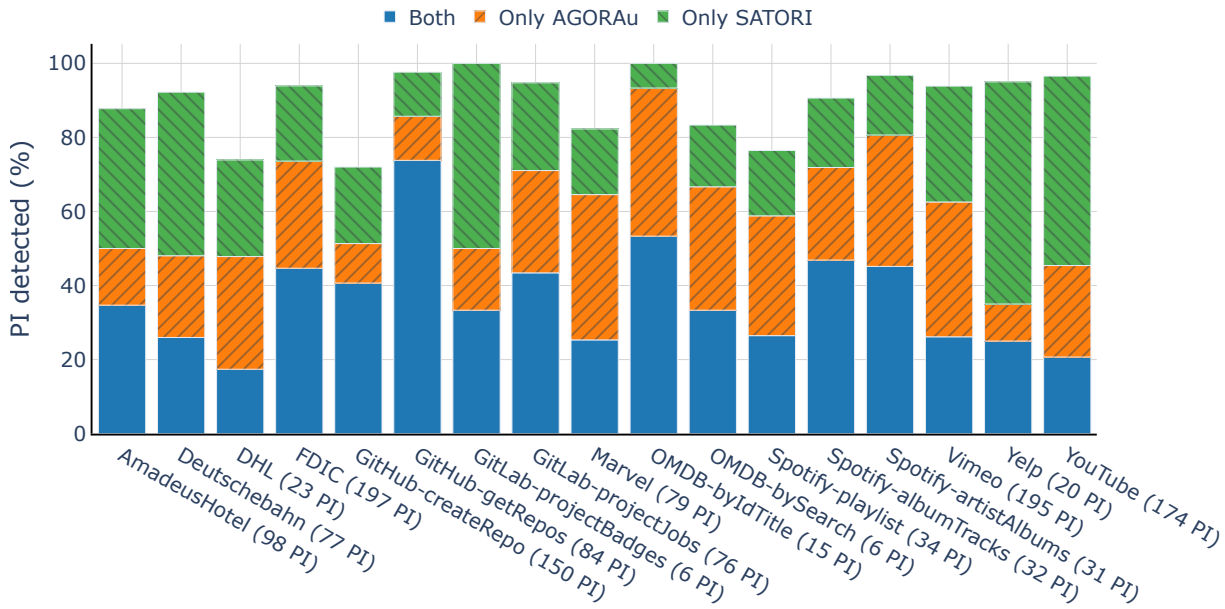


Figure 8.4: Per API oracle generation overlapping.

`boolean_always_true`, `boolean_always_false` and `array_number_ascending_order`). However, it is noteworthy that most of the test oracles in which AGORA+$_U$ is the best alternative are less common, with 6 of them having 34 positive instances or less.

Figure §8.4 shows that the number of positive instances of test oracles generated exclusively by SATORI was greater or equal than the number generated by AGORA+$_U$ in 7 out of 17 operations; and the number of test oracles generated by both approaches was greater or equal than those generated only by AGORA+$_U$ in 6 out of the 10 remaining operations.

Answer to **RQ$_2$**: What is the effectiveness of SATORI in generating test oracles and how does it compare against dynamic oracle generation approaches?

Overall, SATORI outperforms AGORA+$_U$ in terms of precision, recall and F1-Score. However, their overlapping in terms of generated oracles shows that both approaches are complementary, and that SATORI can generate a significant percentage of the test oracles without previously executing the API.

8.3.2 Experiment 2: Artificial Failure Detection

This experiment aims to answer **RQ$_3$** by comparing the effectiveness of the oracles generated by SATORI and AGORA+ in detecting artificial failures.

Experimental setup

Next, we describe the setup for Experiment 2, detailing the baseline, the test oracles implemented, the test case selection criteria, the mutant generation process and the metrics applied to measure performance.

Baseline We evaluated SATORI (using GPT4o), AGORA+, and their combination in detecting API failures. We distinguished failures identified by AGORA+$_U$ and the binary oracles of AGORA+ (*i.e.*, mutants killed by the binary invariants of AGORA+ that were not detected by AGORA+$_U$). We also reported the failures detected by the ground truth, which serves as the upper bound for unary approaches.

Test oracles For each API operation of Experiment 1, we selected the valid test oracles generated by each approach (*i.e.*, confirmed as true positives) which were automatically transformed into executable assertions using PostmanAssertify [79].

Test cases For each API operation, we randomly selected 1K API responses from the set of 10K test cases meeting the following constraints: (1) they were not part of the 50-requests set used as input for AGORA+, (2) they contained at least one result item (we cannot apply mutation operators on empty arrays), and (3) they revealed no failures (Section §8.3.3). Some failure-revealing test oracles (17 oracles from 6 APIs) exposed such a high number of failures in the unmutated API responses that they had to be discarded, despite being true positives.

Mutants To mutate API responses, we systematically seeded *errors* in API responses using the same JSONMutator [59] configuration as in the evaluation of AGORA+ (Chapter §7).

Metrics For each mutated API response of the 1K test cases test suites, we ran the assertions and marked the failure as detected if at least one the test assertions failed. Then, we computed the Failure Detection Ratio (FDR) achieved by the approach on the test suite. We repeated the mutation process 100 times per approach (SATORI, AGORA+) and for the ground truth to minimize the effect of randomness, computing the average percentage of failures detected. In total, the results are based on 1.7M

seeded errors: 17 operations x 1k API responses x 100 repetitions, executed three times (approaches and ground truth).

Experimental results

Table §8.5 shows the number of assertions generated (*i.e.*, true positives) and the Failure Detection Ratio achieved by SATORI, AGORA+, and the combination of both. SATORI achieved an average FDR of 31.1%, ranging from 14.7% to 56.8%. AGORA+$_U$ achieved an average FDR of 38.5%, ranging from 11.5% to 68.1%. The binary oracles of AGORA+ leaded to an average FDR of 51%, increasing the FDR of AGORA+$_U$ an average of 12.5 percentage points, ranging from 0 to 58.1 percentage points. The combination of both SATORI and AGORA+ achieved a FDR of 55.1%, ranging from 20.6% to 93%.

Table 8.5: Failure Detection Ratio per API operation achieved by each approach. A="Assertions"

	AGORA+$_U$		AGORA+ binary only		AGORA+		SATORI		SATORI + AGORA+	
API - Operation	**#A**	**FDR (%)**	**#A**	**FDR (%)**	**#A**	**FDR (%)**	**#A**	**FDR (%)**	**#A**	**FDR (%)**
AmadeusHotel	47	56.4	22	3.8	69	60.2	70	48.5	107	66.9
Deutschebahn	32	19	6	0.8	38	19.8	53	16.8	73	26.5
DHL	10	45.6	3	3.9	13	49.4	10	34.8	19	51.2
FDIC	107	43.8	30	3.4	137	47.2	114	27.4	187	53.2
GitHub-createRepo	75	34.9	117	58.1	192	93	92	39.5	226	93
GitHub-getRepos	69	40.1	61	24.7	130	64.9	72	37.8	143	65.5
GitLab-getProjectsIdBadges	3	30.4	0	0	3	30.4	5	35.8	6	49.4
GitLab-listProjectJobs	42	25.5	11	11.6	53	37.2	48	23.9	83	39.8
Marvel	40	30.3	16	6.3	56	36.6	31	19	69	39.3
OMDB-byIdTitle	14	33.8	1	2.4	15	36.2	9	17.7	16	38
OMDB-bySearch	4	18.6	1	2	5	20.6	2	14.7	5	20.6
Spotify-playlist	18	46.7	22	46.1	40	92.8	15	28.9	46	92.8
Spotify-albumTracks	23	65.6	19	1.6	42	67.2	21	53.9	48	68.4
Spotify-artistAlbums	22	68.1	21	7.4	43	75.6	19	56.8	48	78.3
Vimeo	104	23.2	95	23.9	199	47.1	111	20.5	255	51.6
Yelp	7	11.5	5	12.1	12	23.6	17	15	24	31
YouTube	53	60.8	37	5.3	90	66.1	125	37.3	183	70.7
TOTAL	**670**	**38.5**	**467**	**12.5**	**1137**	**51**	**814**	**31.1**	**1538**	**55.1**

Figure §8.5 shows the overlapping, in terms of FDR, between SATORI and AGORA+. For the mutants that were only detected by AGORA+, we distinguish between those detected by AGORA+$_U$ and those that required resorting to binary oracles. SATORI detected 53.1% of all the failures detected by AGORA+ (66.1% of those detected by AGORA+$_U$), plus those failures that were detected only by SATORI, which shows that an important percentage of the failures can be detected without previously executing the APIs to infer test oracles. The horizontal red lines in the graph indicate the FDR achieved by the ground truth, which represents the upper bound for the FDR

attainable using the unary oracles currently supported by both approaches. On average, the FDR of the unary oracles falls 3.3 percentage points below that of the ground truth, highlighting that combining both approaches can yield results that closely approximate the ground truth.

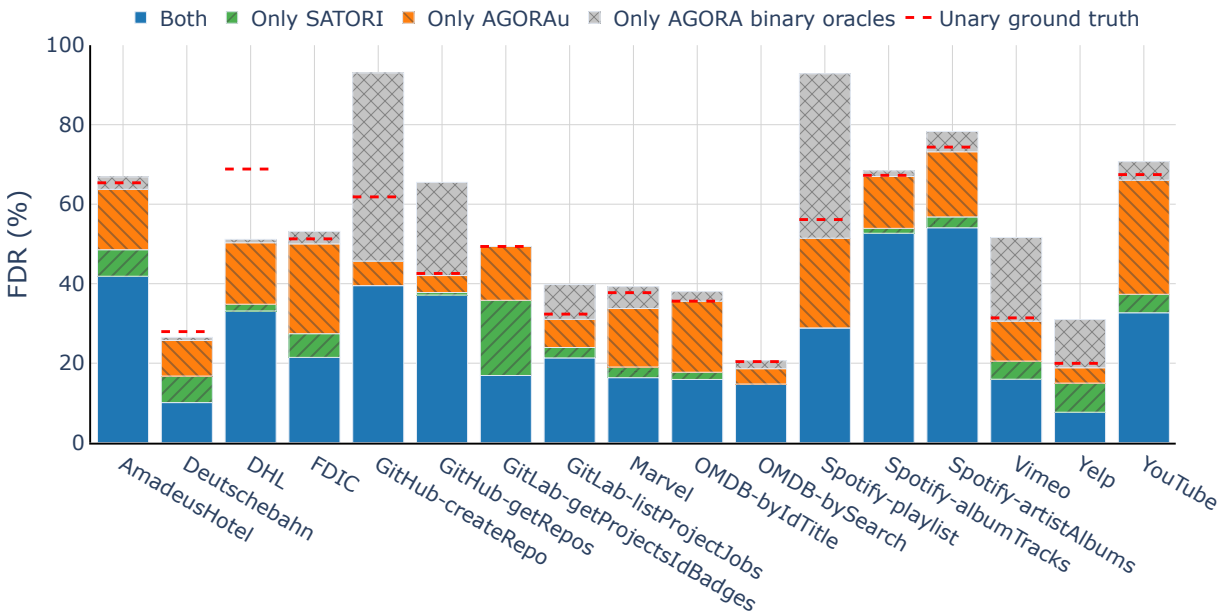


Figure 8.5: FDR overlapping between SATORI and AGORA+.

The FDR improvement achieved by binary test oracles is particularly notable in GitHub, Spotify-playlist, Vimeo, and Yelp. This improvement is primarily driven by the presence of numerous equality (*e.g.*, `input.description == return.description`) and substrings invariants (*e.g.*, `return.name is a substring of return.full_name`) in GitHub, Spotify, and Vimeo. Additionally, arithmetic comparisons (*e.g.*, `return.total >= size(return.businesses[])`) contribute to the improvement in Vimeo and Yelp.

Answer to $\mathbf{RQ_3}$: How effective is SATORI in detecting artificially seeded faults and how does it compare against dynamic approaches?

SATORI detected more than half of the failures detected by AGORA+ without previously executing the API under test, with a FDR ranging between 14.7% and 56.8%. More importantly, both approaches are complementary, achieving a combined FDR of 55.1%.

8.3.3 Experiment 3: Real Failure Detection

This experiment addresses **RQ$_4$** by comparing the effectiveness of SATORI and AGORA+ in detecting real failures.

Experimental setup

We compared the performance of SATORI (using GPT4o) and AGORA+ in detecting real failures by converting the valid test oracles reported by each approach into executable assertions using PostmanAssertify. These assertions were then executed on the original dataset of 10K API requests from Experiment 1. Violations of these test oracles revealed real bugs. For AGORA+, we repeated the experiment 10 times, each using a different random subset of 50 requests for invariant detection and reported the number of executions in which AGORA+ identified each bug. AGORA+ can also detect bugs when an invariant exposes inconsistencies. For example, in the Amadeus Hotel API, the invariant `return.code one of {MISCELLANEOUS, SERVICE_CHARGE, TOTAL_TAX}` indicated inconsistencies in the `code` response field, which, according to the specification, should contain country codes.

Experimental results

Table §8.6 shows the approaches that detected each bug, with SATORI identifying 18 bugs, while AGORA+ detected 13 (but only 7 of them were detected in all of its 10 executions). In total, both approaches detected 22 unique bugs across 8 APIs, 6 of these bugs (bugs 13, 16, 17, 18, 19, and 20) were already detected in the AGORA conference paper [125], 3 of the new bugs (bugs 14, 21, and 22) are confirmed by developers, and bug 14 has been fixed. These bugs are grouped into the following four categories:

Category 1: Invalid String Formats. These bugs occur when a string response field is expected to follow a specific format (*e.g.*, country codes, numbers, or timestamps), but the API returns values that deviate from this format. For example, in the "projectBadges" operation of the GitLab API (bug 14), SATORI and AGORA+ identified instances where the `rendered_image_url` response field, expected to be URL, instead returned invalid URLs containing whitespaces. This bug has been confirmed and fixed by the API providers.

Category 2: Invalid Enum Values. These bugs are found when a response field is expected to hold specific string values (often outlined in the specification), but the API returns either undocumented values (bugs 9, 10, 11, 12, 15, 20, and 21) or entirely dif-

Table 8.6: Faults detected by SATORI and AGORA+.

#Bug	Cat.	API-Operation	AGORA+	SATORI
1	1	AmadeusHotel	1	✓
2	2	Deutschebahn	10	✓
3	1	Deutschebahn		✓
4	1	Deutschebahn		✓
5	1	Deutschebahn	1	✓
6	2	FDIC	10	✓
7	2	FDIC	10	✓
8	2	FDIC	10	✓
9	2	FDIC		✓
10	2	FDIC		✓
11	2	FDIC		✓
12	2	FDIC		✓
13	4	GitHub-createRepo	9	
14	1	GitLab-projectBadges	2	✓
15	2	Marvel	1	✓
16	3	Marvel	10	
17	1	Marvel	10	
18	1	Marvel	6	
19	1	Marvel		✓
20	2	OMDB-bySearch	10	✓
21	2	Vimeo		✓
22	3	Vimeo		✓

ferent ones (bugs 2, 6, 7, and 8). For instance, in the FDIC OAS, the `CONSERVE` and `LAW_SASSER_FLG` fields are expected to use numerical flags ("1" or "0"), yet the API returns "Y" or "N" (bugs 6 and 7). Similarly, in the Vimeo API (bug 21), while the specification lists 14 valid values for the account field, the API also returns an undocumented value ("custom"). The Vimeo API providers confirmed this inconsistency and created an internal issue to update the documentation.

Category 3: Numerical Constraints. These bugs occur when a numerical response field does not comply with expected constraints, such as minimum or maximum values. For example, in the Vimeo API, the `field_of_view` response field should range from 30 to 90. However, SATORI detected three videos with values exceeding the upper limit (bug 22). This bug has been confirmed by the API providers, and they have created an internal issue to update the API documentation.

Category 4: Binary Test Oracles. These bugs occur when a confirmed test oracle involving two variables is violated, making them detectable only by AGORA+. The only bug of this category (bug 13) was found in the "GitHub-createRepo" operation, where the violation of the invariant `input.license_template==return.license.key` revealed 15 cases of repositories being created with incorrect licenses.

Answer to **RQ$_4$**: Can SATORI uncover real faults not discoverable by dynamic approaches?

SATORI effectively detects real API failures, uncovering 18 failures across 7 APIs, compared to 13 failures identified by AGORA+. Although 4 of the failures found by AGORA+ were not detected by SATORI, highlighting their complementary nature, SATORI ultimately uncovered more unique failures than AGORA+.

8.4 Threats to Validity

In this section, we explore the potential threats to the validity of our results, along with the actions taken to mitigate them.

Internal validity. *Are there factors that might affect the results of our evaluation?* For our experiments, we used the OAS specifications of the APIs under test. We used the OAS specifications provided in the AGORA+ supplementary material [28], updating them to reflect the latest version of the web documentation. It is possible that these specifi-

cations have errors and deviate from the API specification. To mitigate this threat, the specification files were reviewed by at least two authors.

The effectiveness of AGORA+ largely depends on the diversity of the input test suite, we used the same test suites as in the AGORA+ paper [28]. The division of these test suites into random sets may also affect the results. To mitigate this threat, we performed this division 10 times, computing the average performance between runs. To address the potential variability of SATORI, we use the default settings for all models and a temperature of 0 (greedy decoding), making the outputs of the models mostly deterministic.

Manually creating the ground truth for all the APIs may be affected by human biases or errors. To mitigate this, each classified instance of these datasets was checked by at least two authors, analyzing the API documentation, or consulting the API providers in case of discrepancies.

External validity. *To what extent can we generalize the findings of our investigation?* We evaluated SATORI using 21 different LLMs and a set of 17 operations from 12 APIs; therefore, our conclusions may not fully generalize beyond this scope. To mitigate this threat, we selected LLMs of varying sizes from different providers, along with a set of widely-used APIs spanning diverse domains and sizes, and used in related studies.

The test oracles supported by SATORI may not generalize beyond the selected APIs. To mitigate this threat, we based these oracles on the unary invariants supported by AGORA+, derived from a systematic analysis of a dataset comprising 40 realistic APIs (702 operations) across diverse domains [122]. However, we note that this list of test oracles is not exhaustive, and SATORI is designed to be easily extended with additional oracles.

8.5 Conclusions

This chapter introduces SATORI, a black-box static approach for generating test oracles for REST APIs from their specification using LLMs. SATORI analyzes the response fields of an API operation, providing them as inputs to a target LLM, which infers a set of pre-defined test oracles. SATORI then converts these inferred oracles into a machine-readable format compatible with an extended version of PostmanAssertify, a tool that transforms the oracles into executable Postman assertions. This integration

makes SATORI readily applicable for practical use.

Evaluation results on a set of 17 operations from 12 industrial APIs show that SATORI can generate hundreds of valid test oracles per operation without executing the API. SATORI achieved an F1-Score of 74.5%. The differences in performance between SATORI and AGORA+ reveal complementary strengths, each approach excels at detecting distinct types of test oracles, with the combination of them achieving a Failure Detection Ratio of 55.1%. SATORI identified 18 bugs across 7 widely used industrial APIs, leading to bug fixes and documentation updates in GitLab and Vimeo, respectively. Operating in black-box mode, SATORI is also easy to integrate with API testing tools that support OAS.

Part IV

Final Remarks

9

Conclusions and Future Work

People think of education as something that they can finish.

Isaac Asimov,
American writer and professor

This chapter concludes the dissertation. Section §9.1 presents general conclusions based on the results obtained throughout the PhD. Section §9.2 examines these results in relation to the research questions outlined in this dissertation. In Section §9.3, we highlight the main limitations of the proposed solutions. Finally, Section §9.4 explores potential directions for future research.

9.1 Conclusions

In this dissertation, we have tackled two major limitations of existing tools for the automated testing of REST APIs: the automated generation of realistic input values and the automated generation of test oracles. The primary contributions of this work are ARTE, an approach that automatically generates realistic input values by analyzing API specifications and previous API responses; AGORA+, an approach that derives test oracles by analyzing the results of past API executions; and SATORI, an approach that generates test oracles from API specifications, without requiring any prior API execution.

To demonstrate the feasibility and effectiveness of these techniques, we presented extensive experimental results. Our approaches uncovered over 40 real bugs in widely used commercial REST APIs, leading to documentation updates and fixes in platforms such as GitHub, Vimeo, GitLab, Amadeus Hotel, and YouTube, among others.

All three approaches operate in a black-box context and can be applied to any API that adheres to REST principles, leveraging the standardization provided by the OpenAPI Specification (OAS). They have been implemented as practical tools that can be integrated into existing API testing pipelines, significantly improving both the validity of generated API requests (thanks to ARTE) and the fault-detection capabilities (enabled by the test oracles produced by AGORA+ and SATORI).

Overall, the contributions presented in this PhD pave the way towards more effective and automated testing of REST APIs, reducing the manual effort required to create high-quality test inputs and oracles, and ultimately improving the reliability of modern web services.

9.2 Discussion

In this section, we analyze how our contributions advance the state-of-the-art by addressing the Research Questions introduced in Section §1.3.

RQ$_1$ *How effectively can realistic test input values be automatically generated for REST APIs?* ARTE outperformed all existing baselines for automated input generation for REST APIs used in the literature. It successfully generated realistic input values for 64.9% of the parameters, produced up to 31.3 percentage points more valid API re-

quests than the best-performing baseline, and detected a total of 149 failures. A key strength of ARTE lies in its ability to learn from previous API responses by automatically synthesizing regular expressions, which allows it to refine and expand the set of generated input values over time.

Despite these results, there is still room for improvement, particularly in generating values for highly API-specific parameters (*e.g.*, database IDs) or confidential fields (*e.g.*, email addresses), which often require access to the database of the system under test. Future extensions of ARTE could explore incorporating additional input generation techniques, such as leveraging LLMs, and enhancing feedback mechanisms—*e.g.*, learning from responses to infer new valid values or analyzing error messages to avoid client-side errors. These enhancements could further increase the ability of ARTE to generate valid and diverse API requests, thereby improving its fault-detection capabilities.

RQ$_2$ *To what extent is it possible to automatically generate test oracles for REST APIs?* Our evaluation results show that AGORA+ and SATORI can each generate up to hundreds of valid test oracles per API operation, demonstrating strong performance in terms of both precision and execution time. The two approaches are complementary, with each excelling at different types of test oracles. For instance, certain oracles, such as checking whether a string field should be a valid URL, can be derived directly from the API specification, without requiring any prior API executions. These are efficiently handled by SATORI, making it a cost-effective solution. In contrast, other oracles, such as inferring precise string lengths or value ranges, typically require observing API behavior and can only be derived using AGORA+. Notably, AGORA+ is the only approach capable of generating relational oracles that involve multiple input or output variables.

Both techniques have proven effective at uncovering real failures in industrial systems, going beyond superficial issues like syntax violations or server errors. Collectively, they have detected over 40 previously unknown bugs in widely used commercial APIs. These bugs span a variety of failure types, including incorrect documentation, missing constraints, and unhandled edge cases, highlighting the practical utility of automatically generated test oracles in improving the reliability of real-world APIs. Despite these results, there remains room for improvement. For example, AGORA+ depends on a diverse test suite and currently supports test oracles involving at most two variables, limiting its ability to capture more complex relationships. SATORI, on the other hand, can generate test oracles directly from the OAS specification without

requiring prior API execution, but it currently supports only single-variable oracles.

9.3 LIMITATIONS

We now discuss the primary limitations of our work.

- *Testing sequences of operations.* The techniques proposed in this dissertation focus on isolated API operations. However, they provide a foundation that could be extended to support *sequences of operations*, which are common in real-world usage scenarios. For example, ARTE could be enhanced to track and reuse values returned by earlier API calls, such as resource identifiers (*e.g.*, user IDs), or timestamps, as inputs for subsequent operations. This would enable the generation of realistic multi-step test cases that better reflect actual API workflows.

  Similarly, AGORA+ and SATORI could be adapted to generate sequence-level test oracles that verify the consistency and correctness of state transitions across multiple operations. For instance, a test oracle could check that certain invariants are preserved throughout a sequence, for example, that the total number of items returned by `GET /cart` reflects the cumulative effects of prior `POST /cart/items` and `DELETE /cart/items/{id}` calls.

  Extending the current techniques to support such scenarios would significantly increase their applicability to complex workflows and stateful APIs, enabling more thorough testing of behavior over time and across different API endpoints.

- *Manual effort required.* While the proposed approaches automate significant aspects of REST API testing, they still involve a degree of manual effort that could be reduced with relatively modest engineering improvements. For instance, the test oracles and the bugs detected by AGORA+ and SATORI must currently be manually reviewed and classified by the user. This step, although necessary to ensure accuracy, could be streamlined with semi-automated validation workflows or user interfaces that guide the classification process. Additionally, AGORA+ requires users to supply diverse test suites as input, which must be manually crafted or sourced from existing test collections. Automating the generation or augmentation of such test suites could further reduce the burden on users.

- *SATORI is limited to unary oracles.* At present, SATORI supports only unary test oracles, meaning it can only reason about individual variables in isolation. This

limits its ability to detect issues involving relationships between multiple variables in the same response. For example, SATORI cannot express oracles like "if `start_date` is present, then `end_date` must also be present and must occur after `start_date`", or "the `total_price` field should equal the sum of individual item prices in the items array". Adding support for relational oracles would significantly enhance the ability of SATORI to detect logical and semantic inconsistencies in complex API responses.

- *Platform dependence.* Both AGORA+ and SATORI currently generate executable test oracles using PostmanAssertify, which outputs assertions in a format compatible with the Postman API testing platform. While Postman was chosen due to its popularity, user-friendliness, and widespread adoption in industry, this choice introduces a level of platform dependence. The current implementation limits integration with alternative testing frameworks or programming environments. However, this is not a fundamental limitation of the approach but rather an engineering choice. Extending the tools to support other formats—such as JUnit (Java) [60], pytest (Python) [80], or REST Assured [83]—would be straightforward and would broaden the applicability of the approaches to a wider range of development ecosystems.

9.4 FUTURE WORK

This section outlines several future research directions that build upon the contributions of this thesis. These potential lines of work aim to address the limitations identified in Section §9.3, while also enhancing the usability, automation, complementarity, and fault-detection capabilities of our proposed approaches.

- *Enhancing ARTE with LLMs and Black-box coverage analysis.* One key avenue for future work is the improvement and extension of ARTE by incorporating LLMs to generate higher-quality test inputs. While recent studies have explored the use of prompt engineering for generating input values for REST APIs [217], they rely on static prompts and lack dynamic adaptation to the feedback provided by the API. In contrast, a more advanced integration could involve analyzing the black-box coverage achieved during testing [239]. Black-box coverage refers to the extent to which the generated test inputs cover different combinations of input and output space. By evaluating how well the current inputs explore the

response space of the API, ARTE could adapt its generation strategy to ensure more exhaustive testing. This would be coupled with adaptive prompt generation and response-aware feedback loops. Here, the system would analyze error messages returned by the API to infer the root causes of invalid requests—such as missing fields, type mismatches, or value constraints—and automatically refine subsequent test inputs to avoid repeating these errors. By focusing on areas where previous inputs have not fully explored the possible response space or where issues were identified, the system would improve the realism and validity of the test inputs, thus reducing manual effort and enhancing test coverage by exploring a broader and more meaningful set of inputs. Additionally, ARTE could be extended to support sequences of operations by reusing values returned in earlier responses (*e.g.*, user IDs) as inputs for subsequent calls. This would enable the generation of realistic multi-step test scenarios that resemble real-world API usage more closely.

- *Automating Test Oracle Classification for Improved Efficiency.* The manual effort required to classify the test oracles generated by AGORA+ and SATORI could be significantly reduced through the application of automatic classification techniques. Several researchers have already built on our work in this area. For instance, Fischer and Klammer [182] used LLMs to automatically classify the invariants reported in the AGORA+ dataset [28], achieving an accuracy of 80.7%. Building on this, a more sophisticated approach could involve using the classified test oracles from both the AGORA+ [28] and OKAMI [74] datasets, each containing over 10k instances, as training data for an automated classification system. This would enable the automatic clustering or ranking of test oracles based on key characteristics, such as their potential to reveal faults. The system could then prioritize for review only those oracles whose classification is uncertain or which are most likely to identify a fault, thereby reducing the manual effort required for validation.

- *Integrate test input generation techniques with test oracle generation techniques.* Lastly, the three approaches could be integrated to further enhance their performance and capabilities. By combining ARTE with the test oracle generation methods, we could significantly increase the diversity of the API requests, while also guiding the testing process towards areas more likely to reveal faults. ARTE could automatically identify test oracles that indicate gaps in the diversity of the test suite. For example, if a test oracle like `return.repository.visibility == “public”` indicates that the API has only returned public repositories so far, ARTE could

generate new input values that specifically target private repositories, thus expanding the range of tested scenarios. This would ensure that the API is not only tested for common cases but also for edge cases that might not be covered by the initial test set. Furthermore, ARTE could leverage test oracles that have already been confirmed as true positives. By generating new input values designed to violate these oracles, ARTE could trigger scenarios that more likely expose faults. For example, if a test oracle confirmed as a true positive states that a specific response field should have a minimum value, ARTE could generate a series of input values that progressively test for edge cases, trying to violate that test oracle, which would reveal the presence of a fault. This would not only enhance fault detection but also refine the overall testing process by focusing on the areas that are most likely to reveal defects.

Part V

Appendices

A

Supplemental Material for: ARTE: Automated Generation of Realistic Test Inputs for Web APIs

Authors: *Juan C. Alonso, Alberto Martin-Lopez, Sergio Segura, José María García, and Antonio Ruiz-Cortés.*

Publication type: *Appendix of Journal publication.*

Journal: *IEEE Transactions on Software Engineering (Volume: 49, Issue: 1, 01 January 2023).*

DOI: `https://doi.org/10.1109/TSE.2022.3150618`

Rank: *3/110 (Top 2.27%, **Q1**).*

Supplemental Material for: ARTE: Automated Generation of Realistic Test Inputs for Web APIs

Juan C. Alonso, Alberto Martin-Lopez, Sergio Segura, *Member, IEEE,* José María García and Antonio Ruiz-Cortés, *Member, IEEE*

APPENDIX A
WORKFLOW OF ARTE

Algorithm 1 depicts the complete workflow of ARTE for an API operation. ARTE starts by processing the information of the parameters selected by the user (line 1). For each parameter (line 2), ARTE extracts a set of predicates from the knowledge base (line 3). These predicates (along with the syntactic constraints from the API specification), are used to extract a set of input values for each parameter (line 5).

ARTE automatically generates regular expressions that are used to filter out syntactically invalid input values and to extract new input values by leveraging additional predicates. These regular expressions are generated from a set of values classified as valid or invalid. This classification can be made either manually or (as in our experimentation) automatically, and it is performed in every iteration of the workflow (lines 7-17), up to a maximum number of times, denoted by the $maxIterations$ parameter, which serves as a stopping criterion. As an example, in our experiments, we generated 1000 API requests per operation, and performed the classification of input values every 50 requests, resulting in a total of 20 iterations.

In each iteration, for each parameter (line 9), its previously obtained input values are classified into valid (line 10) and invalid (line 11). In our experiments, this classification is done automatically based on the API responses obtained (valid values obtain successful responses, while invalid values obtain erroneous ones). If the number of valid ($param.validValues$) and invalid ($param.invalidValues$) values exceed a certain threshold (line 12), denoted as $minSize$ and set to 5 in our experiments, then ARTE attempts to generate a regular expression from them, if the maximum number of tries $maxTries$ (2 in our evaluation) has not been reached (line 13). The generated regular expression is used to filter the already existing set of input values of the parameter (line 14). At the end of the execution, the list of parameters with their extracted input values is returned (line 19).

A.1 Processing test parameters

The workflow of this phase is shown in Algorithm 2. ARTE iterates over all the parameters of an operation in the API specification (line 2), filtering out those parameters that are not in the set of $semanticParameters$ (line 3), i.e., only keeping those for which the user desires to generate input values. Next, we detail the process followed for every parameter in the list.

In the first place, if the parameter name is one character long (line 4), ARTE tries to infer the full parameter name from its description using Natural Language Processing (NLP) techniques (line 5). More specifically, ARTE applies part-of-speech tagging [1], stop words removal [2], and lemmatization [3], selecting the shortest noun from the list that begins with the same letter as the one-character original name. If a match is found (line 6), the parameter name is updated (line 7).

- *J.C. Alonso, A. Martin-Lopez, S. Segura, J.M. García, and A. Ruiz-Cortés are with the Smart Computer systems Research and Engineering Lab (SCORE) and the Research Institute of Informatics Engineering (I3US), Universidad de Sevilla, Spain. E-mail: {javalenzuela, alberto.martin, sergiosegura, josemgarcia, aruiz}@us.es*

Algorithm 1: Complete workflow of ARTE

Input: $operation$: Operation of an API specification
Input: $semanticParameters$: Parameters for which ARTE will extract input values
Input: n: Number of predicates selected when searching for predicates
Input: $minSupport$: Min. support of predicate
Input: $minValues$: Min. number of unique parameter values
Input: $maxTries$: Max. number of consecutive attempts to generate a regex
Input: $minSize$: Min. number of valid and invalid values for generating regex
Input: $minRecall$: Min. recall for accepting regex
Input: $maxPredicates$: Max. number of additional predicates to extract per parameter
Input: $maxIterations$: Max. number of classifications as valid or invalid values
Output: $params$: Parameters with their input values

```
1  params ← PROCESSTESTPARAMETERS(operation, semanticParameters)
2  foreach param in params do
3      param.predicates ← SEARCHFORPREDICATES(param, n, minSupport)
4  end
5  params.inputValues ← SEARCHFORTESTINPUTS(minValues, params)
6  iterations ← 0
7  do
8      iterations ← iterations + 1
9      foreach param in params do
10         param.validValues ← param.validValues ∪ GETVALIDVALUES(param)
11         param.invalidValues ← param.invalidValues ∪ GETINVALIDVALUES(param)
12         if (|param.validValues| ≥ minSize ∧ |param.invalidValues| ≥ minSize) then
13             if param.numTries < maxTries then
14                 param.inputValues ← AUTOMATED-GENERATIONOFREGEX(param, param.inputValues, minRecall, maxPredicates)
15             end
16         end
17     end
18 while iterations < maxIterations;
19 return params
```

Algorithm 2: Processing test parameters

Input: *operation*: Operation of an API specification
Input: *semanticParameters*: Parameters for which ARTE will extract input values
Output: *params*: List of processed parameters

```
1  params ← ∅
2  foreach param in operation.parameters do
3      if param.name in semanticParameters then
4          if LENGTH(param.name) = 1 then
5              newName ← APPLYNLP(param.name,
                 param.description)
6              if newName ≠ ∅ then
7                  param.name ← newName
8              end
9          end
10         param.predicates ← ∅
11         param.inputValues ← ∅
12         param.numTries ← 0
13         param.validValues ← ∅
14         param.invalidValues ← ∅
15         param.numAdditionalPredicates ← 0
16         params ← params ∪ param
17     end
18 end
19 return params
```

During this step, ARTE also initializes the properties of each parameter, namely, the set of predicates associated to it (line 10), the set of extracted input values (line 11), the current number of tries to generate a regular expression (line 12), the set of input values classified as valid (line 13) and invalid (line 14) and the number of additional predicates extracted for each parameter (line 15). The `predicates` property contains the set of predicates extracted for a parameter during the "Search for predicates" and "Automated generation of regular expressions" steps, the `inputValues` set contains the test inputs extracted in the "Search for test inputs" and "Automated generation of regular expressions", and the properties initialized in lines 12 to 15 are used during the "Automated generation of regular expressions" step.

Finally, ARTE adds the parameter to the set of parameters for which input values will be generated (line 16). Once ARTE has iterated over all the parameters of the operation, it returns the set of parameters (line 19) that will be processed in subsequent phases.

A.2 Search for predicates

In this step, a user-defined knowledge database (e.g., DBpedia) is queried to obtain predicates that are representative of the target input parameters. More specifically, a set of keywords are obtained for each parameter, and these are used to obtain the representative predicates. The keywords are extracted by applying a set of matching rules to the parameters' names and descriptions.

```
SELECT DISTINCT ?predicate WHERE {
   ?predicate a rdf:Property

   FILTER regex(str(?predicate), 'keyword', 'i')
} ORDER BY strlen(str(?predicate))
```

Listing 1. SPARQL query used to search for predicates.

Algorithm 3 illustrates the process. ARTE iterates over the matching rules, ordered by priority (line 1). For each rule, it extracts one or more keywords from the parameter name and description (line 2) and tries to obtain a list with a maximum of n predicates containing such words (line 4) by executing the SPARQL query shown in Listing 1. For each of the predicates obtained (line 5), its support (i.e., number of unique RDF triples containing it) is computed (line 6), and if it is greater than a certain threshold $minSupport$ (line 7), set to 20 in our evaluation, then the predicate is immediately returned (line 8) and the process starts over again for the next parameter. If no predicates are found for any of the matching rules, the parameter is ignored and no input values will be generated for it (line 13).

The support of a predicate is obtained by running a COUNT SPARQL query as the one shown in Listing 2. This query may include

Algorithm 3: Search for predicates

Input: *param*: Parameter information
Input: n: Number of predicates selected when searching for predicates
Input: $minSupport$: Min. support of predicate
Input: $rules$: Set of matching rules
Output: *predicate*: A predicate representative of the parameter

```
1  foreach rule in rules do
2      keywords ← EXTRACTKEYWORDS(rule, param)
3      foreach keyword in keywords do
4          predicates ← EXTRACTPREDICATES(keyword, n)
5          foreach predicate in predicates do
6              support ← COMPUTESUPPORT(predicate)
7              if support ≥ minSupport then
8                  return predicate
9              end
10         end
11     end
12 end
13 return ∅
```

filters with the syntactic constraints of the API specification, namely regular expressions and min/max values (line 3 of Listing 2).

It is worth noting that, in this phase, each parameter is assigned only one predicate, except in the case of matching rule number 6, i.e., when the parameter name is split into multiple keywords. In this case, lines 3 to 11 of Algorithm 3 are executed once for each word of the parameter name. For instance, a parameter named `icao_iata` would be split into `icao` and `iata`, in which case, two predicates would be returned, i.e., one per keyword.

```
SELECT COUNT(DISTINCT ?isbn) WHERE {
 ?subject <http://dbpedia.org/ontology/isbn> ?isbn .
 FILTER regex(str(?isbn), '^([0-9]*[-| ]){4}[0-9]*$')
}
```

Listing 2. COUNT SPARQL query with filter by regex.

A.3 Search for test inputs

At this stage, the predicates obtained during the previous phase are used to drive the search for meaningful test inputs. When searching test inputs for a specific parameter, it is important to consider it in the context of all the parameters supported by the API operation.

The complete workflow of this step is shown in Algorithm 4. ARTE starts by executing a SPARQL query (similar to the one shown in Listing 3) containing all the parameters of the operation (line 1). Note that the *params* input contains all the information related to each parameter, including the predicate retrieved during the "Search for predicates" phase and syntactic constrains such as regular expressions for strings and maximum and minimum values for numbers.

```
SELECT DISTINCT ?title ?pages ?isbn WHERE {
 ?subject <http://dbpedia.org/property/title> ?title ;
      <http://dbpedia.org/ontology/numberOfPages> ?pages ;
      <http://dbpedia.org/ontology/isbn> ?isbn .
 FILTER (?pages >= 100)
 FILTER regex(str(?isbn), '^([0-9]*[-| ]){4}[0-9]*$')
}
```

Listing 3. SPARQL query for searching test inputs.

A minimum of $minValues$ values should be obtained for all parameters (100 in our evaluation). This is checked by creating the set $unsatisfiedParametersSet$ which contains those parameters for which this threshold was not achieved (line 2), and comparing the size of this set with the original list of parameters $params$ (line 3). Three scenarios are possible:

(1) All parameters achieve the threshold of $minValues$ ($|unsatisfiedParametersSet| = 0$). In this case, ARTE simply returns the input values extracted for each parameter (line 19).

(2) Some parameters do not achieve the threshold of $minValues$ ($0 < |unsatisfiedParametersSet| < |params|$). When this happens, ARTE makes a recursive call (line 17), and returns the

Algorithm 4: Search for test inputs

Input: $minValues$: Min. number of unique parameter values
Input: $params$: Parameters information (including their representative predicates)
Output: $testInputs$: Input values for each parameter

1 $testInputs \leftarrow$ EXECUTESPARQLQUERY($params$)
// Parameters that do not achieve the threshold
2 unsatisfiedParametersSet $\leftarrow \{p_i \in testInputs : |V_i| < minValues\}$
// Isolate one parameter
3 **if** $|unsatisfiedParametersSet| = |params|$ **then**
4 maxResults $\leftarrow 0$
5 isolatedParameter $\leftarrow \emptyset$
6 **foreach** *uP* **in** *unsatisfiedParametersSet* **do**
7 currentResults $\leftarrow$ EXECUTECOUNTQUERY(unsatisfiedParametersSet $-$ uP)
8 **if** *currentResults* $\geq$ *maxResults* **then**
9 maxResults $\leftarrow$ currentResults
10 isolatedParameter $\leftarrow$ uP
11 **end**
12 **end**
13 $testInputs \leftarrow testInputs \cup$ EXECUTESPARQLQUERY(isolatedParameter)
14 unsatisfiedParametersSet $\leftarrow$ unsatisfiedParametersSet - isolatedParameter
15 **end**
// Execute algorithm recursively with unsatisfied parameters
16 **if** $|unsatisfiedParametersSet| \neq 0$ **then**
17 $testInputs \leftarrow testInputs \cup$ SEARCHFORTESTINPUTS($minValues$, unsatisfiedParametersSet) // Recursive call
18 **end**
19 **return** $testInputs$

union of the test inputs previously obtained (line 1) and the test inputs returned by the recursive call, which will only include those unsatisfied parameters in the corresponding SPARQL query.

(3) None of the parameters achieve the threshold of $minValues$ ($|unsatisfiedParametersSet| = |params|$). In this case, ARTE executes a number of queries equal to the number of parameters, $params$. The purpose of these queries is to count the number of input values obtained when discarding one of the parameters (line 7). The parameter which, when discarded, its associated query yields the highest number of results, is isolated (lines 8-11). Then, ARTE executes a SPARQL query containing only the isolated parameter, from which its test inputs will be obtained (line 13). The isolated parameter is removed from the set of unsatisfied parameters and will no longer be considered as such (line 14). Finally, if the set of unsatisfied parameters is empty, ARTE simply returns the extracted test inputs (line 19), otherwise ARTE makes the recursive call explained in (2) (lines 16-18).

A.4 Automated generation of regular expressions

Finally, ARTE can optionally generate regular expressions for each target parameter based on the input values obtained in the previous phases, by integrating the tool RegexGenerator++ [4], [5], [6]. This allows to refine the search for syntactically valid inputs, improving the effectiveness of the approach.

Algorithm 5 depicts the complete execution flow of this phase. In the first place, ARTE increases the number of tries to generate a regular expression for the parameter (line 1), which serves as a stopping criterion of the overall process (see Algorithm 1, lines 13-15). Then, a regular expression is generated based on the valid and invalid values of the parameter (line 2), and its recall is computed on the set of valid and invalid values (line 3). If the recall is higher than $minRecall$ (line 4), set to 90% in our experiments, then the list of parameter values is filtered such that only those matching the regular expression are kept (line 5). In line 6, if the number of additional predicates extracted for the parameter is less than the maximum number of additional predicates to extract ($maxPredicates$, set to 3 in our evaluation), ARTE tries to extract a new predicate (the next in priority order according to the matching rules defined for the "Search for predicates" phase) using the

Algorithm 5: Automated generation of regular expression

Input: $param$: Parameter information (including its representative predicates)
Input: $inputValues$: Input values
Input: $minRecall$: Min. recall for accepting regex
Input: $maxPredicates$: Max. number of additional predicates to extract per parameter
Output: $inputValues$: Updated list of input values

1 $param.numTries \leftarrow param.numTries + 1$
2 regex $\leftarrow$ GENERATEREGEX($param.validValues$, $param.invalidValues$)
3 recall $\leftarrow$ COMPUTERECALL(regex, $param.validValues$, $param.invalidValues$)
4 **if** *recall* $\geq minRecall$ **then**
5 $inputValues \leftarrow$ FILTERVALUES($inputValues$, regex)
6 **if** $param.numAdditionalPredicates < maxPredicates$ **then**
7 newPredicate $\leftarrow$ SEARCHADDITIONALPREDICATES($param$, regex)
8 **if** *newPredicate* $\neq \emptyset$ **then**
9 newInputValues $\leftarrow$ SEARCHNEWINPUTVALUES($param$, regex, newPredicate)
10 $param.predicates \leftarrow param.predicates \cup$ newPredicate
11 $param.numAdditionalPredicates \leftarrow param.numAdditionalPredicates + 1$
12 $inputValues \leftarrow inputValues \cup$ newInputValues
13 $param.numTries \leftarrow 0$
14 **end**
15 **end**
16 **end**
17 **return** $inputValues$

generated regular expression as a filter (line 7). If a new predicate is successfully extracted (line 8), ARTE performs a new search for input values using this predicate and the generated regular expression as a filter (line 9).

After obtaining new input values for the parameter, the new predicate recently used is added to the set of predicates that have been used to extract values for the parameter (line 10) and the number of additional predicates extracted for the parameter is increased (line 11). The new inputs are added to the set of input values of the parameter (line 12), and the current number of tries to generate a regular expression is reset to 0 (line 13). This is because the new input values may contain data with invalid formats not considered by the regular expression used as filter to extract them. The execution of this phase ends once ARTE returns the updated set of input values associated to the parameter (line 17).

REFERENCES

[1] K. Toutanova, D. Klein, C. D. Manning, and Y. Singer, "Feature-rich part-of-speech tagging with a cyclic dependency network," in *Proceedings of the 2003 conference of the North American chapter of the association for computational linguistics on human language technology-volume 1*. Association for Computational Linguistics, 2003, pp. 173–180. [Online]. Available: https://doi.org/10.3115/1073445.1073478

[2] S. Vijayarani, M. J. Ilamathi, and M. Nithya, "Preprocessing techniques for text mining-an overview," *International Journal of Computer Science & Communication Networks*, vol. 5, no. 1, pp. 7–16, 2015.

[3] V. Balakrishnan and E. Lloyd-Yemoh, "Stemming and lemmatization: a comparison of retrieval performances," *Lecture Notes on Software Engineering*, pp. 262–267, 2014. [Online]. Available: https://doi.org/10.7763/LNSE.2014.V2.134

[4] A. Bartoli, A. D. Lorenzo, E. Medvet, and F. Tarlao, "Inference of regular expressions for text extraction from examples," *IEEE Transactions on Knowledge and Data Engineering*, vol. 28, no. 5, pp. 1217–1230, May 2016. [Online]. Available: https://doi.org/10.1109/TKDE.2016.2515587

[5] A. Bartoli, A. De Lorenzo, E. Medvet, F. Tarlao, and M. Virgolin, "Evolutionary learning of syntax patterns for genic interaction extraction," in *Proceedings of the 2015 on Genetic and Evolutionary Computation Conference*. ACM, 2015, pp. 1183–1190. [Online]. Available: https://doi.org/10.1145/2739480.2754706

[6] A. Bartoli, A. De Lorenzo, E. Medvet, and F. Tarlao, "Can a machine replace humans in building regular expressions? a case study," *IEEE Intelligent Systems*, vol. 31, no. 6, pp. 15–21, 2016. [Online]. Available: https://doi.org/10.1109/MIS.2016.46

B

Acronyms

ACM	Association for Computing Machinery.
AFL	American Fuzzy Lop.
AGORA	Automated Generation of test Oracles for REST APIs.
AI	Artificial Intelligence.
API	Application Programming Interface.
ARTE	Automated generation of Realistic TEst inputs.
ASE	Automated Software Engineering.
BaaS	Backend as a Service.
BERT	Bidirectional Encoder Representations from Transformers.
BoW	Bag of Words.
CI/CD	Continuous Integration and Continuous Delivery/Deployment.
CPU	Central Processing Unit.
CRUD	Create, Read, Update and Delete.
CSV	Comma-Separated Values.
DBaaS	DataBase as a Service.
DBLP	Digital Bibliography and Library Project.
DOI	Digital Object Identifier.
DOM	Document Object Model.
DSRM	Design Science Research Methodology.
EAN	European Article Number.

ESEC/FSE European Software Engineering Conference and Symposium on the Foundations of Software Engineering.

extJWNL Extended Java WordNet Library.

FDIC Federal Deposit Insurance Corporation.

FDR Failure Detection Ratio.

FN False Negative.

FP False Positive.

GaaS Games as a Service.

GPT Generative Pre-trained Transformer.

GPU Graphics Processing Unit.

GRPO Group Relative Policy Optimization.

GRU Gated Recurrent Units.

GUI Graphical User Interface.

HTTP HyperText Transfer Protocol.

IaaS Infrastructure as a Service.

ICSE International Conference on Software Engineering.

ICSOC International Conference on Service Oriented Computing.

IEEE Institute of Electrical and Electronics Engineers.

IF Impact Factor.

IMFAHE International Mentoring Foundation for the Advancement of Higher Education.

IMP International Mentor Program.

IoT Internet of Things.

ISA Ingeniería del Software Aplicada.

ISO International Organization for Standardization.

ISSTA	International Symposium on Software Testing and Analysis.
JCR	Journal Citation Reports.
JISBD	Jornadas de Ingeniería del Software y Bases de Datos.
JSON	JavaScript Object Notation.
KAIST	Korea Advanced Institute of Science and Technology.
LLM	Large Language Model.
LSTM	Long-Short Term Memory.
MIT	Massachusetts Institute of Technology.
NER	Named Entity Recognition.
NLP	Natural Language Processing.
OAS	OpenAPI Specification.
OKAMI	Oracle Knowledge of API Methods for Innovation.
OMDb	Open Movie Database.
PaaS	Platform as a Service.
PLSE	Programming Languages and Software Engineering.
POS	Part of Speech.
PPO	Proximal Policy Optimization.
RAG	Retrieval-Augmented Generation.
RDF	Resource Description Framework.
REST	REpresentational State Transfer.
RLHF	Reinforcement Learning from Human Feedback.
RNN	Recurrent Neural Network.
RQ	Research Question.
SaaS	Software as a Service.
SATORI	Static API Test ORacle Inference.

SCORE	Smart COmputer systems Research and Engineering lab.
SEART	SoftwarE Analytics Research Team.
SIGSOFT	Special Interest Group on Software Engineering.
SOC4AI	Service-Oriented Computing for Artificial Intelligence.
SPARQL	SPARQL Protocol and RDF Query Language.
SQL	Structured Query Language.
SRC	Student Research Competition.
SUT	System Under Test.
TaaS	Testing as a Service.
TCL	Test Coverage Level.
TCM	Test Coverage Model.
TF-IDF	Term Frequency-Inverse Document Frequency.
TN	True Negative.
TOSEM	Transactions on Software Engineering and Methodology.
TP	True Positive.
TSE	Transactions on Software Engineering.
UAT	User Acceptance Testing.
UI	User Interface.
URI	Uniform Resource Identifier.
URL	Uniform Resource Locator.
USI	Università della Svizzera Italiana.
UW	University of Washington.
ViT	Vision Transformer.
W3C	World Wide Web Consortium.
WS-BPEL	Web Services Business Process Execution Language.

YAML Yet Another Markup Language.

XaaS Anything as a Service.

XML Extensible Markup Language.

Bibliography

[1] IEEE Standard for Software and System Test Documentation. *IEEE Std 829-2008*, pages 1–150, 2008. doi: 10.1109/IEEESTD.2008.4578383.

[2] SPARQL 1.1 Overview. W3C Recommendation, W3C, Mar. 2013. https://www.w3.org/TR/2013/REC-sparql11-overview-20130321/.

[3] GitLab issue. Invalid URLs in the getApiV4ProjectsIdBadges operation. https://gitlab.com/gitlab-org/gitlab/-/issues/473603, accessed April 2025.

[4] OMDb API. http://www.omdbapi.com, accessed April 2025.

[5] RESTCountries response fields. https://gitlab.com/restcountries/restcountries/-/blob/master/FIELDS.md, accessed April 2025.

[6] AFL documentation. https://afl-1.readthedocs.io/en/latest/, accessed April 2025.

[7] AFL GitHub repository. https://github.com/google/AFL, accessed April 2025.

[8] AliExpress Web API. https://openservice.aliexpress.com/doc/api.htm, accessed April 2025.

[9] Allure Report - Open-source HTML test automation report tool. https://allurereport.org, accessed April 2025.

[10] Amadeus Flights APIs. https://developers.amadeus.com/self-service/category/flights, accessed April 2025.

[11] Amadeus Hotel APIs. https://developers.amadeus.com/self-service/category/hotels, accessed April 2025.

[12] Amazon EC2. https://aws.amazon.com/ec2/, accessed April 2025.

[13] Repository APIs guru. https://github.com/APIs-guru/openapi-directory, accessed April 2025.

[14] ARTE Journal First, ESEC/FSE 2022. https://2022.esec-fse.org/details/fse-2022-journal-first/15/ARTE-Automated-Generation-of-Realistic-Test-Inputs-for-Web-APIs, accessed April 2025.

[15] Beautiful Soup. https://www.crummy.com/software/BeautifulSoup/, accessed April 2025.

[16] Beet repository. https://github.com/isa-group/Beet, accessed April 2025.

[17] Burp Suite - Application Security Testing Software. https://portswigger.net/burp, accessed April 2025.

[18] Chai Assertion Library. https://www.chaijs.com/api/bdd/, accessed April 2025.

[19] Circle CI. https://circleci.com, accessed April 2025.

[20] Claude 3.7 Sonnet System Card. https://anthropic.com/claude-3-7-sonnet-system-card, accessed April 2025.

[21] Stanford CoreNLP. https://stanfordnlp.github.io/CoreNLP/, accessed April 2025.

[22] Cypress: Testing Frameworks for Javascript. https://www.cypress.io, accessed April 2025.

[23] Daikon AGORA Docker image. https://hub.docker.com/r/javalenzuela/daikon_agora, accessed April 2025.

[24] Daikon front ends and instrumentation. https://plse.cs.washington.edu/daikon/download/doc/daikon.html#Front-ends-and-instrumentation, accessed April 2025.

[25] Daikon AGORA repository. https://github.com/JuanCarlosAlonsoValenzuela/daikon_modified, accessed April 2025.

[26] Daikon developer manual. New formatting for invariants. https://plse.cs.washington.edu/daikon/download/doc/developer.html#New-formatting-for-invariants, accessed April 2025.

[27] Datadog continuous testing. https://www.datadoghq.com/pricing/?product=continuous-testing#products, accessed April 2025.

[28] AGORA+ dataset. https://doi.org/10.5281/zenodo.15066014, accessed April 2025.

[29] ARTE dataset. http://doi.org/10.5281/zenodo.5792081, accessed April 2025.

[30] dblp API. https://dblp.org/faq/13501473.html, accessed April 2025.

[31] DeepL API. https://www.deepl.com/docs-api/, accessed April 2025.

[32] DHL API Developer Portal. https://developer.dhl.com, accessed April 2025.

[33] DHL Location Finder API. https://developer.dhl.com/api-reference/location-finder-unified, accessed April 2025.

[34] Domainr API. https://rapidapi.com/domainr/api/domainr, accessed April 2025.

[35] extJWNL (Extended Java WordNet Library) repository. https://github.com/extjwnl/extjwnl, accessed April 2025.

[36] Foursquare API. https://docs.foursquare.com/developer/reference/places-api-overview, accessed April 2025.

[37] Juan Carlos Alonso, premiado en la segunda conferencia de mayor impacto mundial en Ingeniería del Software. https://www.us.es/actualidad-de-la-us/juan-carlos-alonso-premiado-en-la-segunda-conferencia-de-mayor-impacto-mundial, accessed April 2025.

[38] Gemini - Google DeepMind. https://deepmind.google/technologies/gemini/, accessed April 2025.

[39] Gemini: A Family of Highly Capable Multimodal Models. https://arxiv.org/abs/2312.11805, accessed April 2025.

[40] GeoDBCities API Documentation. https://rapidapi.com/wirefreethought/api/geodb-cities, accessed April 2025.

[41] GitHub Actions. https://github.com/features/actions, accessed April 2025.

[42] Google Compute Engine. https://cloud.google.com/products/compute, accessed April 2025.

[43] GoogleTest. https://google.github.io/googletest/, accessed April 2025.

[44] Google Workspace: Secure Online Productivity & Collaboration Tools. https://workspace.google.com, accessed April 2025.

[45] Hello GPT-4o. https://openai.com/index/hello-gpt-4o/, accessed April 2025.

[46] GraphWalker - Model-based testing. https://github.com/graphwalker, accessed April 2025.

[47] Great Circle Mapper. https://www.greatcirclemapper.net/en/api.html, accessed April 2025.

[48] HTTP status codes. https://www.restapitutorial.com/httpstatuscodes.html, accessed April 2025.

[49] Hugging Face - The AI community building the future. https://huggingface.co, accessed April 2025.

[50] Google, Amazon, Nvidia and other tech giants invest in AI startup Hugging Face, sending its valuation to $4.5 billion. https://www.cnbc.com/2023/08/24/google-amazon-nvidia-amd-other-tech-giants-invest-in-hugging-face.html, accessed April 2025.

[51] Hypothesis documentation. https://hypothesis.readthedocs.io/en/latest/, accessed April 2025.

[52] Hypothesis GitHub repository. https://github.com/HypothesisWorks/hypothesis, accessed April 2025.

[53] International CORE Conference Rankings: ICORE. https://www.core.edu.au/icore-portal, accessed April 2025.

[54] ICORE Conference Portal. https://portal.core.edu.au/conf-ranks/, accessed April 2025.

[55] IMFAHE. https://www.imfahe.org, accessed April 2025.

[56] Apache Jena. https://jena.apache.org/index.html, accessed April 2025.

[57] Jenkins. https://www.jenkins.io, accessed April 2025.

[58] Jest. https://jestjs.io, accessed April 2025.

[59] JSONMutator. https://github.com/isa-group/JSONmutator, accessed April 2025.

[60] JUnit 5. https://junit.org/junit5/, accessed April 2025.

[61] libFuzzer - a library for coverage-guided fuzz testing. https://llvm.org/docs/LibFuzzer.html, accessed April 2025.

[62] Meta Llama 3 License. https://www.llama.com/llama3/license/, accessed April 2025.

[63] The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation. https://ai.meta.com/blog/llama-4-multimodal-intelligence/, accessed April 2025.

[64] Mabl. https://www.mabl.com, accessed April 2025.

[65] Microsoft 365. https://www.microsoft.com/en-us/microsoft-365, accessed April 2025.

[66] Microsoft Azure App Service. https://azure.microsoft.com/en-us/products/app-service, accessed April 2025.

[67] Microsoft Azure Virtual Machines. https://azure.microsoft.com/en-us/products/virtual-machines, accessed April 2025.

[68] Nordic APIs. 20 Impressive API Economy Statistics. https://nordicapis.com/20-impressive-api-economy-statistics/, accessed April 2025.

[69] Numverify API Documentation. https://numverify.com/documentation, accessed April 2025.

[70] OpenAI o3-mini. https://openai.com/index/openai-o3-mini/, accessed April 2025.

[71] OpenAPI Initiative. https://www.openapis.org, accessed April 2025.

[72] OpenAPI Specification v3.1.1. https://spec.openapis.org/oas/v3.1.1.html, accessed April 2025.

[73] OpenAPI Specification. https://swagger.io/specification/, accessed April 2025.

[74] OKAMI dataset. https://huggingface.co/datasets/javalenzuela/okami-dataset, accessed April 2025.

[75] OpenAI API. https://platform.openai.com/docs/overview, accessed April 2025.

[76] PayPal API. https://developer.paypal.com/api/rest/, accessed April 2025.

[77] Postman Guide to API-first. https://www.postman.com/api-first/, accessed April 2025.

[78] Postman API Platform. https://www.postman.com, accessed April 2025.

[79] PostmanAssertify repository. https://github.com/JuanCarlosAlonsoValenzuela/PostmanAssertify, accessed April 2025.

[80] pytest documentation. https://docs.pytest.org/en/stable/, accessed April 2025.

[81] RapidAPI API directory. https://rapidapi.com/products/api-hub/, accessed April 2025.

[82] Test & Monitor APIs with Rapids API Hub. https://rapidapi.com/products/build-apis/test-and-monitor/, accessed April 2025.

[83] REST Assured. http://rest-assured.io, accessed April 2025.

[84] RESTCountries API issue. Empty idd value. https://gitlab.com/restcountries/restcountries/-/issues/220#note_1726639875, accessed April 2025.

[85] RESTCountries API. https://restcountries.com, accessed April 2025.

[86] RESTCountries commit fixing country with negative area bug. https://gitlab.com/restcountries/restcountries/-/commit/ee498c74ad21c93b66a577d63d2c8eacefc58d42, accessed April 2025.

[87] RESTCountries GitLab issue. Country with negative area. https://gitlab.com/restcountries/restcountries/-/issues/219, accessed April 2025.

[88] RESTest repository. https://github.com/isa-group/RESTest, accessed April 2025.

[89] SATORI repository. https://github.com/JuanCarlosAlonsoValenzuela/SATORI, accessed April 2025.

[90] Sauce Labs. https://saucelabs.com, accessed April 2025.

[91] Selenium with Python. https://selenium-python.readthedocs.io/, accessed April 2025.

[92] Selenium. https://www.selenium.dev, accessed April 2025.

[93] SmartBear. https://smartbear.com, accessed April 2025.

[94] Model-based Testing with SpecExplorer. https://www.microsoft.com/en-us/research/project/model-based-testing-with-specexplorer/, accessed April 2025.

[95] Spotify Web API. https://developer.spotify.com/web-api/, accessed April 2025.

[96] SRC Grand Finalists 2023. https://src.acm.org/grand-finalists/2023, accessed April 2025.

[97] El doctorando Juan Carlos Alonso gana el segundo premio en las ACM SRC Grand Finals. https://www.us.es/actualidad-de-la-us/el-doctorando-juan-carlos-alonso-gana-el-segundo-premio-en-la-acm-src-grand, accessed April 2025.

[98] Understanding the API-First Approach to Building Products. https://swagger.io/resources/articles/adopting-an-api-first-approach/, accessed April 2025.

[99] Testim. https://www.testim.io, accessed April 2025.

[100] Transformers documentation. https://huggingface.co/docs/transformers/en/index, accessed April 2025.

[101] Transformers library. https://github.com/huggingface/transformers, accessed April 2025.

[102] Travis CI: Simple, Flexible, Trustworthy CI/CD Tools. https://www.travis-ci.com, accessed April 2025.

[103] TRUST4AI website. https://trust4ai.github.io/trust4ai/, accessed April 2025.

[104] Twitch API. https://dev.twitch.tv/docs/api/, accessed April 2025.

[105] Uber Guest Trips API. https://developer.uber.com/docs/guest-rides/introduction, accessed April 2025.

[106] Vimeo REST API. https://developer.vimeo.com/api, accessed April 2025.

[107] Vimeo API. Working with Authentication. https://developer.vimeo.com/api/authentication, accessed April 2025.

[108] Write API test scripts in Postman. https://learning.postman.com/docs/writing-scripts/test-scripts/, accessed April 2025.

[109] Yannet Interian, University of San Francisco. https://www.usfca.edu/faculty/yannet-interian, accessed April 2025.

[110] Yelp API. https://docs.developer.yelp.com/docs/getting-started, accessed April 2025.

[111] YouTube API. List videos operation. https://developers.google.com/youtube/v3/docs/videos/list, accessed April 2025.

[112] Swagger Request Validator. https://mvnrepository.com/artifact/com.atlassian.oai/swagger-request-validat, accessed January 2025.

[113] Asos API. https://rapidapi.com/apidojo/api/asos2, accessed May 2021.

[114] Referential API. https://rapidapi.com/referential/api/referential, accessed May 2021.

[115] M. Abdin, J. Aneja, H. Behl, S. Bubeck, R. Eldan, S. Gunasekar, M. Harrison, R. J. Hewett, M. Javaheripi, P. Kauffmann, et al. Phi-4 Technical Report. *arXiv preprint arXiv:2412.08905*, 2024.

[116] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, et al. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774*, 2023.

[117] A. Afzal, C. Le Goues, and C. S. Timperley. Mithra: Anomaly Detection as an Oracle for Cyberphysical Systems. *IEEE Transactions on Software Engineering*, pages 1–1, 2021. doi: 10.1109/TSE.2021.3120680.

[118] P. Alian, N. Nashid, M. Shahbandeh, and A. Mesbah. Semantic Constraint Inference for Web Form Test Generation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2024, pages 932–944, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400706127. doi: 10.1145/3650212.3680332. URL https://doi.org/10.1145/3650212.3680332.

[119] J. C. Alonso. Automated generation of realistic test inputs for web APIs. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2021, pages 1666–1668, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450385626. doi: 10.1145/3468264.3473491. URL https://doi.org/10.1145/3468264.3473491.

[120] J. C. Alonso. Automated Generation of Test Oracles for RESTful APIs. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2022, pages 1808–1810, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450394130. doi: 10.1145/3540250.3559080. URL https://doi.org/10.1145/3540250.3559080.

[121] J. C. Alonso, A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés. Identificación Automática de Relaciones Metamórficas en Sistemas de Búsqueda. In *XXV Jornadas de Ingeniería del Software y Bases de Datos*, 2021.

[122] J. C. Alonso, A. Martin-Lopez, S. Segura, J. M. Garcia, and A. Ruiz-Cortes. ARTE: Automated Generation of Realistic Test Inputs for Web APIs. *IEEE Transactions on Software Engineering*, 2022. doi: 10.1109/TSE.2022.3150618.

[123] J. C. Alonso, S. Segura, and A. Ruiz-Cortés. Generación Automática de Oráculos de Prueba para APIs RESTful. In *XXVI Jornadas de Ingeniería del Software y Bases de Datos*, 2022.

[124] J. C. Alonso, A. Martin-Lopez, S. Segura, J. M. Garcia, and A. Ruiz-Cortés. ARTE: Automated Generation of Realistic Test Inputs for Web APIs. In *XXVII Jornadas de Ingeniería del Software y Bases de Datos*, 2023.

[125] J. C. Alonso, S. Segura, and A. Ruiz-Cortés. AGORA: Automated Generation of Test Oracles for REST APIs. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2023, pages 1018–1030, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702211. doi: 10.1145/3597926.3598114. URL https://doi.org/10.1145/3597926.3598114.

[126] J. C. Alonso, S. Segura, and A. Ruiz-Cortés. AGORA: Automated Generation of Test Oracles for REST APIs. In *XXVIII Jornadas de Ingeniería del Software y Bases de Datos*, 2024.

[127] J. C. Alonso, G. Bavota, A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés. SATORI: Static Test Oracle Generation for REST APIs. In *40th IEEE/ACM International Conference on Automated Software Engineering*, 2025. To be submitted.

[128] J. C. Alonso, M. D. Ernst, S. Segura, and A. Ruiz-Cortés. Test Oracle Generation for REST APIs. *ACM Trans. Softw. Eng. Methodol.*, Mar. 2025. ISSN 1049-331X. doi: 10.1145/3726524. URL https://doi.org/10.1145/3726524. Just Accepted.

[129] P. Ammann and J. Offutt. *Introduction to Software Testing*. Cambridge University Press, 2016.

[130] A. Arcuri. RESTful API Automated Test Case Generation with EvoMaster. *ACM Transactions on Software Engineering and Methodology*, 28(1):1–37, 2019. doi: 10.1145/3293455.

[131] V. Atlidakis, P. Godefroid, and M. Polishchuk. RESTler: Stateful REST API Fuzzing. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pages 748–758, 2019. doi: 10.1109/ICSE.2019.00083.

[132] V. Atlidakis, P. Godefroid, and M. Polishchuk. Checking Security Properties of Cloud Services REST APIs. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*, pages 387–397, 2020. doi: 10.1109/ICST46399.2020.00046.

[133] J. Ayerdi, V. Terragni, A. Arrieta, P. Tonella, G. Sagardui, and M. Arratibel. Generating Metamorphic Relations for Cyber-Physical Systems with Genetic Programming: An Industrial Case Study. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2021, pages 1264–1274, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450385626. doi: 10.1145/3468264.3473920. URL https://doi.org/10.1145/3468264.3473920.

[134] V. Balakrishnan and E. Lloyd-Yemoh. Stemming and lemmatization: a comparison of retrieval performances. *Lecture Notes on Software Engineering*, pages 262–267, 2014. doi: 10.7763/LNSE.2014.V2.134.

[135] O. Banias, D. Florea, R. Gyalai, and D.-I. Curiac. Automated Specification-Based Testing of REST APIs. *Sensors*, 21(16), 2021.

[136] E. Barlas, X. Du, and J. C. Davis. Exploiting input sanitization for regex denial of service. In *Proceedings of the 44th International Conference on Software Engineering*,

ICSE '22, pages 883–895, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450392211. doi: 10.1145/3510003.3510047. URL https://doi.org/10.1145/3510003.3510047.

[137] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering*, 41(5): 507–525, 2015. doi: 10.1109/TSE.2014.2372785.

[138] A. Bartoli, A. De Lorenzo, E. Medvet, F. Tarlao, and M. Virgolin. Evolutionary Learning of Syntax Patterns for Genic Interaction Extraction. In *Proceedings of the 2015 on Genetic and Evolutionary Computation Conference*, pages 1183–1190. ACM, 2015. doi: 10.1145/2739480.2754706.

[139] A. Bartoli, A. De Lorenzo, E. Medvet, and F. Tarlao. Can a machine replace humans in building regular expressions? A case study. *IEEE Intelligent Systems*, 31(6):15–21, 2016. doi: 10.1109/MIS.2016.46.

[140] A. Bartoli, A. D. Lorenzo, E. Medvet, and F. Tarlao. Inference of Regular Expressions for Text Extraction from Examples. *IEEE Transactions on Knowledge and Data Engineering*, 28(5):1217–1230, 2016. ISSN 1041-4347. doi: 10.1109/TKDE.2016.2515587.

[141] B. Beizer. *Software testing techniques (2nd ed.)*. Van Nostrand Reinhold Co., USA, 1990. ISBN 0442206720.

[142] S. Bird, E. Klein, and E. Loper. *Natural language processing with Python: analyzing text with the natural language toolkit*. " O'Reilly Media, Inc.", 2009.

[143] C. Bizer, J. Lehmann, G. Kobilarov, S. Auer, C. Becker, R. Cyganiak, and S. Hellmann. DBpedia-A crystallization point for the Web of Data. *Journal of web semantics*, 7(3):154–165, 2009.

[144] C. Bizer, T. Heath, and T. Berners-Lee. *Linked Data - The Story So Far*, pages 115–143. Association for Computing Machinery, New York, NY, USA, 1 edition, 2023. ISBN 9798400707940. URL https://doi.org/10.1145/3591366.3591378.

[145] A. Blasi, A. Goffi, K. Kuznetsov, A. Gorla, M. D. Ernst, M. Pezzè, and S. D. Castellanos. Translating code comments to procedure specifications. In *ISSTA 2018, Proceedings of the 2018 International Symposium on Software Testing and Analysis*, pages 242–253, Amsterdam, Netherlands, July 2018.

[146] A. Blasi, A. Gorla, M. D. Ernst, M. Pezzè, and A. Carzaniga. MeMo: Automatically identifying metamorphic relations in Javadoc comments for test automation. *Journal of Systems and Software*, 181:111041:1–13, Nov. 2021.

[147] A. Blasi, A. Gorla, M. D. Ernst, and M. Pezzè. Call Me Maybe: Using NLP to Automatically Generate Unit Test Cases Respecting Temporal Constraints. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, ASE '22, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9781450394758. doi: 10.1145/3551349.3556961. URL https://doi.org/10.1145/3551349.3556961.

[148] M. Böhme, C. Geethal, and V.-T. Pham. Human-In-The-Loop Automatic Program Repair. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*, pages 274–285, 2020. doi: 10.1109/ICST46399.2020.00036.

[149] M. Bozkurt and M. Harman. Automatically Generating Realistic Test Input from Web Services. In *Proceedings of 2011 IEEE 6th International Symposium on Service Oriented System (SOSE)*, pages 13–24. IEEE, 2011. doi: 10.1109/SOSE.2011.6139088.

[150] H. B. Braiek and F. Khomh. On testing machine learning programs. *Journal of Systems and Software*, 164:110542, 2020. ISSN 0164-1212. doi: https://doi.org/10.1016/j.jss.2020.110542. URL https://www.sciencedirect.com/science/article/pii/S0164121220300248.

[151] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language Models are Few-Shot Learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[152] C. Cadar, D. Dunbar, and D. Engler. KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, OSDI'08, pages 209–224, USA, 2008. USENIX Association.

[153] A. Carzaniga, A. Goffi, A. Gorla, A. Mattavelli, and M. Pezzè. Cross-checking oracles from intrinsic software redundancy. In *Proceedings of the 36th International Conference on Software Engineering*, ICSE 2014, pages 931–942, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450327565. doi: 10.1145/2568225.2568287. URL https://doi.org/10.1145/2568225.2568287.

[154] F. J. Cavero, J. C. Alonso, and A. Ruiz-Cortés. From Static to Intelligent: Evolving SaaS Pricing with LLMs. In *1st International Workshop on Service-Oriented Computing for AI Applications*, 2024.

[155] D. Chen, P. Hartout, P. Pellizzoni, C. Oliver, and K. Borgwardt. Endowing Protein Language Models with Structural Knowledge. *arXiv preprint arXiv:2401.14819*, 2024.

[156] T. Chen, K. Heo, and M. Raghothaman. Boosting static analysis accuracy with instrumented test executions. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2021, pages 1154–1165, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450385626. doi: 10.1145/3468264.3468626. URL https://doi.org/10.1145/3468264.3468626.

[157] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.

[158] S. R. Choudhary, A. Gorla, and A. Orso. Automated Test Input Generation for Android: Are We There Yet? (E). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 429–440, 2015. doi: 10.1109/ASE.2015.89.

[159] K. Claessen and J. Hughes. QuickCheck: a lightweight tool for random testing of Haskell programs. *SIGPLAN Not.*, 35(9):268–279, Sept. 2000. ISSN 0362-1340. URL https://doi.org/10.1145/357766.351266.

[160] D. Clerissi, G. Denaro, M. Mobilio, and L. Mariani. Plug the Database & Play With Automatic Testing: Improving System Testing by Exploiting Persistent Data. In *2020 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 66–77, 2020. doi: 10.1145/3324884.3416561.

[161] D. Clerissi, G. Denaro, M. Mobilio, and L. Mariani. DBInputs: Exploiting Persistent Data to Improve Automated GUI Testing. *IEEE Transactions on Software Engineering*, 50(9):2412–2436, 2024. doi: 10.1109/TSE.2024.3439002.

[162] J. Cobb, J. A. Jones, G. M. Kapfhammer, and M. J. Harrold. Dynamic Invariant Detection for Relational Databases. In *Proceedings of the Ninth International Workshop on Dynamic Analysis*, WODA '11, pages 12–17, New York, NY,

USA, 2011. Association for Computing Machinery. ISBN 9781450308113. doi: 10.1145/2002951.2002955. URL https://doi.org/10.1145/2002951.2002955.

[163] L. Copeland. *A Practitioner's Guide to Software Test Design*. Artech House, Inc., USA, 2003. ISBN 158053791X.

[164] D. Corradini, M. Pasqua, and M. Ceccato. Automated Black-box Testing of Mass Assignment Vulnerabilities in RESTful APIs, 2023. URL https://arxiv.org/abs/2301.01261.

[165] P. Cousot and R. Cousot. Abstract Interpretation Frameworks. *Journal of Logic and Computation*, 2, 08 1992. doi: 10.1093/logcom/2.4.511.

[166] R. A. DeMillo, R. J. Lipton, and F. G. Sayward. Hints on Test Data Selection: Help for the Practicing Programmer. *Computer*, 11(4):34–41, 1978. ISSN 0018-9162. doi: 10.1109/C-M.1978.218136.

[167] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang. Large Language Models Are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2023, pages 423–435, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702211. doi: 10.1145/3597926.3598067. URL https://doi.org/10.1145/3597926.3598067.

[168] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, 2019. URL https://arxiv.org/abs/1810.04805.

[169] E. Dinella, G. Ryan, T. Mytkowicz, and S. K. Lahiri. TOGA: A Neural Method for Test Oracle Generation. In *Proceedings of the 44th International Conference on Software Engineering*, ICSE '22, pages 2130–2141, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450392211. doi: 10.1145/3510003.3510141. URL https://doi.org/10.1145/3510003.3510141.

[170] Z. Dong, M. Böhme, L. Cojocaru, and A. Roychoudhury. Time-travel testing of Android apps. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ICSE '20, pages 481–492, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450371216. doi: 10.1145/3377811.3380402. URL https://doi.org/10.1145/3377811.3380402.

[171] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

[172] W. Dou, Z. Cui, Q. Dai, J. Song, D. Wang, Y. Gao, W. Wang, J. Wei, L. Chen, H. Wang, H. Zhong, and T. Huang. Detecting Isolation Bugs via Transaction Oracle Construction. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 1123–1135, 2023. doi: 10.1109/ICSE48619.2023.00101.

[173] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[174] S. Dustdar and W. Schreiner. A Survey on Web Services Composition. *International journal of web and grid services*, 1(1):1–30, 2005. doi: 10.1504/IJWGS.2005.007545.

[175] H. Ed-douibi, J. L. C. Izquierdo, and J. Cabot. Automatic Generation of Test Cases for REST APIs: A Specification-Based Approach. In *2018 IEEE 22nd International Enterprise Distributed Object Computing Conference*, pages 181–190, 2018.

[176] J. L. Elman. Finding structure in time. *Cognitive science*, 14(2):179–211, 1990.

[177] M. D. Ernst. Static and dynamic analysis: Synergy and duality. In *WODA 2003: Workshop on Dynamic Analysis*, pages 24–27, Portland, OR, USA, May 2003.

[178] M. D. Ernst, J. Cockrell, W. G. Griswold, and D. Notkin. Dynamically discovering likely program invariants to support program evolution. *IEEE Transactions on Software Engineering*, 27(2):99–123, Feb. 2001.

[179] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69(1):35–45, 2007. ISSN 0167-6423. doi: https://doi.org/10.1016/j.scico.2007.01.015. URL https://www.sciencedirect.com/science/article/pii/S016764230700161X. Special issue on Experimental Software and Toolkits.

[180] R. T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.

[181] M. R. Fine. *Beta testing for better software*. John Wiley & Sons, 2002.

[182] S. Fischer and C. Klammer. Automating Invariant Filtering: Leveraging LLMs to Streamline Test Oracle Generation.

[183] G. Fraser and A. Arcuri. Evosuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*, pages 416–419, 2011.

[184] A. Gamez-Diaz, P. Fernandez, A. Ruiz-Cortés, P. J. Molina, N. Kolekar, P. Bhogill, M. Mohaan, and F. Méndez. The Role of Limitations and SLAs in the API Industry. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1006–1014, 2019. doi: 10.1145/3338906.3340445.

[185] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, H. Wang, and H. Wang. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv preprint arXiv:2312.10997*, 2, 2023.

[186] G. Gay, S. Rayadurgam, and M. P. Heimdahl. Improving the Accuracy of Oracle Verdicts through Automated Model Steering. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering*, ASE '14, pages 527–538, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450330138. doi: 10.1145/2642937.2642989. URL https://doi.org/10.1145/2642937.2642989.

[187] L. Gazzola, M. Goldstein, L. Mariani, I. Segall, and L. Ussi. Automatic Ex-Vivo Regression Testing of Microservices. In *Proceedings of the IEEE/ACM 1st International Conference on Automation of Software Test*, AST '20, pages 11–20, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450379571. doi: 10.1145/3387903.3389309. URL https://doi.org/10.1145/3387903.3389309.

[188] C. Giuffrida, L. Cavallaro, and A. S. Tanenbaum. Practical automated vulnerability monitoring using program state invariants. In *2013 43rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 1–12, Los Alamitos, CA, USA, jun 2013. IEEE Computer Society. doi: 10.1109/DSN.2013.6575318. URL https://doi.ieeecomputersociety.org/10.1109/DSN.2013.6575318.

[189] P. Godefroid. Fuzzing: Hack, Art, and Science. *Commun. ACM*, 63(2):70–76, 2020.

[190] P. Godefroid, B.-Y. Huang, and M. Polishchuk. Intelligent REST API Data Fuzzing. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 725–736, 2020. ISBN 9781450370431. doi: 10.1145/3368089.3409719.

[191] P. Godefroid, D. Lehmann, and M. Polishchuk. Differential Regression Testing for REST APIs. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2020, pages 312–323, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450380089. doi: 10.1145/3395363.3397374. URL https://doi.org/10.1145/3395363.3397374.

[192] A. Goffi, A. Gorla, M. D. Ernst, and M. Pezzè. Automatic generation of oracles for exceptional behaviors. In *ISSTA 2016, Proceedings of the 2016 International Symposium on Software Testing and Analysis*, pages 213–224, Saarbrücken, Genmany, July 2016.

[193] A. Golmohammadi, M. Zhang, and A. Arcuri. Testing RESTful APIs: A Survey. *ACM Trans. Softw. Eng. Methodol.*, 33(1), Nov. 2023. ISSN 1049-331X. doi: 10.1145/3617175. URL https://doi.org/10.1145/3617175.

[194] S. Grant, H. Cech, and I. Beschastnikh. Inferring and Asserting Distributed System Invariants. In *Proceedings of the 40th International Conference on Software Engineering*, ICSE '18, pages 1149–1159, New York, NY, USA, 2018. Association for Computing Machinery. ISBN 9781450356381. doi: 10.1145/3180155.3180199. URL https://doi.org/10.1145/3180155.3180199.

[195] G. Grefenstette. Tokenization. In *Syntactic wordclass tagging*, pages 117–133. Springer, 1999.

[196] T. Gu, C. Sun, X. Ma, C. Cao, C. Xu, Y. Yao, Q. Zhang, J. Lu, and Z. Su. Practical GUI testing of Android applications via model abstraction and refinement. In *Proceedings of the 41st International Conference on Software Engineering*, ICSE '19, pages 269–280. IEEE Press, 2019. doi: 10.1109/ICSE.2019.00042. URL https://doi.org/10.1109/ICSE.2019.00042.

[197] R. Guha and D. Brickley. RDF Schema 1.1. W3C Recommendation, W3C, Feb. 2014. https://www.w3.org/TR/2014/REC-rdf-schema-20140225/.

[198] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948*, 2025.

[199] J. Haltermann and H. Wehrheim. Machine Learning Based Invariant Generation: A Framework and Reproducibility Study. In *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*, pages 12–23, Los Alamitos, CA, USA, apr 2022. IEEE Computer Society. doi: 10.1109/ICST53961.2022.00012. URL https://doi.ieeecomputersociety.org/10.1109/ICST53961.2022.00012.

[200] Z. Hatfield-Dodds and D. Dygalo. Deriving Semantics-Aware Fuzzers from Web API Schemas. In *2022 IEEE/ACM 44th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, pages 345–346, 2022. doi: 10.1145/3510454.3528637.

[201] Y. He, L. Zhang, Z. Yang, Y. Cao, K. Lian, S. Li, W. Yang, Z. Zhang, M. Yang, Y. Zhang, and H. Duan. TextExerciser: Feedback-driven Text Input Exercising for Android Applications. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1071–1087, 2020. doi: 10.1109/SP40000.2020.00071.

[202] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[203] A. Hogan. *Web of data*. Springer, 2020.

[204] S. B. Hossain and M. Dwyer. TOGLL: Correct and Strong Test Oracle Generation with LLMs, 2024. URL https://arxiv.org/abs/2405.03786.

[205] Y. Huai, Y. Chen, S. Almanee, T. Ngo, X. Liao, Z. Wan, Q. A. Chen, and J. Garcia. Doppelgänger Test Generation for Revealing Bugs in Autonomous Driving Software. In *Proceedings of the 45th International Conference on Software Engineering*, ICSE '23, pages 2591–2603. IEEE Press, 2023. ISBN 9781665457019. doi: 10.1109/ICSE48619.2023.00216. URL https://doi.org/10.1109/ICSE48619.2023.00216.

[206] R. Huang, W. Sun, Y. Xu, H. Chen, D. Towey, and X. Xia. A Survey on Adaptive Random Testing. *IEEE Transactions on Software Engineering*, 47(10):2052–2083, 2021. doi: 10.1109/TSE.2019.2942921.

[207] A. R. Ibrahimzada, Y. Varli, D. Tekinoglu, and R. Jabbarvand. Perfect is the Enemy of Test Oracle. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2022, pages 70–81, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450394130. doi: 10.1145/3540250.3549086. URL https://doi.org/10.1145/3540250.3549086.

[208] D. Jackson. *Software Abstractions: Logic, Language, and Analysis*. The MIT Press, 2012. ISBN 0262017156.

[209] D. Jacobson, G. Brail, and D. Woods. *APIs: A Strategy Guide*. O'Reilly Media, Inc., 2011. ISBN 1449308929, 9781449308926.

[210] Y. Jia and M. Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2010.

[211] C. G. Kapugama, V.-T. Pham, A. Aleti, and M. Böhme. Human-in-the-Loop Oracle Learning for Semantic Bugs in String Processing Programs. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2022, pages 215–226, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450393799. doi: 10.1145/3533767.3534406. URL https://doi.org/10.1145/3533767.3534406.

[212] S. Karlsson, A. Causevic, and D. Sundmark. QuickREST: Property-based Test Generation of OpenAPI Described RESTful APIs. In *International Conference on Software Testing, Validation and Verification*, pages 131–141, 2020.

[213] M. E. Khan and F. Khan. A comparative study of white box, black box and grey box testing techniques. *International Journal of Advanced Computer Science and Applications*, 3(6), 2012.

[214] M. Kim, Q. Xin, S. Sinha, and A. Orso. Automated Test Generation for REST APIs: No Time to Rest Yet. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2022, pages 289–301, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450393799. doi: 10.1145/3533767.3534401. URL https://doi.org/10.1145/3533767.3534401.

[215] M. Kim, D. Corradini, S. Sinha, A. Orso, M. Pasqua, R. Tzoref-Brill, and M. Ceccato. Enhancing REST API Testing with NLP Techniques. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2023, pages 1232–1243, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702211. doi: 10.1145/3597926.3598131. URL https://doi.org/10.1145/3597926.3598131.

[216] M. Kim, S. Sinha, and A. Orso. Adaptive REST API Testing with Reinforcement Learning. In *2023 38th IEEE/ACM International Conference on Automated*

Software Engineering (ASE), pages 446–458, Los Alamitos, CA, USA, sep 2023. IEEE Computer Society. doi: 10.1109/ASE56229.2023.00218. URL https://doi.ieeecomputersociety.org/10.1109/ASE56229.2023.00218.

[217] M. Kim, T. Stennett, D. Shah, S. Sinha, and A. Orso. Leveraging Large Language Models to Improve REST API Testing. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*, ICSE-NIER'24, pages 37–41, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400705007. doi: 10.1145/3639476.3639769. URL https://doi.org/10.1145/3639476.3639769.

[218] M. Kim, T. Stennett, S. Sinha, and A. Orso. A Multi-Agent Approach for REST API Testing with Semantic Graphs and LLM-Driven Inputs, 2024. URL https://arxiv.org/abs/2411.07098.

[219] M. Kim, S. Sinha, and A. Orso. LlamaRestTest: Effective REST API Testing with Small Language Models, 2025. URL https://arxiv.org/abs/2501.08598.

[220] S. Krishna. *Introduction to database and knowledge-base systems*, volume 28. World Scientific, 1992.

[221] J. Kukucka, L. Pina, P. Ammann, and J. Bell. CONFETTI: amplifying concolic guidance for fuzzers. In *Proceedings of the 44th International Conference on Software Engineering*, ICSE '22, pages 438–450, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450392211. doi: 10.1145/3510003.3510628. URL https://doi.org/10.1145/3510003.3510628.

[222] S. Lahiri and S. Roy. Almost Correct Invariants: Synthesizing Inductive Invariants by Fuzzing Proofs. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2022, pages 352–364, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450393799. doi: 10.1145/3533767.3534381. URL https://doi.org/10.1145/3533767.3534381.

[223] T. Le, T. Tran, D. Cao, V. Le, T. N. Nguyen, and V. Nguyen. KAT: Dependency-Aware Automated API Testing with Large Language Models . In *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*, pages 82–92, Los Alamitos, CA, USA, May 2024. IEEE Computer Society. doi: 10.1109/ICST60714.2024.00017. URL https://doi.ieeecomputersociety.org/10.1109/ICST60714.2024.00017.

[224] H. K. Leung and P. W. Wong. A study of user acceptance tests. *Software quality journal*, 6:137–149, 1997.

[225] M. Li, J. Cao, Y. Tian, T. O. Li, M. Wen, and S.-C. Cheung. COMET: Coverage-guided Model Generation For Deep Learning Library Testing. *ACM Trans. Softw. Eng. Methodol.*, 32(5), July 2023. ISSN 1049-331X. doi: 10.1145/3583566. URL https://doi.org/10.1145/3583566.

[226] X. Li, S. Wang, S. Zeng, Y. Wu, and Y. Yang. A Survey on LLM-based Multi-Agent Systems: Workflow, Infrastructure, and Challenges. *Vicinagearth*, 1(1):9, 2024.

[227] Y. Li, Z. Yang, Y. Guo, and X. Chen. DroidBot: a lightweight UI-guided test input generator for Android. In *Proceedings of the 39th International Conference on Software Engineering Companion*, ICSE-C '17, pages 23–26. IEEE Press, 2017. ISBN 9781538615898. doi: 10.1109/ICSE-C.2017.8. URL https://doi.org/10.1109/ICSE-C.2017.8.

[228] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, et al. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437*, 2024.

[229] P. Liu, X. Zhang, M. Pistoia, Y. Zheng, M. Marques, and L. Zeng. Automatic text input generation for mobile testing. In *Proceedings of the 39th International Conference on Software Engineering*, ICSE '17, pages 643–653. IEEE Press, 2017. ISBN 9781538638682. doi: 10.1109/ICSE.2017.65. URL https://doi.org/10.1109/ICSE.2017.65.

[230] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692*, 2019.

[231] Y. Liu, Y. Li, G. Deng, Y. Liu, R. Wan, R. Wu, D. Ji, S. Xu, and M. Bao. Morest: Model-Based RESTful API Testing with Execution Feedback. In *Proceedings of the 44th International Conference on Software Engineering*, ICSE '22, pages 1406–1417, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450392211. doi: 10.1145/3510003.3510133. URL https://doi.org/10.1145/3510003.3510133.

[232] Z. Liu, C. Chen, J. Wang, X. Che, Y. Huang, J. Hu, and Q. Wang. Fill in the Blank: Context-aware Automated Text Input Generation for Mobile GUI Testing.

In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 1355–1367, 2023. doi: 10.1109/ICSE48619.2023.00119.

[233] L. Madeyski, W. Orzeszyna, R. Torkar, and M. Józala. Overcoming the Equivalent Mutant Problem: A Systematic Literature Review and a Comparative Experiment of Second Order Mutation. *IEEE Transactions on Software Engineering*, 40(1):23–42, 2014. doi: 10.1109/TSE.2013.44.

[234] R. Mahmood, J. Pennington, D. Tsang, T. Tran, and A. Bogle. A Framework for Automated API Fuzzing at Enterprise Scale. In *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*, pages 377–388, Los Alamitos, CA, USA, apr 2022. IEEE Computer Society. doi: 10.1109/ICST53961.2022.00018. URL https://doi.ieeecomputersociety.org/10.1109/ICST53961.2022.00018.

[235] C. Manning and H. Schutze. *Foundations of statistical natural language processing*. MIT press, 1999.

[236] L. Mariani, M. Pezzè, O. Riganelli, and M. Santoro. Link: Exploiting the Web of Data to Generate Test Inputs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, pages 373–384, 2014.

[237] A. Martin-Lopez and J. C. Alonso. Testing of RESTful Web APIs. In *Service-Oriented Computing - ICSOC 2022 Workshops: ASOCA, AI-PA, FMCIoT, WESOACS 2022, Sevilla, Spain, November 29 - December 2, 2022 Proceedings*, pages 411–413, Berlin, Heidelberg, 2023. Springer-Verlag. ISBN 978-3-031-26506-8. doi: 10.1007/978-3-031-26507-5_43. URL https://doi.org/10.1007/978-3-031-26507-5_43.

[238] A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés. A Catalogue of Inter-Parameter Dependencies in RESTful Web APIs. In *International Conference on Service-Oriented Computing*, pages 399–414, 2019. doi: 10.1007/978-3-030-33702-5_31.

[239] A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés. Test Coverage Criteria for RESTful Web APIs. In *A-TEST*, pages 15–21, 2019.

[240] A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés. RESTest: Black-Box Constraint-Based Testing of RESTful Web APIs. In *International Conference on Service-Oriented Computing*, pages 459–475, 2020.

[241] A. Martin-Lopez, A. Arcuri, S. Segura, and A. Ruiz-Cortés. Black-Box and White-Box Test Case Generation for RESTful APIs: Enemies or Allies? In *International Symposium on Software Reliability Engineering*, 2021.

[242] A. Martin-Lopez, S. Segura, C. Müller, and A. Ruiz-Cortés. Specification and Automated Analysis of Inter-Parameter Dependencies in Web APIs. *IEEE Transactions on Services Computing*, 2021. doi: 10.1109/TSC.2021.3050610.

[243] A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés. RESTest: Automated Black-Box Testing of RESTful Web APIs. In *International Symposium on Software Testing and Analysis*, 2021.

[244] A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés. Online Testing of RESTful APIs: Promises and Challenges. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2022, pages 408–420, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450394130. doi: 10.1145/3540250.3549144. URL https://doi.org/10.1145/3540250.3549144.

[245] A. Mastropaolo, S. Scalabrino, N. Cooper, D. N. Palacio, D. Poshyvanyk, R. Oliveto, and G. Bavota. Studying the Usage of Text-To-Text Transfer Transformer to Support Code-Related Tasks. In *Proceedings of the 43rd International Conference on Software Engineering*, ICSE '21, pages 336–347. IEEE Press, 2021. ISBN 9781450390859. doi: 10.1109/ICSE43902.2021.00041. URL https://doi.org/10.1109/ICSE43902.2021.00041.

[246] P. McMinn, M. Shahbaz, and M. Stevenson. Search-Based Test Input Generation for String Data Types Using the Results of Web Queries. In *2012 IEEE Fifth International Conference on Software Testing, Verification and Validation*, pages 141–150. IEEE, 2012.

[247] T. Mikolov, K. Chen, G. Corrado, and J. Dean. Efficient Estimation of Word Representations in Vector Space, 2013. URL https://arxiv.org/abs/1301.3781.

[248] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean. Distributed Representations of Words and Phrases and their Compositionality. In C. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 26. Curran Associates, Inc., 2013. URL https://proceedings.neurips.cc/paper_files/paper/2013/file/9aa42b31882ec039965f3c4923ce901b-Paper.pdf.

[249] F. Molina, R. Degiovanni, P. Ponzio, G. Regis, N. Aguirre, and M. Frias. Training Binary Classifiers as Data Structure Invariants. In *Proceedings of the 41st International Conference on Software Engineering*, ICSE '19, pages 759–770. IEEE Press,

2019. doi: 10.1109/ICSE.2019.00084. URL https://doi.org/10.1109/ICSE.2019.00084.

[250] F. Molina, P. Ponzio, N. Aguirre, and M. Frias. EvoSpex: An Evolutionary Algorithm for Learning Postconditions. In *Proceedings of the 43rd International Conference on Software Engineering*, ICSE '21, pages 1223–1235. IEEE Press, 2021. ISBN 9781450390859. doi: 10.1109/ICSE43902.2021.00112. URL https://doi.org/10.1109/ICSE43902.2021.00112.

[251] F. Molina, M. d'Amorim, and N. Aguirre. Fuzzing Class Specifications. In *Proceedings of the 44th International Conference on Software Engineering*, ICSE '22, pages 1008–1020, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450392211. doi: 10.1145/3510003.3510120. URL https://doi.org/10.1145/3510003.3510120.

[252] F. Molina, A. Gorla, and M. d'Amorim. Test Oracle Automation in the Era of LLMs. *ACM Trans. Softw. Eng. Methodol.*, Jan. 2025. ISSN 1049-331X. doi: 10.1145/3715107. URL https://doi.org/10.1145/3715107. Just Accepted.

[253] D. Molinelli, A. Martin-Lopez, E. Zackrone, B. Eken, M. D. Ernst, and M. Pezzè. Tratto: A Neuro-Symbolic Approach to Deriving Axiomatic Test Oracles, 2025. URL https://arxiv.org/abs/2504.04251.

[254] G. J. Myers, C. Sandler, and T. Badgett. *The Art of Software Testing*. Wiley Publishing, 3rd edition, 2011. ISBN 1118031962.

[255] E. Nguyen, M. Poli, M. Faizi, A. Thomas, M. Wornow, C. Birch-Sykes, S. Massaroli, A. Patel, C. Rabideau, Y. Bengio, et al. HyenaDNA: Long-Range Genomic Sequence Modeling at Single Nucleotide Resolution. *Advances in neural information processing systems*, 36:43177–43201, 2023.

[256] J. W. Nimmer and M. D. Ernst. Static verification of dynamically detected program invariants: Integrating Daikon and ESC/Java. In *RV 2001: Proceedings of the First Workshop on Runtime Verification*, Paris, France, July 2001.

[257] J. W. Nimmer and M. D. Ernst. Automatic generation of program specifications. In *ISSTA 2002, Proceedings of the 2002 International Symposium on Software Testing and Analysis*, pages 232–242, Rome, Italy, July 2002.

[258] A. J. Offutt. Investigations of the software testing coupling effect. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 1(1):5–20, 1992.

[259] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training Language Models to Follow Instructions with Human Feedback. *Advances in neural information processing systems*, 35: 27730–27744, 2022.

[260] M. Palomo-Duarte, A. García-Domínguez, I. Medina-Bulo, A. Alvarez-Ayllón, and J. Santacruz. Takuan: A Tool for WS-BPEL Composition Testing Using Dynamic Invariant Generation. In B. Benatallah, F. Casati, G. Kappel, and G. Rossi, editors, *Web Engineering*, pages 531–534, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg. ISBN 978-3-642-13911-6.

[261] L. Pan, S. Cohney, T. Murray, and V.-T. Pham. EDEFuzz: A Web API Fuzzer for Excessive Data Exposures. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ICSE '24, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400702174. doi: 10.1145/3597503.3608133. URL https://doi.org/10.1145/3597503.3608133.

[262] M. Papadakis, Y. Jia, M. Harman, and Y. Le Traon. Trivial compiler equivalence: A large scale empirical study of a simple, fast and effective equivalent mutant detection technique. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, volume 1, pages 936–946. IEEE, 2015.

[263] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, and M. Harman. Mutation testing advances: an analysis and survey. In *Advances in computers*, volume 112, pages 275–378. Elsevier, 2019.

[264] A. Pecchia, S. Russo, and S. Sarkar. Assessing Invariant Mining Techniques for Cloud-Based Utility Computing Systems. *IEEE Transactions on Services Computing*, 13(1):44–58, 2020. doi: 10.1109/TSC.2017.2679715.

[265] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee. A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems*, 24(3):45–77, 2007.

[266] Qwen, A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Tang, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu. Qwen2.5 Technical Report, 2025. URL https://arxiv.org/abs/2412.15115.

[267] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever, et al. Improving Language Understanding by Generative Pre-Training, 2018.

[268] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, et al. Language Models are Unsupervised Multitask Learners. *OpenAI blog*, 1(8):9, 2019.

[269] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer, 2023. URL https://arxiv.org/abs/1910.10683.

[270] L. Richardson, M. Amundsen, and S. Ruby. *RESTful Web APIs*. O'Reilly Media, Inc., 2013. ISBN 1449358063, 9781449358068.

[271] M. Romero-Arjona, J. A. Parejo, J. C. Alonso, A. B. Sánchez, A. Arrieta, and S. Segura. AI-Driven Fairness Testing of Large Language Models: A Preliminary Study. In *1st International Workshop on Fairness in Software Systems*, 2025.

[272] G. Salton and C. Buckley. Term-weighting approaches in automatic text retrieval. *Information processing & management*, 24(5):513–523, 1988.

[273] G. Salton, A. Wong, and C.-S. Yang. A vector space model for automatic indexing. *Communications of the ACM*, 18(11):613–620, 1975.

[274] D. Schuler and A. Zeller. Covering and uncovering equivalent mutants. *Software Testing, Verification and Reliability*, 23(5):353–374, 2013.

[275] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[276] S. Segura, G. Fraser, A. B. Sanchez, and A. Ruiz-Cortés. A Survey on Metamorphic Testing. *IEEE Transactions on Software Engineering*, 42(9):805–824, 2016. doi: 10.1109/TSE.2016.2532875.

[277] S. Segura, J. A. Parejo, J. Troya, and A. Ruiz-Cortés. Metamorphic Testing of RESTful Web APIs. *IEEE Transactions on Software Engineering*, 44(11):1083–1099, 2018. doi: 10.1109/TSE.2017.2764464.

[278] S. Segura, J. C. Alonso, A. Martin-Lopez, A. Durán, J. Troya, and A. Ruiz-Cortés. Automated Generation of Metamorphic Relations for Query-Based Systems. In *2022 IEEE/ACM 7th International Workshop on Metamorphic Testing (MET)*, pages 48–55, 2022. doi: 10.1145/3524846.3527338.

[279] M. Shahbaz, P. McMinn, and M. Stevenson. Automated Discovery of Valid Test Strings from the Web Using Dynamic Regular Expressions Collation and Natural Language Processing. In *2012 12th International Conference on Quality Software*, pages 79–88, 2012. doi: 10.1109/QSIC.2012.15.

[280] M. Shahbaz, P. McMinn, and M. Stevenson. Automatic Generation of Valid and Invalid Test Data for String Validation Routines Using Web Searches and Regular Expressions. *Science of Computer Programming*, 97:405–425, 2015. doi: 10.1016/j.scico.2014.04.008.

[281] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*, 2024.

[282] R. Sharnagat. Named entity recognition: A literature survey. *Center For Indian Language Technology*, 1, 2014.

[283] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. *arXiv preprint arXiv:2408.03314*, 2024.

[284] D. Stallenberg, M. Olsthoorn, and A. Panichella. Improving Test Case Generation for REST APIs through Hierarchical Clustering. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering*, ASE '21, pages 117–128. IEEE Press, 2022. ISBN 9781665403375. doi: 10.1109/ASE51524.2021.9678586. URL https://doi.org/10.1109/ASE51524.2021.9678586.

[285] N. Sunman, Y. Soydan, and H. Sözer. Automated Web application testing driven by pre-recorded test cases. *J. Syst. Softw.*, 193(C), Nov. 2022. ISSN 0164-1212. doi: 10.1016/j.jss.2022.111441. URL https://doi.org/10.1016/j.jss.2022.111441.

[286] G. Team, A. Kamath, J. Ferret, S. Pathak, N. Vieillard, R. Merhej, S. Perrin, T. Matejovicova, A. Ramé, M. Rivière, et al. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.

[287] V. Terragni, G. Jahangirova, P. Tonella, and M. Pezzè. Evolutionary Improvement of Assertion Oracles. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2020, pages 1178–1189, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450370431. doi: 10.1145/3368089.3409758. URL https://doi.org/10.1145/3368089.3409758.

[288] H. Tian, G. Wu, J. Yan, Y. Jiang, J. Wei, W. Chen, S. Li, and D. Ye. Generating Critical Test Scenarios for Autonomous Driving Systems via Influential Behavior Patterns. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, ASE '22, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9781450394758. doi: 10.1145/3551349.3560430. URL https://doi.org/10.1145/3551349.3560430.

[289] K. Toutanova, D. Klein, C. D. Manning, and Y. Singer. Feature-rich part-of-speech tagging with a cyclic dependency network. In *Proceedings of the 2003 Conference of the North American Chapter of the Association for Computational Linguistics on Human Language Technology - Volume 1*, pages 173–180, 2003. doi: 10.3115/1073445.1073478.

[290] L. Tunstall, L. Von Werra, and T. Wolf. *Natural Language Processing with Transformers*. " O'Reilly Media, Inc.", 2022.

[291] T. Vassiliou-Gioles. A simple, lightweight framework for testing RESTful services with TTCN-3. In *2020 IEEE 20th International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, pages 498–505, 2020. doi: 10.1109/QRS-C51114.2020.00089.

[292] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[293] S. Vatsal and H. Dubey. A Survey of Prompt Engineering Methods in Large Language Models for Different NLP Tasks, 2024. URL https://arxiv.org/abs/2407.12994.

[294] E. Viglianisi, M. Dallago, and M. Ceccato. RestTestGen: Automated Black-Box Testing of RESTful APIs. In *International Conference on Software Testing, Verification and Validation*, 2020.

[295] S. Vijayarani, M. J. Ilamathi, and M. Nithya. Preprocessing techniques for text mining-an overview. *International Journal of Computer Science & Communication Networks*, 5(1):7–16, 2015.

[296] M. von Stein, D. Shriver, and S. Elbaum. DeepManeuver: Adversarial Test Generation for Trajectory Manipulation of Autonomous Vehicles. *IEEE Trans. Softw. Eng.*, 49(10):4496–4509, Oct. 2023. ISSN 0098-5589. doi: 10.1109/TSE.2023.3301443. URL https://doi.org/10.1109/TSE.2023.3301443.

[297] D. Vrandečić and M. Krötzsch. Wikidata: A Free Collaborative Knowledge base. *Commun. ACM*, 57(10):78–85, 2014. ISSN 0001-0782.

[298] H. Vu, T. Fertig, and P. Braun. Verification of Hypermedia Characteristic of RESTful Finite-State Machines. In *Companion Proceedings of the The Web Conference 2018*, WWW '18, pages 1881–1886, Republic and Canton of Geneva, CHE, 2018. International World Wide Web Conferences Steering Committee. ISBN 9781450356404. doi: 10.1145/3184558.3191656. URL https://doi.org/10.1145/3184558.3191656.

[299] T. Wanwarang, N. P. Borges, L. Bettscheider, and A. Zeller. Testing Apps With Real-World Inputs. In *Proceedings of the IEEE/ACM 1st International Conference on Automation of Software Test*, pages 1–10, 2020. ISBN 9781450379571.

[300] C. Watson, M. Tufano, K. Moran, G. Bavota, and D. Poshyvanyk. On Learning Meaningful Assert Statements for Unit Test Cases. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ICSE '20, pages 1398–1409, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450371216. doi: 10.1145/3377811.3380429. URL https://doi.org/10.1145/3377811.3380429.

[301] H. Wu, L. Xu, X. Niu, and C. Nie. Combinatorial Testing of RESTful APIs. In *Proceedings of the 44th International Conference on Software Engineering*, ICSE '22, pages 426–437, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450392211. doi: 10.1145/3510003.3510151. URL https://doi.org/10.1145/3510003.3510151.

[302] J. Wu, W. Gan, Z. Chen, S. Wan, and P. S. Yu. Multimodal Large Language Models: A Survey. In *2023 IEEE International Conference on Big Data (BigData)*, pages 2247–2256. IEEE, 2023.

[303] R. Yandrapally, S. Sinha, R. Tzoref-Brill, and A. Mesbah. Carving UI Tests to Generate API Tests and API Specification. In *Proceedings of the 45th International Conference on Software Engineering*, ICSE '23, pages 1971–1982. IEEE Press, 2023. ISBN 9781665457019. doi: 10.1109/ICSE48619.2023.00167. URL https://doi.org/10.1109/ICSE48619.2023.00167.

[304] H. Yu, Y. Lou, K. Sun, D. Ran, T. Xie, D. Hao, Y. Li, G. Li, and Q. Wang. Automated Assertion Generation via Information Retrieval and Its Integration with

Deep Learning. In *Proceedings of the 44th International Conference on Software Engineering*, ICSE '22, pages 163–174, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450392211. doi: 10.1145/3510003.3510149. URL https://doi.org/10.1145/3510003.3510149.

[305] X. Zeng, D. Li, W. Zheng, F. Xia, Y. Deng, W. Lam, W. Yang, and T. Xie. Automated test input generation for Android: are we really there yet in an industrial case? In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, FSE 2016, pages 987–992, New York, NY, USA, 2016. Association for Computing Machinery. ISBN 9781450342186. doi: 10.1145/2950290.2983958. URL https://doi.org/10.1145/2950290.2983958.

[306] J. Zhai, Y. Shi, M. Pan, G. Zhou, Y. Liu, C. Fang, S. Ma, L. Tan, and X. Zhang. C2S: Translating Natural Language Comments to Formal Program Specifications. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2020, pages 25–37, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450370431. doi: 10.1145/3368089.3409716. URL https://doi.org/10.1145/3368089.3409716.

[307] M. Zhang and A. Arcuri. Adaptive Hypermutation for Search-Based System Test Generation: A Study on REST APIs with EvoMaster. *ACM Trans. Softw. Eng. Methodol.*, 31(1), sep 2021. ISSN 1049-331X. doi: 10.1145/3464940. URL https://doi.org/10.1145/3464940.

[308] M. Zhang, A. Belhadi, and A. Arcuri. JavaScript Instrumentation for Search-Based Software Testing: A Study with RESTful APIs. In *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*, pages 105–115, Los Alamitos, CA, USA, apr 2022. IEEE Computer Society. doi: 10.1109/ICST53961.2022.00022. URL https://doi.ieeecomputersociety.org/10.1109/ICST53961.2022.00022.

[309] Y. Zhang, X. Gao, G. J. Duck, and A. Roychoudhury. Program Vulnerability Repair via Inductive Inference. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2022, pages 691–702, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450393799. doi: 10.1145/3533767.3534387. URL https://doi.org/10.1145/3533767.3534387.

[310] T. Zohdinasab, V. Riccio, and P. Tonella. DeepAtash: Focused Test Generation for Deep Learning Systems. In *Proceedings of the 32nd ACM SIGSOFT In-*

ternational Symposium on Software Testing and Analysis, ISSTA 2023, pages 954–966, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702211. doi: 10.1145/3597926.3598109. URL https://doi.org/10.1145/3597926.3598109.